Contents

1 pricklypear.borg **4**

2 ai-authoring.borg **6**

3 architecture.borg **12**

4 auth.borg **16**

5 branching.borg **27**

6 calendar.borg **33**

7 dabar.borg **34**

8 flashcards.borg **37**

9 image.borg **40**

10 lisp-interpreter.borg **57**

11 migrations.borg **63**

12 multi-user.borg **66**

13 notes.borg **70**

14 object-model.borg **75**

15 philosophy.borg **77**

16 product-experience.borg **79**

17 single-image.borg **84**

18 storage.borg **88**

19 supervisor.borg **92**

20 workflow.borg **95**

21 common/lib/auth_types.ml **97**

22 common/lib/branch.ml **99**

23 common/lib/manifest.ml **100**

24 common/lib/protocol.ml **102**

25 spikes/awa_probe/probe.ml **103**

26 spikes/awa_probe_probe.ml **104**

27 spikes/awa_ssh/spike.ml **105**

28 spikes/mirage_solo5/config.ml **107**

29 spikes/mirage_solo5/unikernel.ml **108**

30 common/test/common_test.ml **109**

31 image/lib/agent_tools.ml **111**

32 image/lib/ai_store.ml **129**

33 image/lib/auth_password.ml **136**

34 image/lib/auth_store.ml **137**

35 image/lib/boot.ml **145**

36 image/lib/crud.ml **146**

37 image/lib/http.ml **154**

38 image/lib/http_agent.ml **163**

39 image/lib/http_branches.ml **172**

40 image/lib/http_code.ml **180**

41 image/lib/http_core.ml **188**

42 image/lib/http_data.ml **200**

43 image/lib/http_pages.ml **209**

44 image/lib/lineedit.ml **225**

45 image/lib/llm_client.ml **226**

46 image/lib/loader.ml **237**

47 image/lib/packages.ml **239**

48 image/lib/primitives.ml **241**

49 image/lib/prims_pure.ml **245**

50 image/lib/repl_endpoint.ml **257**

51 image/lib/session.ml **259**

52 image/lib/session_types.ml **279**

53 image/lib/store.ml **280**

54 image/lib/store_db.ml **294**

55 image/bin/main.ml **297**

56 lisp/lib/ast.ml **298**

57 lisp/lib/effect.ml **302**

58 lisp/lib/env.ml **306**

59 lisp/lib/eval.ml **307**

60 lisp/lib/reader.ml **312**

61 image/test/ai_store_test.ml **318**

62 image/test/auth_password_test.ml **320**

63 image/test/packages_test.ml **321**

64 image/test/primitives_test.ml **322**

65 lisp/bin/read_test.ml **326**

66 lisp/bin/repl.ml **327**

67 lisp/test/effect_test.ml **329**

68 lisp/test/eval_test.ml **330**

1 pricklypear.borg

pricklypear

A multi-user Lisp operating system in the Smalltalk tradition: OCaml image binary, Postgres object store, htmx UI. Many humans, each with session + personal garden. Deploy topology may be one binary (single-image.borg); multi-user product is auth.multi-user-product + multi-user.isolation.

[status: partial]

> – *agent note:* it31 plan-pass #7 (product-truth reconciliation): Demoted project status implemented -> partial. The substrate (5 packages, 3 migrations, scripts/dev.sh, 92 alcotest cases, end-to-end smoke path SSH/line-protocol -> authd -> image -> REPL -> define -> call) IS real and green. That is not the same as the v1 product experience bar. Project stays partial until a user can: (1) log in without developer ceremony, (2) use base functions in a live image, (3) create/edit functions from the environment (not only raw REPL typing), and (4) optionally accept AI-proposed functions on a branch. New planned product surfaces added this pass: auth.login-experience, auth.web-login, auth.user-management, image.system-browser, image.browser-repl, image.function-editor, image.inspector, ai-authoring.*, and branching.ast-nodes. Systems sections remain mostly partial/ implemented as before; type-system stays planned (typed fields deferred). Solo5 target remains a v1.5 hardening axis.
>
> – *agent note:* it34 overnight foundation: monolithic pricklypear.borg split into chapter files for literate book structure. Root inlines each chapter project in book order. Backup: pricklypear.borg.monolith.bak.

philosophy

Book chapter – full text inlined from philosophy.borg

architecture

Book chapter – full text inlined from architecture.borg. SUPERSEDED by single-image.borg (it35) for topology, package-layout, and transport-choices.

single-image

Book chapter – the single-image pivot (it35). One process, single-tenant, no Mirage/Solo5, no supervisor, no authd. Successor to architecture/supervisor/multi-user.

object-model

Book chapter – full text inlined from object-model.borg

branching

Book chapter – full text inlined from branching.borg

lisp-interpreter

Book chapter – full text inlined from lisp-interpreter.borg

storage

Book chapter – full text inlined from storage.borg

multi-user

Book chapter – RETIRED by the single-image pivot (it35). Process-isolation model dropped; the system is single-tenant. Namespaces/budgets/permissions stay as Lisp-level safeguards. See single-image.borg. Kept for historical context; status dropped.

auth

Book chapter – full text inlined from auth.borg

supervisor

Book chapter – RETIRED by the single-image pivot (it35). Kept for historical context; status dropped. See single-image.borg for the successor topology.

image

Book chapter – full text inlined from image.borg

product-experience

Book chapter – full text inlined from product-experience.borg

notes

Book chapter – outline-first notes (org/roam/obsidian-shaped); full text inlined from notes.borg

calendar

Book chapter – month/day calendar; full text inlined from calendar.borg

flashcards

Book chapter – flashcards / spaced repetition; full text inlined from flashcards.borg

dabar

Book chapter – BSB study libraries (dabar); full text inlined from dabar.borg

migrations

Book chapter – full text inlined from migrations.borg

workflow

Book chapter – full text inlined from workflow.borg

ai-authoring

Book chapter – full text inlined from ai-authoring.borg

2 ai-authoring.borg

pricklypear-ai-authoring

Pricklypear book chapter: ai-authoring
[status: partial]

ai-authoring

AI as a first-class author of functions: propose, surgical AST rewrite/freeze, growth-from-seed, human merge gate, provenance
[status: partial]

> – *agent note:* it33: design expanded with growth-style coding tools (llm-inner/outer/seed) discussed with roerick. Still PARTIAL: blank-slate propose-function ships; surgical freeze rewrites and recursive seed growth are planned. Effects gates reference lisp-interpreter.effects. Runtime LLM-as-value holes are explicitly secondary to coding-tool freeze-on-accept.

AI is an author, not a privileged runtime. Machine- written code uses the same store, branch, and merge machinery as human code. Default trust model: AI writes only to a user (or agent) branch; promoting into system/* still requires admin merge policy.

Coding styles supported by the tool surface:

1) Blank-slate propose (propose-function 'name "spec") Whole new/replaced binding from a prompt.

2) Surgical freeze rewrite (CODING TOOL – preferred) (llm-inner 'name "concept") (llm-outer 'name "concept") LLM rewrites one AST component; on accept the result is FROZEN into function_defs (not left as an unevaluated hole). Emacs-ish replace-region with generated code, but tree-native.

3) Growth-from-seed (CODING TOOL) (llm-seed "build an app that ...") Recursively: create smallest clean core, then llm-outer until requirements are satisfied. Bottom-up / inside-out Lisp style operationalized.

4) Runtime generative eval (SECONDARY / later) Eval-time expansion of AI holes without freeze. Useful, but not the primary coding workflow.

Non-goals for first cut of new tools: - AI does not bypass author-runs-as-author - AI does not write system/* without merge policy - full addressable node table not required for whole-function inner/outer (paths into JSONB AST suffice); node IDs are a later upgrade (branching.ast-nodes)

tools

How an AI proposes or patches functions into the store
[status: implemented]

> – *agent note:* it32: IMPLEMENTED blank-slate path. image/lib/llm_client.ml OpenAI-compat via curl; Session.propose_function; prim propose-function; POST /api/ai/propose; author agent:<model>. Surgical rewrite + seed growth live in sibling subsections. 2026-07-11: agent-portal is the unified entry; propose remains a thin authoring helper on top of (agent ...).

v1 SHIPPED: - propose-function Lisp prim + /api/ai/propose - whole-function body from model, define on current branch - author = agent:<model> - never merges to system/* - local or remote OpenAI-compat endpoint via env LLM_API_BASE / LLM_API_KEY / LLM_MODEL

Shared client contract for ALL AI tools: - chat/completions (stream optional; SSE for live UI) - model returns code/AST only for author modes - parse

gate before any store write - effect inference after parse (lisp-interpreter.effects) - policy gate before freeze

Primary guest surface is agent-portal: durable sessions + one tool-capable loop ((agent-say sid prompt)). Full message history is resubmitted each provider call. propose-function / llm-* remain specialized authoring helpers outside the conversation store.

Sibling tools (exist as prims; product UX secondary): - ast-rewrite: llm-inner / llm-outer / llm-rewrite - growth-seed: llm-seed recursive outer growth

agent-portal

One durable agent: sessions + messages in Postgres, one tool-capable loop, full history resubmitted each provider call

[status: implemented]

> – *agent note:* 2026-07-12 IMPLEMENTED. Migration 005 + image/lib/ai_store.ml + Session.run_loop (full history resubmit each chat_turn) + prims agent-session-new/say/history/sessions/session-delete + temporary aliases agent/agent-task + POST /api/agent session_id + libs/ai wrappers. Verified: dune build image; ai_store alcotest round-trip; live create/append multi-turn user rows in PG; HTTP resume by session_id; delete clears thread. Token streaming deferred (Synthetic hang); non-stream turns + progress events. Full LLM tool-loop smoke needs LLM_API_KEY in image env (dev.sh should forward LLM_* from the parent shell). BORGE-1783840832 resolved.

==========
PRODUCT (finished shape – small and closed) ==========

An agent conversation is a durable thread owned by a user: - seed (system persona) fixed at create - ordered messages (user / assistant / tool) - optional tools allowlist on the session - every model call resubmits [system seed] + history

There is ONE loop. Tools are not a second product: - model may answer in text, or request allowlisted tools - kernel runs tools, appends results, continues - one-shot Q&A is the same loop with a short history

NOT in this product cut: - separate (agent) vs (agent-task) kernels - in-memory-only multi-turn that dies with the request - token streaming as a requirement (progress events yes) - chat UI polish (REPL/pp_eval + thin HTTP enough) - propose-function / llm-* merge into sessions (those stay specialized authoring helpers)

==========
PROVIDER CONTRACT (do not invent a second one) ==========

OpenAI-compatible POST /chat/completions:

{ model, messages: [...full history...], tools?: [...] }

The provider does not remember prior turns. "Multi-turn" means WE store messages and resubmit them every call. Tool rounds are ordinary messages in that list: assistant + tool_calls, then role=tool results, then another completions call with the longer list.

Default transport: non-stream chat_turn (reliable on Synthetic). Progress via on_event: Token | Tool of {name; summary} | Done | Fail HTTP may SSE those control events. Token stream:true is optional later, not part of acceptance for this cut.

==========
SCHEMA (migration 005_ai_sessions.sql) ==========

Additive-only. Named to avoid clash with fleet ‘sessions‘ and auth ‘auth_sessions‘.

ai_sessions: id uuid PK default gen_random_uuid() owner text NOT NULL – username (whoami) title text – optional; first user line ok seed text NOT NULL – system prompt; not a message row branch_ref text – branch

at create / last say tools boolean NOT NULL DEFAULT true model text – snapshot of LLM_MODEL at create created_at timestamptz NOT NULL DEFAULT now() updated_at timestamptz NOT NULL DEFAULT now()

ai_messages: id uuid PK default gen_random_uuid() session_id uuid NOT NULL REFERENCES ai_sessions(id) ON DELETE CASCADE seq int NOT NULL – dense order from 0 role text NOT NULL – user|assistant|tool – (system is session.seed only) content text – nullable for pure tool-call – assistant turns tool_calls jsonb – [{id,name,arguments}] or null tool_call_id text – set when role=tool created_at timestamptz NOT NULL DEFAULT now() UNIQUE (session_id, seq)

Indexes: ai_sessions (owner, updated_at DESC) ai_messages (session_id, seq)

Invariants: - seed lives ONLY on ai_sessions (no role=system rows) - messages are append-only in v1 (no edit/delete mid-thread except whole-session delete) - owner isolation: list/get/delete scoped to whoami - tools flag is session-level; per-turn override later

========== STORAGE INTERFACE (image/lib/ai_store.ml) ==========

Pure store module over Store.t / pgx. No LLM calls here. Maps rows <-> Llm_client.chat_message for the runner.

Types: type session = { id; owner; title : string option; seed; branch_ref : string option; tools : bool; model : string option; created_at; updated_at }

API (all Lwt): create_session store ~owner ~seed ?title ?branch_ref ?tools ?model () -> session get_session store ~owner ~id -> session option (* owner check; None if missing or not owned *) list_sessions store ~owner ?limit () -> session list (* updated_at DESC; default limit 50 *) delete_session store ~owner ~id -> bool list_messages store ~session_id -> chat_message list (* ordered by seq; roles user|assistant|tool only *) append_messages store ~session_id (chat_message list) -> unit (* assigns next seq values; touches updated_at *) touch_session store ~id ?title () -> unit messages_for_api session history -> chat_message list (* Msg System seed :: history *)

chat_message encoding: Msg {User|Assistant; content} -> role, content, tool_calls=null, tool_call_id=null Assistant_tools {content; tool_calls} -> role=assistant, content, tool_calls=jsonb Tool_result {tool_call_id; content} -> role=tool, content, tool_call_id System messages are NEVER stored; only injected at API build.

Errors: missing session, owner mismatch, bad role on write. Use Store.current_conn so with-transaction works if needed.

========== ONE RUNNER (after storage; same subsection product) ==========

Agent.run_loop ~session ~tools ~on_event ~max_steps : string Lwt.t

Algorithm: 1. history = list_messages(session.id) 2. loop step: msgs = messages_for_api session history turn = chat_turn ~messages:msgs ~tools:(if session.tools then allowlist else []) match turn with | Final text -> append [Msg Assistant text]; on_event Done; return text | Need_tools {content; tool_calls} -> append Assistant_tools; for each call: run allowlisted tool; append Tool_result; on_event Tool {...}; continue (cap max_steps) | Turn_error e -> on_event Fail; return error string

Guest entry (prims; wire after store): (agent-session-new seed) -> sid (agent-say sid prompt) -> text append user msg, run_loop, return final assistant text (agent-history sid) -> list of role/content pairs (agent-sessions) -> recent threads for whoami (agent-session-delete sid) -> ok

Library (libs/ai) becomes thin: ai/ask = new-or-sticky session + say (tools may stay on) ai/do = same (explicit tools session) Deprecated as peer kernels (keep temporary aliases): (agent seed prompt) -> new session + one say (agent-task prompt) -> new tools session + one say

propose-function / install-function remain OUTSIDE this loop as one-shot authoring helpers (they may share llm_client.chat_turn but not ai_sessions).

==========
CAPABILITY / PROGRESS / NON-GOALS ==========

Capability: prims labeled Ai (existing effect lattice). Progress: on_event / optional SSE control events. Not required: OCaml 5 effects, WebSocket mid-turn input, message edit, shared multi-user threads, token streaming.

MAKE ORDER 1. migration 005 + scripts/dev.sh apply list 2. image/lib/ai_store.ml + dune + round-trip test 3. run_loop on ai_store (replace in-memory agent_task msgs) 4. prims + libs/ai aliases 5. HTTP session_id on /api/agent (thin)

ast-rewrite

Surgical coding tools: llm-inner / llm-outer / llm-rewrite with freeze-on-accept into stored AST

[status: implemented]

> – *agent note:* it33: NEW. Roerick's coding-tool model: not primarily runtime hole expansion; editor/session command that pings LLM to write one AST component, previews, freezes into function_defs. Emacs replace-region analogy. Depends on llm_client + AST json round-trip (already ship) + effects inference (planned).

PURPOSE Iterative code growth/refactor against live stored trees. Human (or command) supplies a concept; model rewrites ONE component; accept freezes into Postgres AST + live env.

PRIMS / COMMANDS

(llm-rewrite name prompt . options) General form. Options may include: mode: 'replace | 'wrap ; default replace path: structural path ; default whole body as: new-name ; non-destructive rename target branch: branch-ref ; work on a branch first accept: 'auto | 'preview ; default preview in UI, ; REPL may auto with flag

(llm-inner name prompt . options) Sugar: mode=replace on function body (or selection path). "Improve/replace the organ."

(llm-outer name prompt . options) Sugar: mode=wrap on whole function. "Put tissue around the organ." Old behavior preserved via call-through or careful inline; prefer call-through to name-old or nested original subtree.

FREEZE-ON-ACCEPT SEMANTICS (critical) 1. Load current function AST from store/env 2. Select target subtree (body / path / whole) 3. Build surgical prompt (see PROMPT CONTRACT) 4. Ping LLM local or remote 5. Parse response as s-expr or JSON AST only 6. Splice into tree (replace or wrap) 7. infer_effects on result 8. policy check (default allow Pure|ReadDb|Html; escalate WriteDb|Branch|Publish|Ai) 9. Preview diff (pretty + optional AST) 10. On accept: - persist function_defs.body JSONB - rebind live env - author = agent:<model> - optional provenance run id 11. On reject: no store mutation

This is NOT leaving (llm-inner ...) in the program. After accept, the program contains real Lisp AST only.

PROMPT CONTRACT (surgical AST author) System: You are editing a Pricklypear Lisp AST. Return ONLY one replacement form: a single s-expression (or JSON AST). No markdown, no essay. You write ONE component of a larger program tree.

User payload includes: - mode replace|wrap - function name - user concept / intent - current function pretty - current function AST json - selected path + selected subtree pretty/json - available prims summary - allowed effect set - for wrap: instruction to preserve behavior and call through original unless inlining is clearer

Model output: - full lambda when replacing whole binding body - subtree only when replacing a selection hole - wrapper lambda when mode=wrap

VALIDATION GATES - reader/json parse success - splice shape compatibility - effect policy - optional smoke eval of (name sample-args) if provided

UI SURFACE - function editor command: Rewrite? / Wrap? - structural selection path when structure mode exists - REPL prims for power users - always show diff before freeze unless accept:'auto

DEFAULTS (safety) - prefer work on current branch; offer agent/* branch - prefer preview over silent overwrite - keep old body in diff buffer / one-level undo - non-destructive 'as new-name always available

NON-GOALS (this subsection) - recursive app growth (see growth-seed) - runtime hole expansion without freeze - multi-file repo patches

growth-seed

llm-seed: recursive bottom-up growth via smallest core + repeated llm-outer until requirements satisfied

[status: implemented]

> – *agent note:* it33: NEW. Roerick coding style operationalized: start from one little function, grow outward. llm-seed orchestrates propose/inner for the seed then recursive llm-outer with a planner prompt until the high-level requirement is met or caps hit. This is how LLMs can be made to work inside-out instead of only top-down whole-app dumps.

PURPOSE Operationalize inside-out Lisp development for AI: seed a minimal clean function, then wrap/grow until the stated requirements are satisfied.

TOP-LEVEL COMMAND

(llm-seed requirement . options)

Options (sketch): name: optional root name hint branch: default fork to user/ai-<stamp> or agent/* max-steps: default small (e.g. 4..8) budget: token/time caps effects-allow: default Pure|ReadDb|Html stop: 'ask | 'auto-judge

ALGORITHM (orchestrator, not one giant prompt)

Step 0 – Plan (optional short call) Given requirement, emit a growth outline: core behavior, successive wrappers, final page? Human may edit plan later; v1 can be implicit.

Step 1 – Seed (smallest clean core) Prompt specialty: "Write the SMALLEST and CLEANEST pure-ish function that captures the essential nucleus of: <req>. One lambda only. No wrapping, no IO unless required." Freeze as seed name (e.g. notes-core). Run a tiny smoke if possible.

Step 2..N – Grow outward (llm-outer loop) While not satisfied and steps < max-steps: a. Judge prompt (or human): "Does current function set satisfy <req>? If not, what SINGLE outer responsibility should be added next (validation, IO, page, composition)?" b. If satisfied -> stop c. Else llm-outer on current tip with that single responsibility (surgical wrap prompt) d. Freeze wrapper as new tip (or replace tip name while keeping -vN history / old symbol) e. Record step in a growth log (session + optional data_rows type 'ai-growth-step)

Step Final – Optional publish surface If requirement implies a page and effects allow: define page handler + (route ...) Still no auto-merge to system/*.

JUDGE / STOP CONDITIONS - model judge returns satisfied=true - human stops - max-steps reached - effect escalation would exceed allowlist - parse/policy failures exceed retry budget

WHY RECURSIVE OUTER NOT ONE-SHOT APP DUMP - preserves a working center at every step - each layer is runnable/testable in the image - matches bottom-up Lisp craft - forces LLMs into step-sized commits - diffs remain reviewable

PROMPT SPECIALIZATIONS Seed prompt: smallest nucleus, prefer Pure Outer prompt: one responsibility only, preserve prior behavior via call-through Judge prompt: boolean satisfied + next missing capability in one short structured response

FAILURE MODES / GUARDRAILS - onion wraps forever -> max-steps + dedupe judge - outer reimplements instead of wraps -> prompt + validation heuristic (must reference prior name) - effect creep -> policy gate per step - nondeterminism -> provenance + branch + growth log

RELATIONSHIP TO OTHER TOOLS llm-seed USES: - blank-slate propose or llm-inner for seed - llm-outer for each growth ring - effects inference/policy each freeze - review-merge for promotion to system/* llm-seed is an orchestrator over ast-rewrite.

NON-GOALS - fully autonomous unattended product engineering - hidden multi-minute agent swarms without logs - replacing human taste for when to re-inner/ collapse wrappers after growth

review-merge

Human gate: inspect AI changes, then merge with existing policy
[status: planned]

> – *agent note:* it33: still planned. Applies to propose, ast-rewrite freezes, and growth-seed branches alike. Growth log should appear in review UI when seed exists.

Review loop: 1. AI commits one or more function_defs on a branch (propose, llm-inner/outer freezes, or llm-seed steps) 2. Human opens system browser / functions list / editor / inspect to read AST + effect badges 3. Human runs/tests via browser-repl or scripts/repl.sh 4. For growth-seed: step through growth log diffs 5. Human merges branch -> target via existing (merge ...) policy (admin-only into system/*) 6. Reject path: leave branch, redefine, or undo freeze

No separate "AI merge" privilege. Merge politics stay in branching.merge.

provenance

Attribution, audit, and trust labels for machine-authored code
[status: partial]

> – *agent note:* it32/it33: PARTIAL. author=agent:<model> on propose. ast-rewrite/growth-seed must record the same plus optional run id and parent step for seed rings.

v1 SHIPPED: author=agent:<model> on AI writes; visible in /bindings and inspect.

REQUIRED for rewrite/seed tools: - author agent:<model> on every freeze - optional metadata: run_id, parent_function, growth_step, prompt_hash, mode inner|outer|seed - growth-seed writes an inspectable step log

DEFERRED: visual agent badge styling, dedicated audit table. Trust rule remains branch + merge policy + effect allowlists.

3 architecture.borg

pricklypear-architecture

Pricklypear book chapter: architecture – SUPERSEDED by single-image.borg (it35) for topology, package-layout, and transport-choices. This chapter's multi-process / Mirage unikernel diagram is the RETIRED design. Kept for historical context; the live architecture is single-image.

[status: deprecated]

– *agent note:* it35 plan pass: the topology below (hypervisor -> supervisor -> per-user image unikernels + authd) is retired. The live topology is ONE image binary + Postgres + nginx. See single-image.borg. This chapter is kept so the book records the prior design and the reason for the pivot; its section statuses are not authoritative after it35.

architecture

The overall picture: hypervisor -> supervisor unikernel -> per-user image unikernels -> Postgres

[status: partial]

– *agent note:* 2026-07-12 status-reconcile: Doctrinal multi-unikernel picture superseded operationally by single-image; package-layout still accurate for lisp/common/migrations/image.

– *agent note:* it23 plan-pass #5: architecture is partially realized. Topology describes a supervisor + per-user image + auth + PG layout that IS built, except the production Solo5 + hypervisor-VM- request path is deferred (Solo5 spike still pending; v1 uses posix_spawn from Lwt_unix, not hypervisor VM requests). The SSH front door's awa-ssh path is ALSO real (it21) but is opt-in – the default v1 path is a non-crypto line protocol per BORGE-1783439434 (awa 0.3.1's static-auth-db constraint blocks user identity propagation through the awa path). The image's internal REPL endpoint matches the spec – the supervisor forwards SSH channels to it. The A/B rollback per image, always-on boot_mode, and health-check auto-restart loop are deferred sub-features of supervisor.fleet/health. Topology section as a whole is partial: implemented parts (the diagram) are accurate; deferred parts are called out explicitly in the agent notes in those subsections.

topology

Physical and logical layout

[status: partial]

– *agent note:* 2026-07-12 status-reconcile: RUNNING topology is single-image (see single-image.borg): one image process + Postgres + nginx. Multi-process supervisor/authd/fleet diagram is historical. Solo5/A-B still deferred.

– *agent note:* it23: topology diagram is implemented-by-construction. PG cluster is real. Auth unikernel connects as pricklypear_auth role (verified it5). Supervisor runs migrations + manages fleet + exposes admin HTTP + SSH front door (it4+it6). Per-user Image instances boot via Fleet.boot with PP_USER_ID manifest env vars (it4+it6). The 'flow for user adds function via SSH' (5 steps) is verified end-to-end by /tmp/pp_e2e_it6.py – awa-ssh handshake replaced with non-crypto line protocol in v1 (see BORGE-1783316371

> + BORGE-1783439434); image REPL endpoint accepts the forwarded session; define persists to function_defs and registers live. The 'flow for supervisor changes a primitive' (4 steps, A/B boot) is the deferred half – v1 has no A/B boot staging; primitive changes require a manual image rebuild + supervisor restart. Marked partial: the diagram-shaped intent is accurate as built; the A/B staging + auto-rollback is v1.5/v2 work. Carried-forward TODOs: Solo5 mirage target spike, A/B image staging, always-on boot_mode pin, health-check auto-restart loop.

Hypervisor box (KVM or Solo5 host) | +– Postgres (shared object store, namespaced, | SERIALIZABLE defaults; role-isolated | for auth tables) | +– Auth unikernel (separate; see section auth) | +– verify_ssh_key, verify_password | +– mint_token, verify_token (HMAC sign) | +– audit_log on every verify | +– connects as pricklypear_auth role (only | role that can read system_keys) | +– internal RPC, called by supervisor | +– Supervisor unikernel (Solo5 in prod, mirage/unix in dev) | +– manages fleet of per-user image instances +– A/B rollback per image +– health-check (ping image's SSH/HTTP) +– admin HTTP (httpaf-lwt-unix + pure-html) +– SSH front door (line protocol default; | awa-ssh OPT-IN side port) +– calls auth before booting/attaching | +– alice's image (boot-on-connect, loads | alice's branch bindings) +– bob's image (always-on, pinned) +– ... one per active session

Flow for "user adds a function via SSH": 1. Connect to supervisor SSH front door (v1: line protocol or OPT-IN awa-ssh); supervisor calls authd.verify_ssh_key -> user_id (or denied). 2. Supervisor boots/attaches the user's image, forwards the SSH channel to the image's REPL endpoint. 3. Define function -> reader parses s-expr -> AST -> stored as JSONB row in function_defs. 4. Registered in in-memory dispatch table, live immediately, no rebuild. 5. New htmx route /call/fn-name serves the result (the route is itself a stored function).

Flow for "supervisor changes a primitive": 1. Edit lib/primitives.ml in image source 2. Supervisor stages image v(N+1), A/Bs it 3. Health-check passes -> new default 4. Health-check fails -> auto-rollback to vN

package-layout

Directory and package structure

[status: implemented]

> – *agent note:* it23: package-layout matches the spec exactly. migrations/001_branches.sql + 002_auth.sql + 003_data.sql. lisp/ (pure, no I/O; 30 alcotest cases). common/ (pure types; 10 alcotest cases). image/ at lib/{store,primitives,session, repl_endpoint,http,lineedit,boot,crud}.ml + bin/unikernel.ml (mirage/unix). supervisor/ at lib/{fleet,health,process,store,ssh,ssh_awa, ssh_awa_shims,ssh_keys,auth_client,admin,boot}.ml + bin/supervisor.ml. auth/ at lib/{verify,tokens, admin_api,store,http,boot}.ml + bin/authd.ml + test/auth_test.ml (4 alcotest cases). proto/ in spec is common/ in the actual code (the boot_manifest, health protocol, auth_types modules live there). scripts/dev.sh orchestrates PG + authd + supervisor. Spec text is accurate as the actual code structure, modulo proto/ -> common/ rename.

pricklypear/ the whole system | +– migrations/ SQL, shared data | +– 001_branches.sql code store: branches, | function_defs, type_defs, | view_bindings, sessions | +– 002_auth.sql users, credentials, sessions, | auth_log, system_keys | (NOT data tables – those come | from type_defs accepted to main) | +– proto/ shared format defs | +– boot_manifest.json per-user boot config | +– health.proto health-check format | +– lisp/ the interpreter | PURE, no I/O dep | +– lib/ast.ml expr + value types | +– lib/reader.ml s-expr -> ast |

+– lib/env.ml lexical env | +– lib/eval.ml the evaluator | +– bin/repl.ml dev REPL, no image | +– test/eval_test.ml alcotest, no svc | +– image/ Mirage unikernel | per-user instance | +– lib/store.ml PG client, branch | +– lib/primitives.ml ~20 impure fns | +– lib/session.ml per-conn REPL env | +– lib/repl_endpoint.ml internal REPL protocol | (supervisor forwards SSH | channel here; not full SSH) | +– lib/http.ml httpaf-lwt-unix + pure-html | +– lib/crud.ml data_rows CRUD prims | +– lib/lineedit.ml readline stub (linenoise later) | +– lib/boot.ml reads manifest | +– bin/unikernel.ml mirage/unix entry | +– supervisor/ unikernel, root | manages fleet | +– lib/fleet.ml per-user instances | +– lib/health.ml ping images | +– lib/process.ml boot/kill (Solo5 | VM-request model) | +– lib/store.ml PG (manifests, sessions) | – Lwt via pgx_lwt + Io_unix | +– lib/ssh.ml line-protocol front door | +– lib/ssh_awa.ml OPT-IN awa-ssh side port | +– lib/auth_client.ml RPC client to authd | (verify_ssh_key, tokens) | +– lib/admin.ml httpaf-lwt-unix + pure-html, | fleet dashboard routes | +– bin/supervisor.ml entry point | +– auth/ the auth unikernel | (separate; see section auth) | +– lib/verify.ml verify_ssh_key, verify_password | +– lib/tokens.ml mint, verify_token, HMAC sign | +– lib/admin_api.ml create_user, add_credential (admin) | +– lib/store.ml PG (users, credentials, sessions, | auth_log, system_keys) – own PG role | +– lib/http.ml httpaf-lwt-unix, internal RPC | +– bin/authd.ml entry point | +– test/verify_test.ml crypto tests, no services | +– common/ shared OCaml lib | +– lib/manifest.ml boot manifest type | +– lib/branch.ml branch ref type | +– lib/protocol.ml health msg types | +– lib/auth_types.ml user, credential, token types | (shared supervisor <-> auth) | +– config/ infra config +– scripts/ dev helpers

transport-choices

Why each transport + framework choice
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: httpaf-lwt-unix + pure-html + pgx_lwt are the live stack. Supervisor SSH/awa path retired/optional. Solo5 deferred.
>
> – *agent note:* it23: transport choices are realized as documented, with the same SSH caveat: the spec text says 'awa-ssh listens on :22' which IS the design; v1 uses a non-crypto line protocol on :PP_SSH_PORT as the default (deferred from it6) and the real awa-ssh path is opt-in on PP_AWA_SSH_PORT (it21) – see BORGE-1783439434 for the awa 0.3.1 constraint. htmx via httpaf-lwt-unix + pure-html IS realized on both supervisor (admin.ml) and image (http.ml) – pages use HTML.Hx.* htmx attributes (it11 UI pages, it14 render-html prim). Admin HTTP routes are ~3 currently (GET /, /fleet, /health). Postgres via pgx_lwt with custom Io_unix shim (BORGE-1783389549). SERIALIZABLE-by-default via Store.with_transaction (it16). ONE v1 stack on both unikernels (httpaf-lwt-unix + pure-html + pgx_lwt) – confirmed; doctrinal details rewritten it31 to match. No Dream. Marked partial: default SSH is still line-protocol; real awa default depends on awa 0.5.x (BORGE-1783439434); Solo5 transport swap deferred.

SSH (supervisor front door – see image.ssh-interface): - v1 default: non-crypto line protocol on PP_SSH_PORT; OPT-IN real awa-ssh on PP_AWA_SSH_PORT (see BORGE-1783439434) - Supervisor authenticates via authd verify_ssh_key (NOT wyo.auth), forwards to image's internal REPL endpoint - Image does NOT run full SSH, just a line protocol on an internal port

htmx (image interface): - v1: httpaf-lwt-unix + pure-html (no Dream). Future Solo5 path: gluten-mirage + httpaf (there is no httpaf-mirage package on opam). - Routes ARE functions in the store - pure-html provides HTML.Hx.* htmx attribute support (the type-safe htmx attrs we want)

Admin (supervisor): - v1: httpaf-lwt-unix + pure-html (same stack as image) - ~3 routes live today (/, /fleet, /health); kill/rollback routes deferred - Hand-written small router

Postgres: - v1: pgx_lwt + custom Io_unix shim (POSIX). pgx_lwt_mirage is installed but needs a real Tcpip.Stack.V4V6 – Solo5/v1.5 axis only. - SERIALIZABLE by default for mutations - LISTEN/NOTIFY available for cache invalidation later - Lwt throughout unikernels (no Eio in the hot path; Eio only for pure-unix dev tools)

ONE stack on both unikernels in v1: httpaf-lwt-unix + pure-html + pgx_lwt(+Io_unix). No Dream. Future Solo5 stack swap stays behind the same Store/Http interfaces: gluten-mirage + httpaf + pgx_lwt_mirage. Reasoning: - Keep one mental model across supervisor/image - htmx is rendering chrome; shell/REPL is primary - pure-html gives HTML.Hx.* without Dream

4 auth.borg

pricklypear-auth

Pricklypear book chapter: auth
[status: partial]

auth

Multi-user identity: credentials, sessions, login, and binding each human to their garden
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Web login + admin + personal garden identity ship in-image. argon2, full multi-user process isolation, rich credential kinds still open.

> – *agent note:* 2026-07-12: PRODUCT CORRECTION. Multi-user is the system, not an optional phase and not cancelled by the single-image deploy pivot. Single-image simplified process topology (one binary to ship); it does NOT mean single-human. Complete auth = many users, each with credentials + session + personal branch + authorship; process-per-user isolation remains the isolation model (multi-user.isolation) when the fleet is live. nginx basic-auth is a temporary gate, not the product. Spec below is the complete multi-user auth system for implementation.

Auth is its own unikernel, separate from the supervisor. This is deliberate, not just componentization:

- AUDIT/TRUST BOUNDARY: auth holds credentials and verifies them. If compromised, the whole system is compromised. This image should be small, boring, never touched. The supervisor changes often; auth changes rarely. Separating means supervisor changes never risk auth code. - BLAST RADIUS: a memory bug in the supervisor cannot reach into auth's address space to read credentials. (Both are unikernels with their own address spaces; no shared memory.) - RESTART INDEPENDENCE: supervisor restarts (new fleet logic shipped) don't kill active sessions (auth keeps running, tokens stay valid). Auth restarts (rare) are explicit. - DEPENDENCY HYGIENE: auth pulls crypto primitives (argon2, ed25519, HMAC). Supervisor pulls process management. Each unikernel's dep surface stays small and Mirage-idiomatic.

Auth's job is FOUR PURE FUNCTIONS of the credential store (in Postgres): verify(cred) -> May<user_id> lookup(user_id) -> May<User> mint(user_id) -> token verify_token(token) -> May<user_id> Plus boot/attach provisioning, which is the supervisor's job, NOT auth's. Auth knows nothing about the fleet.

Auth is the ONLY thing that holds the session token signing key. The supervisor never sees raw credentials (passwords, keys) past the initial verify call – it only sees user_ids and signed tokens. Good separation.

credentials-store

The users, credentials, sessions, auth_log tables in Postgres
[status: implemented]

> – *agent note:* All tables shipped in migrations/002_auth.sql: users, credentials, auth_sessions [renamed from sessions to avoid the name clash with the image's sessions table, see BORGE-1783389549 note], auth_log, system_keys. The 'pricklypear_auth' PG role is GRANTed SELECT/INSERT/UPDATE ONLY on auth tables (not the branches/function_defs/data_rows tables) – defense-in-depth role isolation as the spec describes. auth/lib/store.ml

connects as ‘pricklypear_auth‘. All id columns have DEFAULT gen_random_uuid() (it5 migration fix). auth_log accumulates ssh_key verify rows verified end-to-end through /tmp/pp_e2e_it6.py.

HARDENING TODO carried forward (documented in spec text below): move the session_signing key from system_keys (on-disk in PG) to auth-image MEMORY ONLY (env var on boot). v1 keeps the key in system_keys for restart survival; the spec calls this an accepted personal/small-team risk now.

Tables (added by migrations, see migrations):
users (id uuid PK, username text UNIQUE, is_admin bool, created_at timestamptz)
credentials (id uuid PK, user_id uuid FK, kind text, – ’ssh_key’|’password’|’web_token’|’api_key’ public_data jsonb, – SSH key material for verification; – null for password (hash is secret) secret_data jsonb, – argon2 hash for password; – reference for api_key created_at timestamptz, last_used_at timestamptz)
sessions (id uuid PK, – the token’s opaque id user_id uuid FK, token_hash text, – HMAC of token; lookup target expires_at timestamptz, created_at timestamptz, revoked_at timestamptz NULL)
auth_log (id uuid PK, user_id text NULL, kind text, success bool, source_ip text, happened_at timestamptz)
system_keys (id text PK, – ’session_signing’ key_data bytea, – the HMAC signing key, raw created_at timestamptz)
KEY STORAGE (in-Postgres, role-isolated): - The session signing key lives in system_keys, read on auth boot. - system_keys is in a dedicated PG role that ONLY the auth image connects as. A DB dump from any other role does not include it (defense-in-depth, free). - KNOWN RISK: a DB dump from auth’s role, or an auth-image compromise, leaks the signing key -> all web sessions forgeable. Accepted for now (personal/small-team OS). - HARDENING GOAL (when this becomes stable & useful): move the signing key to auth-image MEMORY ONLY (env var on boot, never touches disk). Tokens survive auth restart only if the same key re-supplied. Invalidating all sessions on key rotation is acceptable then. - Other credentials (password hashes via argon2id, SSH public keys) in Postgres permanently; that’s correct and stable (these are public verification material, not signing keys).

verification

verify(cred) / lookup / mint / verify_token – the four pure functions
[status: partial]

– *agent note:* 2026-07-12 status-reconcile: Password verify works (interim sha256$). SSH-key/authd path retired with auth package. argon2id still deferred (BORGE-1783831698).

– *agent note:* Implementation it5/verified it6 onward for the SSH-key path; token path verified by auth/test/auth_test.ml (4 alcotest cases for HMAC determinism, random-opaque_id 32B, round-trip, malformed-token rejection).

IMPLEMENTED: - ‘verify_ssh_key(presented_key) -> May<user_id>‘ full path: auth/lib/verify.ml + auth/lib/store.ml SELECT the credential WHERE kind=’ssh_key’ AND public_data=jsonb; returns Ok uid or Denied. End-to-end verified: denied key ? {"ok":false}; admin key ? {"ok":true,"user_id":"<uuid>"}. - ‘mint(user_id)‘ + ‘verify_token(token)‘ via auth/lib/tokens.ml: HMAC-SHA256 with the system_keys signing key; opaque_id is 32 random bytes; token format is base64(opaque_id || sig); verify recomputes HMAC + compares. Mirage_crypto_rng initialized via Mirage_crypto_rng_unix.initialize (module Fortuna) on authd boot.

> NOT YET IMPLEMENTED (deferred – v1.5 / requires an opam install): - ‘verify_password(username, password)‘: STUB only – raises ‘Not_implemented‘ because argon2 is NOT on the poohstack opam switch. No web-login password verification. ACCEPTABLE for v1 because the SSH-first model means the primary credential IS the SSH key; password verification is the future web-login path. v1.5 will install ‘argon2‘ opam package + wire a real argon2id verify.
>
> Section marked ‘partial‘ until at least verify_password lands (the four pure functions the spec enumerates include both ssh_key AND password cred kinds).

SSH KEY VERIFICATION (for supervisor SSH front door): 1. Supervisor does SSH handshake, receives the presented public key. 2. Supervisor calls auth: verify_ssh_key(presented_key) -> May<user_id> 3. Auth: parse key, SELECT FROM credentials WHERE kind='ssh_key' AND public_data = key. Return user_id or denied. 4. Supervisor calls auth.lookup(user_id) -> User record (for is_admin, etc.). 5. Auth logs to auth_log.

PASSWORD VERIFICATION (for web login): 1. User POSTs username+password to the supervisor's /login (or the auth image's own endpoint – TBD; either way). 2. Auth: SELECT secret_data FROM credentials WHERE kind='password' AND user_id= (SELECT id FROM users WHERE username=$1). argon2-verify against the presented password. Return user_id or denied. 3. Auth logs to auth_log.

TOKEN MINTING (after web login): 1. Auth: generate random token bytes (32B), HMAC-sign with the session_signing key from system_keys, opaque_id = random, signed = HMAC(opaque_id). 2. Auth: INSERT INTO sessions (id=opaque_id, user_id, token_hash=signed, expires_at=...). 3. Auth: return token = base64(opaque_id || signed). Client stores as cookie.

TOKEN VERIFICATION (on web requests): 1. Client sends token cookie. Routing layer (supervisor or relayd-style proxy) calls auth.verify_token(token). 2. Auth: split token, recompute HMAC, compare; if match, lookup sessions row, check not revoked and not expired. Return user_id or denied. 3. Auth logs to auth_log.

TOKEN FORMAT (minimal, NOT JWT): opaque_id (32B random) || HMAC(opaque_id) We control both ends; don't need JWT's cross-system interop claims. Hash stored in sessions table enables revocation.

bootstrap-admin

First-admin creation via env var on auth first boot

[status: implemented]

> – *agent note:* Implementation it5/verified it6 onward. auth/lib/boot.ml checks: if users table is empty AND PP_BOOTSTRAP_ADMIN_KEY env var is set, INSERT into users (username='admin', is_admin=true) + INSERT into credentials (kind='ssh_key', public_data=the key, user_id=admin). Verified end-to-end: /tmp/pp_e2e_it6.py sends the SAME admin SSH key as authd was supplied via env var on first boot; the verify_ssh_key call returns the admin's user_id. (When users table is non-empty, the env var is ignored on subsequent boots – idempotent bootstrap.) Subsequent users are NOT part of this subsection's done bar – see auth.user-management (planned; admin_api still stubs).

Bootstrapping the first user, with no users yet:

1. Operator sets env var on the auth unikernel at first boot: PRICKLYPEAR_BOOTSTRAP_ADMIN_KEY=<ssh-ed25519 AAAA...> 2. Auth boot: if users table is empty AND the env var is set, INSERT into users (username='admin', is_admin=true) and into credentials (kind='ssh_key', public_data=the key, user_id=admin). 3. Auth boot:

warn loudly in logs that bootstrap was used, and recommend clearing the env var afterward. 4. After first boot, the env var is ignored.

Subsequent users: added by an admin via the supervisor's admin UI (which itself authenticates via auth) calling an auth primitive (create-user username, add-credential user_id ssh_key, etc.). These primitives require is_admin=true on the calling session.

No interactive bootstrap flow needed; the env-var pattern is simple, standard, and audit-logged.

audit-log

Every verification logged to auth_log
[status: implemented]

> – *agent note:* Implementation it5: auth/lib/store.ml's 'audit_log' INSERTs a row (user_id, kind, success, source_ip=NULL, happened_at default now()) per verify_ssh_key call. Verified: '/tmp/pp_e2e_it6.py' succession of one successful admin verify + one denied verify leave behind two auth_log rows (kind='ssh_key', success=t then f). Use cases from the spec text below – synthesized log for anomalies (cred misuse, brute-force, insider abuse) – are achievable with the existing log shape; a future stats page on the supervisor admin could surface these.

Every call to verify_ssh_key, password verification, mint, and verify_token writes a row to auth_log: (user_id NULL if unknown/failed, kind: 'ssh_key'|'password'|'mint'|'verify_token', success bool, source_ip text, happened_at timestamptz default now())

WHY from day one (despite personal-OS scope): - Cheap to add (one INSERT per verify call). - Invaluable when something goes wrong (compromised key? malicious insider? audit compliance if ever needed?). - Painful to retrofit (verification call sites scattered; adding log calls everywhere later means finding every one). - Standard practice; non-controversial.

The log is append-only in practice (no UPDATE/DELETE on auth_log). Retention policy deferred; for now, keep forever (cheap rows).

rpc-from-supervisor

How supervisor calls auth: internal signed/mTLS RPC
[status: deprecated]

> – *agent note:* 2026-07-12 status-reconcile: Supervisor+authd RPC retired by single-image pivot. Auth is in-process.

> – *agent note:* Implementation it6/verified it6 onward. supervisor/lib/auth_client.ml does an HTTP POST to authd's /verify_ssh_key endpoint (hand-rolled HTTP/1.1 over plain Lwt_unix socket; reads response bytes until '}' or EOF; parses JSON; returns Ok uid or Denied). Fail-closed: any transport error logs + returns Denied. End-to-end verified by /tmp/pp_e2e_it6.py through the dev.sh stack: python connects to supervisor SSH stub :17822 ? supervisor calls authd ? authd verifies ? image spawned. The supervisor itself never holds raw credentials in memory past the initial in-flight RPC call. (See BORGE-1783389549 for the note that hand-rolled HTTP-over-socket was needed because Lwt_io channels triggered EBADF mixed with raw Lwt_unix.read/write on the same fd.)
>
> DEFERRED from spec text: the actual spec doctrinal text below describes a signed/mTLS RPC. v1 uses plain HTTP+JSON on the internal network (loopback in dev, the inter-unikernel fabric in prod). The internal network IS the trust boundary in v1: images,

supervisor, and authd share an isolated fabric; spoofing the authd address requires being on the fabric.

Section marked 'partial' until (1) a real mTLS or signed-token RPC layer lands (a future hardening iteration that would let authd REJECT unauthenticated supervisor RPCs – currently the supervisor's identity is implicit via network position). Also deferred: token-result passing through the supervisor (v1 returns the user_id straight from the SSH verify RPC, no separate token minting on the SSH path because the SSH session IS the auth).TOKEN verification RPC IS implemented (authd /verify_token endpoint) for the future web-login path.

The supervisor calls auth over an internal RPC (HTTP+JSON, or a tighter binary protocol – HTTP is simpler and auth doesn't need throughput). On the internal hypervisor network.

AuthENTICATION OF THE RPC ITSELF: - mTLS between supervisor and auth: both present certs signed by a shared CA (the CA is the hypervisor itself, or a generated root for the deployment). - OR simpler: a shared static secret in a header, both images load from env vars, rotated manually. Less secure but v1-OK for a trusted internal network. - Either way: auth refuses calls from anything that doesn't present the right credential. The supervisor is the only legitimate caller.

Endpoints auth exposes (internal only): POST /verify_ssh_key (key) -> {user_id or denied} POST /verify_password (user, pw) -> {user_id or denied} POST /mint_token (user_id) -> {token, expires_at} POST /verify_token (token) -> {user_id or denied} GET /lookup/:user_id -> {user} (admin only) POST /admin/create_user (admin only) POST /admin/add_credential (admin only)

The supervisor holds a long-lived client cert (or shared secret) for these calls. It is the only caller; auth refuses all others.

v1 SHIPPED: plain HTTP+JSON on internal network, /verify_ssh_key and token paths live. DEFERRED: mTLS/shared-secret RPC auth of the caller; full admin endpoint wiring (see auth.user-management).

multi-user-product

COMPLETE multi-user product: many humans, each with identity, session, garden, authorship

[status: partial]

– *agent note:* 2026-07-12 status-reconcile: Session+branch multi-user works (many users, personal gardens, admin gates). Process isolation / fleet remains deferred; argon2 deferred. Not full complete-system isolation story.

– *agent note:* 2026-07-12: authoritative multi-user auth product. Replaces "single-tenant only" reading of single-image.auth. Single-image = one binary to deploy; multi-user = many people.

==========
MULTI-USER IS THE SYSTEM ==========

Pricklypear is a multi-user Lisp OS. Multiple humans share one deployment. Each human has: - a user row + credentials - a browser (and/or API) session - a personal branch garden: <username>/personal - authorship on their defines (function_defs.author) - isolation from other users' gardens and (when fleet is live) from other users' processes

single-image.borg simplifies HOW WE SHIP THE BINARY (one process to build/restart). It does NOT mean one human. multi-user.isolation (per-user image process) remains the isolation architecture for the complete system.

==========
PERSONAS ==========

Operator/admin – first user; can create users, merge to system/main, manage credentials, revoke sessions. Human user – logs in, works on <user>/personal, inherits system/main defaults, cannot admin-merge to system. Agent (as user) – authenticated as a human or service account; installs Lisp on that identity's branch; branch-locked for the task duration. Anonymous – only public routes (/login, health as policy).

==========
USER STORIES (complete set) ==========

US-BOOTSTRAP – First admin Empty users table ? bootstrap admin from env (PP_BOOTSTRAP_USER / PP_BOOTSTRAP_PASSWORD) OR one-time /setup form that creates admin then disables itself. is_admin=true. Site thereafter requires auth.

US-LOGIN – Browser sign-in GET /login ? username + password form. POST /login ? verify ? mint session ? Set-Cookie pp_session (HttpOnly; Secure; SameSite=Lax; Path=/). Redirect /. Bad credentials ? generic error + auth_log.

US-LAND – Land in my computer After login (or valid cookie): 1. Resolve user_id, username, is_admin from session. 2. Ensure branch <username>/personal exists as EMPTY child of system/main (inheritance, not full copy). 3. Image runtime identity = that user. 4. Checkout <username>/personal if not already. 5. (whoami) / chrome meta / define author = username. Alice never receives Bob's personal as current branch.

US-LOGOUT – Sign out POST /logout ? revoke auth_sessions row ? clear cookie ? /login.

US-SESSION – Stay signed in Every non-public request: load session by cookie ? valid unexpired unrevoked ? attach identity; else redirect /login (HTML) or 401 (API).

US-INVITE – Admin creates user Admin UI/API: create_user(username, is_admin?), set password credential, optional temp password. New user can US-LOGIN and gets own personal garden.

US-AGENT – Agent or laptop as a user Authenticate via same session cookie (cookie jar) OR API key credential (Authorization: Bearer ?) bound to a user_id. install_function / define write that user's current branch. Agent branch lock forbids checkout/ fork/merge mid-task. Author may be agent:<model> with provenance; capability still runs as the user.

US-ISOLATION – Process isolation (complete multi-user) Logging in boots or attaches that user's image process (supervisor fleet; multi-user.isolation). Alice's wedge does not take Bob offline. Auth never sees fleet; fleet never sees raw passwords – only user_id from verify.

US-COTENANT – Logical multi-user in one process (allowed implementation shape when fleet is not running): One image process serves many sessions; identity is per-request; personal branch is per-session; no cross- user write to another's personal. Weaker blast radius than process isolation but correct namespacing. Complete system MUST support US-ISOLATION; cotenant is not a substitute for the isolation model long-term.

US-PERMS – Authorization - define/route/deftype ? current branch only - merge to system/* ? admin only - merge to own owner/* ? free for that owner - admin user CRUD ? admin session only - data_rows shared (existing rule) unless future row-level owner column is added

US-AUDIT – Failures are boring and logged auth_log for login success/fail, logout, admin acts. No username enumeration in errors.

==========
END-TO-END STORY ==========

1. Deploy; bootstrap admin. 2. Admin opens https://host/ ? /login ? signs in ? admin/personal home. 3. Admin creates user "alice" + password. 4. Alice signs in ? alice/personal (empty garden), inherits system/main tools. 5. Alice

edits ui/menu ? pin on alice/personal only. 6. Bob signs in ? bob/personal; does not see Alice's pin as own; may inherit same main defaults. 7. Agent with Alice session installs helper ? alice/personal. 8. Alice logout; cookie dead. 9. With fleet: Alice's image process is hers; Bob's is his.

========== OBJECTS (complete) ==========

users(id, username UNIQUE, is_admin, created_at) credentials(id, user_id, kind, public_data, secret_data, created_at, last_used_at) kind: password | api_key | ssh_key | ? auth_sessions(id, user_id, token_hash, expires_at, created_at, revoked_at, user_agent, source_ip) auth_log(id, user_id NULL, kind, success, source_ip, happened_at)

Runtime image session (per connection / request): user_id, username, is_admin store.branch_id ? <username>/personal (default) author for define = username | agent:<model>

Password hashing: argon2id (complete system). Interim hash only acceptable until argon2 is available; must be upgraded in place.

Session token: opaque random id; store only HMAC/hash in DB; cookie carries raw id. Revocable. Expiry required.

========== HTTP SURFACE (complete) ==========

Public: GET /login POST /login GET /health (policy: public or authed – choose public for ops probes)

Authenticated (any user): POST /logout all product routes (/, /todos, /notes, /api/eval, ?) identity-dependent branch + author

Admin: /admin/users (list, create, set password, revoke sessions) merge policy to system/* already enforced in Lisp prims

API: Cookie session OR Authorization: Bearer <api_key> same identity attachment as browser

nginx: TLS termination; reverse proxy to image(s). Complete system does NOT rely on htpasswd as the user database. htpasswd may exist only as emergency break-glass.

========== FLEET ATTACH (complete multi-user isolation) ==========

On successful login / SSH verify: supervisor (or attach layer) resolves user_id ? Fleet.boot_or_attach(user) ? image with env: PP_USER_ID, PP_USERNAME, PP_BRANCH_ID, PP_BRANCH_NAME Proxy HTTP/REPL to that instance.

Auth component: verify + mint only. Never boots images. Image: trusts identity from supervisor/session attach; does not re-verify password every request if session token already verified at the edge of the image.

========== RELATION TO BRANCHES (already built) ==========

system/main – package defaults (seed from libs/*.pp) <user>/personal – empty-child garden; pins are "own" Effective Functions list = deduped closest-wins Agent locked to current branch during agent-task

Auth does not replace branching; it answers WHO is on which garden.

========== NON-GOALS ==========

- OAuth / OIDC / social login - Email magic links (unless added later as a credential kind) - Sharing one password (htpasswd) among humans as the permanent multi-user model

login-experience

What log in means: I present credentials and land in MY live image

[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: Login binds identity, ensure_personal + checkout personal, whoami is session user. Fleet

attach deferred (single-image).

Product bar (from multi-user-product): 1. Present credential (password browser, API key, or SSH key when SSH front door is live). 2. Verify ? user_id (never proceed on failure). 3. Mint or resume session; attach identity. 4. Ensure <username>/personal; set runtime branch. 5. With fleet: boot/attach that user's image. 6. User works; logout/revoke ends session.

Success: "I logged into my computer as me." Failure modes never leak other users' existence beyond generic invalid-credentials.

web-login

Browser password login, session cookie, middleware, logout

[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: GET/POST /login, pp_session cookie, middleware, logout ship in image/lib/http.ml + auth_store.ml. Interim password hash sha256$ (BORGE-1783831698); argon2 still deferred.

Complete browser path (part of multi-user-product):

GET /login Public form: username, password. No product chrome that assumes identity.

POST /login Body: username, password (+ CSRF token when forms are cookie-authenticated for other posts). verify_password against credentials.kind=password. On success: insert auth_sessions; Set-Cookie pp_session=<opaque>; redirect /. On failure: auth_log; same error for bad user/pass.

Middleware (request gate – see auth.route-visibility) Always ATTEMPT identity resolve (cookie then Basic). Classify path (system public / system private / user route visibility). Private + no identity ? login redirect (HTML) or 401 (API). Public may proceed anonymously. Identity present ? attach user even on public paths (optional auth).

POST /logout Revoke session; clear cookie; redirect /login.

Password: argon2id verify/hash in complete system. Cookie flags: HttpOnly; Secure; SameSite=Lax; Path=/.

nginx: TLS + reverse_proxy only for this path. htpasswd is not the user database.

HARD PUBLIC (kernel, not user-route table): /login /health /logout /welcome /book /book.pdf /src.tgz /code /code/* /assets /assets/* (public release face – product-experience.public-release) User-route public/private: auth.route-visibility.

route-visibility

Public vs private HTTP routes: store-backed visibility + OCaml request gate

[status: planned]

– *agent note:* 2026-07-12 plan: Today is_public_path is a hard OCaml allowlist (/login /health /logout); every other path including all Lisp (route ...) pages requires auth. Product needs opt-in public user pages (share links, public study, landing) without making /data /admin /api/eval world-readable. Extend existing launch gate – do not invent Express-style middleware chains. Visibility is data on view_bindings; OCaml enforces before handler runs.

==========

WHY ==========

The image already has a single request gate (launch in image/lib/http.ml): hard public allowlist, else require session/Basic. That is correct kernel shape – but it cannot express "this Lisp page is a public share link."

Product needs: - Default-private user routes (todos, notes, gardens) - Opt-in public user routes (marketing, shared deck view) - Kernel paths that are NEVER public via Lisp - Optional auth: public page still richer when logged in

NOT this subsection: - Per-row ACLs / Alice-vs-Bob data isolation (multi-user.permissions – later) - Capability bits on functions (separate) - OAuth / JWT (still out; wyo.auth / session cookie)

==========

GATE ALGORITHM (authoritative) ==========

For every HTTP request to the image:

1. Parse path (strip query). 2. Resolve identity OPTIONALLY: cookie pp_session ? else HTTP Basic ? else None. Do not fail yet; just compute user_opt. 3. Classify path into one of:

A. SYSTEM_HARD_PUBLIC /login /health /logout /welcome /book /book.pdf /src.tgz /code and /code/* /assets and /assets/* (extend only by OCaml change; see product-experience.public-release)

B. SYSTEM_HARD_PRIVATE Always require identity (Lisp cannot open these): /admin and /admin/* /data and /data/* /api/eval /api/agent /api/agent-task /api/ai/* /api/edit/* /api/recipe/* /bindings /binding/* /edit/* /repl /branches and /branches/* /types /pages /page/* /inspect/* /agent (Implementation: prefix/exact table in http.ml; when unsure, DEFAULT PRIVATE.)

C. USER_ROUTE Session.lookup_route path succeeds ? visibility from route record: "private" (default) | "public"

D. UNKNOWN No system page, no user route ? still require auth before 404 (do not leak route existence to anonymous via different status; v1 may 302 login then 404 after login – acceptable). Prefer: unauth ? login redirect for HTML; 401 for API Accept.

4. Decision: - If class A ? proceed (login form has no product chrome). - If class B or C-private or D: user_opt None ? deny (302 /login?next= or 401) user_opt Some ? attach_from_user ? proceed - If class C-public: user_opt Some ? attach_from_user ? proceed user_opt None ? proceed as ANONYMOUS (below)

5. PP_AUTH_DISABLED=1 ? proceed all (dev escape; unchanged).

No multi-layer middleware DSL. One gate function.

==========

ANONYMOUS SESSION (public + no identity) ==========

- (whoami) ? "anonymous" (string) or document VNil – pick one in make; prefer VStr "anonymous" easy tests. - Branch: system/main (read package defaults). Do NOT create or checkout a personal garden for anonymous. - author for any accidental write: "anonymous" - WRITE POLICY v1: CRUD prims (create/update/delete) and define/route/fork/merge/checkout are DENIED for anonymous (raise clear error or return error VStr). Public pages are read-oriented. public-write (POST forms that mutate) is PHASE 2 – not v1. - Chrome: product ui/nav MAY render; avoid admin links. If ui/nav fails, emergency chrome only.

==========

SCHEMA (additive migration 006_route_visibility.sql) ==========

ALTER TABLE view_bindings ADD COLUMN IF NOT EXISTS visibility text NOT NULL DEFAULT 'private'; – allowed values: 'private' | 'public' – no CHECK required if app validates; optional CHECK OK

Fork copy of view_bindings MUST copy visibility (UPDATE fork INSERT column lists).

list_routes / register_route SELECT/INSERT visibility. Default private for all existing rows (DEFAULT).

scripts/dev.sh migration loop: add 006_route_visibility.

==========

LISP API ==========

(route path handler) ? register private route (default). If route already exists

and only handler changes, KEEP prior visibility (so re-seed does not clobber an intentional public). If brand-new route ? private.

(route path handler 'public) (route path handler "public") ? register/update route with visibility=public. Third arg also accepts 'private / "private".

(public-route path handler) ? sugar for (route path handler 'public)

(route-visibility path) ? "public" | "private" | () () if no visible route on current branch chain.

Agent tools: register_route gains optional visibility field (default private).

========== INHERITANCE ==========
Same recursive CTE as today: nearest branch wins for a path. Child can override parent public?private or private?public by re-routing on personal.

========== UI ==========
/pages table: column Visibility (public|private). Optional: make-public / make-private forms later; v1 Lisp/REPL is enough.

========== SECURITY NOTES ==========
- Public handler still runs full Lisp: author must not leak secrets in public pages. Document footgun. - System hard-private list is deny-by-default for kernel surfaces; never consult view_bindings to open /admin. - data_rows remain shared storage; public route ? row ACL. Sharing one deck is handler logic (filter by id), not "all flash-card rows are public." - /api/eval stays hard-private (agents use Basic auth).

========== PHASES ==========
Phase 1 (this subsection make): migration 006, store/session route record, route prim arity, gate algorithm, anonymous read, /pages badge, smokes above. Phase 2: public POST / public-write visibility value; next= return path after login; CSRF for cookie posts. Phase 3: share tokens / signed URLs for single-resource public access without opening a whole route (flashcards).

========== ACCEPTANCE NARRATIVE ==========
Dev marks (route "/hello" "hi-page" 'public). Logged-out browser opens /hello and sees the page. Same browser on /todos still hits login. /data still login. After login, /hello shows whoami in chrome. Re-seed of private routes does not wipe an intentional public flag when only the handler is updated.

user-management

Admin creates and manages users and credentials

[status: implemented]

- *agent note:* 2026-07-12 status-reconcile: /admin/users create user, set password, revoke sessions; bootstrap_admin_if_empty. api_key/ssh_key credential UX still thin.

- *agent note:* admin_api create/list paths existed under authd; complete system needs in-product /admin/users UI + password credentials + revoke session, wired to the same users table.

Complete admin surface: create_user(username, is_admin?) set_password(user_id, password) add_credential(user_id, kind, ?) ; api_key, ssh_key list_users / get_user revoke_sessions(user_id) revoke_credential(id)

First admin: bootstrap-admin (env) when users empty. All later humans: this API/UI only.

UI: /admin/users (admin session required). Enforcement: is_admin on session; 403 otherwise.

Credentials order of support in complete system: password (web-login), api_key (agents/pp_eval), ssh_key (SSH front door when live).

identity-and-branch

Session identity drives personal garden and authorship
[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: ensure_personal_branch + checkout_personal at session make; define authorship uses username; agent_branch_lock exists.

On every authenticated request / image attach:
ensure_branch owner=username name=personal parent=system/main (EMPTY child; CTE inherit) store.branch_id = that id manifest.username = username manifest.branch_name = username/personal
(define name body) persists with author=username (or agent:<model> when agent installs under policy).
Functions UI: effective one-name-one-row; origin own vs inherited (already implemented).
Agent-task: agent_branch_lock blocks checkout/fork/merge (already implemented); must run under the user's session.
Changing user mid-process (cotenant): rebind identity + checkout personal for that user; never leave previous user's pins writable.

5 branching.borg

pricklypear-branching

Pricklypear book chapter: branching
[status: partial]

branching

Branches as the fundamental organizational primitive: code branches, shared data, fork/merge
[status: partial]

> – *agent note:* Section overall 'partial' as of personal-workspace plan: fork = implemented, merge = partial, schema-authority = partial, personal-workspace = implemented (boot + seed-on-main), package-updates / promote / branch-browser-ui = planned. Per-subsection status below carries the per-feature detail.

Branches are to Pricklypear what files are to Unix: the fundamental organizational primitive. Everything is a branch; everything else is built on top of that.

STORAGE (one Postgres, not one DB per branch): - branches(id, parent_id, owner, name, ...) - function_defs / type_defs / view_bindings each keyed by (branch_id, name|route) - data_rows has NO branch_id – shared data No separate schemas/tables per branch. No Postgres INHERITS (see storage.no-inheritance).

KEY DISTINCTION: branches are for CODE, not data. - BRANCHED: function definitions, type schemas, view/route bindings. - SHARED: data_rows (todos, notes, bible, ?). Alice's experimental branch renders against REAL shared data. She forks code, not data.

TWO FORK SHAPES: - EMPTY child (personal garden): parent pointer only; inheritance via recursive CTE. Main package updates stay visible unless overridden. - FULL fork (experiment): copy code-rows from parent (cheap snapshot). Parent updates do not appear until merge.

ROLES: system/main package defaults (vendor channel) <user>/personal garden (user channel; boot default) <user>/<name> experiments (full fork)

Effective definition lookup (CLOS override): walk parent chain, closest definition wins. Writes (define/route) always hit the ACTIVE branch only.

ast-nodes

Design: addressable AST nodes as substrate for surgical edits and AI patches
[status: planned]

> – *agent note:* it31 plan-pass #7: NEW planned design subsection, elevating BORGE-1783398327 out of bugs-only limbo. Not required for v1 product cut (whole-function define/edit/AI propose is enough). Captures the 'nodes as substrate, named branchable subtrees as versioning unit' proposal so future plan/make work has a home. No schema migration until explicitly planned.

TODAY: two trees, two models - Branch tree: rows in branches, fork/merge, recursive CTE lookup, unit = whole function - AST tree: JSONB blob in function_defs.body, in-memory eval walk, no node ids, edit = replace whole body

PROPOSAL (design only): Give AST nodes identity so AI/human tools can patch subexpressions without rewriting whole functions, while keeping branches as the versioning model over named subtrees.

Sketch (non-normative until a later plan pass): nodes(id, branch_id, kind, payload, parent_id, position, ...) function_defs may become a view/root pointer over a node tree, or remain a cache of the root expression.

Non-goals for first product cut: - Do not block function-editor or ai-authoring tools on this redesign - Whole-function propose/edit ships first

fork

Forking a branch

[status: implemented]

> – *agent note:* Implementation it9 (2026-07-06); hardened with a SERIALIZABLE transaction in it16. The 3-step implementation matches the spec text below exactly: INSERT branches row, INSERT function_defs copy, INSERT type_defs copy, INSERT view_bindings copy. All 4 statements run in a single ‘Store.with_transaction‘ (it16) – atomic, pgx-rolled-back on exception. The primitive is a normal Lwt-typed prim registered via ‘register_session_control_prims‘ (it12); its return value is the new branch UUID. Smoke test ‘/tmp/pp_fork_it9.py‘ exercises the full lifecycle; ‘/tmp/pp_override_it9.py‘ exercises the branch isolation half of fork semantics.

(fork 'main 'alice/experiment)

Implementation: 1. INSERT INTO branches (id, parent_id, owner, name, ...) VALUES (new_uuid, main_id, 'alice', 'experiment', ...) 2. INSERT INTO function_defs (branch_id, name, body, author, updated_at) SELECT new_uuid, name, body, author, updated_at FROM function_defs WHERE branch_id = parent_id 3. Similarly for type_defs, view_bindings

Cost: O(num code rows in parent). For ~100 fns and ~20 types, a few hundred row inserts. Lands in well under a second.

Forking does NOT copy data table rows. The forked branch's functions operate on the same shared event rows as the parent.

merge

Merging a branch into its parent

[status: partial]

> – *agent note:* Implementation it10 (2026-07-06); hardened with a SERIALIZABLE transaction in it16. The current implementation does the basic case: INSERT INTO function_defs SELECT ... FROM function_defs WHERE branch_id=source ON CONFLICT (branch_id, name) DO UPDATE – full-source-to-target upsert of function_defs. Returns ‘merged N rows‘ count via INSERT RETURNING. Policy enforced at the Lisp prim layer (session.ml): admin-only merges TO system branches (owner="system"), per-user can merge into their own branches freely. Verified end-to-end by /tmp/pp_merge_it10.py (both happy-path and policy-rejection).
>
> NOT YET IMPLEMENTED (deferred – these are nicer-to-haves over the basic merge): (1) diff-based merge (only fns whose updated_at > parent's last_synced_with_child, OR whose name was added in child). (2) per-function conflict detection (parent ALSO modified same name since fork ? conflict surfacing). (3) type_defs schema-merge governance (only main accepts schema changes via migration-governed merge). (4) the real htmx inspector UI for per-function conflict resolution.
>
> Section marked ‘partial‘ until at least (1) and (2) land; (3) and (4) are explicitly v2+ features.

(merge 'alice/experiment 'alice/main)

Policy: - Default: admin-only merges to main, per-user can merge into their own branches freely. - Expandable later (PR/review flow).

Implementation: 1. Compute diff: code rows in alice/experiment whose updated_at > parent's last_sync_with_child, OR whose name was added in child. 2. For each changed function, apply to parent. Conflict if parent ALSO modified same name since fork. Per-function conflict (coarse, not line-level): pick a version, or write a new one. 3. Schema changes (type_defs) are merge-governed separately: only main accepts schema changes. A branch can propose; main accepts via a special merge that runs the migration.

The merge needs a real htmx UI (in the inspector): list changed functions, show diffs (whole function bodies since functions are atomic units), let user resolve per-function.

schema-authority

Types in main, branches read against main's schema

[status: partial]

> – *agent note:* Partially implemented as of it14 (2026-07-06): the ‘(deftype 'name 'field1 ...)‘ prim lands type declarations into the ‘type_defs‘ table on the current branch; ‘Store.list_types‘ walks the recursive-CTE override chain so ancestor type declarations are visible to descendant branches (the read side of "branches read against main's schema"). The /types system page surfaces them in the UI.
>
> NOT YET IMPLEMENTED (deferred – explicitly v2+): (1) the migration-governed schema-change accept flow ("only main accepts schema changes via a special merge that runs the migration"). Currently any admin-author on the system branch can run (deftype ...) directly. (2) schema_version bump + cache-invalidation broadcast on schema change (LISTEN/NOTIFY or schema_version check). (3) governance policy for proposed-vs-accepted schema changes across non-system branches.
>
> The read side (recursive-CTE type lookup) IS implemented and spec-text-aligned. Marked ‘partial‘ until at least the migration-governed merge-to-main accept flow and the schema_version invalidation hook land.

Schema authority is 'main'. Branches CANNOT change the underlying table shape directly; they propose schema changes which only take effect when merged to main.

When main accepts a schema change: 1. Run migration on shared data table (additive-only: see migrations.additive-only) 2. Bump schema_version 3. All branches see new shape on next function lookup (cache invalidation: schema_version check, or LISTEN/NOTIFY)

Branch code that ignores new fields: fine, reads subset. Branch code that expects old fields: errors at runtime (additive-only means removed fields don't happen, but renamed/reused-semantic fields could; document expectations).

This makes type_defs the slow-changing, merge-governed layer; function_defs the fast-changing, per-branch layer. Two tiers of change, mirroring the OCaml-slow/Lisp-fast split.

personal-workspace

Boot on empty <user>/personal; seed package defaults only onto system/main

[status: implemented]

> – *agent note:* Shipped a934445 / 42f755b: Store.ensure_branch creates EMPTY children; seed_stdlib uses Store.with_branch system/main; boot ensure_personal ? seed main ? checkout personal ? rehydrate via Define AST (no pp_expr round-trip).

PROBLEM: reseeding libs/*.pp onto the active branch wiped live (define) edits after every deploy.

MODEL: - system/main = package defaults (vendor channel). Boot seed_stdlib ALWAYS writes libs/*.pp here only (std, ui, ai, todo, bible, dabar today). - <user>/personal = garden (user channel). Empty child of system/main (parent pointer only). Image boots here after seed. User (define) and (route) write personal rows only. - Inheritance: recursive CTE; personal overrides shadow main; unpinned names auto-see main updates.

BOOT ORDER: 1. ensure_branch owner=username name=personal parent=system/main (no code copy) 2. seed_stdlib with store.branch_id = system/main 3. checkout personal + rehydrate env from CTE 4. refresh ui/nav shell

load-library on CURRENT branch still overwrites that branch (intentional). On personal that PINS the whole package – prefer small overrides.

(checkout "system/main") / (checkout "admin/personal") rehydrate sync; header shows current branch.

package-updates

How software updates work: kernel vs main packages vs personal pins

[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Kernel vs main seed vs personal pins behavior is real (seed_stdlib?system/main, personal inherits). unoverride prim + conscious update UX still planned.

THREE CHANNELS (not one monolithic version):

1. KERNEL – OCaml binary (HTTP, prims, loader, chrome CSS). Updated by git push ? promote ? restart. No branch involved.

2. PACKAGE DEFAULTS – function_defs on system/main from libs/*.pp. Updated by boot seed after deploy. Anyone who INHERITS a name (no personal row) gets the new body automatically.

3. GARDEN PINS – function_defs rows on personal (or any child). Never written by seed. Survive every deploy. Shadow main until unoverride/merge.

DATA (data_rows) is never versioned by branch and never wiped by push. New package code must tolerate older row shapes (defaults in readers) or ship an explicit data migrator.

UPDATE RULE: inherited name ? auto-update on seed overridden name ? user must accept (unoverride or merge); show "main newer"

CONSCIOUS UPDATE UX (planned on branch browser): per name: Use main's version | Keep mine | Diff per package: Reset package pins to main unoverride prim: delete personal row for name so inheritance resumes (MISSING today)

load-library WARNING (planned): when current branch is not system/main, warn that loading pins the package on the garden.

Anti-patterns: - reseed onto personal - auto-merge main?personal on boot - full copy-on-fork for personal (freezes forever) - treat data_rows as package payload

promote

Push garden work to system/main and/or git master (libs/*.pp)

[status: planned]

Users will edit on personal and want to ship work upstream. Two destinations – both matter:

A. LIVE main – (merge "admin/personal" "system/main") Upserts function_defs personal ? main now. Admin-only into system/* (existing policy). NOT durable alone: next seed reloads libs/*.pp and can overwrite main unless git matches.

B. GIT master – pp_sync pull <pkg> while on personal exports to libs/<pkg>/lib.pp; operator commits and git push wyo. Next seed makes main durable.

FULL SHIP LOOP: edit on personal ? dogfood ? optional (merge personal main) for instant live ? pp_sync pull selected packages ? git commit + push ? seed refreshes system/main from files ? unoverride pins on personal so inheritance is clean (don't keep stale full-package pins)

PROMOTE IS SELECTIVE: Ship package functions (ui/*, notes/*, ?), NOT the entire garden (no personal-canary dump). UI: multi-select packages/names, then [ ] Merge into system/main [ ] Export for git (operator runs pp_sync/commit)

Image does not git-push itself (no repo credentials in the image). Export API + laptop/habitat operator owns the commit.

Policy: - merge to system/main: admin only - merge to own branches: free - data_rows never part of promote

branch-browser-ui

Git-like light /branches browser: list, open, stats, checkout, promote

[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: GET /branches bare table exists. Checkout/stats/detail/promote UI still planned.

TODAY: GET /branches is a bare table (owner/name, parent uuid prefix, created). No checkout, stats, detail, or promote.

GOAL: tiny Gitea-ish branch page in classic htmx chrome – browse, open, stats, checkout, later promote – no SPA.

ROUTES: GET /branches GET /branches?ref=owner/name detail POST /branches/checkout ref=? POST /branches/fork from + name POST /branches/merge src + dst (later) POST /branches/promote selective (later)

LIST PAGE: Header strip: On: <current> ? + quick links Table columns: ? | branch | parent | own code | overrides | routes | age | actions own code = COUNT function_defs WHERE branch_id (no CTE). overrides = own names also on parent. actions: Open ? Checkout (disabled if current) Labels: system/main "defaults"; */personal "garden" Create card: fork-from select + new name + Create (full fork for experiments; personal stays empty)

DETAIL PAGE (?ref=): Title, parent link, Checkout, back to list Stats: own fns, overrides, own routes, visible fn count (CTE), last define, authors Sections (htmx partials): Overrides (default for personal) – name, author, updated, new|override, view, diff All own code Routes (own view_bindings) Activity: ORDER BY updated_at on own defs (revision log later) Diff vs parent v1: two <pre> columns of pp_expr (identical / changed / only-here / only-parent)

CHECKOUT: POST ? Session.checkout_to ? 303 /branches or HX-Redirect. Defines already persisted; no "uncommitted" confirm.

PROMOTE PANEL (detail, admin): package multi-select + name multi-select [ ] Merge into system/main [ ] Mark for export (pp_sync pull list) Never dump whole garden by default.

UPDATES BADGE (package-updates): override where main.updated_at > personal ? "main newer" + Accept (unoverride) | Keep | Diff

STORE HELPERS (own-branch, no CTE unless noted): count_functions_on_branch count_routes_on_branch list_own_function_names list_overrides (own ? parent names) lookup_function_on_branch (no CTE) delete_function_on_branch (unoverride)

PHASES: A list: ? current, checkout POST, open, own counts B detail: overrides + link /binding/<name> C diff vs parent (two pre) D fork form + promote panel + unoverride E activity / revision log

NON-GOALS (light): git SHAs/commits, PR review UI, blame, per-branch data, fancy fork-tree viz

6 calendar.borg

pricklypear-calendar

Pricklypear book chapter: calendar
[status: partial]

calendar

Month/day calendar: events + due todos + daily notes
[status: partial]

model

Naive local dates; shared data_rows; Lisp package
[status: implemented]
CHOICES (v1): - Day click opens day agenda (not auto-create) - Naive local YYYY-MM-DD (no TZ UI) - Seeded on boot after todo - Month + day only (no week view yet)
DATA: calendar-event title day start_time end_time all_day notes location color source
OVERLAYS (read): todo-item.due on day notes daily-YYYY-MM-DD presence
PRIMS: date-today parse-int date-weekday date-format
ROUTES: /calendar ?ym= ?day= Package: calendar/*

7 dabar.borg

pricklypear-dabar

Pricklypear book chapter: dabar – BSB study library (reader + Strong's + interlinear)
[status: planned]

> – *agent note:* 2026-07-11: New chapter. Dabar is NOT a separate unikernel. It is Lisp library packages + data_rows seeds living inside the image. Feature/data donor for study UX is the sibling repo wyo.tech/dabar (Dream app; no .borg of its own). Semantic search is OUT OF SCOPE.

dabar

Bible study layer on the live image: plugin libs, seeds, routes
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Packaging, reader baseline, nav, Strong's, dogfood interlinear, search via find-text ship. Real FTS + richer study UX remain.

PRODUCT: Dabar on Pricklypear is a multi-user, live Bible study environment built as *libraries* inside the user image – not a port of the Dream dabar server.

PILLARS: 1. Plugin-shaped Lisp packages (libs/*/lib.pp + load-library). 2. Canon + lexicons in Postgres data_rows (shared across branches). Personal notes/bookmarks are a SEPARATE module (not this chapter). 3. htmx partials for reading/study panels (no SPA). 4. Seeds are offline scripts; recipes only install code.

NAMING (locked): - bible/ – core reader (exists): verse, chapter, search, /bible, partials, dropdowns. - dabar/ – study layer: Strong's, interlinear, study chrome that *uses* bible/*. Do NOT rename existing bible/* without a migration plan.

RELATION TO wyo.tech/dabar: - Donor of data formats, Strong's CSV, interlinear field shapes, UX ideas (filters, prev/next, random). - That repo has no borge book; this chapter is SoT for the Pricklypear port. Link from root pricklypear.borg.

packaging

Library layout, boot policy, deps on std/bible
[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: libs/bible + libs/dabar seed_stdlib; seed scripts present; routes /dabar /strongs.

LAYOUT: libs/bible/lib.pp – reader (shipped) libs/dabar/lib.pp – study (to add) scripts/seed_bible_bsb.py scripts/seed_strongs.py (from dabar strongs.csv) scripts/seed_interlinear.py (from dabar extract)

BOOT: seed_stdlib loads: std, ui, ai, todo, bible dabar: POST /api/recipe/dabar or (load-library "dabar") – not auto-boot until study layer is dogfood-stable.

DEPS: dabar assumes bible/* and html/* (std) already bound. Document in lib header; multi-file require is deferred.

INVARIANTS: - No dabar-specific OCaml except generic find/FTS if needed. - Routes under /bible stay bible-owned; /strongs and /dabar/* may be dabar-owned. - Kernel stays thin.

reader

Existing bible/ reader – baseline for dabar
[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: BSB reader + chapter UI + search panel; prev/next/random landed in bible/* used by dabar.

– *agent note:* Reader shipped: types bible-book/bible-verse, seed BSB full text, /bible dropdowns + htmx panel, find-row/find-text. Status partial until prev/next/random and FTS land (WP1/WP4).

BASELINE (implemented): - ref BSB:<Abbrev>:<c>:<v> - (bible/verse book ch v), (bible/chapter ?), (bible/search q) - /bible + /partials/bible-chapter - html/* selects for book/chapter/verse
STILL IN READER WP1: - (bible/prev-chapter) / (bible/next-chapter) + UI - (bible/random) + Surprise control - empty/missing chapter UX

navigation

Prev/next chapter and random verse
[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: bible/prev-ref next-ref random-row + UI links including /dabar?random=1.

Use bible/chapter-counts + book ordinal (bible-book rows). Wrap policy v1: stop at ends (no wrap), or wrap OT/NT – pick stop-at-ends for clarity. Random: SQL ORDER BY random() LIMIT 1 via thin prim or store helper; expose (bible/random) in bible/ or dabar/.

strongs

Strong's lexicon seed, lookup, pages
[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: strongs-entry type, strongs/get, /strongs page, seed_strongs.py.

SOURCE: wyo.tech/dabar/src/tool/sql/strongs/strongs.csv (and dabar Schema.Strongs shape: number, lemma, xlit, pronounce, description).
TYPE: strongs-entry (or strongs) number, lemma, xlit, pronounce, description id/ref: fields.number unique (H430 / G26).
PACKAGE: prefer dabar/strongs-* or strongs/* under dabar load; public names (strongs/get) OK if namespaced carefully.
UI: /strongs list/search stub; detail by number; links from interlinear tokens.

interlinear

Word-level interlinear fragments + toggles
[status: partial]

– *agent note:* 2026-07-12 status-reconcile: bible-interlinear type + dabar/inter-html verse study path; dogfood seed only, not full corpus.

SOURCE: dabar inter_verse (bsb_sort, base_text, variant_text, translit, parsing, parsing_str, heb_str, grk_str, verse_id, heading, crossref, bsb_version).
STORAGE v1: per-verse JSON on a type bible-interlinear ref, tokens (JSON array of word objects), heading?, crossref? keyed by same BSB:Abbrev:C:V ref as bible-verse. Avoid millions of token rows unless needed.
UI: htmx toggle on /bible or /dabar/verse; Strong's hrefs to /strongs?n=.
Full corpus seed may be large – allow –book John for dogfood.

search-fts

Postgres full-text search over verse text (not semantic)
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: bible/search via find-text substring – not Postgres FTS/tsvector yet.

OPTIONAL WP: expression index / tsvector on data_rows.fields-»'text' for type bible-verse, or generated column in a later migration. Prim: find-text-fts or specialized bible search SQL. NON-GOAL: pgvector, embedder, /api/search/semantic.

work-packages

Implementation order
[status: planned]
WP0 This chapter + root inline + image.packages cross-link WP1 Navigation: prev/next + random (bible/) WP2 Strong's seed + dabar/strongs + /strongs WP3 Interlinear seed + toggle + Strong's links WP4 FTS (optional parallel) WP5 Multi-file library / require (when packages grow)
NOT THIS CHAPTER: personal notes/bookmarks – separate module.
ACCEPTANCE v1 "dabar on PP": - bible reader green on boot - load-library dabar adds study without breaking /bible - Strong's lookup + one interlinear dogfood path - no semantic search - no notes/bookmarks here

non-goals

Explicit exclusions
[status: planned]
- Semantic search / pgvector / llama embedder - Personal notes / bookmarks (separate module) - Porting Dream dabar server into the image - Audio - Path-param routes /bible/John/3 until platform supports - Multi-translation UI - New unikernel or service binary for dabar

8 flashcards.borg

pricklypear-flashcards

Pricklypear book chapter: flashcards – decks + spaced-repetition cards as Lisp + data_rows

[status: planned]

> – *agent note:* 2026-07-12: New product chapter. Flashcards are NOT a separate service and NOT Anki import-first. Substrate = data_rows types flash-deck + flash-card; behavior = libs/flashcards/lib.pp; UI = html/? + form POST study ratings; agents use pp_eval flash package API. SRS v1 is a small SM-2-ish scheduler in Lisp (due dates as YYYY-MM-DD strings). Data is branch-shared like all data_rows. Orthogonal to notes (thinking tool) and bible/dabar (canon study) – this chapter is deliberate memorization.

flashcards

Decks, cards, study queue, SM-2-ish review, hypermedia UI, agent API

[status: planned]

========== PRODUCT INTENT ==========
A first-class memorization app on the live image, inspired by: - Anki: decks, front/back cards, spaced review grades - SuperMemo SM-2: ease + interval + due (simplified) - Pricklypear: objects in Postgres, UI as Lisp routes, agents study/create via eval – no parallel REST API

NOT a port of Anki desktop. NOT file-backed .apkg as the authority (import may come later). NOT OCaml system pages – this is a product package (Lisp).

KILLER PROPERTY ON THIS OS: Cards are data_rows; the study algorithm is editable Lisp on your branch; agents can bulk-add or quiz you via pp_eval; the new /data browser can inspect flash-card rows when the package UI is wrong.

========== LAYERING ==========
- libs/flashcards/lib.pp – package flash/? (seed on system/main) - data_rows types – flash-deck, flash-card (shared) - Routes via (route ?) – /flashcards and study/edit paths - Optional nav entry – ui/menu pin or seed menu link - OCaml – only existing CRUD/date prims; no new kernel prims required for v1 if date-today (calendar) or string due compares suffice

========== DATA MODEL (v1) ==========
Types (deftype on load; fields are documentation + form UI):

flash-deck title string description string color string (optional CSS token / hex) archived "true" | "" (hide from default list)

flash-card deck_id uuid string of flash-deck front string (prompt) back string (answer) tags string (comma-separated free text v1) suspended "true" | "" (excluded from study) ;; SM-2-ish scheduling state (all strings in jsonb v1): ease string float default "2.5" interval string int days default "0" reps string int default "0" lapses string int default "0" due string YYYY-MM-DD (empty or past => due now) last_reviewed string YYYY-MM-DD or empty created string YYYY-MM-DD optional

NO separate flashcard-review log table in v1 (history deferred). NO cloze / image occlusion / media blobs in v1. NO per-user row ACL – data_rows are shared (existing rule). Multi-user isolation of card content is a later multi-user.permissions concern, not this chapter's first cut.

Orphan cards (missing deck_id) show under an "(orphan)" virtual deck in the UI; agents can reassign.

==========
SRS ALGORITHM (v1 – SM-2-ish, Lisp, editable) ==========
Ratings (buttons + flash/review rating arg): again | hard | good | easy

State on each card: ease (min 1.3), interval days, reps, lapses, due.

On review with rating R, today = date-today (or flash/today helper): again: reps = 0; lapses += 1; interval = 0; due = today ease = max(1.3, ease - 0.2) hard: interval = max(1, floor(interval * 1.2)) ease = max(1.3, ease - 0.15) reps += 1; due = today + interval good: if reps = 0 then interval = 1 else if reps = 1 then interval = 6 else interval = floor(interval * ease) reps += 1; due = today + interval easy: if reps = 0 then interval = 4 else interval = floor(interval * ease * 1.3) ease = ease + 0.15 reps += 1; due = today + interval

New cards: due empty or due <= today, reps=0 ? introduce in study. Suspended cards never enter the queue. Order: due ASC, then created/id; limit batch (default 20) per study page.

Implementation lives in flash/schedule (or flash/apply-rating) as pure-ish Lisp over field dicts so personal gardens can override.

Date math: reuse calendar/image date prims if present (date-today, and a day-add helper). If day-add is missing, file issue / add a small flash/add-days using existing prims only – do NOT invent a second calendar system.

==========
PACKAGE API (flash/ prefix) ==========
Decks: (flash/decks) -> list of (id . fields) (flash/deck id) -> fields | () (flash/deck-create title [description]) -> id (flash/deck-update id fields-or-keys?) (flash/deck-archive id)

Cards: (flash/cards deck-id [limit]) -> list (flash/card id) -> fields | () (flash/add deck-id front back [tags]) -> id (flash/update id ?) (flash/suspend id) / (flash/unsuspend id) (flash/delete id) ; hard delete v1 OK (confirm in UI)

Study: (flash/today) -> "YYYY-MM-DD" (flash/due [deck-id] [limit]) -> due cards (not suspended) (flash/review id rating) -> updated fields | error string rating in again|hard|good|easy (flash/stats [deck-id]) -> simple counts (due, new, learning, suspended) as printable structure

HTML / routes (handlers are flash/?-page functions): (flash/install) or load-side effects: (route "/flashcards" "flash/home-page") (route "/flashcards/deck" "flash/deck-page") (route "/flashcards/study" "flash/study-page") (route "/flashcards/card" "flash/card-edit-page") ; optional Exact query keys are implementation discretion; document in lib.pp header. Prefer ?deck=uuid & ?id=uuid.

Naming: package directory libs/flashcards/, binding prefix flash/ (short, like todo/). Types flash-deck / flash-card (not flashcards-deck).

==========
HYPERMEDIA UI ==========
/flashcards - list decks with due counts - form: new deck (title) - link to study-all (due across decks) optional

/flashcards/deck?id= - deck title, description - card table (front truncated, due, ease) - add card form (front, back, tags) - Study this deck

/flashcards/study?deck= (deck optional = all) - if no due: empty state + links - else one card: show FRONT [Reveal] ? show BACK + rating buttons - POST rating ? flash/review ? redirect next due - plain form POST (same as todos); htmx partials optional polish

Mobile-first: big rating buttons, minimal chrome noise.

Use existing product theme classes (card, btn, table). No new OCaml CSS. Optional small flash-local classes in ui/theme later.

==========
SEED / NAV ==========
- seed_stdlib: load-library "flashcards" after calendar (or after todo – order free as long as std/html exists) - recipe optional: (load-library "flashcards") from home - ui/menu: add Flashcards ? /flashcards on system seed when easy; personal may pin. Not a kernel nav hard-code.

==========

AGENT ACCESS ==========
Image is the API: python3 scripts/pp_eval.py '(load-library "flashcards")' python3 scripts/pp_eval.py '(flash/add deck-id "Q" "A")' python3 scripts/pp_eval.py '(flash/due)' python3 scripts/pp_eval.py '(flash/review id "good")' No flash-specific REST. Bulk import via agent loop over flash/add.
==========
NON-GOALS (v1) ==========
- Anki .apkg import/export - Cloze deletions, image occlusion, audio - FSRS / full SM-18 - Review history graph / heatmap (nice later) - Shared multiplayer decks / permissions - Sync protocol beyond data_rows + pp_sync package export - OCaml system pages for study (package owns UX) - Branch-scoped cards (data is shared by design)
==========
PHASES ==========
Phase 1 (this chapter make): types + flash/? CRUD + due/review + /flash-cards UI + seed Phase 2: stats page, tags filter, keyboard shortcuts, htmx reveal Phase 3: notes ? card capture, bible verse ? card, Anki import spike
==========
ACCEPTANCE NARRATIVE ==========
Human: open /flashcards ? create deck ? add two cards ? Study ? reveal ? rate Good/Again ? due dates move ? later session only shows due cards. Agent: flash/add + flash/due without UI. /data/flash-card still inspectable as raw rows.

model

flash-deck + flash-card data_rows; SM-2-ish fields on card
[status: planned]

srs

Lisp scheduler flash/apply-rating; ratings again|hard|good|easy
[status: planned]

ui

Routes /flashcards, deck, study; form POST ratings
[status: planned]

agent-access

pp_eval flash/? as the API
[status: planned]

9 image.borg

pricklypear-image

Pricklypear book chapter: image
[status: partial]

image

The per-user unikernel: Lisp interpreter, REPL endpoint, htmx interface
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Boot/repl/browser-repl/theme/packages ship. Inspector/editor/branch UI polish + some prims still partial.

> – *agent note:* it32: image still partial overall. browser-repl + function-editor implemented; system-browser + inspector partial; product-experience owns about/vocabulary/first-run. ssh still line-protocol default; linenoise still deferred.

boot

Reading the manifest, loading initial env
[status: implemented]

> – *agent note:* it23: image/lib/boot.ml + bin/unikernel.ml. Reads manifest from env vars (PP_USER_ID, PP_USERNAME, PP_BRANCH_ID, PP_BRANCH_NAME, PP_IMAGE_VERSION, PP_HTTP_PORT, PP_REPL_PORT, PP_DB_* env vars). PG connect (non-fatal on failure). load_defs_async populates env from function_defs via recursive-CTE walk (it8). REPL endpoint + HTTP server started async. Health endpoint /health. The spec's 7th step 'marks self healthy, signals supervisor' is NOT explicitly wired – supervisor uses retry-on-connect backoff (it6) to detect image readiness instead. The spec describes the PROD flow as 'manifest fetched from PG by the supervisor before booting' – v1 dev flow uses env vars because boot-on-connect uses posix_spawn (it4); prod Solo5 + hypervisor VM request would carry the manifest forward, deferred. The 8-step boot sequence is implemented; the PG-fetched manifest is the prod-flow upgrade path.

On boot, the image: 1. Reads its boot manifest (from common/lib/manifest.ml type, fetched from Postgres by the supervisor before booting). 2. Manifest specifies: - user_id (who this image is; asserted by supervisor after auth verified the credential – see section auth. The image trusts this because supervisor+auth did the verification; the image does no auth itself.) - username (for the REPL prompt and author tracking) - branch_id (default branch, usually user/main) - boot_mode ('on-connect' | 'always-on') - image_version (for A/B tracking) - config (budgets, feature flags) 3. Loads all function_defs and type_defs for branch_id from Postgres into the in-memory env (or, with no-caching policy, sets up the branch-aware lookup that reads on each call). 4. Starts the REPL endpoint (internal line protocol; the supervisor forwards SSH channels here – see ssh-interface). NOT a full SSH server. 5. Starts the htmx server (httpaf-lwt-unix in v1) on configured port. 6. Registers health endpoint. 7. Marks self healthy, signals supervisor.

The image is STATELESS across reboots w.r.t. user data: anything that must survive is in Postgres. In-memory REPL state (define'd locals in a session) does NOT survive reboot unless the user persisted via (store) or similar.

ssh-interface

The SSH front door: supervisor authenticates, forwards to image REPL
[status: deprecated]

> – *agent note:* 2026-07-12 status-reconcile: Supervisor SSH front door retired with single-image. Optional line-protocol REPL may remain on image; browser /repl + /api/eval are primary.
>
> – *agent note:* it23: the supervisor-fronts-SSH pattern is implemented (supervisor/lib/ssh.ml). v1 uses a non-crypto line protocol on PP_SSH_PORT (default 17822 dev / 2222 prod): accept TCP, banner, read key as first line, auth via authd's /verify_ssh_key HTTP RPC, DENIED or OK+ boot/attach image, byte-proxy TCP between SSH socket and image REPL socket. Real awa-ssh path (it21) is OPT-IN via PP_AWA_SSH_PORT; real SSH transport + pubkey auth works end-to-end but the exec_callback's REPL proxy bridge has a Cstruct<->Lwt_io bug, AND awa 0.3.1's static-auth-db constraint means the awa path can't propagate user identity into the image (see BORGE-1783439434). The line protocol IS the spec's design minus the cryptographic transport. Marked partial: the cryptographic SSH handshake isn't default; waiting on awa 0.5.x upgrade (its auth-event callback exposes username to exec_callback). Per the spec's own tradeoff section 'supervisor-to-image hop is internal (trusted hypervisor network), so no end-to-end SSH crypto between user terminal and image' – that has held. Image does NOT run full SSH – matches spec design.

THE SSH PATH (end to end):
1. User runs: ssh alice@pricklypear.host 2. SUPERVISOR listens on port 22 (awa-ssh, Mirage-compatible) and accepts the connection. 3. SSH handshake + key exchange (awa-ssh, pure OCaml, no libssh). 4. Authentication: supervisor calls pricklypear authd (NOT wyo.auth – deliberate v1 choice; see workflow.conventions) via verify_ssh_key RPC to resolve the presented public key to a user_id. Key -> user mapping is authd's job. 5. Supervisor boots (boot-on-connect) or attaches (always-on) the user's image unikernel. 6. Supervisor opens a channel to the image's internal REPL endpoint (see repl_endpoint below) and FORWARDS the SSH channel's PTY stream to it. The user's terminal is now talking to the image's REPL. 7. Image presents the REPL prompt (see repl-experience). 8. On disconnect, supervisor notes session end; boot-on-connect images die, always-on images keep running.

WHY SUPERVISOR-FRONTS-SSH (not each image on its own port): - One door, one auth, one image per user. Matches 'SSH-first OS' – it's THE login, not N logins. - Users don't need to know their image's port. - authd integration happens in one place (supervisor), not N places (each image). - The image doesn't need full SSH – just a REPL endpoint. Simpler image.

TRADEOFF: supervisor-to-image hop is internal (trusted hypervisor network), so no end-to-end SSH crypto between user terminal and image. For a personal OS on a trusted hypervisor, this is acceptable. If it becomes a concern, the image can run its own awa-ssh on an internal port and the supervisor does TCP forwarding of the encrypted SSH stream (then the image decrypts). Defer this until it's a felt concern.

repl-endpoint

Image's internal REPL endpoint, not full SSH
[status: implemented]

> – *agent note:* it23: image/lib/repl_endpoint.ml. TCP listener on PP_REPL_PORT. Accepts a connection, prints banner 'Prickly-pear REPL. Type :quit to exit.', prompts with prompt (branch_name

+ '> '). Read-eval-print loop driven by Lwt_io channels. Session.t IS shared across connections (one Session.t per image process), so '(define x 42)' in one socket persists to the next – matches the it18 smoke-test pattern. Graceful disconnect (it10): all post-disconnect cleanup wrapped in Lwt.catch, EBADF classified as 'peer disconnected' not fatal. No full SSH in the image – matches the spec's design. The supervisor connects and forwards SSH-to-image TCP bytes via Lwt_unix.read/write.

The image does NOT run a full SSH server. Instead it exposes an internal REPL endpoint – a simple line protocol over a TCP socket (or Mirage channel) on an internal port. The supervisor connects to this and forwards the user's PTY stream.

Protocol (minimal, framed): - Connect: supervisor opens TCP to image:REPL_PORT - Auth: none needed (internal network; the supervisor already authenticated the user) - Exchange: lines of UTF-8. User input flows in; REPL output flows out. - The image's session.ml tracks this connection as a session; on disconnect, session ends.

WHY NOT FULL SSH IN THE IMAGE: - Removes awa-ssh dependency from the image (keeps image smaller; awa crypto is non-trivial). - One SSH implementation (in supervisor) to maintain, not N. - authd resolution stays in supervisor.

The endpoint is in image/lib/repl_endpoint.ml. It owns: accepting the supervisor's connection, wiring PTY stream to the line editor (lineedit.ml), and the REPL read-eval-print loop itself.

repl-experience

What the user sees: prompt, multi-line, line editing, meta-commands
[status: partial]

– *agent note:* it23: prompt + multi-line + meta-commands + session-state are implemented in image/lib/repl_endpoint.ml. Currently the prompt is '<branch-name» ' (e.g. 'main> '), NOT the spec's 'alice/main>' 2-token form. Multi-line: the reader's is_complete is consulted so a '(define square (lambda ...' spanning lines waits for the closing paren. Meta-commands (define, fork, checkout, merge, list-bindings) ARE implemented as Lisp special forms or prims (define is the only AST-level intercept since its name arg is a Var not to be evaluated before binding; others are Lwt prims on the it12 ABI). Session state: define persists to PG (it8); REPL locals are session-local, persist across REPL socket reconnections within an image's life; checkout mutates branch_id (it11). NOT IMPLEMENTED: real line EDITING (image/lib/lineedit.ml is a single-line stub). ocaml-linenoise is the candidate; v1 just uses raw input – no cursor movement, no history, no Ctrl-A/E. Editing is desired but not v1-blocking (the image IS usable via a wrapper that sends prepared lines). (edit name), (describe 'type), (load "file"), (help) – meta-commands not yet implemented (ergonomic gaps, see lisp-interpreter.primitives' agent note). Marked partial: the REPL usability core + meta-commands ship; real linenoise-style editing + the 4 missing meta-commands are the deferred v1.5 polish layer.

THE PROMPT: alice/main> – showing user + current branch alice/main (def) | – mid-multiline-define

READING INPUT: - The REPL reads s-expressions. It keeps reading lines until parens are balanced and strings closed (standard Lisp REPL behavior). The reader tells us 'incomplete' so we know to prompt for more, not to evaluate yet. - One REPL turn = one s-expression (which may span many lines). - Comments: ; to end of line, as in the reader.

LINE EDITING: - Real line editing matters for a dev tool. Raw SSH PTY + a line-editing layer. - Library choice deferred to build time: ocaml-linenoise is the leading candidate (small, Mirage-compatible-ish). Pure OCaml. - Supports: cursor movement, history (up/down), Ctrl-A/E, backspace, delete. - Search/history-reverse-search: nice-to-have, not v1-blocking.

EVALUATION + PRINTING: - Evaluate the s-expression in the session env. - On success: pretty-print the resulting value. VList -> (a b c), VLambda -> #<lambda>, VRef -> #<ref event:uuid>, VInt -> 42, etc. - On error: print the error with location if possible (line/col from the reader), then a continuation prompt. Don't kill the session.

META-COMMANDS (Lisp forms, not a separate command lexer – the reader handles them, the eval loop recognizes them as special forms): (define name expr) bind + persist to store (edit name) print htmx URL for AST editor (fork parent child) create a branch (checkout branch) switch session's branch (merge source target) reviewer flow (describe 'type) show type schema (list-bindings) current env (load "file") eval a file's contents (help) list specials Any other Lisp form evaluate + print result

SESSION STATE: - (define ...) persists to the store: survives image reboot (loaded on next boot of the branch). - REPL-local let bindings (e.g. (let ((x 5)) ...) at top level): session-local, not persisted. Like bash variables. - Switching branch (checkout) within a session keeps REPL locals but changes which function defs are visible. (Decision: this is the 'working directory' model – you carry your locals across branch switches.)

htmx-interface

httpaf-lwt-unix + pure-html, routes AS functions in the store

[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: System pages + user GET/POST routes + request-param/redirect used by notes/todos. Keep partial: hx-* Lisp prims incomplete; Solo5 HTTP deferred.

> – *agent note:* it23 baseline: system pages + GET-only user routes + render-html dispatch shipped. it33 PLAN: user-route POST + request-context prims + redirect return (see details USER-ROUTE HTTP METHODS / REQUEST CONTEXT). Dogfood: interactive todo recipe forms. hx-* Lisp prims and Solo5 HTTP remain deferred.

The image's htmx endpoint serves two kinds of pages:

1. SYSTEM pages (always present, part of the OCaml image): /health health check /inspect/:id browse any stored value /branches list + fork/merge UI /types type browser (type_defs) /data ? data rows browser (full CRUD; see image.data-browser) /bindings function browser (per branch)

2. USER pages (defined as functions in the store): registered via (route "/path" fn-name). The route handler runs with a REQUEST CONTEXT (method, path, params) and returns a value the system renders (HTML, redirect, text, etc).

Routes ARE functions in the store. Adding a route = (route "/foo" 'foo-handler). This makes the user-facing UI part of the live image, editable via the same mechanisms as everything else.

ROUTE VISIBILITY (auth.route-visibility – planned): User routes default private (session required). (route path handler 'public) opts into anonymous GET. Kernel paths (/data /admin /api/eval /bindings /edit ?) are hard-private in OCaml and cannot be opened by Lisp. Gate runs before handler dispatch (extend launch, do not invent a middleware framework).

pure-html (the dream_html HTML generation core, usable WITHOUT Dream) is used for HTML generation (per poohstack convention; use Hx.* not raw strings at the OCaml layer).

==========
USER-ROUTE HTTP METHODS (it33 – PLAN, not yet shipped) ==========

GOAL: Lisp pages can accept browser forms. Todo and every other dogfood app need POST without leaving the image.

METHODS on user routes (view_bindings paths): - GET – already shipped (call handler, render value) - POST – REQUIRED this plan - PUT / PATCH / DELETE – deferred (not needed for form dogfood; may map later; do not invent now)

MATCHING: exact path string only (status quo). No path params (/todos/:id) in this plan. Method does NOT select a different handler – same route path ? same handler name for GET and POST. Handler branches on (request-method).

SYSTEM routes keep exclusive claim on their paths: /api/*, /edit/*, /health, /bindings, ? never fall through to user routes (status quo dispatch order).

BODY (POST): - Read full body (reuse existing body reader in http.ml). - Content-Type application/x-www-form-urlencoded: parse into string key?value map (reuse form_field style decode; empty values allowed; last wins on dup keys). - Other content-types: raw body available via (request-body); params map empty unless also on query. - Size: soft cap 64 KiB for v1; oversize ? 413 text error (do not eval handler).

QUERY (GET and POST): - Parse target query string into the same params map. - Conflict rule: form field WINS over query key of same name (POST body is the intentional write path).

==========
REQUEST CONTEXT (dynamic, handler-scoped) ==========

Design choice: DYNAMIC BINDING, not a mandatory handler argument. Zero-arg handlers keep working: (define hi-page (lambda () (render-html "?"))) Interactive handlers read request via prims while the HTTP dispatch is inside call_handler:

(request-method) -> VStr "GET" | "POST" (request-path) -> VStr path without query (request-param k) -> VStr value | VNil if missing (request-params) -> VStr JSON object of all params (request-body) -> VStr raw body ("" on GET)

Implementation sketch (mirrors with-transaction scope): - Session.t gains mutable current_request : request option - type request = { method_ : [‘GET|‘POST]; path : string; params : (string * string) list; (* or Hashtbl *) body : string; } - http.ml builds request from meth + path + query + body - Session.call_handler ~request sets current_request, runs (handler-name) zero-arg apply, clears in finalize - Prims registered at session-make (image layer, like whoami) read Session.current_request; if None: (request-method) etc raise clean User_error "no active request" (REPL/eval outside HTTP)

WHY NOT pass (lambda (req) ?) as required arity: - Breaks every existing zero-arg page (hello, current todo) - Forces a VDict / record convention early - Dynamic prims match (whoami) / with-transaction style Optional later: also pass req dict as single arg if handler arity is 1 – NOT in this plan.

==========
HANDLER RETURN VALUES ==========

Keep existing render rules, add redirect:

1. render-html tagged VStr (___html___:?) -> 200 text/html, body = raw HTML inside chrome (status quo; chrome wrap stays for full pages) 2. bare VStr -> 200 text/html, <p> escaped text 3. other values -> 200 text/html, <pre> pp_value 4. NEW: (redirect path) -> tagged VStr ___redirect___:path -> HTTP 303 See Other, Location: path, empty/minimal body Path must be absolute-on-host ("/todos") for v1; reject off-site (no "http://?") – User_error at prim or 500 if somehow returned.

PRG: POST handlers that mutate SHOULD (redirect ?) so refresh does not re-POST. Dogfood todo does this.

Htmx partials: if request has header HX-Request: true, v1 MAY still wrap

chrome (acceptable); optional later: skip chrome when HX-Request. Not required for form POST dogfood.

Handler errors (Interp_error / missing handler): - GET: 500 HTML error page (status quo) - POST: 500 HTML error page (same); do not redirect

==========

DOGFOOD: INTERACTIVE TODO (acceptance for this plan) ==========

Upgrade Session.recipe_todo so /todos is a real form app:

Type: todo-item {title, done} (unchanged)

Functions (illustrative; exact HTML may vary): todo/add – (lambda (title) (create ?)) todo/list – (lambda () (rows 'todo-item 50)) todo/toggle – flip done via update-field / get+dict-set todo/page – method-aware page: POST: (todo/add (request-param "title")) then (redirect "/todos") GET: render-html with: - <form method="POST" action="/todos"> <input name="title"> + submit - list of items (title + done) - optional small POST forms/buttons for toggle (same path with action=toggle&id=? OR separate route /todos/toggle – pick one in make; prefer same-path + request-param "action")

Route: (route "/todos" "todo/page") only is enough if action dispatch is param-based. Extra routes OK if clearer.

Smoke path: POST /api/recipe/todo POST /todos title=milk -> 303 /todos GET /todos -> body contains milk Functions browser shows package todo

==========

MAKE CHECKLIST (implementation order) ==========

1. Session.request type + current_request field + accessors 2. Register request-* and redirect prims (session-scoped) 3. http.ml: parse query+form into request; allow POST on user-route fallthrough; call_handler ~request 4. http.ml: dispatch redirect sentinel -> 303 Location 5. Upgrade recipe_todo + home copy if needed 6. Smoke script (e.g. /tmp/pp_post_routes_it33.py) 7. Keep GET zero-arg hello recipe green

OUT OF SCOPE THIS PLAN: - path parameters (/items/:id) - multipart file upload - JSON body auto-parse to dict prims - CSRF tokens (localhost/dev trust; file issue if prod needs) - (hx-get)/(hx-post) Lisp prims (still embed in HTML strings) - Solo5 / gluten-mirage HTTP - PUT/PATCH/DELETE user routes

==========

ROUTE VISIBILITY (see auth.route-visibility – planned) ==========

Authority for "may this request run this page?" is the OCaml launch gate, not Lisp inside the handler.

- User routes default private (session or Basic required). - (route path handler 'public) ? anonymous GET allowed. - System hard-private prefixes never become public via view_bindings (/admin /data /api/eval /bindings /edit ?). - System hard-public remains /login /health /logout. - Optional auth: resolve identity even on public paths. - Anonymous: system/main, whoami anonymous, no writes v1. - Migration 006: view_bindings.visibility text default 'private'. list_routes/register_route carry the flag. - /pages shows visibility badge.

v1 SHIPPED (pre-it33): - httpaf-lwt-unix HTTP server - system pages listed above (except /inspect) - user routes via (route ...) + render-html (GET only)

DEFERRED beyond it33: - /inspect/:id (see image.inspector) - (hx-get)/(hx-post) Lisp prims - Solo5 HTTP via gluten-mirage + httpaf - richer system-browser chrome (own subsection) - public-write routes / CSRF (route-visibility phase 2)

primitives-layer

image/lib/primitives.ml: the ~20 impure fns registered at boot

[status: partial]

– *agent note:* 2026-07-12 status-reconcile: list-branches + current-branch now real (not VNil). Keep partial: query/describe/diff/hx helpers still missing or thin.

– *agent note:* it31 plan-pass #7: demoted implemented -> partial in lockstep with lisp-interpreter.primitives. Registration path is real: image/lib/primitives.ml + crud.ml + session-control prims + env auto-bind of VPrim names at session-make. Interpreter stays pure (lisp/ tests with no services). Section is partial because the registered universe still includes stubs/missing prims listed under lisp-interpreter.primitives (list-branches VNil stub, missing query/describe/inspect/diff/current-branch, hx helpers incomplete, create returns VStr not VRef, no author-scoped CRUD filtering). Promote back to implemented only when that table is honest and complete-or-trimmed.

See lisp-interpreter.primitives for the full primitive list. This subsection covers the IMAGE layer's responsibility: registering them with the interpreter at boot.

The interpreter is pure and knows nothing about Postgres, htmx, or SSH. The IMAGE layer: 1. Opens a Postgres connection (v1: pgx_lwt + custom Io_unix shim; Solo5 later may use pgx_lwt_mirage behind the same Store.t). 2. Constructs an initial env with VPrim values bound to names like 'get', 'create', etc. 3. The interpreter's Prim call dispatches by name to the OCaml function the image registered. 4. Each primitive handles its own transaction (SERIALIZABLE by default) and permission checks (per multi-user.permissions; author- scoped filtering still partial).

This separation is what makes the interpreter testable without any services running (pure lisp/test/eval_test.ml, no DB, no HTTP).

v1 SHIPPED registration path: real. DEFERRED: full prim table completeness (see lisp-interpreter.primitives); Solo5 pgx path.

system-browser

The live environment UI: browse branches, functions, types as a place you live
[status: partial]

– *agent note:* 2026-07-12 status-reconcile: Functions/pages/home/nav/theme shell live. Keep partial: branch browser promote/stats and full inspector polish still open.

– *agent note:* Chrome + pages + page inspector ship. 2026-07-11 plan B: Functions browser with package grouping, search, used-by pages; prefix packages (todo/list). Page?function nav fixes already live. 2026-07-12 PRODUCT SHELL RULE: product chrome is Lisp (ui/nav, ui/menu, ui/home). OCaml must not ship a silent second product nav/home. Silent default_nav_nodes / default_home_content are debt – emergency chrome only, labeled when used. Load-order fix (parents-first rehydrate) landed 653f5e5 so personal ui/menu pins can depend on system menu/* helpers. 2026-07-12 PRODUCT THEME (Path B, BORGE-1783837363): product presentation is Lisp data too – css/* sheet compiler + ui/theme + ui/css. Reject CSS-string-in-lambda hybrid and silent full OCaml design-system twin. See image.product-theme.

VOCABULARY (must not conflate): - Page / endpoint = route URL (e.g. /hello) - Function = named binding (e.g. hi-page or todo/list) - Package = in-branch name prefix (todo/), NOT a branch - Branch = versioned code world (system/main) A page RUNS a function; you EDIT functions, OPEN pages. Packages ORGANIZE functions inside one branch.

v1 SHIPPED: - Chrome: whoami, branch, uptime, nav - Home entry points + AI propose form - /about concepts guide - /pages list + /page/<path> inspector - Flat /bindings table (name, effects, author, edit, inspect)

FUNCTIONS BROWSER (/bindings) – Smalltalk-ish drilldown:

PHILOSOPHY: - Majority of programs are Lisp bindings in function_defs. - Kernel primitives are OCaml (performant, trusted); still BROWSABLE so the system is legible, but read-only. - Optional future: mark some OCaml modules as "image libs" for hot paths; not required for this cut.

MOBILE FIRST (three-step drilldown, not three columns): URL state via query params (works on phones): Step 1 GET /bindings ? list packages only (big tap targets) Always includes virtual package "(kernel)" Workspace packages from prefix grouping + "(root)" Step 2 GET /bindings?pkg=todo ? list functions in that package Kernel: list prim names in catalog Step 3 GET /bindings?pkg=todo&fn=list ? detail: source/docs, effects, author, used-by, edit/inspect links (workspace) OR read-only kernel doc (primitives) Desktop MAY show a wider table when no pkg selected (grouped overview), but pkg/fn drilldown is the canonical model and must work at ~390px width. Back links: Functions ? package ? (back to packages).

KERNEL CATALOG: - Virtual package name: "(kernel)" (not a real binding prefix) - Source: prim registry + short docs (name, summary, effects) - Kind badge: primitive | function - No Save on primitives; link to About/docs if needed

WORKSPACE (existing + keep): - Group by package_of_name; search q=; author filter - used-by pages; create package+local form - Edit only for function_defs rows

UI SHELL AS LISP (authoritative – 2026-07-12):

RULE: If a human thinks of it as "the environment UI" (navbar, home workspace, app menus), it is Lisp on a branch – not a parallel OCaml product skin.

- Product shell bindings (required when seed runs): ui/menu – menu tree (prefer zero-arg lambda that returns a list; eager lists are fragile if they call menu/* at define-time) ui/nav – (lambda () (render-html (menu/bar ?))) ui/home – (lambda () ?) home body HTML ui/theme – css/sheet data (tokens + rules); Lisp, not a CSS string blob ui/css – (lambda () (css/render ui/theme)) adapter only; session caches its string - libs/ui/lib.pp seeds system/main (load-library "ui" / seed_stdlib). Personal may pin overrides; inherit main when no pin. Prefer thunks over eager value pins. - Chrome nav uses (ui/nav); GET / uses (ui/home); full-page <style> uses rendered (ui/css) – see image.product-theme. - Identity strip only (brand, username, branch, uptime) stays OCaml – not product structure. - Kernel OCaml pages stay OCaml until deliberately moved: /login, /health, auth middleware, /bindings,/branches, /edit, /inspect, emergency recovery. These are host UI, not a second apps menu. - EMERGENCY chrome (nav/home/theme fail or unbound): minimal, labeled failure (must not look like a healthy product menu or full design system). Silent rich OCaml default_nav / default_home / full inline CSS twin is REJECTED architecture – debt to remove. - Rehydrate load order: parents first, child overrides last (so personal ui/menu can call system menu/link; personal theme/* tokens can override system tokens). - Edit shell via Functions ? package ui (mobile drilldown), including ui/theme and theme/* tokens.

LAYERING SUMMARY: Lisp = the computer you live in (shell + theme + apps) OCaml = host + gate (eval, store, auth, health, emergency)

DEFERRED: - Remove silent default_nav_nodes / default_home_content and shrink html_response style blob to emergency_css (image.product-theme make) - ui/menu seed always lambda (load-order safe by default) - Move softer system pages (/about copy) into Lisp packages - Branch switcher; fork/merge UI - True three-column desktop layout (optional later) - Import/require, private symbols, rename-with-rewrite - Authoring new prims from the UI - Dedicated /theme form UI (Functions edit is enough v1)

DATA BROWSER: full CRUD over data_rows is specified in image.data-browser (OCaml system pages, not a Lisp package). /types remains the type_defs index and links into /data/<type>.

data-browser

Generic OCaml system UI for full CRUD over shared data_rows by type
[status: planned]

> – *agent note:* 2026-07-12 plan: promote deferred 'Data rows browser by type' from system-browser into its own subsection. Full CRUD (list/create/ read/update/delete) as host/gate OCaml pages – parallel to /types and /bindings, not a Lisp product package. Product apps (todo/notes/bible) keep their own UIs; this is the always-on object browser.

WHY: /types shows declared shapes; /inspect/row dumps one row as pretty JSON. Neither is a place to live in the object store. Product packages (todo, notes, bible) have purpose-built UIs, but agents and humans still need a generic browser when there is no package UI, the package is broken, or the type is ad-hoc.

LAYERING (locked): - OCaml system pages in image/lib/http.ml (+ thin helpers in crud.ml / store.ml as needed). Host/gate tier – same as /types, /bindings, /inspect, /edit. - NOT a Lisp package. Product shell stays Lisp; this is emergency/generic substrate UI that must work with empty or broken gardens. - Mutations call existing Crud/Store SQL paths. Do not invent a second data access layer. - data_rows are SHARED across branches (no branch_id). UI must say so on every list/detail surface.

========== ROUTES (full CRUD) ==========

Index (Read types + counts): GET /data - Table: type_name | declared fields | row count | link - Types = visible type_defs on current branch (Store.list_types) - Also surface type_names that have data_rows but no type_def (orphan data) so nothing is invisible - Banner: "Data rows are shared across branches" - Link back/forward with /types (types = schema index; /data = instances)

List (Read many): GET /data/<type> - Query: limit (default 50, max 200), offset (default 0) - Columns: id (short) + each declared field (truncated) + updated_at if cheap; link id ? detail - Pagination: Prev / Next when more rows exist - Actions: [New row] ? /data/<type>/new - Empty state: no rows yet + create CTA - Order: created_at ASC (match Crud.list) unless a later plan picks something smarter

Create: GET /data/<type>/new - Form: one input per declared field (text inputs v1) - If type has ZERO declared fields OR user wants freeform: textarea name=fields_json for raw JSON object - POST target: /data/<type> POST /data/<type> - Content-Type application/x-www-form-urlencoded - Build fields jsonb from declared field names present in the form; if fields_json posted and non-empty, that wins (power path) - author = Session username - On success: 303 Location: /data/<type>/<id> - On failure: 400 with error card + back link

Detail / Update / Delete: GET /data/<type>/<id> - Show all fields (declared first, then extra jsonb keys not in type_defs – do not hide unknowns) - Edit form prefilled (same field rules as create) - Delete form POST /data/<type>/<id>/delete (confirm via button label; no JS required) - Meta: id, type, author, created_at, updated_at when available - Links: back to type list, /types, REPL hint POST /data/<type>/<id> - Update fields (full replace of fields jsonb from form, same merge rules as create). Uses Crud.update path. - 303 back to detail on success POST /data/<type>/<id>/delete - Crud.delete; 303 to /data/<type> - Missing row: 404 card, not silent success

URL parsing notes: - type_name may contain only safe path segments in v1 (letters, digits, hyphen, underscore). Reject or 400 weird paths; no

path-param injection into SQL (always parameterized – already true in Crud). - id is UUID string. - Prefer longest-match dispatch: /data/<type>/new /data/<type>/<id>/delete /data/<type>/<id> /data/<type> /data so "new" is never treated as an id.

========== FIELD / JSON RULES (v1) ==========

- type_defs.fields is a JSON array of field-name strings (no field types yet – object-model.type-system deferred). - Form values are strings; store as JSON strings in the fields object. Do NOT guess number/bool coercion in v1 unless the existing value is already that shape and a later polish wants round-trip fidelity. - Raw JSON textarea: must parse as a JSON object; reject arrays/scalars with a clear error. - Extra keys on existing rows: display on detail; include in edit form as additional inputs OR preserve via hidden merge so update does not drop them. REQUIRED: updates must not silently strip unknown keys.

========== RELATIONSHIP TO EXISTING PAGES ==========

- /types: keep as type_defs browser; each row links to /data/<type>. Optional: add row count column on /types once count helper exists (nice-to-have, not blocking). - /inspect/row/<type>/<id>: redirect (302/303) to /data/<type>/<id> OR thin alias rendering the same page. Prefer redirect so one canonical URL. - Product apps unchanged: /todos, /notes, /bible stay Lisp. - Nav: emergency/system links may include Data ? /data; product ui/menu MAY add Data later (Lisp choice, not required for make).

========== STORE / CRUD HELPERS (implementation discretion) ==========

Add only what the UI needs if missing: - count rows by type_name (for index) - list type_names present in data_rows (orphans) - list rows with limit+offset (Crud.list today is limit-only; extend with offset or a store helper) - optional: fetch author/created_at/updated_at for detail Keep Lisp prim surface stable unless a prim gap blocks the browser (prefer OCaml-only helpers for offset/count).

========== AUTH / SAFETY ==========

- Same session gate as other authenticated system pages. - Not admin-only in v1 (any logged-in user can browse/mutate shared data – matches Lisp CRUD prims today). - No CSRF token infrastructure in v1 (status quo for other POSTs); cookie session + same-site assumptions hold. - Never interpolate type/id into SQL strings.

========== NON-GOALS (v1) ==========

- Field type system / validation beyond JSON object shape - Full-text search UI (find-text exists for Lisp; wire later) - Inline htmx row edit without full page (optional polish) - Bulk delete / CSV import-export - Per-row branch scoping (data is shared by design) - Lisp-redefinable browser UI - SPA / client-side routing

========== ACCEPTANCE NARRATIVE ==========

Human can: open /data ? pick a type ? page through rows ? open a row ? edit a field ? save ? create a new row ? delete a row ? land back on the type list. All without REPL. Agent can use the same URLs or keep using CRUD prims.

product-theme

Product presentation as Lisp data: css/* sheet compiler, ui/theme + ui/css, emergency CSS only in OCaml

[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: Path B shipped: libs/std css/* + libs/ui ui/theme + ui/css; BORGE-1783837363 resolved.

– *agent note:* 2026-07-12 Path B locked (BORGE-1783837363). Rejects: (1) CSS-string-in-lambda hybrid (worst of both languages); (2) silent full design-system twin hard-coded in http.ml html_response as the product theme. Honest Lisp: styles are sexps; css/render emits CSS; session caches string; kernel injects or falls back to labeled emergency_css. Parallel to ui/nav product-shell rule.

WHY: Today almost all product CSS is a giant inline string in image/lib/http.ml html_response – inaccessible from the garden, unversioned as Lisp, dual to the product-shell rule that already moved nav/home into libs/ui. Widget packages emit class names that only work because that OCaml blob knows them. That is a black box.

DECISION (Path B – honest Lisp): Presentation is Lisp data, same substrate as shell and apps. A small css/* compiler (sibling of html/* in std) turns sheets into CSS text. ui/theme holds the sheet; ui/css is a thin (lambda () (css/render ui/theme)) adapter for session refresh. Humans and agents edit sexps / tokens via Functions – not escaped CSS inside a lambda, and not a second language document as the native store.

REJECTED: - Path hybrid: (define ui/css (lambda () "body{?}")) as the primary form (CSS-in-string wearing Lisp hat) - Path A as native store: bare CSS document / data_rows stylesheet type as the only theme representation (allowed later as import/export sugar, not authority) - Static /assets/*.css as the product theme path - Silent full OCaml design-system twin beside Lisp theme - Per-request uncached re-eval of ui/css without shell refresh discipline (mirror ui/nav caching lessons)

==========

css/* (libs/std/lib.pp – seed with std, before ui) ==========

Parallel to html/*: structure in Lisp, render to string.

Core bindings (v1 – keep small): (css/sheet rule?) ? sheet value (list-tagged data) (css/use other) ? splice/concat another sheet (css/raw "?") ? escape hatch for rare CSS (css/render sheet) ? CSS string

Rule encoding (selectors as strings; declarations data): - (selector prop val prop val?) selector may be a symbol (body, header) or string (".card", "nav a", "details.ol-node > .ol-sum") - ((sel1 sel2 ?) prop val ?) multi-selector group ? "sel1, sel2 { ? }" - ((media "max-width:40rem") rule?) nested @media block - Values: number | string | symbol (symbol resolves as token binding if bound, else as CSS ident) - Do NOT invent a full selector AST or px-type system in v1. Selectors stay strings/symbols; props are alternating key/value data.

Emit rules (css/render): - prop symbols/strings ? CSS property names (margin ? margin; font-family ? font-family; accept underscore?hyphen if needed) - numbers: print without quotes (0, 1.4) - strings: as CSS values (caller includes units) - stable, readable output; order preserved - css/raw segments pass through unchanged

Composition: (css/sheet (css/use ui/theme-base) (css/use todo/theme) ; optional later (body background theme/bg))

html/* analogy docstring in std: "sheets ? css/render; no raw rule soup in apps" (css/raw remains the deliberate guest).

==========

ui/theme + tokens + ui/css (libs/ui/lib.pp) ==========

Required product-shell bindings (with ui/menu ui/nav ui/home):

theme/* tokens (examples; exact set is implementation discretion so long as the classic-Mac look ports): theme/bg theme/ink theme/panel theme/chrome theme/link theme/err theme/ok ?

ui/theme A css/sheet (or zero-arg lambda returning a sheet) expressing the full product design system currently hard-coded in http.ml (body, header, nav, cards, tables, forms, outline, todos, calendar, bible, weather class contracts,

mobile breakpoints). Prefer data + theme/* tokens over magic hex literals so personal gardens can (define theme/bg ?).

ui/css (lambda () (css/render ui/theme)) or (lambda () (css/render (ui/theme))) if theme is a thunk. Adapter only – not where humans paste CSS.

Class contract: - Shared classes used by multiple packages live in ui/theme (todo-box, ol-node, cal-grid, card, ?). - Package-local extras may keep a small (html/tag "style" ?) or later (todo/theme) composed via css/use – weather/style today is acceptable local. - Moving the monolith out of OCaml does not require splitting packages on day one.

Seed: seed_stdlib still load-library std then weather then ui ? – css/* must exist before ui/theme renders. (std before ui already holds.)

==========

Session cache + HTTP inject ==========

Mirror ui/nav:

Session.t gains mutable ui_css : string option Some s ? product stylesheet text None ? use emergency_css

refresh_ui_shell (rename meaning: refresh shell): existing (ui/nav) path unchanged plus eval (ui/css) ? string | raw-html-as-text ? cache; on miss/throw ? ui_css <- None

Invalidate / refresh when: define/load of ui/css, ui/theme, ui/nav, ui/home (and ideally theme/* token defines – at minimum any define that refresh_ui_shell already re-runs after shell names; token-only defines may require explicit ui/css refresh or re-eval on next request launch) checkout / branch switch (no stale other-branch CSS) load-library that may redefine shell

html_response gains ~css parameter (or reads session): HTML.style [] "%s" css Authenticated full pages pass Session.ui_css | emergency_css.

fragment_response / HX partials: do NOT re-inject the full stylesheet (page already has it). Same spirit as chrome-skip for fragments.

login_form_page (pre-session): keeps a tiny OCaml-only mini stylesheet. Must not require Lisp theme. Not the product design system.

==========

Emergency CSS (OCaml kernel – labeled, minimal) ==========

emergency_css is host recovery, not a product twin: - monospace / high-contrast / readable - styles emergency-banner, emergency-nav, basic links - MUST look broken or sparse on purpose if product theme fails (parallel emergency-nav rule) - After make: the 200-line classic-Mac blob MUST leave html_response; only emergency_css (+ login mini) remain as OCaml string stylesheets.

Optional: when falling back, surface a one-line labeled banner in chrome ("product theme unbound – emergency css") consistent with emergency-nav labeling.

==========

Authoring / multi-user / sync ==========

- Edit via Functions ? package ui ? ui/theme | theme/* (same mobile drill-down as other shell bindings). - Personal pin: (define theme/bg "#e8f0e8") or full ui/theme override on user/personal; inherits main when no pin. - (export-package "ui") includes theme tokens + ui/theme + ui/css; pp_sync pull/push works unchanged. - Agents: pp_eval define/function-source – no new REST. - v1 non-goals: /theme textarea page, CSS file import pipeline, Tailwind/Sass build step, data_rows theme type as authority, per-user CSS without branch pins.

==========

MAKE ORDER ==========

1. css/* in libs/std + unit/smoke sexp?CSS 2. Transcribe http.ml product blob ? ui/theme (+ tokens) in libs/ui; define ui/css adapter 3. Session ui_css + refresh_ui_shell extension 4. html_response ~css inject from session 5. Shrink OCaml to emergency_css (+ login mini) 6. Smokes: seed marker, personal override, failure fallback, fragment no-reinject 7. borge drift / resolve BORGE-1783837363

STATUS: planned until make lands and silent full OCaml product CSS twin is gone.

packages

In-branch prefix packages for organizing functions (not branch ownership)
[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: Prefix packages, load-library, seed_stdlib (std/ui/ai/outline/notes/todo/calendar/bible/dabar/?), export/function-source, bible dogfood. Study extras under dabar.borg.
>
> – *agent note:* 2026-07-11 plan: packages = name prefixes with / separator. Orthogonal to multi-user.namespaces (branches). No new tables in v1. Reader already allows / in symbols (is_symbol_start). 2026-07-11: BIBLE LIBRARY dogfood locked – BSB in data_rows, ref format BSB:Abbrev:C:V, bulk seed script, libs/bible/lib.pp, URLs via query book/chapter/verse or ref= (see BIBLE LIBRARY). Study layer (Strong's, interlinear, dabar package) is specified in chapter dabar.borg (root pricklypear.borg inlines it) – not expanded here.

PROBLEM: flat binding names collide once many tools live on one branch (todo list vs calendar list).

MODEL (v1): - Package is a PREFIX of a function name, separator '/'. - Full binding key remains (branch_id, name) with name the full string "todo/list". No packages table required. - package_of_name "todo/list" = "todo"; local = "list". - package_of_name "todo/ui/row" = "todo/ui"; local = "row". - package_of_name "hi-page" = "" (root). - Call and define use the full name: (todo/list), (define todo/list ?). - Pages still route to full handler names: (route "/todos" "todo/list").

ORTHOGONAL TO BRANCHES: - Branch = version + ownership (multi-user.namespaces). - Package = organization inside one branch. - UI must never call a branch a "package" or vice versa.

NOT IN V1: - (require 'todo) / import aliases - Private package symbols - Automatic rename of call sites - Cross-branch qualified names (system/main:greet) – still multi-user.namespaces deferred item - Privileged global package registry

AI / EDITOR: - Prefer namespaced names for new tools (todo/list not list). - AI propose form: optional package field + local name, or single name field that may contain /. - Function editor URL remains /edit/<full-name> with / in the path segment (encode if needed) or query name=; implementation may use /edit?name=todo/list if path routing is awkward – pick one and document in agent note.

LIBRARY LOADER (implemented): Same Lisp source text in git repos and in the live system. OCaml provides only a pure loader (no app string blobs).

On-disk layout (shipped in release tarball as libs/): libs/std/lib.pp – list/*, html/*, css/*, menu/*, data/* libs/todo/lib.pp – deftype + todo/* + routes + partials libs/hello/lib.pp – optional hi-page recipe as Lisp libs/bible/lib.pp – bible/* API + /bible (canon is data_rows) libs/ui/lib.pp – ui/menu ui/nav ui/home ui/theme ui/css libs/ai/lib.pp – ai/ask ai/do wrappers

Loader API: (load-string SRC) read_all forms; eval_one each (define persists via existing intercept; route/ deftype via prims). Stop on first error. (load-library NAME) load file libs/<NAME>/lib.pp relative to PP_LIBS_DIR or ./libs (release CWD).

Boot / recipes: seed_stdlib ? load-library "std" then "ui" then "ai" recipe_todo ? load-library "todo" recipe_hello ? load-library "hello" recipe_bible ? load-library "bible" (types+fns+route ONLY; does NOT bulk-insert verses)

Release build MUST copy libs/ into the tarball.

==========
LIBRARY SYNC CLI (2026-07-11 – implement) ==========
PROBLEM: live function_defs drift from git libs/*.pp when users or AI edit in the browser. Push of kernel does NOT overwrite DB. Developers need explicit sync so laptop git matches language state.

MODEL: - Local truth for recipes: libs/<pkg>/lib.pp - Live truth for running language: function_defs on current branch - CLI (scripts/pp_sync.py) uses authenticated /api/eval (pp_eval) and export prims – no direct Postgres from laptop required.

EXPORT PRIMS (image): (function-source "ui/menu") ? VStr pretty body only (lambda/list/?) for define (export-package "ui") ? VStr multi-form lib.pp text: ;; package ui – exported from live branch (define ui/menu ?) (define ui/nav ?) ? sorted by name Root package "" / "(root)" exports unprefixed names only. Kernel prims are never exported. (package-bindings "ui") ? VList of VStr full names in that package on this branch

CLI: scripts/pp_sync.py status [pkg] Compare libs/<pkg>/lib.pp define names vs live package-bindings. Report only-local / only-live / both (optional body hash later). scripts/pp_sync.py pull <pkg> Write libs/<pkg>/lib.pp from (export-package pkg). Creates directory if missing. Does not auto-commit git. scripts/pp_sync.py push <pkg> (load-library pkg) via pp_eval – file ? live (upsert defines). scripts/pp_sync.py pull-all Pull every non-empty package that exists live or under libs/.

ENV: same as pp_eval (PP_URL PP_USER PP_PASS, ~/.config/pricklypear/env).

OUT OF SCOPE v1: - data_rows sync (todos/bible text) - automatic merge conflict resolution - function revision history table - Doltgres

VERIFY: - (export-package "ui") contains define ui/nav - pull ui rewrites libs/ui/lib.pp; status shows both - push ui after edit reloads live

==========
BIBLE LIBRARY (BSB dogfood – PLAN locked 2026-07-11) ==========
GOAL: read BSB from the live image via Lisp + /bible page. Canon lives in Postgres data_rows (shared across branches). Functions live in package bible/ via load-library.

DECISIONS: - Translation v1: BSB only (attribution in page footer / lib header). - Storage: data_rows, NOT giant define / NOT .sexp corpus at runtime. - Ref string (human + share + seed key): BSB:<Abbrev>:<chapter>:<verse> e.g. BSB:John:3:16 BSB:1Cor:13:4 BSB:Ps:23:1 - Book token in ref = stable ABBREV (no spaces). Display name on bible-book rows separately. - data_rows.id stays UUID PK (migration 003). Ref is NOT the PK. fields.ref holds BSB:Abbrev:C:V uniquely. - Optional seed optimization: deterministic UUID v5 from ref so tooling can recompute id; Lisp users still call bible/* not raw get-by-uuid. Either UUID-default or v5 is fine if ref unique.

TYPES (deftype from libs/bible/lib.pp): bible-book: name, abbrev, ordinal, testament (~66 rows; menus / prev-next / name?abbrev map) bible-verse: translation, book, abbrev, chapter, verse, text, ref chapter/verse as JSON numbers or digit strings – pick one in seed and stick to it (prefer numbers in JSON). ref = "BSB:John:3:16" always.

LOOKUP (needs more than blind rows scan): (get 'bible-verse uuid) alone is not enough for humans. Provide ONE of (make picks; prefer smallest): A. Crud/store helper + prim: (find-row 'bible-verse "ref" "BSB:John:3:16") (find-rows 'bible-verse "abbrev" "John" ["chapter" 3]) ; chapter list B. Dedicated thin prims bible-get-ref / bible-chapter (worse – special-case kernel). Prefer A (general field find). GIN on fields already exists; add UNIQUE index for idempotent seed: CREATE UNIQUE INDEX data_rows_bible_ref_uidx ON data_rows ((fields->>'ref')) WHERE type_name = 'bible-verse'; (migration 004 additive – only if make needs it).

LISP API (libs/bible/lib.pp): (bible/ref book chapter verse) -> "BSB:John:3:16" book may be name or abbrev; normalize via bible-book catalog (bible/verse book chapter verse) -> text VStr | VNil (bible/verse-row book chapter verse)

-> fields JSON VStr | VNil (bible/chapter book chapter) -> VList of verse fields/sexps (bible/books) -> list from bible-book rows ordered by ordinal (bible/search q) -> VList hits (v1: SQL ILIKE or limited scan; may be prim-backed; OK if slow/limited first cut) (bible/page) -> GET renders chapter; POST goto redirects (route "/bible" "bible/page")

URL SURFACE (exact path /bible only – no path params in v1): GET /bible GET /bible?book=John&chapter=3 GET /bible?book=John&chapter=3&verse=16 GET /bible?ref=BSB:John:3:16 GET /bible?q=love (optional search) POST /bible book= & chapter= & verse= -> redirect to GET with query (PRG) Do NOT require colon-ids in path segments. ref= query is the shareable explicit form (URL-ok in query).

ABBREV TABLE (seed must use these tokens in ref): Gen Exod Lev Num Deut Josh Judg Ruth 1Sam 2Sam 1Kgs 2Kgs 1Chr 2Chr Ezra Neh Esth Job Ps Prov Eccl Song Isa Jer Lam Ezek Dan Hos Joel Amos Obad Jonah Mic Nah Hab Zeph Hag Zech Mal Matt Mark Luke John Acts Rom 1Cor 2Cor Gal Eph Phil Col 1Thess 2Thess 1Tim 2Tim Titus Phlm Heb Jas 1Pet 2Pet 1John 2John 3John Jude Rev (If vendor data uses different codes, map in seed script; document mapping next to script. Display names full English.)

BULK SEED SCRIPT (required – not HTTP recipe): Path: scripts/seed_bible_bsb.py (or .ksh driver + py) Inputs: - BSB source file(s) (JSON/CSV/USFM – vendor format; script owns parsing; do not commit illegal redistrib if license forbids; prefer redistributable BSB dump + NOTICE) - PG connect: same as dev (PP_DB_* / /tmp:5433) Behavior: 1. Ensure type_defs exist OR document that load-library bible must run first (deftype). Prefer: script can INSERT type_defs rows for bible-book/bible-verse if missing (author system). 2. Upsert ~66 bible-book rows (stable id or fields.abbrev unique). 3. Upsert all bible-verse rows: fields.ref unique; ON CONFLICT DO UPDATE text (requires unique index on ref – migration or special conflict target). 4. Idempotent: second run = no duplicate verses. 5. Print counts: books N, verses M. NOT in recipe_bible HTTP handler (too slow / wrong layer).

INSTALL STORY: A. scripts/seed_bible_bsb.py # once per DB B. (load-library "bible") # or POST /api/recipe/bible C. (bible/verse "John" 3 16) D. open /bible?book=John&chapter=3

HOME UI (optional make): Card "Install bible library" -> POST /api/recipe/bible Note on card: "Run seed script for BSB text if empty".

RENDER: v1: simple HTML chapter list + optional verse highlight is OK. Optional: ?view=sexp returns chapter as Lisp list via bare value. Attribution footer: Berean Standard Bible / license line.

PERSONAL LAYER: notes/bookmarks are NOT bible/dabar – a separate module owns them. Canon text stays shared.

MAKE CHECKLIST: 1. Abbrev map + seed script I/O against sample or full BSB 2. migration 004 unique index on verse ref (if upsert needs it) 3. find-row / find-rows (or equivalent) for ref + chapter 4. libs/bible/lib.pp API + route /bible 5. recipe_bible + optional home card 6. Smokes in verify stanza above

OUT OF SCOPE V1: - multi-translation UI - path params /bible/John/3/16 (map later from same ref) - full-text search engine - audio / cross-refs apparatus - pure-sexp file corpus as runtime source of truth

DEFERRED (packages general): library_files table, git export, multi-file manifests, load from HTTP.

Kernel prims (OCaml) stay under (kernel) browser package. Majority of app code remains Lisp in .pp files.

browser-repl

Evaluate Lisp from the browser against the live session env

[status: implemented]

– *agent note:* it32: IMPLEMENTED. GET /repl + POST /api/eval

invoke Session.eval_one on the live env. Multi-line via textarea; errors return JSON without killing session; (define ...) persists. scripts/dream_product_smoke.py covers define+call.

- GET /repl – input + Eval button - POST /api/eval {expr} -> {ok,value|error} - Same Session.eval_one path as SSH REPL - Not a second interpreter

function-editor

Load, edit, and save a binding's AST from the UI

[status: partial]

– *agent note:* it32: base editor implemented. 2026-07-11 plan: missing-fn empty state + htmx create-in-place; route-aware choices when name matches a page handler path. Status back to partial until those ship.

URL: /edit/<function-name> (function name, NEVER a route path as the object of edit – routes are pages).

v1 SHIPPED: plain textarea editor + save on current branch when the binding already exists.

REQUIRED NEXT – missing binding empty state: When GET /edit/<name> and no function_defs row for name:

1. Do NOT present only a raw TODO stub as if the user is already editing a real function.

2. Show an explicit card: Function "<name>" does not exist.

3. Route awareness: if a page route "/<name>" (or path equal to name) exists with handler H: There is a page /<name> that currently runs function H. Primary: [Edit H] -> /edit/H Secondary:[Create function <name>] Optional: [Create <name> and point /<name> at it] (define + route update; power move) If no such route: only Create (and link back to Functions).

4. Create-in-place via htmx (no full-page dead end): POST /api/edit/create {name} (and optional rebind_route) -> 200 HTML partial: the editor form for the new binding (same fields as GET /edit/<name> after create) hx-target=#editor-root hx-swap=innerHTML Default body stub: (lambda ()\n "TODO: implement <name>") After create, user saves via existing POST /edit/<name>.

5. Creating a function does not silently rename routes unless the user chose "point page at it".

6. Namespaced names (image.packages): - Accept full names with '/' (todo/list) in name field. - Create form MAY split package + local name and join with '/'. - URL routing for /edit/todo/list: either path segments after /edit/ joined back to todo/list, or /edit?name=todo%2Flist. Prefer path join if http router is easy; document choice.

DEFERRED: optimistic concurrency / stale-body reject, structural AST tree-editor (branching.ast-nodes).

inspector

Inspect values, rows, ASTs, and page?function graphs live

[status: partial]

– *agent note:* it32: PARTIAL. GET /inspect/binding/<name> and /inspect/row/<type>/<id> ship. 2026-07-11 plan: page call graph – endpoint ? handler + static user-function callees.

v1 SHIPPED: binding AST pretty+JSON; data row JSON pretty.

REQUIRED NEXT – page ? functions graph: GET /page/<path> (also linked from /pages "inspect"):

A. Routing panel - Page URL (canonical route key) - Handler function name - Links: open page, edit handler, change handler (deferred UI)

B. Functions this page uses (STATIC analysis) - Always include the handler as root - Walk handler body AST; collect App heads that are Vars naming a visible function_def on the current branch - Depth: transitive closure of user bindings (cap depth e.g. 8 to avoid cycles; mark cycles) - Do not list pure prims as first-class rows; optional dim note "also uses prims: render-html, ..." is OK - Each row: name, role (handler|callee), edit + inspect links

C. Entry body - Pretty source or pp_expr of the handler (read-only here; edit goes to /edit/<handler>)

This answers: "which functions does this endpoint touch?"

REVERSE INDEX (function ? pages) – now REQUIRED for the Functions browser (system-browser), not only deferred: - For each visible route, compute handler + static callee set. - Invert to map function-name -> list of route keys. - Expose on /bindings rows as "used by" links to /page/<path>. - Same static analysis as page inspector (no runtime traces).

DEFERRED: eval-result inspect, type-def deep inspect, runtime request traces, mutate actions from inspector.

10 lisp-interpreter.borg

pricklypear-lisp-interpreter

Pricklypear book chapter: lisp-interpreter
[status: partial]

lisp-interpreter

Pure Scheme-flavored Lisp-1 interpreter, AST-as-source, primitives layer
[status: partial]

> – *agent note:* Section overall 'partial' as of it18 plan-pass: ast = implemented, reader = implemented, eval = partial (Lwt-typed eval shipped it12; the per-call step-budget / cooperative stepping described in eval subsection text is deferred), primitives = implemented (the ~21 prim set + dict prims + render-html + CRUD + select-for-update + deftype + branching control prims).

ast

The expr and value types
[status: implemented]

> – *agent note:* lisp/lib/ast.ml ships exactly the type schema documented below (Var/Lit/App/Let/Letrec/If/Lambda/Prim/Seq/Quote on the expr side; VInt/VStr/VBool/VNil/VPair/VLambda/VPrim/VRef/VList on the value side). AST serializes to/from JSON via Ast.expr_of_json / expr_to_json (Yojson) for storage in function_defs.body::jsonb. Homoiconicity preserved (the reader produces the same shape eval consumes; printer round-trips). Closed value type universe – no VDict was ever introduced (v1 used VStr-JSON + dict prims instead; see BORGE-1783432745).

type expr = | Var of string | Lit of value | App of expr * expr list | Let of string * expr * expr | Letrec of (string * expr) list * expr | If of expr * expr * expr | Lambda of string list * expr | Prim of string * expr list | Seq of expr * expr | Quote of expr (* for macros later *)

type value = | VInt of int64 | VStr of string | VBool of bool | VNil | VPair of value * value (* cons cells *) | VLambda of string list * expr * env | VPrim of string (* primitive name *) | VRef of type_name * id (* tagged ref into store *) | VList of value list (* sugar for nested pairs *)

The AST IS the source (homoiconicity). Stored as JSONB in function_defs.body. The reader (s-expr parser) produces expr values; the printer round-trips them. No parser/printer asymmetry.

Closures capture their defining env (lexical scope). VLambda carries env. No dynamic scope.

VRef carries its target type so the interpreter can verify derefs at call time before querying the store.

reader

S-expression reader: source text -> AST
[status: implemented]

> – *agent note:* lisp/lib/reader.ml. Recognizes standard s-expr surface syntax: (foo bar baz), 'symbol, "string", 123, #t, #f, (). Comments ';' to end-of-line. Special forms (lambda/let/letrec/if/define/quote/fork/merge/route/checkout/deftype) are recognized at EVAL time, not read time – the reader produces App (Var name,

> ...) and eval dispatches on the head. is_complete correctly tracks line-comment completeness at EOF (it1 fix). The '\\' escape handler in read_string is correct as of it13 (was a stale advance-return bug; see BORGE-1783431485). pp_value round-trips VPair chains as proper lists and improper tails with '.' notation. 2026-07-11: symbol charset already allows '/' (package names like todo/list are Vars). Keep that; add explicit smoke/tests when image.packages lands. Do NOT treat '/' as a reader special form.

The reader parses Lisp surface syntax to expr values. Standard s-expressions: (foo bar baz), 'symbol, "string", 123, #t, #f, ().

Surface syntax is just an editor concern: a function stored in Postgres is its AST (JSONB), and the s-expr syntax is one of several possible editors (along with the htmx tree-editor that edits the AST directly).

Special forms recognized at read or eval time (decision: eval time, the reader is generic): (lambda (args) body) (let ((name val)...) body) (letrec ((name val)...) body) (if test then else) (define name expr) (* top-level binding *) (quote x) (deftype name (fields ...)) (fork parent child) (merge source target) (app "prim-name" args) (* explicit prim call *)

Comments: ; to end of line. Docstrings: a leading string in a body position is the function's doc.

eval

The evaluator: env * expr -> value
[status: partial]

> – *agent note:* it31 plan-pass #7: corrected stale it12 note.
>
> SHIPPED: - 'Eval.eval : env -> expr -> value Lwt.t' (it12 Lwt-ABI). - Pure prims via register_sync; DB prims return promises. - Special forms: lambda/let/letrec/if/define/quote/seq/prim/ with-transaction. - Per-call step budget IS implemented in lisp/lib/eval.ml: type budget { mutable steps; limit }, default_limit = 100_000, tick at every recursive eval step, raises Budget_exceeded. Earlier agent notes claiming "NO step budget" were stale. - 30 alcotest cases in lisp/test/eval_test.ml.
>
> STILL PARTIAL: (1) Quote of compounds: quote still accepts only Lit/Var; (quote (a b)) and nested App forms raise User_error "can only quote literals/symbols" (BORGE-1783383905). Spec intent is full s-expr quote; this blocks quoted data as first-class values. (2) Memory budget tracking (multi-user.budgets layer) deferred. (3) Explicit per-subcall budget raise + audit logging deferred.
>
> Status stays partial primarily for quote semantics + memory budget, NOT because step budgeting is missing.

The evaluator is PURE. It takes an env and an expr, returns a value or raises an interpreter error. No I/O in the interpreter. All side effects go through Prim calls, which the IMAGE layer wires to actual Postgres/SSH/HTTP side-effects.

Cooperative evaluation: the eval loop is stepped; each Prim invocation yields back to the image's event loop. This gives per-session budgeting: - A step counter per call, configurable per session - When exceeded, that ONE call aborts - The session continues This prevents "user's bad function wedges the image" entirely.

(define name expr) at top-level adds binding to current session's env AND persists to store. Edit = redefine (update the row). No versioning at the binding level beyond what branches provide.

Application: evaluate operator + args, apply. Primitives are dispatched by name through a table the IMAGE layer registers at boot. The interpreter knows nothing about Postgres or htmx.

Special forms handled in eval, not reader: lambda, let, letrec, if, define, quote. The reader just produces App/Variants of these as plain AST.

primitives

The ~20 impure functions the interpreter calls into
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: list-branches + current-branch now real. Keep partial: query/describe/diff/hx helpers + author-scoped CRUD still open; quote compounds open (BORGE-1783383905).
>
> – *agent note:* it31 plan-pass #7: demoted implemented -> partial. Primitive DISPATCH + a useful core set are shipped, but the section's own details still enumerate missing/stub prims, so 'implemented' was an overclaim.
>
> v1 SHIPPED (image/lib/primitives.ml + crud.ml + session.ml): pure math (+ - * = < > <= >= not), list/pair (list car cdr cons null? pair? count), string (string-append str), reflection (type-of), session-aware (whoami current-author list-bindings), CRUD (get create update update-field delete rows select-for-update deftype), dict-over- JSON (dict-get dict-set dict-keys), branching (fork merge checkout), htmx (route render-html), with-transaction special form + private ___with_transaction___ prim. All on Lwt-typed ABI (it12). 48 alcotest cases + multiple smoke tests.
>
> STUB / MISSING (keep section partial until these land or are explicitly removed from the doctrinal table): (1) (query 'type where: ...) – only (rows 'type [limit]) (2) (describe 'type) / (inspect binding) (3) (diff source target) (4) (current-branch) (5) (list-branches) – registered but returns VNil stub (6) (hx-get)/(hx-post) – hx-get is a weak string-tag stub; hx-post missing; authors currently embed Hx attrs inside render-html (7) create returns VStr uuid, not VRef as details text says (8) author-scoped data filtering on CRUD (permissions gap)
>
> image.primitives-layer demoted in lockstep.

Primitives live in image/lib/primitives.ml. They are REAL OCaml, typed at the boundary, fast. Anything that touches Postgres, htmx, or the OS is a primitive, never user-Lisp.

CRUD: (create 'type {fields...}) -> VRef (get 'type id) -> record value (update 'type id {fields...}) -> VRef (delete 'type id) -> VNil (query 'type where: ...) -> VList of records (select-for-update 'type id) -> record (locked)

Branching: (fork parent child-name) -> branch-id (merge source target) -> status (current-branch) -> branch-id (checkout branch-id) -> VNil (* session switches *) (list-branches) -> VList (diff source target) -> VList of changes

htmx/Routes: (route path fn) (* register route *) (render-html node) (* pure-html node *) (hx-get url) (hx-post url) (* attribute helpers *)

Reflection: (type-of value) -> type-tag (describe 'type) -> schema (inspect binding) -> AST + metadata (list-bindings) -> VList

Sessions: (whoami) -> username (current-author) -> username (* for perm checks *) (call-fn name args...) (* resolves+calls *)

All primitives that touch data consult the CURRENT-AUTHOR (from the binding being executed) for permission filtering, per multi-user model.

Effect tags for each prim are defined in lisp-interpreter.effects (capability lattice).

effects

Capability effects for Lisp programs: classify pure vs DB vs AI vs publish; infer; gate; display. Separate from OCaml 5 host effects and from Lwt async plumbing.

[status: implemented]

> – *agent note:* it34 IMPLEMENTED: lisp/lib/effect.ml + 12 tests + /bindings badges + AI propose policy (PP_AI_ALLOW_WRITE elevate). it33 design sketch (roerick asked for effects + async clarity). IMPORTANT SPLIT: (1) ASYNC composition is ALREADY SOLVED at runtime: Eval.eval returns value Lwt.t; DB/AI prims return promises; pure prims use register_sync. (define u (get 'note id)) composes. (2) EFFECTS as policy/visibility are NOT solved: the language does not yet label or enforce Pure/ReadDb/WriteDb/Ai/Publish. (3) OCaml 5 algebraic effects are a possible FUTURE host runtime for the interpreter, NOT a substitute for Lisp-level capability effects. Do not migrate off Lwt for this feature. This subsection is the source-of-truth design; implement later as prim effect table + AST inference + UI badges + AI/route gates.

==========

0. THREE PROBLEMS PEOPLE CONFUSE (read this first) ==========

A) ASYNC / COMPOSITION (mostly DONE) Can a function call (get ...) and use the result? YES today: (define u (get 'note id)) (dict-get u "title") Mechanism: eval : env -> expr -> value Lwt.t Pure prims: register_sync wraps Lwt.return DB/AI prims: return promises directly This is the "async problem" for data-in-Lisp. It is runtime plumbing, not an effect system.

B) CAPABILITY EFFECTS (NOT DONE – this section) What powers does this function need? Pure calc vs read DB vs write DB vs AI vs publish page. Needed for: UI badges, AI safety, page policy, testing without services, author-runs-as-author story.

C) HOST ALGEBRAIC EFFECTS (OPTIONAL LATER) OCaml 5 Effect.perform / handlers inside the interpreter implementation. Could one day replace some Lwt coloring. Does NOT give Lisp programs effects for free. Guest language still needs B.

Do not block B on C. Do not re-solve A while building B.

==========

1. WHY EFFECTS ==========

Today every lambda looks the same: (lambda (x) (* x x)) ; pure (lambda () (get 'note id)) ; read DB (lambda () (create 'note "{}")) ; write DB (lambda () (propose-function ...)); AI + write code (lambda () (render-html "...")) ; page-ish

The OS cares which is which. Effects make that first-class, inspectable, enforceable.

==========

2. EFFECT LATTICE (capability sets) ==========

Base labels (finite, small):

Pure no capability prims; only pure computation ReadDb get, rows, select-for-update, list-branches, ... WriteDb create, update, update-field, delete, deftype, ... Branch fork, merge, checkout Publish route (and later define-page helpers) Ai propose-function / LLM client Html render-html (soft; display intent, not security) Txn body expects with-transaction scope Meta define-persist side channel? (see note)

A function's effect set is a SET of labels, e.g. {ReadDb, Html} {WriteDb, Ai} {}

Ordering for policy (allowlist checks): Pure ? anything ReadDb ? ReadDb?WriteDb (writes imply reads usually) optional sugar: Db = ReadDb?WriteDb

NOTE on (define ...): Top-level define persists code. That is a Meta/code-store write performed by the REPL/session layer, not always an effect inside

the defined body. Distinguish: - effects OF a function body (what it does when called) - effects OF defining it (session install + persist)

==========

3. PRIM EFFECT TABLE (initial map) ==========

Pure: + - * = < > <= >= not list car cdr cons null? pair? count string-append str type-of dict-get dict-set dict-keys hx-get hx-post ; string tags only

ReadDb: get rows select-for-update list-branches current-branch list-bindings whoami current-author ; session read; treat ReadDb or soft Ambient

WriteDb: create update update-field delete deftype

Branch: fork merge checkout

Publish: route

Html: render-html

Ai: propose-function (Ai usually pairs with WriteDb on code store via define path)

Txn-related: with-transaction (special form) introduces Txn scope select-for-update meaningful only under Txn

Image layer owns the table (primitives live in image/). Lisp package may define the effect datatype only.

==========

4. INFERENCE (syntactic, conservative) ==========

effects(expr) = union over subexpressions + effects of any prim head used in App + effects of known global bindings referenced (lookup function_defs metadata / env annotation)

Rules of thumb: - overapproximate: if unsure, add effect - higher-order unknown fn => Effects.top or {ReadDb,WriteDb,Ai,Publish,Branch} policy knob - lambda body effects = function effects - quote bodies contribute no runtime effects

On successful (define name body): store inferred effect set next to function (new column function_defs.effects jsonb OR side table OR derive on the fly from body AST – prefer derive-on-read for v1 to avoid migration, cache later)

v1 implementation preference: derive-on-read from AST + prim table (no schema change) optional cache field later

==========

5. GATES / POLICY (where effects pay rent) ==========

a) UI badges /bindings and editor show: pure | db-read | db-write | ai | page

b) AI propose policy (default) allowed = {Pure, ReadDb, Html} denied = {WriteDb, Branch, Publish, Ai} unless user elevates (tune experimentally; document in product-experience)

c) Route publish policy (optional v1) (route path fn) warns or rejects if fn has Branch/Ai pages should be ReadDb/Html/Pure by default

d) Sandbox eval (future) browser anonymous do-it: Pure only

e) Transactions if body has select-for-update or multi WriteDb, linter suggests with-transaction

==========

6. ASYNC STORY (what is already true) ==========

Solved: - eval returns value Lwt.t - prim_fn = value list -> value Lwt.t - composition of DB reads in Lisp works - with-transaction threads dynamic connection

Not the same as effects: Lwt answers "when / how concurrency is scheduled" Effects answer "what capabilities are required"

Example: (get 'note id) is BOTH async (returns promise through eval) AND ReadDb (capability)

You already need both; only async exists today.

==========

7. RELATION TO OCAML 5 EFFECTS ==========

OCaml 5 algebraic effects MAY later implement host-side intercept for DB/AI inside the interpreter (handlers). That is a runtime rewrite of image/lisp internals.

Explicit non-goals for first effects delivery: - do not replace Lwt in Mirage/httpaf/pgx stack - do not expose Effect.perform to Lisp users - do not require monadic syntax in Lisp

Guest programs keep looking like normal Lisp. Effects are metadata + checks, not new call syntax (unless we add optional declare forms later).

Optional sugar later: (define-pure name body) (define-page path body) Desugar to define + check inferred ? declared.

==========

8. IMPLEMENTATION PHASES ==========

Phase 0 (docs/UI copy): explain Pure vs DB vs AI in /about Phase 1: Effect module + prim -> effect set table Phase 2: infer_effects : expr -> EffectSet.t Phase 3: badges on /bindings + /edit + inspect Phase 4: AI propose allowlist gate (configurable) Phase 5: route publish warnings Phase 6 (later): session capability masks, setuid authority effect Phase 7 (optional research): OCaml effect handlers under eval

==========

9. DATA SHAPES (sketch) ==========

type effect = | Pure | ReadDb | WriteDb | Branch | Publish | Ai | Html | Txn

type effect_set = effect list (* unique sorted *)

val prim_effects : string -> effect_set val infer : prim_table_meta -> expr -> effect_set val subset : effect_set -> effect_set -> bool val pp : effect_set -> string (* "pure" / "read-db+html" *)

No change to value or eval types required for Phases 1-5.

==========

10. SUCCESS CRITERIA ==========

- User can see whether a function is pure or touches DB/AI - AI-proposed functions can be policy-gated by effects - Pure functions remain testable without Postgres - Async composition remains intact (no regression of Lwt ABI) - Spec/docs stop conflating async with effects

11 migrations.borg

pricklypear-migrations

Pricklypear book chapter: migrations
[status: partial]

migrations

Schema evolution: supervisor-applied, additive-only, schema authority in main

> – *agent note:* plan-pass #4 (it20): migrations shipped. supervisor/lib/store.ml's migration runner (Run_pending_migrations + split_sql_statements) applies 001_branches.sql + 002_auth.sql + 003_data.sql idempotently tracked via schema_migrations table, blocking image boot until applied (verified it4 supervisor smoke test + it5 auth smoke test + it6 e2e). Spec-consistent: 11 tables across 3 migrations, all additive. The type_defs-merged-to-main -> emitted-DDL migration (the second ordering paragraph in applied-by-supervisor's doc text) is NOT yet built – v1 uses a single generic data_rows table for all user-declared types (migration 003), sidestepping the per-type-DDL emission machinery. That sub-feature is what keeps this section at 'partial' rather than 'implemented'. See BORGE-1783433175 for the comprehensive plan-pass rationale.

[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Additive SQL migrations 001-005 real. DDL emission from type_defs still deferred.

applied-by-supervisor

Supervisor owns schema state before any image boots
[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Single-image: scripts/dev.sh (and deploy hooks) apply 001-005 SQL, not supervisor. type_defs?DDL emission still absent; keep partial.

> – *agent note:* it20: IMPLEMENTED – supervisor's Run_pending_migrations in supervisor/lib/store.ml creates schema_migrations if missing, reads applied versions, applies pending .sql files (001, 002, 003 declared statically) in order. Each migration is split via the hand-rolled split_sql_statements (handles single-quoted strings, dollar-quoted $$ blocks, – comments – see BORGE-1783527733). Idempotent: re-applying is a no-op (no duplicate schema_migrations rows). Verified end-to-end by /tmp/pp_e2e_it6.py through scripts/dev.sh. The NOT-yet-built half: migrations EMITTED FROM type_defs merged to main – v1 uses a single shared data_rows table (migration 003) instead of per-type-DDL emission, so the type-declaration -> DDL migration machinery doesn't exist.

The SUPERVISOR, not the image, runs migrations. It owns schema state because it owns image lifecycle. Images assume "the schema is current".

Migration source: SQL files in migrations/ (version-numbered), PLUS migrations emitted from type_defs that have been merged to main (the type-declaration -> DDL emission).

Initial migrations: 001_branches.sql – code store tables (branches, function_defs, type_defs, view_bindings, sessions, schema_migrations) 002_auth.sql

– auth tables (users, credentials, sessions [NAMED auth_sessions to avoid clash], auth_log, system_keys) + CREATE ROLE pricklypear_auth with SELECT-only on its own tables; system_keys GRANTed ONLY to that role. The supervisor runs the migration but never connects as pricklypear_auth; only the auth image does. 003_data.sql – shared data_rows 004_bible_ref_index.sql – bible lookup indexes 005_ai_sessions.sql – agent conversation store (ai_sessions, ai_messages; NOT fleet sessions and NOT auth_sessions). Spec: ai-authoring.agent-portal. 006_route_visibility.sql – view_bindings.visibility text NOT NULL DEFAULT 'private' (public|private). Spec: auth.route-visibility. Additive column only; existing routes stay private.

Applied idempotently: tracked in schema_migrations table. On supervisor boot: 1. Read applied versions from schema_migrations. 2. Apply any pending migrations in order. 3. Only then: start serving image-boot requests.

A migration in flight blocks new image boots until it completes (or fails). This guarantees images always see a consistent schema.

Migration failure: supervisor refuses to boot images, alerts admin, requires manual intervention. No half-applied state. (Additive-only policy makes failure recovery simpler.)

additive-only

Schema changes can ADD, not remove or rename

[status: implemented]

> – *agent note:* it20: IMPLEMENTED by policy – all 3 migrations (001_branches.sql, 002_auth.sql, 003_data.sql) are strictly additive: CREATE TABLE statements only, no DROP COLUMN, no RENAME, no ALTER COLUMN TYPE except column additions with sensible defaults. Migration 002 added DEFAULT gen_random_uuid() on all id columns during the it5 fix. No non-additive migrations exist in migrations/. The runtime type_defs -> data_rows story (v1: a shared generic table holds all user-declared types as JSONB) is itself additive – new types add rows, not DDL. (See BORGE-1783432745.)

Schema evolution policy for type_defs in main: ADDITIVE ONLY. You can: - Add a new type (new table) - Add a new field to an existing type (ALTER TABLE ADD COLUMN, with NOT NULL forbidden unless a DEFAULT is specified)

You CANNOT: - Remove a field (no DROP COLUMN) - Rename a field (no RENAME) - Change a field's type (no ALTER COLUMN TYPE except widening)

WHY: - Additive changes are backward-compatible with running images. An image running old code that doesn't know about a new field still works; it just ignores the column. - Removing/renaming breaks running images that reference the field, with no compile-time error to catch it (Lisp is dynamic). - The cost is cruft accumulation: dead fields pile up over time. Tradeoff accepted for correctness.

When a field truly must go: a manual migration (off-cycle, supervisor downtime, admin-run) that also updates affected branch code. This is rare and treated as an incident, not routine.

This policy is set early because retrofitting it later (after non-additive migrations exist) is expensive. It is a foundation constraint.

branching-interaction

How migrations interact with branches

[status: partial]

> – *agent note:* it31 plan-pass #7: corrected factual error in it20 note. IMPLEMENTED – migrations only touch shared data tables (data_rows has no branch_id; shared across branches per branching.data). A branch can propose a type change via (deftype ...); recursive-CTE lookup makes ancestor type_defs visible. schema_vers DOES exist on type_defs in migrations/001_branches.sql (integer NOT NULL DEFAULT 1, plus migrated boolean) – earlier notes wrongly claimed the column was missing. NOT IMPLEMENTED – runtime validation against schema_vers / main field set ("field X not in current main schema"), cache invalidation on schema change, and type_defs-merge-emitted DDL. v1 data_rows still accepts any jsonb fields bag for any type_name. schema_vers column is schema-ready; enforcement is the remaining partial.

Migrations only apply to the SHARED data tables (event, user, etc.), governed by main's type_defs (which are merge-governed into main).

Branches CANNOT migrate data tables. A branch can PROPOSE a type change in its type_defs, but the underlying table doesn't change until that proposal is merged to main and the supervisor runs the resulting migration.

A branch running code that references a field added in main but not yet handled by the branch's own code: works (the column exists; branch code ignores it on write, reads null on read if no default).

A branch running code that references a field that DOESN'T exist (because the branch's type_defs is behind main's): runtime error at the primitive layer. The user gets a clear message: "field X not in current main schema; merge your schema change or sync to main".

Schema version is tracked per type (schema_vers in type_defs). Branches see the version they declared; main sees its current version. The primitive layer can validate against either at query time.

12 multi-user.borg

pricklypear-multi-user

Pricklypear book chapter: multi-user – MANY HUMANS. Process-per-user isolation (this chapter) + auth identity (auth.borg multi-user-product) are the complete multi-user system. single-image.borg is about shipping ONE binary, not one human. Do not read single-image as cancelling multi-user.

[status: partial]

> – *agent note:* 2026-07-12: PRODUCT CORRECTION. Multi-user is not deprecated. it35 single-image pivot retired the pain of coordinating three always-on daemons for deploy; it did not cancel multi-user product. Isolation-by- process remains the isolation model. Auth multi-user product is specified in auth.multi-user-product.

multi-user

Per-user image instances, namespaces via branch ownership, budgets, author-runs-as-author

[status: partial]

> – *agent note:* it31 plan-pass #7: section overall partial. isolation = implemented; namespaces = partial (no cross-branch qualified calls); budgets = partial (step budget SHIPPED in eval, memory/raise-budget deferred – earlier notes wrongly said planned/ missing); permissions = partial (author column + admin merge yes, runtime setuid + row scoping no). User-facing multi-user model is in place at process/branch level; finer permission semantics remain.

isolation

Process-level isolation: one image per user

[status: implemented]

> – *agent note:* Implementation it6 / verified it6 onward: the supervisor's 'Fleet.boot' (supervisor/lib/fleet.ml) spawns a per-user image (unix process) on SSH connect and sets PP_USER_ID/USERNAME/BRANCH_ID/BRANCH_NAME env vars so each image boots as the verified user. Auth is the separate authd unikernel (auth/) that verifies the SSH key via the auth RPC; the supervisor never sees raw credentials, just gets back the user_id or denial. The image trusts the supervisor's identity claim via PP_USER_ID (the internal network is the trust boundary). End-to-end verified by /tmp/pp_e2e_it6.py: admin's SSH key ? auth verifies ? supervisor spawns image inst-001 ? REPL proxy ? (define square (lambda (x) (* x x))) ? (square 7) -> 49. An infinite loop OR bad function in alice's image takes only alice's image offline; the supervisor's other connections (admin, other users) are unaffected (each is its own Lwt.async per supervisor/lib/ssh.ml's per-connection dispatch).

Multi-user is achieved by giving each user their OWN image unikernel, not by namespacing within one shared image. This is the Unix/POSIX model: - Each user has a "home" (their branch bindings) - Logging in = booting your image - Each image is a separate process

Wins: - Alice's infinite loop wedges Alice's image only - Alice's bad function definition takes only her image offline; she recovers by rolling back to her last-good branch - The interpreter doesn't need cooperative scheduling for cross-user safety (only for in-session budgeting) - Per-user A/B rollback (finer than whole-image)

Costs: - N images always running if "always-on" pinned (RAM committed) - Boot-on-connect default mitigates this: idle users' images aren't running

Auth: a SEPARATE auth unikernel (see section auth) verifies credentials and resolves user identities. The supervisor never sees raw credentials – it passes them to auth and gets back a user_id (or denial). The image trusts the supervisor's identity claim because: the supervisor asserted it AND the supervisor got it from auth, AND image/supervisor are on the internal network. The image itself does no credential verification.

namespaces

Branch ownership namespaces; packages are a separate in-branch axis
[status: partial]

> – *agent note:* Branch half implemented: no privileged 'common'; key (branch_id, name); fork/override/merge verified. 2026-07-11: clarify ORTHOGONAL image.packages (prefix packages like todo/list) – not this subsection's job. Cross-branch qualified calls still deferred.

TWO AXES (do not conflate):

1. BRANCH namespace (this subsection): There is no privileged 'common' namespace. There is 'main' (system baseline) and per-user branches. Versioning/ownership IS branch ownership. A binding's key is (branch_id, name). When Alice forks main to alice/main, she gets her own 'greet' independent of main's greet (until she merges).

2. PACKAGE prefix (image.packages): Within ONE branch, organize functions with '/' in the name (todo/list). That is organization, not ownership. See image.packages.

Cross-branch calls (still deferred): a function can explicitly reference another branch's binding via a qualified name (e.g. system/main:greet – note colon or other separator TBD, must not collide with package '/'). Normally a function calls bindings in its own branch.

The 'common namespace problem' is thus reframed as the 'merge to main politics problem': - Default: admin-only merges to main - All users can fork freely - Users can merge into their own branches freely - Politics expand later (PR/review workflow)

This sidesteps the up-or-down political decision the original 'publish to common' model forced.

budgets

Per-call step budgets to bound runaway user code
[status: partial]

> – *agent note:* it31 plan-pass #7: corrected stale it23 note.
>
> SHIPPED: (1) Process isolation – each user gets their own image process via Fleet.boot; a wedged image only takes that user offline. (2) Per-call evaluator step budget – lisp/lib/eval.ml ticks a budget at every recursive eval step; default_limit 100_000; raises Budget_exceeded for that call. Earlier notes claiming "step budget NOT shipped" were wrong after the budget code landed in eval.
>
> STILL DEFERRED (keeps section partial): - Memory-budget tracking per session/call - Explicit primitive to raise budget for a legitimate long sub-call, with audit logging - Configurable per-session default budget from manifest/config - Supervisor-side kill/restart automation when an image is wedged despite budgets (health auto-restart loop)
>
> Step budget + process isolation are the two load-bearing layers for runaway-code defense today.

Each Lisp primitive invocation runs with a step counter. When exceeded, that ONE call aborts; the session continues. This prevents "one user's bad function wedges the whole image" at the interpreter level (defense in depth alongside process-level isolation).

Default budget: configurable per session, sensible initial value (e.g. 100k eval steps). User can request more for legitimate long-running operations via a primitive that explicitly raises the budget for a sub-call (and logs it).

The interpreter checks the counter at every Prim invocation and every recursive eval depth. Cheap: one integer increment per step.

Memory: also tracked. A session exceeding memory budget aborts the offending call. (Deferred; step budget is the v1 mechanism.)

This is per-CALL, not per-session: a session can make many calls, each independently bounded.

permissions

Author-runs-as-author: functions execute with their author's data perms
[status: partial]

> – *agent note:* Partially implemented as of it19 plan-pass #3.
>
> IMPLEMENTED (the OCaml-side governance half): - Admin-only merges to system branches IS enforced (it10; see branching.merge) – this is half of the "admin owns main / merges require admin approval" permissions story. - function_defs and type_defs carry an ‘author‘ text column, populated from manifest.username at (define ...) / (deftype ...) / (route ...) time. Store.lookup_function + list_function_names return author alongside the body; it's surfaced in the /binding/<name> system page UI. - ‘(current-author)‘ prim is registered (it7) returning the session user – the FIRST HALF of the spec's author-runs-as- author pattern (reading the author for visibility).
>
> NOT YET IMPLEMENTED (the runtime-permission half): - ‘(current-author)‘ is the SESSION user, NOT "the executed binding's author" – the interpreter does NOT thread the AUTHOR of the currently-executing binding through eval. So a function authored by alice and called by bob runs as bob (the session user), NOT as alice. This is a meaningful security gap from the spec's setuid model: alice-authoring a function that reads her calendar and bob calling it would read bob's calendar today (or fail), not alice's. - Data prims (get/create/update/delete/rows) do NOT consult current-author for permission filtering – they operate on the shared ‘data_rows‘ table without per-author row scoping. In v1 every image sees every row in its data_rows-table shape; isolation depends on per-user PG roles or per-user tables, which isn't wired yet (data_rows is schema-shared).
>
> Section marked ‘partial‘ until the runtime-author-threading + per-author row scoping lands. v1 ships the SECURITY MODEL halfway: governance policy (admin-only merges) yes; runtime setuid semantics no. Documented honestly so a perusing user knows what NOT to assume about a privileged function's blast radius today.

A function executes with ITS AUTHOR's data permissions, not the caller's. This is the Unix setuid model: if alice authors a function that reads her calendar, and bob calls it, the function reads alice's calendar (not bob's), and bob sees only what alice's function returns.

Implications: - The interpreter's eval carries a (current-author) field, read from the EXECUTED binding's author, not the session user. - Data primitives (query, get, create, etc.) consult current-author for permission filtering. - A

careless privileged function is a security hole, same as Unix setuid. Honest and familiar. - Functions can expose capabilities to other users by name without exposing raw rows.

Authoring permissions (separate concern): - A binding in alice/main is callable by anyone in the same image (read access implicit). - Only alice can edit bindings in alice/* branches. - main is owned by admin; merges require admin approval (default).

This model is the Unix model, by choice. It maps cleanly to "SSH-first, multi-user OS" framing. If we wanted a Lisp-machine feel (shared trusted environment), we'd pick differently, but SSH-first implies Unix-style multi-user.

13 notes.borg

pricklypear-notes

Pricklypear book chapter: notes – outline-first personal knowledge (org-roam / org-mode / Obsidian-shaped) as Lisp + data_rows

[status: planned]

> – *agent note:* 2026-07-11: New chapter. Notes are NOT a markdown vault on disk and NOT a separate service. Substrate = sexp outlines in data_rows; behavior = libs/outline + libs/notes; UI = html/* + htmx partials; agents use the vault via eval/pp_eval (image is the API). Priority locked with roerick: OUTLINES first (org structure), then roam links/backlinks/dailies/capture; graph view deferred. Orthogonal to dabar personal notes (study bookmarks) – this chapter is the general thinking tool / Zettelkasten.

notes

Outline-first notes OS: tree documents, wiki links, dailies, capture, hypermedia structure edit

[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Phase-1/2 product shipped (outline, storage, UI, capture, daily, unified todos). links-and-roam + phase-3 comfort still open – chapter not fully done.

==========
PRODUCT INTENT ==========
Build a personal knowledge environment on the live image inspired by: - org-mode: hierarchical OUTLINES as the document model - org-roam: stable ids/slugs, backlinks, dailies, capture - Obsidian: fast browse, wiki links, search (not FS religion)

NOT a port of Electron Obsidian or Emacs. NOT file-backed markdown under libs/ or the VPS disk. Notes are durable OBJECTS in Postgres (data_rows), edited live.

KILLER PROPERTY ON THIS OS: Every note/outline is a Lisp value. Agents and humans use the same functions via REPL / pp_eval / htmx. No notes REST API to invent – the image is the API.

==========
PRIORITY ORDER (locked) ==========
1. Outline tree (structure, fold, structure edit) 2. Wiki links [[slug]] + backlinks 3. Daily notes 4. Capture / inbox 5. Tags / search polish 6. Graph visualization (optional, late)

==========
TWO PACKAGES ==========
outline/ – pure tree ADT + print/parse/html + structure ops (no DB; testable offline) notes/ – persistence, routes, link index, dailies, capture (depends on outline/, html/*, find-row/find-text)

==========
OUT OF SCOPE (v1) ==========
- Full org-mode syntax (drawers, babel, agenda, clock) - Full CommonMark / markdown WYSIWYG - Force-directed graph as launch requirement - File vault sync (Obsidian folder) - Doltgres / DB engine swap for note history - Per-note function_defs (notes are data, not code) - Real-time multiplayer CRDT on one outline - Branching data_rows (notes stay shared vault until multi-user vault field is designed)

outline-model

Sexp outline ADT: node tree with stable ids – org structure as data
[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: libs/outline/lib.pp: node tree, find/update/delete/move, todo cycle, HTML render.

CANONICAL SHAPE (implementation may use lists/dicts):
A node is a record with: id – stable string (uuid or short id), required title – string, required body – string, optional prose (may contain [[slug]]) todo – "" | "TODO" | "DONE" | future keywords kids – list of child nodes (order matters)
Suggested Lisp construction helpers:
(outline/node title) (outline/node title body) (outline/node* props kids) ; props = alist or dict JSON
Tree = root node (the note's outline field stores this tree).
WHY SEXPS: Nested lists ARE outlines. No heading-level parser. pp_expr / pretty printers already exist for readability. Structure edit = pure functions on values.
PATHS: Prefer id-based addressing over index paths so reorder does not break ops: (outline/find tree id) -> node | nil (outline/update tree id f) -> tree (outline/insert-child tree parent-id new-node) -> tree (outline/delete tree id) -> tree (outline/promote tree id) / demote / move-up / move-down
FOLD STATE: v1: browser <details> open attribute only (ephemeral). Optional later: collapsed-ids set in note props or session.
PRINT/PARSE: Store as JSON in data_rows.fields (Yojson-friendly: nested objects/arrays), OR as a single sexp string field. Recommendation: JSON tree in fields.outline object for robust DB tooling; Lisp constructors convert to/from internal list representation.
PURE PACKAGE: libs/outline/lib.pp must not touch data_rows so it can be unit-tested / reused by other apps (e.g. project plans).

storage

data_rows types for notes and optional link index; shared vault
[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: note type via notes/*, upsert/get/list/search/delete on data_rows.

TYPE note (fields): title – display title slug – stable human id for URLs and [[slug]] (unique among notes; enforce via find-row + app logic; optional unique index migration later) outline – JSON tree (root node object) OR sexp string; see outline-model. Prefer JSON object. tags – string (comma-separated) or JSON array string kind – "note" | "daily" | "project" | future day – "YYYY-MM-DD" for dailies, else "" body – optional root-level prose; may be empty if all content lives in outline nodes
TYPE note-link (optional but recommended early): from_slug – source note from_id – source node id (or "" for whole-note) to_slug – target note to_id – target node id optional label – optional
On notes/save: 1. upsert note row by slug 2. parse [[wikilinks]] from all node bodies (+ title?) 3. replace note-link rows for from_slug
SHARED VAULT: data_rows are branch-shared (existing storage rule). v1 = one vault per Postgres. Multi-user isolation later via author field filter or vault id – not blocking v1.
NOT IN function_defs: Note content is not code. pp_sync does not pull notes. Language packages notes/* and outline/* ARE code and sync with pp_sync like todo/ui.
HISTORY: v1 overwrite on save (updated_at). Revision log is a future chapter; do not block on Doltgres.

STORAGE ALTERNATIVES (decision 2026-07-11): Phase 1 PRIMARY: whole outline as JSON (or list-encoded JSON) on the note row – document = tree (org buffer model). Lisp structure ops on in-memory lists; encode/decode at DB boundary (json-emit / json-parse prims).

NOT phase 1: PostgreSQL ltree. ltree is a solid contrib for path queries (a.b.c, ancestors/descendants) when every headline is its own row. It does not replace sexp trees for org-like structure editing of one note. Path rewrites on promote/demote/move-subtree are painful if ltree is sole truth.

Phase 2+ IF needed: note_node adjacency (parent_id, pos, note_slug) for huge notes / concurrent section edit; optional ltree OR closure table as a query index, not the only representation. Recursive CTEs (already used for branches) remain the default SQL hierarchy tool.

Do not install ltree only to start notes.

links-and-roam

Wiki links [[slug]], forward links, backlinks partial

[status: planned]

SYNTAX (v1 only): [[slug]] – link to note slug [[slug|label]] – optional display label (if easy) No #heading deep links required v1; optional #node-id later.

PARSE: notes/parse-links body -> list of slugs notes/reindex-links slug outline-tree

RENDER: notes/render-body body -> HTML string html-escape then substitute links notes/render-outline tree -> nested foldable HTML each title/summary + rendered body + children

BACKLINKS: (notes/backlinks slug) -> list of {from_slug, from_id, title} UI partial /partials/notes-backlinks?slug=

MISSING TARGETS: Link to unknown slug still renders as link; open page offers create-stub capture.

UNLINKED MENTIONS (phase 3): find-text on body for title string – optional.

dailies-and-capture

Daily outlines and quick capture (org-roam-dailies / org-capture shaped)

[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: notes/daily, notes/capture, /capture, inbox ensure, navbar capture.

DAILY: slug convention: daily-YYYY-MM-DD kind = "daily", day = YYYY-MM-DD (notes/daily) / (notes/daily "2026-07-11") get-or-create root outline with title "Daily YYYY-MM-DD" UI: /notes/daily , menu link under apps or notes

CAPTURE: (notes/capture title body) -> new slug (slugify title + short id), tags include inbox, kind note, outline = single node with body Optional: (notes/capture-to slug title body) appends a child node under root (inbox into a project outline)

INBOX REVIEW: /notes?tag=inbox list partial – phase 2.

hypermedia-ui

Routes, partials, structure-edit via htmx; menu entry

[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: /notes, /capture, outline partials, node-op POST, menu integration.

ROUTES (full page = chrome; /partials/ and HX-Request = fragment): GET /notes GET /notes?slug=NAME GET /notes/daily GET /notes/edit?slug=NAME ; optional dedicated editor POST /notes ; create / save / capture GET /partials/notes-list GET /partials/notes-outline?slug= GET /partials/notes-backlinks?slug= POST /partials/notes-op ; structure edit params: slug, op, id, ?

STRUCTURE OPS (POST notes-op): op = insert-child | delete | promote | demote | move-up | move-down | set-title | set-body | set-todo Always reindex links if body changes. Response: outline partial HTML (hx-swap target #outline)

RENDER: Prefer <details class="ol-node"> fold (no heavy JS tree). Ops as small buttons on each row (htmx).

MENU: Add (menu/link "notes" "/notes") under apps in default ui/menu when notes is dogfood-stable; or document manual ui/menu edit + pp_sync pull.

COMPOSITION: notes/page embeds partials like todo/page embeds todo/list-html – same hypermedia discipline.

agent-access

Agents use notes via eval – no separate notes HTTP API

[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: Agents use pp_eval /api/eval on notes/*; no separate notes REST. Dedicated agent tools optional later.

LAPTOP / PI: scripts/pp_eval.py '(notes/daily)' scripts/pp_eval.py '(notes/search "sync")' scripts/pp_eval.py '(notes/get "pp-design")'

IN-IMAGE AGENT: Optional phase 2 tools on agent-task: notes_search, notes_get, notes_capture Do not grant blind outline/delete without policy.

PRINCIPLE: Adding a notes feature = adding Lisp functions. Remote agents gain them automatically through eval.

phases

Implementation phases and acceptance

[status: partial]

> – *agent note:* 2026-07-12 status-reconcile: Phase 1 + most of phase 2 (structure ops, daily, capture, menu) shipped. Roam/backlinks + phase 3 comfort remain.

PHASE 1 – Outline core (ship first): - libs/outline/lib.pp pure tree + html - libs/notes/lib.pp deftype note, upsert/get/list - store outline JSON on note - /notes index + /notes?slug= foldable view - whole-outline save (textarea sexp or JSON editor OK) - load-library notes recipe

PHASE 2 – Roam + structure: - note-link reindex, backlinks partial - [[slug]] render - htmx structure ops (promote/demote/move/insert) - daily + capture - menu entry

PHASE 3 – Comfort: - tags filter, better search UI - light typography - unlinked mentions optional - agent-task tools

PHASE 4 – Optional: - graph page - node-level rows if single-blob hurts - revision history - per-user vault field

ACCEPTANCE FOR "v1 notes": Phase 1 + backlinks + daily + capture (phase 2 subset). Graph not required.

unified-todos

Basecamp lists + org node todos; unified /todos with smart views

[status: implemented]

– *agent note:* 2026-07-12 status-reconcile: libs/todo full lists+items+ARCHIVED; notes node todos scrape/cycle; /todos views. Promote dual-copy still later.

Real lists: todo-list + todo-item (list_id, title, notes, state, due, position). state in todo/states (Lisp). Note todos: outline node.todo; scrape + write-back. Smart views: everything, from-notes, default Inbox. /todos nav + body; cycle state; create list; add item. No dual-copy of note todos until promote (later).

14 object-model.borg

pricklypear-object-model

Pricklypear book chapter: object-model
[status: partial]

object-model

Typed records in Postgres, schema declared in Lisp, SQL-via-primitives, no ORM
[status: partial]

> – *agent note:* it23 plan-pass #5: object-model v1 foundation is built, the typed-field half is deferred. WHAT'S REAL: '(deftype 'name 'field1 'field2 ...)' prim (it14) inserts into type_defs (fields stored as JSON array of field-name strings – not typed). Data lives in the shared 'data_rows' table (one row per instance, no per-type PG table is emitted – see migration 003 + BORGE-1783433175). CRUD prims are implemented: (create 'type "{fields-json}") (it13), (get 'type id) (it13), (update 'type id "{fields-json}") (it13), (delete 'type id) (it13), (rows 'type [limit]) (it13), (select-for-update 'type id) (it17), (update-field 'type id "field" "value") (it18). The /types htmx page lists type_defs (it14). NO ORM anywhere – all SQL is hand-written in image/lib/store.ml + image/lib/crud.ml. WHAT'S DEFERRED: typed field declarations (:string/:int/:bool/:ref user/:array (ref user)/ :timestamptz/:json) – type_defs stores field NAME lists only, no field TYPE. Runtime field-level validation against main's schema is unbuilt (v1 data_rows accepts any jsonb fields bag for any type_name). The 'system creates or ALTERs underlying Postgres table on declaration/merge' machinery (the type-merge-governance flow) is NOT built (v1 uses the single shared data_rows table to sidestep the emitted-DDL complexity). '(query 'event where: ...)' parameterized SQL suger is not built (v1 has (rows 'type [limit]) instead – returns all rows of a type, no WHERE). ':ref user' first-class values with type tags (lazy or eager deref) are not built – v1 refs are just plain strings/UUIDs. Marked partial: the 'objects are typed records + no methods + CRUD prims + no ORM' shape is implemented; the typed-field + query-sugar + schema-validation are v1.5/v2 work.

An "object" is a typed record with an id, a set of named typed fields, stored in Postgres. There are NO methods on objects. Methods are functions in the store that take an object id as argument. This is functions + data, not OOP, matching the poohstack shape.

Types are DECLARED IN LISP, not in SQL migrations:

(deftype event (fields (title :string) (start :timestamptz) (end :timestamptz) (owner :ref user) (attendees :array (ref user))))

The declaration lives in function_defs-equivalent table (type_defs, branched like code). On declaration/merge to main, the system creates or ALTERs the underlying Postgres table.

The type is itself inspectable data: the htmx UI can render "list all types, show fields, browse instances" nearly free. This is the Smalltalk-image browser feeling, achieved via a metamodel rather than via live mutable objects.

CRUD primitives (real OCaml, typed at boundary): (create 'event {...}) -> row insert (get 'event id) -> row fetch (update 'event id {...}) -> row update (delete 'event id) -> row delete (query 'event where: ...) -> parameterized SQL (select-for-update ...) -> explicit row lock

References are first-class values: a :ref user field holds an id + type tag. Dereferencing is explicit: (get 'user (event-owner ev)).

The Lisp still has plain values (int, string, bool, lambda, list, ref) for logic that doesn't touch the DB. Pure functions for "does this event conflict?" take event values, return bool, no DB. The DB and the Lisp have a clean seam.

NO ORM. SQL is the query language. (query ...) desugars to parameterized SQL; the primitive layer parses results into typed Lisp values.

type-system

Field types and reference semantics

[status: planned]

> – *agent note:* it23 plan-pass #5: type-system is planned (v1 has NO typed-field implementation). type_defs stores field NAME lists only (e.g. 'name email age' as a JSON array of strings) – no field TYPE declarations. The 7 field types in the spec text (:string, :int, :bool, :timestamptz, :ref <type>, :array <type>, :json) are described-as-design but have no runtime enforcement: any jsonb fields bag is accepted for any type_name by v1 data_rows. v1.5 plan: extend (deftype ...) syntax to take field-type pairs, persist field types in type_defs.fields, and run runtime field-level validation on create/update. v2 plan: ':ref user' as a first-class tagged value (eager deref first; lazy stub-ref later if perf requires). No inheritance, no subtyping is preserved as a principle via storage.no-inheritance (which IS implemented – recursive-CTE override walk). Marked planned because no typed-field code exists as of it23

Field types (initial cut, grow as needed): :string text :int int8 :bool bool :timestamptz timestamptz :ref <type> uuid FK to <type>_table.id :array <type> <type>[] (homogeneous) :json jsonb (schemaless escape hatch)

A (ref user) value is a tagged id: the value carries its target type so the interpreter can verify derefs at call time. Dereferencing returns the full record or a stub-ref that lazy-loads on field access (decision deferred; start eager).

Cycles in the type graph are allowed (user has :ref calendar; calendar has :ref user) because refs hold ids, not nested values.

No inheritance, no subtyping. Composition via references, not via type hierarchies. This matches the poohstack (relational, no ORM, no inheritance) and avoids the Postgres INHERITS trap (see storage.no-inheritance).

15 philosophy.borg

pricklypear-philosophy

Pricklypear book chapter: philosophy
[status: partial]

philosophy

Design principles
[status: implemented]

> – *agent note:* it23 plan-pass #5: all 10 design principles are embodied by the built system. Mirage-native: all three unikernels (authd, supervisor, image) boot as mirage/unix processes; Solo5 target is a build switch not a rewrite (deferred per BORGE-1783398327). Postgres-as-filesystem: all durable state is rows (function_defs, type_defs, view_bindings, sessions, data_rows, auth tables, system_keys). Lisp-1 homoiconic: reader produces App(Var,...), AST JSON- serialized persist_function round-trips. Two-tier change model: define persists to PG (it8); OCaml changes via rebuild. 2026-07-12: product shell is Lisp (ui/nav ui/home); OCaml emergency only – see image.system-browser. Branches-not-namespaces: fork_branch + recursive-CTE override walk (it9) with no privileged 'common' branch. Separate- processes isolation: Fleet.boot spawns per-user image processes (it4). Author-runs-as-author: manifest.author threaded through persist + current-author prim (it7). SERIALIZABLE-by-default: Store.with_transaction uses Pgx.Isolation.Serializable as the default (it16). Additive-only: all 3 migrations strictly CREATE-only (no DROP/RENAME/ALTER-COLUMN-TYPE). Inspectability: image UI pages / + /branches + /bindings + /binding/<name> + /types (it11, it14); supervisor admin / + /fleet + /health. Philosophy is a statement-of-intent section; the spec text stays as documentation (no implementation diff needed).

1. Mirage-native. The OS is a unikernel. No Unix userland in the hot path. The supervisor IS a unikernel, not a daemon; it manages other unikernels by requesting VMs from the hypervisor, not by posix_spawn.

2. Postgres is the filesystem. All durable state lives there. The image is ephemeral and rebuildable. Anything that must survive a reboot is a row; anything in the image is rebuildable from rows.

3. Lisp-1 interpreter, homoiconic. The AST IS the source. Functions stored as JSONB. Editing the AST IS editing the code. Closures, let, lambda, application are first-class. Macros inherit for free later.

4. Two-tier change model. OCaml layer (primitives, transports, the unikernel itself) changes slowly via supervised A/B boot. Lisp layer (user functions AND product environment UI – nav, home, app menus, user routes) changes fast via row updates. Schema layer is merge-governed into 'main'. OCaml must not ship a silent second product skin; host/gate/emergency only. See image.system-browser shell rule.

5. Branches, not namespaces. There is no privileged 'common' namespace. There is 'main' (the system baseline) and per-user branches. Code branches, shared data. Forking is cheap; merging is the politically-sensitive operation.

6. Separate processes for isolation. Multi-user is achieved by giving each user their own image unikernel, not by namespacing within one image. Alice's bad function wedges Alice's image, not Bob's.

7. Author-runs-as-author. A function executes with ITS AUTHOR's data permissions, not the caller's. This is the Unix setuid model and it is honest and familiar.

8. SERIALIZABLE by default. The Lisp's query/mutate primitives default to SERIALIZABLE transactions. Weaker isolation is opt-in. Most user code never touches this; the system does the safe thing.

9. Additive-only schema evolution. Types in 'main' can grow fields but not remove or rename them. Cruft accumulates; correctness is preserved. This sidesteps most migration pain at small ongoing cost.

10. Inspectability is non-negotiable. Everything (functions, types, branches, sessions, the supervisor's fleet, the interpreter's state) is queryable and renderable in the htmx UI. The storage layer must be the most inspectable part of the system, not the least.

16 product-experience.borg

pricklypear-product-experience

Pricklypear book chapter: product-experience
[status: partial]

product-experience

Human-facing product layer: concepts, access paths, legible vocabulary, first-run, horizon
[status: partial]

> – *agent note:* it32: PARTIAL overall. concepts-guide, vocabulary, first-run-recipe implemented; access-paths: browser login now in-image (auth.web-login path); horizon planned design-only. 2026-07-12: product shell layering – environment UI is Lisp; OCaml is host/gate/emergency only (see image.system-browser). 2026-07-12: product theme Path B planned (image.product-theme, BORGE-1783837363) – presentation is Lisp css/* sheets, not a hard-coded OCaml CSS twin.

Pricklypear is a multi-user live computing environment: log into your image, write Lisp functions stored as data, branch code while sharing data, bind functions to URLs, let AI propose functions onto a branch.

Product experience is the thin skin that makes the dense substrate legible. Without it, only authors understand the system.

LAYERING (aligned with image.system-browser + image.product-theme, 2026-07-12): - Environment UI humans live in (nav, home, app menus, theme/presentation, user routes) is Lisp on branches (libs/*.pp seed + personal pins). That is the product. - OCaml hosts Lisp: eval, store, auth gate, health, kernel system pages (/bindings, /edit, ?), and a labeled emergency shell if ui/nav, ui/home, or ui/css fails. - OCaml must not ship a silent second product nav/home or a silent full design-system CSS twin in http.ml. Dual silent defaults are rejected architecture. - Theme is Path B (honest Lisp): css/* sheets + tokens, not CSS-string-in-lambda as the native form.

concepts-guide

In-app about page: glossary, access, first tour, horizon summary
[status: implemented]

> – *agent note:* it32: IMPLEMENTED. GET /about + home link. Markdown docs/concepts-and-horizon.md removed; .borg is source of truth.

GET /about (kernel OCaml page today; may move to Lisp later) MUST explain:

WHAT: multi-user live Lisp environment; image=CPU, Postgres=disk; Smalltalk tradition without OOP image dump.

GLOSSARY: - Image: running process (REPL + HTTP); single-image v1 - Store: Postgres durable truth - Function/binding: named live value; define persists - Branch: versioned code world; data shared - Type/row: deftype + data_rows CRUD - Route/page: URL bound to a function - Shell (ui/menu, ui/nav, ui/home, ui/theme, ui/css): Lisp product chrome + presentation - Theme tokens (theme/bg, ?): branch-visible style knobs - AI propose: agent authors on current branch only

ACCESS: - scripts/dev.sh start (PG + image) - Browser: /login (app session) then / /repl /edit /bindings - scripts/pp_eval.py or Basic auth for agents

- scripts/repl.sh / line-protocol REPL when enabled - nginx basic-auth may sit at edge; not the user DB

FIRST TOUR: log in, define hi-page, route /hello, open page, edit ui/menu for navbar, optional propose-function.

Link from home nav and home body. Full horizon detail in product-experience.horizon.

access-paths

Documented ways humans reach a live image

[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: Browser login, admin users, HTTP basic /api/eval, dev.sh PG+image, verify_auth path. Fleet/default real SSH deferred.

> – *agent note:* 2026-07-12: browser password login + session cookie ship in-image (auth.web-login / multi-user-product Option B). Still partial: edge nginx basic-auth, REPL path, fleet attach.

v1 SHIPPED paths (single-image): 1. Browser: GET/POST /login ? cookie pp_session ? land on username/personal; POST /logout revokes 2. Admin: /admin/users (create user, revoke sessions) 3. Agents: HTTP Basic or session cookie ? /api/eval (scripts/pp_eval.py) 4. scripts/dev.sh start – PG + image 5. scripts/verify_auth.sh product gate 6. Optional scripts/repl.sh / line-protocol REPL

Product chrome after login is Lisp (ui/nav, ui/home, ui/theme/ui/css). Auth gate and emergency shell/CSS are OCaml kernel only.

DEFERRED: - Fleet boot/attach per user (process isolation path) - Default real SSH (awa 0.5.x path) - Drop nginx htpasswd as primary human gate (break-glass only)

vocabulary

Friendly UI labels over systems jargon

[status: implemented]

> – *agent note:* it32: IMPLEMENTED. Nav/home/pages use Functions, Pages, Data, Workspace. Lisp prim names unchanged. Smoke checks home vocabulary.

UI vocabulary (human): Functions <- function_defs / bindings Pages <- view_bindings / routes Data <- type_defs + data_rows Workspace <- current branch + whoami About <- concepts guide

HARD RULE: Page ? Function. Page = URL endpoint (/hello). Function = named code (hi-page). Edit targets are always function names (/edit/hi-page). Page inspect is /page/hello (route + call graph). Never nav-link /edit/<route-basename> as if it were a function.

PACKAGE vs BRANCH: Package = in-branch prefix (todo/) for organizing functions. Branch = versioned workspace (system/main). UI labels: "package" for prefixes, "branch"/"workspace" for version worlds – never swap the words. Functions browser groups by package; branch chrome stays whoami + current branch.

Lisp/REPL vocabulary stays technical (define, route, deftype, rows). The browser skin translates.

first-run-recipe

One-click or one-form 'make a hello page' onboarding

[status: implemented]

– *agent note:* it32: IMPLEMENTED. POST /api/recipe/hello + home button create hi-page + /hello route. Smoke: GET /hello works.

Recipe "hello page": 1. Define function hi-page -> render-html greeting 2. (route "/hello" "hi-page") 3. Return link to /hello and /edit/hi-page
Exposed as: - Button/form on home - POST /api/recipe/hello
Non-goals: full app wizard (that's AI app-sketch later).

public-release

Hosted public face on the image: layout components, welcome, code browser, book PDF, source tarball
[status: implemented]

– *agent note:* 2026-07-12: Public release ships ON the image (not brochure static site). layout/* = reusable page blocks (NOT named public/* – public is auth visibility only). Code browser is kernel read-only FS + CodeMirror 5 mllike. PDF via borge book print at deploy; tarball same allowlist as /code.

– *agent note:* 2026-07-12 IMPLEMENTED+smoke: dune build image OK; unauth GET /welcome 200 layout-hero; /code 200 CM5 tree; /code/raw/pricklypear.borg 200; /book.pdf 200 (~1.1M); /src.tgz 200 (~0.9M); path-as-is ../ escape 403; /bindings 303?login; / 303?/welcome. scripts/public_artifacts.sh writes book.pdf+src.tgz. Seed layout+welcome on system/main.

========== GOAL ==========
Release pricklypear on the hosted instance itself – not GitHub-as-homepage, not a sister brochure site. The live image IS the public site:
/welcome – landing (Lisp layout/* + welcome/page) /code – read-only source browser (kernel + CM5) /book – book explainer (Lisp) /book.pdf – borge book print artifact /src.tgz – pruned source tarball /login – enter the private workspace
Workspace home (/) stays HARD PRIVATE. Anon hitting / redirects to /welcome (or login with next=); logged-in users keep product home (ui/home).
========== LAYOUT COMPONENTS (libs/layout) ==========
Namespace layout/* – NOT public/*. "public" is reserved for route visibility / gate language.
Building blocks (pure HTML strings via html/*): layout/hero title tagline actions-html layout/card title body layout/cards list-of-card-html layout/section title body layout/cta href label layout/theme – css/sheet rules for .layout-*
Seeded on system/main; used by welcome and book pages. Personal gardens may override layout/* like any package.
========== WELCOME + BOOK PAGES (libs/welcome) ==========
welcome/page – compose layout/* into landing welcome/book – short PDF + tarball explainer (public-route "/welcome" "welcome/page") (public-route "/book" "welcome/book")
Also register hard-public kernel paths so the front door works even if seed/route table is empty (belt + suspenders): /welcome /book are hard-public and dispatch to Lisp when bound, else a minimal kernel fallback page.
CTAs: Explore source ? /code ; Download book ? /book.pdf ; Source tarball ? /src.tgz ; Log in ? /login.
========== CODE BROWSER (kernel, image/lib/http_code.ml) ==========

Read-only filesystem browser over PP_SOURCE_ROOT (default: repo root / PP_ROOT / cwd with libs+image+lisp).

Routes (all hard-public GET): /code – HTML shell: file tree + CodeMirror /code?f=path – open file in editor /code/raw/<path> – raw text (for fetch into CM) /assets/cm/* – vendored CodeMirror 5 assets

CodeMirror 5 (vendored, no npm at runtime): codemirror.min.js + codemirror.min.css mode/mllike (text/x-ocaml for .ml/.mli) mode/markdown for .md plain for .borg .pp .sql .sh readOnly: true, lineNumbers: true

SANDBOX: - realpath must stay under PP_SOURCE_ROOT - allowlist extensions: .borg .ml .mli .pp .md .sql .sh .ksh .dune (and dune files named "dune") - skip dirs: _build .git .pi .ralph node_modules _opam .opamswitch spikes (optional) supervisor (deprecated – skip) image/static/codemirror is ok - max file size ~2 MiB - GET only; no write/eval

Default open file: pricklypear.borg when present.

========== BOOK PDF + TARBALL ==========

Built at deploy/dev time, not per-request:

borge book print . -o pricklypear-book scripts/public_artifacts.sh ? PP_PUBLIC_DIR/book.pdf (or pricklypear-book.pdf) src.tgz (same allowlist as /code)

Image serves: GET /book.pdf application/pdf GET /src.tgz application/gzip from PP_PUBLIC_DIR (default /tmp/pp-public or beside source). Missing artifact ? 404 with short HTML hint, not crash.

========== AUTH GATE EXTENSIONS ==========

SYSTEM_HARD_PUBLIC gains: /welcome /book /book.pdf /src.tgz /code and /code/* /assets and /assets/*

Still HARD PRIVATE: / /bindings /edit /repl /data /api /admin /branches /types /pages /agent ?

Anon / ? redirect /welcome (product choice for public host). Logged-in / ? ui/home as today.

========== ENV ==========

PP_SOURCE_ROOT – tree for /code and tarball input PP_PUBLIC_DIR – book.pdf + src.tgz PP_LIBS_DIR – existing; layout+welcome seed from here

horizon

Long-term possible futures built on this substrate (design only)

[status: planned]

> – *agent note:* it32: design-only. Migrated from docs/concepts-and-horizon.md. Not an implementation checklist; guides prioritization and pitching.

Possible futures (not commitments):

A. Personal computing environment Always-on personal image; notes/CRM/automations as live functions; browser as desktop.

B. Multi-user community OS system/main commons; personal garden branches; merge reviews; author-runs-as-author capabilities.

C. AI-native software workshop AI cohabits image; proposes types+fns+routes as app sketches; binding-level diffs; provenance; later addressable AST patches.

D. Internal tools platform Team branches; shared business objects; live inspectable tools without redeploy for Lisp-tier changes.

E. Education / research lab Legible multi-user systems + PL teaching with forks/merges and visible isolation.

F. End-user programmable products Friendly skin on same substrate; templates as starter branches; AI drafts, users edit.

G. Hardened multi-tenant service Solo5, real SSH + web login, argon2, mTLS, fleet policies, true setuid data scoping.

H. Dabar – live Bible study libraries BSB reader + Strong's + interlinear as load-library packages (bible/ + dabar/) on the image; not a separate OS. Spec: dabar.borg (root-inlined chapter). Semantic search deferred.

Milestone ladder: now – hacker env + browser login + Lisp product shell (define/edit/route/AI; ui/nav ui/home; auth session) next – product theme Path B (css/* + ui/theme); kill silent OCaml product defaults (nav/home/CSS); emergency-only chrome+css; shell denser; branch promote UI later – environment feel (more system UI in Lisp, permissions, multi-user fleet isolation) horizon – community + AI coworker + Solo5 product

Branch UX + software updates: see branching.borg subsections personal-workspace (implemented), package-updates / promote / branch-browser-ui (planned). Defaults on system/main; garden on <user>/personal; promote is selective merge + git.

Principles to preserve: live system over file tree; store is truth; code branches / data shared; functions compose; AI is an author not god mode; inspectability; isolation by process.

17 single-image.borg

pricklypear-single-image

Book chapter: the single-image pivot. Supersedes the supervisor + per-user-image + authd topology described in architecture.borg, supervisor.borg, and multi-user.borg. One process. One binary. One deployment. Multi-user at the session/branch layer (not process isolation).
[status: implemented]

> – *agent note:* 2026-07-12 status-reconcile: single-image pivot IS the running system. supervisor/ and auth/ packages retired; image owns auth + HTTP + REPL; scripts/dev.sh applies migrations and starts PG+image only. Multi-user remains at the session/branch layer (auth.multi-user-product), not process isolation. Deploy docs still mention legacy rc.d names in places – runtime is one image binary.

> – *agent note:* it35 plan pass: this chapter is the architectural pivot requested by roerick on 2026-07-11. The MirageOS unikervisor + per-user spawn model is retired. The image becomes a plain OCaml 'unix' binary: one process serving htmx HTTP directly, talking to Postgres, reading its identity from env. No supervisor. No authd sidecar. No fleet. No Solo5. No multi-user process isolation. 2026-07-12: make pass completed – supervisor/auth packages retired; image is the sole binary. Multi-user remains via auth sessions + personal branches.

single-image

The pivot: one process, single-tenant, no Mirage abstraction
[status: implemented]

thesis

What the system IS after the pivot
[status: implemented]

Pricklypear is ONE OCaml 'unix' binary – the image – that: 1. Serves htmx HTTP directly on a port (httpaf-lwt-unix + pure-html). This is the only user-facing surface. 2. Talks to Postgres directly (pgx_lwt + Io_unix). Postgres is the durable object store; the image is ephemeral and rebuildable from it. 3. Reads its identity (branch, author) from process env at boot – no boot manifest, no supervisor injection. 4. Optionally exposes a line-protocol REPL on a second port for operator access. No SSH front door. The browser REPL (/repl) and HTTP eval (/api/eval) are the primary interactive surfaces.

What the system is NOT, after the pivot: - NOT a MirageOS unikernel. No Solo5 target. No mirage/unix target. No hypervisor VM-request spawn. 'dune build' + restart is how you change the image. - NOT multi-user at the process level. One tenant. The branch model (fork/merge) stays for experimentation, but there is no per-user image instance, no fleet, no per-user isolation. - NOT fronted by a supervisor. No authd sidecar. The image is the only Pricklypear process.

This is a deliberate simplification. The unikervisor model was sound in theory but every deployment pain in it1-it34 (EADDRINUSE on respawn, stale route tables, orphaned image PIDs, the awa-ssh identity block, manual spawn-vs-supervised-spawn confusion) traces back to having three coordinated processes. One process cannot have those failures.

removed

What is deleted from the codebase and spec

[status: implemented]

Packages deleted: - supervisor/ (fleet, health, process, ssh, ssh_awa, ssh_awa_shims, ssh_keys, auth_client, admin, boot, store) – ALL of it. - auth/ (verify, tokens, admin_api, store, http, boot, authd.ml) – ALL of it as a SEPARATE process. The users table stays; the verify logic (password/pubkey) either folds into the image as a thin prim or is replaced by nginx basic-auth at the edge.

Spec chapters demoted (status -> dropped, cross-link here as the successor): - supervisor.borg whole chapter: dropped - multi-user.borg isolation by process: dropped; namespaces/budgets/permissions stay as Lisp-level safeguards within the one image - architecture.borg topology + transport-choices + package-layout: rewritten by this chapter

Migrations trimmed: - 002_auth.sql: keep 'users'; drop 'credentials', 'auth_sessions', 'auth_log', 'system_keys' UNLESS the image does its own verify (then keep only what the folded verify reads). Additive-only – no destructive migration; orphaned tables are inert.

rc.d services collapsed: - pricklypear_supervisor, pricklypear_authd -> DELETE - pricklypear (single image) -> ADD One service. One binary. nginx fronts it.

scripts/dev.sh simplified: - start -> PG + image (two things, not three) - stop/restart/status -> the image only

kept

What survives the pivot unchanged in shape

[status: implemented]

Untouched: - lisp/ pure interpreter; no I/O dep - common/ manifest.ml, branch.ml (trim auth_types.ml, protocol.ml health) - image/lib/ store, primitives, session, http, crud, repl_endpoint, lineedit - libs/*.pp all Lisp libraries (std, ui, ai, todo, bible, dabar, notes, outline) - migrations/ 001_branches, 003_data, 004_bible (002_auth trimmed, not dropped) - scripts/seed_* bible/strongs/interlinear seeds

Refactored (not deleted): - image/lib/boot.ml reads PP_USERNAME/PP_BRANCH_NAME/PP_BRANCH_ID from env directly; drops the supervisor-manifest path - image/bin/unikernel.ml renamed image/bin/main.ml; plain 'unix' entry, no mirage - image/lib/dune '(executable ... public_name image)'; drop mirage/unix stanza - dune-project drop mirage/unix enabled flag

topology

The new physical layout

[status: implemented]

OpenBSD host (wyo.town / Vultr): +– Postgres (shared object store, SERIALIZABLE | defaults) | +– pricklypear image (ONE 'unix' binary) | +– httpaf-lwt-unix on PP_HTTP_PORT (htmx UI, | | /api/eval, /repl, /dabar, /todos, /notes) | +– optional line-protocol REPL on | | PP_REPL_PORT (operator only) | +– pgx_lwt to Postgres (functions, data_rows, | branches) | +– nginx (edge) +– TLS / basic-auth -> proxy_pass http://127.0.0.1:PP_HTTP_PORT

Flow for "user adds a function": 1. Open the browser (nginx front door). 2. /repl or /edit/<name>: define persists to function_defs, registered live. 3. New htmx route serves it immediately.

Flow for "operator changes a primitive": 1. Edit image/lib/primitives.ml. 2. 'dune build' + git push (post-receive rebuilds + promotes the release). 3. 'rcctl restart pricklypear' – one process, one restart. No fleet, no A/B, no respawn race.

Flow for "operator deploys": 1. git push wyo master. 2. post-receive: dune build -> promote to /usr/local/libexec/pricklypear/releases/<sha> -> 'current' symlink. 3. rcctl restart pricklypear.

auth

Identity without requiring authd as a second always-on binary
[status: implemented]

> – *agent note:* 2026-07-12: NOT single-tenant-only. Multi-user product is auth.multi-user-product. This subsection only constrains whether verify runs in-process or as a separate authd.

MULTI-USER is required (many humans, personal gardens). See auth.multi-user-product for the complete user stories.

This pivot only answers: where does verify run?

Option A – edge gate only (insufficient as product): nginx basic-auth / htpasswd. NOT the multi-user user database. Acceptable only as temporary break-glass.

Option B – in-image verify (fits one binary): Password/session verify in the image process against users + credentials + auth_sessions. /login cookie middleware. No separate authd required for web login. Still multi-user: many sessions, many personal branches.

Option C – authd process (fits fleet multi-user): Separate verify service as originally designed; supervisor attaches per-user images. Complete isolation story.

Complete system: multi-user product stories + either B or C for verify placement + multi-user.isolation for process isolation when fleet is live.

deployment

One binary, one rc.d service, one restart
[status: implemented]

rc.d service 'pricklypear': /etc/rc.d/pricklypear start -> /usr/local/libexec/pricklypear/current/bin/image with env from /etc/pricklypear/pricklypear.env stop -> kill the single PID restart -> stop + start (no EADDRINUSE because one process, one port, clean shutdown)

post-receive hook (existing, simplified): 1. dune build @all (lisp + common + image only; no supervisor, no auth) 2. promote to releases/<sha> 3. swap 'current' symlink 4. rcctl restart pricklypear

nginx (existing, simplified): - one server block: auth_basic + proxy_pass to 127.0.0.1:PP_HTTP_PORT - drop the /admin -> 17880 block (supervisor gone)

What this fixes (the deployment pains of it1-it34): - No stale image PID holding a port after restart - No supervisor/child respawn race (EADDRINUSE) - No "boot-on-SSH" spawn that needs a trigger - No two DB roles to keep in sync - No first-run ceremony from the supervisor's auth-client handshake

migration-plan

Order of operations for the make pass
[status: implemented]

Phase 1 – make the image standalone: 1. image/bin/unikernel.ml: drop mirage, plain unix main reading PP_* from env 2. image/lib/boot.ml: drop manifest path, read env 3. image/lib/dune + dune-project: drop mirage/unix 4. Verify 'dune build image' green; image runs alone

Phase 2 – delete the other two packages: 5. rm -r supervisor/ auth/ (+ dune stanzas) 6. common/: trim auth_types.ml, protocol.ml 7. Verify 'dune build @all' green (lisp + common + image only)

Phase 3 – deploy machinery: 8. Add /etc/rc.d/pricklypear (single service) 9. Update post-receive hook (build 3 packages, restart one service) 10. nginx: drop /admin block; keep / proxy 11. scripts/dev.sh: PG + image only

Phase 4 – prod cutover: 12. Push, deploy, start single service 13. Delete supervisor + authd rc.d entries 14. Verify /dabar /todos /notes /repl live

Rollback (if needed): revert the commit; the old supervisor+authd binaries still exist in prior releases until pruned. No data migration needed – 002_auth tables are additive and inert if unread.

18 storage.borg

pricklypear-storage

Pricklypear book chapter: storage
[status: partial]

storage

Postgres schema: branches, function_defs, type_defs, sessions. SERIALIZABLE defaults. NO inheritance.
[status: implemented]

> – *agent note:* Iteration 30 (2026-07-06) plan-pass #6: section status lifted partial -> implemented. All four subsections are now 'implemented': schema (it2), no-inheritance (it9), transactions (SERIALIZABLE defaults it16 + select-for-update it17 + with-transaction it29 + dynamic-tx-scope it29). The ONE remaining v1.5 ergonomics addition is the opt-in weaker-isolation '(query ... isolation: 'read-committed)' passthrough (the with-transaction prim already accepts the isolation arg shape, defaulting to SERIALIZABLE; the weaker-isolation passthrough is additive when shipped). This is not a v1 necessity; remember per-subsection details below for the fine-grained status narrative.

schema

Core tables (in migrations/001_branches.sql)
[status: implemented]

> – *agent note:* Schema lives in migrations/001_branches.sql (branches, function_defs, type_defs, view_bindings, sessions, schema_migrations) and migrations/003_data.sql (the shared 'data_rows' table for user-declared types). All SQL applied cleanly by the supervisor's migration runner (which owns 'split_sql_statements' to work around pgx's multi-statement prepared-statement protocol limitation – see BORGE-1783527733). SERIALIZABLE-by-default lives in image/lib/store.ml's 'Store.with_transaction'. Verified end-to-end against the temp PG 18.4 cluster at /tmp/pp-pgsup: 11 tables created, migrations idempotent on re-run.

– The shared code store CREATE TABLE branches (id uuid PRIMARY KEY, parent_id uuid REFERENCES branches(id), owner text NOT NULL, name text NOT NULL, created_at timestamptz NOT NULL DEFAULT now(), last_synced timestamptz, – for merge diffing UNIQUE (owner, name));

CREATE TABLE function_defs (branch_id uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE, name text NOT NULL, body jsonb NOT NULL, – serialized Lisp AST doc text, author text NOT NULL, updated_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY (branch_id, name));

CREATE TABLE type_defs (branch_id uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE, name text NOT NULL, fields jsonb NOT NULL, – field declarations migrated bool NOT NULL DEFAULT false, schema_vers int NOT NULL DEFAULT 1, author text NOT NULL, updated_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY (branch_id, name));

CREATE TABLE view_bindings (branch_id uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE, route text NOT NULL, – e.g. "/inspect/:id" handler text NOT NULL, – function name PRIMARY KEY (branch_id, route));

CREATE TABLE sessions (id uuid PRIMARY KEY, user_id text NOT NULL, branch_id uuid NOT NULL REFERENCES branches(id), pid int, – image instance pid boot_mode text NOT NULL, – 'on-connect'|'always' health text NOT NULL DEFAULT 'unknown', started_at timestamptz NOT NULL DEFAULT now(), last_seen timestamptz);

– Shared data tables (event, user, etc.) are – created by migrations emitted from type_defs – merged to main. Their shape is whatever main – says. They are NOT branched.

CREATE TABLE schema_migrations (version int PRIMARY KEY, applied_at timestamptz NOT NULL DEFAULT now(), description text);

The data tables themselves (e.g. events, users) live alongside these system tables, created by the supervisor's migration runner when type_defs are accepted into main.

SERIALIZABLE is the transaction isolation default for ALL Lisp-driven mutations. Weaker isolation is opt-in per query.

no-inheritance

Why we do NOT use Postgres INHERITS
[status: implemented]

> – *agent note:* Confirmed in code (it1-it16): no INHERITS in any migration; plain tables with 'branch_id' FK + PRIMARY KEY (branch_id, name). The override semantics the spec describes is realized via a recursive-CTE walk up the parent chain (closest-ancestor-wins), used in 'Store.lookup_function', 'Store.list_function_names', 'Store.list_routes', 'Store.list_types', and the CRUD data-row lookups. Override-vs- union behavior verified end-to-end by /tmp/pp_override_it9.py: alice/experiment overrides 'greet'; system/main's 'greet' is unchanged (branch isolation), alice's session sees alice's version (override wins).

Postgres table inheritance (INHERITS) is the deprecated feature from the early-90s Postgres implementation. We deliberately do NOT use it.

THE CORE REASON: branches are an OVERRIDE primitive, not a UNION primitive. When Alice forks and overrides 'greet', her branch's view should see ONLY her greet, not main's greet merged in. Postgres inheritance does union (both rows visible), not override.

Modeling override on top of inheritance forces us to write override resolution ourselves anyway (pick the child's row when it exists), so inheritance buys nothing but its quirks.

Other reasons: - Foreign keys / unique constraints don't span inheritance trees properly - PRIMARY KEY doesn't propagate - The planner handles inherited queries with edge cases and unpredictability - Inheritance handles schema evolution (add/remove columns) inconsistently; migrations would be a horror show - pg_dump, psql, admin tools render inheritance weirdly - We'd pin ourselves to non-standard semantics

Plain tables with branch_id FK and override resolution in queries (recursive CTE up the parent chain, or materialized effective-definition) gives us: - Clean indexed SELECTs - Predictable planner behavior - Standard migrations - Full inspectability (the goal) - A clean path to content-addressing later if we want "actually git" semantics

Future options stay open. Inheritance forecloses them.

transactions

Transaction isolation: SERIALIZABLE by default
[status: implemented]

> – *agent note:* Iteration 30 (2026-07-06) plan-pass #6: status lifted partial -> implemented. All v1 sub-features of this section are

> now built and smoke-tested: SERIALIZABLE-by-default (it16), explicit row locks via ‘(select-for-update ’type id)‘ (it17), and dynamic transaction scoping via ‘(with-transaction (lambda () body))‘ (it29) which gives select-for-update real cross-statement locking scope. Rollback behavior verified end-to-end on live PG.
>
> The ONE remaining unbuilt sub-feature is the opt-in weaker-isolation ‘(query ... isolation: ’read-committed)‘ passthrough – a v1.5 ergonomics addition, not a v1 necessity. The ‘with-transaction‘ prim already accepts the ‘isolation‘ arg shape (defaults to SERIALIZABLE); lifting the v1.5 passthrough is purely additive when shipped.
>
> Cache coherence (initial, NO caching – each call reads fresh from PG) is the current reality and is implemented as documented. LISTEN/NOTIFY-based invalidation is still deferred (later).

All Lisp-driven mutations run in SERIALIZABLE transactions by default. Postgres detects conflicts at commit and aborts one, forcing retry. For a multi-user personal OS, throughput is not the concern; correctness is.

Opt-in weaker isolation: (query ... isolation: ’read-committed) Available but discouraged. Most user code never touches this; the system does the safe thing by default. (v1.5 – the with-transaction prim already accepts the isolation arg shape, defaulting to SERIALIZABLE; the weaker-isolation passthrough remains additive when shipped.)

Explicit row locks: (select-for-update ’type id) Exposed as a primitive for finer control. Persists across subsequent prims in the same with-transaction body (it29).

Cache coherence (initial): NO caching of stored function defs in image memory. Each call reads the AST fresh from Postgres. For a personal OS where most calls are query+render, the function read alongside the query is acceptable overhead.

Cache coherence (later): LISTEN/NOTIFY-based invalidation. When a function is updated, the updating image notifies; others drop from cache. Mechanism well-trodden, deferred until measured.

with-transaction

Dynamic transaction scoping: (with-transaction (lambda () body))
[status: implemented]

> – *agent note:* Iteration 29 (2026-07-06): Option A from BORGE-1783438027 landed. Adds session-level dynamic transaction binding via a mutable ‘current_tx : Pg.t option‘ field on ‘Store.t‘; prims use ‘Store.current_conn t‘ so they see the in-tx conn automatically. New ‘with_transaction‘ special form in lisp/eval.ml dispatches the lambda body through a privately-named ‘___with_transaction___‘ prim registered from session.ml (keeps lisp/ pure-I/O-free). Nested transactions raise a clean Interp_error. Body value passes through verbatim. Borge issue BORGE-1783438027 status: triage -> resolved.

Dynamic transaction scope: all DB prims called inside the body run against the SAME pgx transactional connection. Defaults to SERIALIZABLE isolation.

(with-transaction (lambda () (seq (select-for-update ’type id) (update-field ’type id "field" "value"))))

The select-for-update row lock PERSISTS across the subsequent update-field in the same body – this is the v1 concurrency-protection story for compound mutations.

Nested transactions raise a clean Interp_error. The body value is returned verbatim (no wrapper tags/boxing).

The Lambda body is evaluated lazily; the dynamic binding is set on ‘Store.t.current_tx‘ before the body runs and cleared via ‘Lwt.finalize‘ after (whether BODY succeeds or raises – pgx still sees the propagated exception and rolls back).

19 supervisor.borg

pricklypear-supervisor

Pricklypear book chapter: supervisor – RETIRED by the single-image pivot (it35). Kept for historical context. See single-image.borg for the successor topology.

[status: deprecated]

> – *agent note:* it35 plan pass: this whole chapter is retired. The supervisor package will be deleted in the make pass. One image, one process, one deployment. No fleet, no boot-on-SSH, no A/B staging. The deployment pains of it1-it34 (EADDRINUSE on respawn, stale route tables, orphaned image PIDs, manual-vs-supervised spawn confusion) all trace to three coordinated processes. Status -> dropped.

supervisor

The root unikernel: fleet management, A/B rollback, health checks, admin HTTP

[status: deprecated]

> – *agent note:* 2026-07-12 status-reconcile: Retired by single-image pivot. Package directory is stub-only.

> – *agent note:* plan-pass #4 (it20): supervisor/lib/{boot,fleet,health,process,store,ssh,auth_client,admin}.ml + supervisor/bin/supervisor.ml shipped it4. 5 admin routes served via httpaf-lwt-unix + pure-html (no Dream). Boots cleanly on temp PG cluster, runs migrations, serves fleet dashboard/health/fleet JSON. End-to-end SSH?auth?image supervision verified by /tmp/pp_e2e_it6.py through scripts/dev.sh. Partial (not implemented): Solo5 target – currently mirage/unix only (deferred per [architecture.mirage-vs-unix]). A/B rollback /fleet/:id/rollback route NOT built (Fleet only has boot/kill/list). VM-request boot path NOT built – Fleet spawns via posix_spawn only (dev model). See BORGE-1783433175.

unikernel-target

Solo5 in production, mirage/unix in dev

[status: deprecated]

> – *agent note:* 2026-07-12 status-reconcile: Retired with supervisor package.

> – *agent note:* it20: PARTIAL – currently mirage/unix only. The Process module (supervisor/lib/process.ml) abstracts boot behind a boot()/kill() interface; in dev that's posix_spawn over a configured PP_IMAGE_BIN path. The Solo5/hypervisor-VM-request path is NOT built – Solo5 target itself is deferred per spec [architecture.mirage-vs-unix] ("Solo5 is a build switch, not a rewrite"). The abstraction boundary the spec describes (one interface, two implementations) is in place in process.ml. No real switchover has been exercised.

The supervisor IS a unikernel, not a daemon. It boots directly on the hypervisor (Solo5 target) and manages the rest. No OpenBSD userland in the hot path; the hypervisor trusts exactly one thing.

Process model: the supervisor does NOT posix_spawn child images (Solo5 has no spawn). Instead it requests new VMs from the hypervisor for each

per-user image instance. This is the Solo5/KVM model: boot a unikernel = request a VM.

In dev: mirage/unix target, supervisor runs as a normal process on the dev box, images too. Same code, different target. The fleet/process module abstracts the boot mechanism (VM-request in prod, posix_spawn in dev) behind one interface.

fleet

Managing per-user image instances
[status: deprecated]

> – *agent note:* 2026-07-12 status-reconcile: Retired with supervisor package.
>
> – *agent note:* it20: PARTIAL. IMPLEMENTED – supervisor/lib/fleet.ml maintains an in-memory instance table keyed by instance_id (inst-001, -002, ...). boot/kill/list/find_by_user/mark_health operations work; ports allocated sequentially (http_port_base=18080, repl_port_base=17878) per Fleet.spawn. Verified by /tmp/pp_e2e_it6.py: SSH?auth?image spawn flow registers inst-001 with the verified user_id. NOT IMPLEMENTED – fleet table persistence to PG sessions table (currently in-memory only; supervisor reboot loses it), rollback(instance_id, to_version) A/B op (no route), always-on boot_mode pin (only the boot-on-connect default exists by behavior). The spec text's '(user, branch, image_version, ..., boot_mode, last_seen)' columns are partially populated (instance_id, user_id, ports, started_at mapped; branch/image_version/boot_mode/last_seen omitted).

The supervisor maintains a fleet table: (user, branch, image_version, instance_id, boot_mode, health_status, started_at, last_seen)

Operations: - boot(user, branch, image_version) -> instance On SSH connect (default boot-on-connect) or always-on pin. - kill(instance_id) Stop a wedged or unwanted image. - list-fleet() -> all instances + health - rollback(instance_id, to_version) A/B rollback for ONE user's image, not the whole system.

Lifecycle: - boot-on-connect: image boots when user SSHes in, dies when session ends. Default. Doesn't persist in-memory REPL state across disconnects, but the store survives. - always-on: image pinned running. Power users who want long-lived REPL state across SSH disconnects. RAM always committed.

The supervisor itself never holds user-visible state in memory beyond the fleet table (which is also in Postgres and recoverable). It's stateless w.r.t. user data; Postgres is the truth.

health

Pinging images, marking healthy/unhealthy
[status: deprecated]

> – *agent note:* 2026-07-12 status-reconcile: Retired with supervisor package; image /health remains.
>
> – *agent note:* it20: PARTIAL. IMPLEMENTED – supervisor/lib/health.ml is an HTTP /health client (plain Lwt_unix socket + minimal HTTP/1.0 GET). Parses the JSON response via the Protocol module (the { ok, branch, uptime, error } shape is shared with common/lib/protocol.ml and used by both the supervisor's own /health and the image's /health). Fleet.mark_health updates an instance's health_status

field. NOT IMPLEMENTED – periodic background health-check loop (every 10s). Currently health is checked on-demand (e.g. before returning Fleet.list), not via a background Lwt.async tick. Unhealthy-then-autorestart logic for always-on instances is NOT built (no always-on mode to autorestart).

The supervisor health-checks each running image: - Periodically (every 10s) ping the image's HTTP health endpoint. - Response within timeout -> healthy. - No response in N tries -> unhealthy. - Unhealthy X seconds -> supervisor kills and either restarts (if always-on) or marks dead (if boot-on-connect; will reboot on next SSH).

Health check protocol (in common/lib/protocol.ml): GET /health -> { ok: true, branch: ..., uptime: ... } or { ok: false, error: ... }

On boot of a new image version: - Boot image v(N+1) alongside vN (or instead of, simpler). - Wait for health-check to pass within T seconds. - If healthy: v(N+1) becomes new default. - If not: rollback to vN, log failure, alert admin.

This makes "user upgraded primitives, broke their image" a 10-second recovery, not a crisis.

admin-http

Minimal admin surface, httpaf-lwt-unix + pure-html, no Dream
[status: deprecated]

– *agent note:* 2026-07-12 status-reconcile: Retired with supervisor package.

– *agent note:* it20/it31: PARTIAL. IMPLEMENTED – supervisor/lib/admin.ml serves via httpaf-lwt-unix + pure-html (no Dream, consistent with image/). Three routes live: GET / (fleet dashboard), GET /fleet (JSON), GET /health (supervisor health JSON). Explicit content-length headers. NOT IMPLEMENTED – POST /fleet/:id/kill and POST /fleet/:id/rollback?to=vN; admin auth. Doctrinal text updated it31 away from httpaf-mirage fiction.

The supervisor exposes a small admin surface over HTTP via httpaf-lwt-unix + pure-html (same v1 stack as the image). Hand-written router.

v1 SHIPPED routes: GET / fleet dashboard GET /fleet list instances + health (JSON) GET /health supervisor's own health

DEFERRED routes: POST /fleet/:id/kill POST /fleet/:id/rollback?to=vN

UI: plain HTML tables (pure-html), minimal htmx. Auth (later): admin token via authd, or localhost only in dev. Not wyo.auth.

Solo5 note: when the supervisor moves to Solo5, swap httpaf-lwt-unix for gluten-mirage + httpaf behind the same admin handler shape.

20 workflow.borg

pricklypear-workflow

Pricklypear book chapter: workflow
[status: partial]

workflow

Build/test workflow and conventions
[status: partial]

> – *agent note:* it31 plan-pass #7: conventions doctrinal text rewritten to the adopted v1 stack (Lwt, pgx_lwt+Io_unix, httpaf-lwt-unix, pure-html, own authd). build remains implemented. Section stays partial only while Solo5/prod-path conventions and any remaining package-layout drift are still open; day-to-day build workflow is green.

build

How to build and test each component
[status: implemented]

> – *agent note:* The full build/test workflow described in this subsection's doctrinal text is exercised every iteration: - 'dune build @all' exit 0 across all 5 packages (lisp, common, image, supervisor, auth) + the spikes/awa_ssh spike target. - 'dune runtest –force' runs 92 alcotest cases (lisp 30 + common 10 + auth 4 + image 48). - 'scripts/dev.sh start|stop|restart|status|clean' brings up the temp PG cluster (initdb if first run) + authd + supervisor, mirroring the "Full dev boot" path documented below (with PG + authd + supervisor in one command). Image/supervisor/ui binaries all run on mirage/unix per spec [architecture.mirage-vs-unix]; Solo5 still a deferred build switch. The 13 end-to-end smoke tests under /tmp/pp_*_it*.py exercise every layer end-to-end against the temp PG cluster at /tmp/pp-pgsup (port 5433).

Lisp interpreter (no services needed): cd lisp/ && dune build && dune runtest Tests are pure alcotest cases, no DB, no HTTP.

Image (needs Postgres for full run, but builds without): cd image/ && dune build Run: dune exec bin/unikernel.ml (mirage/unix) Needs PG connection from env.

Supervisor (needs Postgres for full run): cd supervisor/ && dune build Run: dune exec bin/supervisor.ml Needs PG connection, can boot images.

Common: cd common/ && dune build && dune runtest

Migrations: Applied by supervisor on boot, or manually via: psql -f migrations/NNN_xxx.sql

Full dev boot: 1. start Postgres 2. dune exec supervisor/bin/supervisor.ml (supervisor runs migrations, then listens) 3. ssh into a user's image (supervisor boots the image on connect)

conventions

Coding conventions for this project
[status: partial]

> – *agent note:* it31 plan-pass #7: doctrinal text rewritten to adopted v1 stack. Deliberate choices now normative in the details below: Lwt (not Eio) in unikernels; pgx_lwt + custom Io_unix (not pgx_lwt_mirage

in v1); httpaf-lwt-unix (httpaf-mirage does not exist on opam; future Solo5 path is gluten-mirage + httpaf); own authd unikernel (not wyo.auth) as the SSH/token/audit trust boundary. Eio remains allowed only for pure-unix dev tooling (lisp/bin/repl.ml, scripts). Section stays partial until Solo5 package conventions and any remaining transport text elsewhere are fully aligned; the day-to-day v1 conventions are now honest.

v1 ADOPTED STACK (normative): - OCaml / dune (per poohstack); OCaml 5.x on the poohstack opam switch - Lwt for ALL unikernel I/O (supervisor, authd, image). Mirage's runtime is Lwt-based; the evaluator is Lwt-typed (eval returns value Lwt.t). Eio only for pure-unix dev tooling (e.g. lisp/bin/repl.ml), never inside unikernels. - pure-html (dream_html HTML core, no Dream) for HTML; use HTML.Hx.* not raw htmx attribute strings. txt ~raw:true for intentional raw-HTML injection. - httpaf-lwt-unix for image + supervisor + authd HTTP in v1. (There is no httpaf-mirage package on opam. Future Solo5 path: gluten-mirage + httpaf.) - Postgres via pgx_lwt + a custom Io_unix shim in v1 (POSIX-flavored libs). pgx_lwt_mirage exists on the switch but needs a real Tcpip.Stack.V4V6 device, only available inside a true Mirage unikernel – that swap is a Solo5/v1.5 axis, not v1. - Auth via pricklypear's own authd unikernel (auth/), NOT wyo.auth. authd is the credential/token/audit trust boundary; supervisor calls it over internal HTTP RPC; images trust supervisor-asserted identity via the boot manifest. - SERIALIZABLE transactions by default for mutations - No ORM, no Postgres INHERITANCE, no premature abstraction; plain parameterized SQL - Additive-only schema migrations - The .borg file is the source of truth; if the implementation disagrees, the implementation is wrong – unless a plan pass rewrites the spec - Design changes go through borge plan/make/review; during make, file 'borge issue new' instead of editing .borg

DEFERRED CONVENTIONS (not v1-normative yet): - Solo5 target + Mirage-native transports (pgx_lwt_mirage, gluten-mirage, real VM boot) - mTLS/signed internal RPC between supervisor/authd - wyo.auth integration (explicitly not chosen for v1)

21 common/lib/auth_types.ml

```
(** Pricklypear Common -- Auth types.

    Shared between the supervisor (which calls auth via RPC) and the auth
    unikernel (which serves the RPC). Per spec [auth]: auth is the only
    thing that verifies credentials and mints tokens; the supervisor never
    sees raw credentials past the verify call. *)
open Yojson.Safe

type user_id = string
type credential_id = string

type user = {
  id : user_id;
  username : string;
  is_admin : bool;
  created_at : string;
}

type credential_kind =
  | K_ssh_key
  | K_password
  | K_web_token
  | K_api_key

let credential_kind_to_string = function
  | K_ssh_key -> "ssh_key"
  | K_password -> "password"
  | K_web_token -> "web_token"
  | K_api_key -> "api_key"

let credential_kind_of_string = function
  | "ssh_key" -> Some K_ssh_key
  | "password" -> Some K_password
  | "web_token" -> Some K_web_token
  | "api_key" -> Some K_api_key
  | _ -> None

(* A signed session token: opaque_id || HMAC(opaque_id).
   The opaque_id is the sessions.id; the HMAC is checked by auth. *)
type token = {
  opaque_id : string;     (* 32-byte random, base64 *)
  signature : string;     (* HMAC, base64 *)
}

let token_to_string (t : token) : string =
  (* joined with "." so we can split -- base64 is url-safe (no .) *)
  t.opaque_id ^ "." ^ t.signature

let token_of_string (s : string) : (token, string) result =
  match String.index_opt s '.' with
  | None -> Error "malformed token (no separator)"
  | Some i ->
      let opaque_id = String.sub s 0 i in
      let signature = String.sub s (i + 1) (String.length s - i - 1) in
      if opaque_id = "" || signature = "" then
        Error "malformed token (empty part)"
      else Ok { opaque_id; signature }

(* RPC request/response types *)
type verify_ssh_key_req = { key : string }
type verify_ssh_key_resp = Ok of user_id | Denied

let verify_ssh_key_resp_to_json = function
  | Ok uid -> `Assoc [ ("ok", `Bool true); ("user_id", `String uid) ]
  | Denied -> `Assoc [ ("ok", `Bool false) ]

let verify_ssh_key_resp_of_json (j : Yojson.Safe.t) : (verify_ssh_key_resp,
     string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    match List.assoc "ok" assoc with
```

```
    | `Bool true ->
        (match List.assoc "user_id" assoc with
         | `String uid -> Ok (Ok uid)
         | _ -> failwith "user_id")
    | `Bool false -> Ok Denied
    | _ -> failwith "ok"
  with Failure msg -> Error msg
```

22 common/lib/branch.ml

```
(** Pricklypear Common -- Branch reference.

    Shared between supervisor, auth, and image. A branch is identified by
    (owner, name); each has a parent (the branch it was forked from). The
    root branch is [main], which has owner="system", name="main", no parent.

    Per spec [branching]: branches are the fundamental organizational
    primitive. Code lives in branches (keyed by branch_id in Postgres);
    shared data lives in main's data tables. *)
open Yojson.Safe

type t = {
  id : string;            (* uuid, the primary key in branches table *)
  parent_id : string option;
  owner : string;         (* "system" for main, username otherwise *)
  name : string;          (* "main", "alice/main", "alice/experiment", ...
    *)
  created_at : string;    (* ISO 8601 timestamp *)
  last_synced : string option;  (* for merge diffing *)
}

let make ~id ~parent_id ~owner ~name ~created_at ?(last_synced = None) () :
    t =
  { id; parent_id; owner; name; created_at; last_synced }

let is_main (b : t) : bool = b.owner = "system" && b.name = "main"

let qualified_name (b : t) : string =
  if is_main b then "main" else b.owner ^ "/" ^ b.name

let to_json (b : t) : Yojson.Safe.t =
  `Assoc [
    ("id", `String b.id);
    ("parent_id", match b.parent_id with
                  | Some p -> `String p
                  | None -> `Null);
    ("owner", `String b.owner);
    ("name", `String b.name);
    ("created_at", `String b.created_at);
    ("last_synced", match b.last_synced with
                    | Some s -> `String s
                    | None -> `Null);
  ]

let of_json (j : Yojson.Safe.t) : (t, string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not an assoc" in
    let find k = List.assoc k assoc in
    let id = match find "id" with `String s -> s | _ -> failwith "id" in
    let parent_id = match find "parent_id" with
                    | `String s -> Some s | `Null -> None
                    | _ -> failwith "parent_id" in
    let owner = match find "owner" with `String s -> s | _ -> failwith
     "owner" in
    let name = match find "name" with `String s -> s | _ -> failwith "name"
     in
    let created_at = match find "created_at" with `String s -> s | _ ->
     failwith "created_at" in
    let last_synced = match find "last_synced" with
                      | `String s -> Some s | `Null -> None
                      | _ -> failwith "last_synced" in
    Ok { id; parent_id; owner; name; created_at; last_synced }
  with Failure msg -> Error msg
```

23 common/lib/manifest.ml

```
(** Pricklypear Common -- Boot manifest.

    Per spec [image.boot]: the supervisor reads a per-user manifest from
    Postgres (or constructs it on SSH connect) and passes it to the image
    on boot. The image uses it to know which branch to load, which user
    it's running as, whether it's boot-on-connect or always-on, and what
    budgets/flags apply.

    The manifest carries an IDENTITY CLAIM the image trusts because the
    supervisor asserted it (and the supervisor got the user_id from auth
    after verifying the SSH key -- see [auth.verification]). The image
    itself does no credential verification. *)
open Yojson.Safe

type boot_mode =
  | Boot_on_connect    (* image dies when session ends; default *)
  | Boot_always_on     (* image pinned running; survives SSH disconnects *)

let boot_mode_to_string = function
  | Boot_on_connect -> "on-connect"
  | Boot_always_on -> "always-on"

let boot_mode_of_string = function
  | "on-connect" -> Some Boot_on_connect
  | "always-on" -> Some Boot_always_on
  | _ -> None

type config = {
  step_budget_limit : int;       (* eval step budget per call, default
     100000 *)
  memory_budget_mb : int;        (* deferred; recorded here *)
  feature_flags : (string * bool) list;
}

let default_config : config = {
  step_budget_limit = 100_000;
  memory_budget_mb = 64;
  feature_flags = [];
}

let config_to_json (c : config) : Yojson.Safe.t =
  `Assoc [
    ("step_budget_limit", `Int c.step_budget_limit);
    ("memory_budget_mb", `Int c.memory_budget_mb);
    ("feature_flags",
     `Assoc (List.map (fun (k, v) -> (k, `Bool v)) c.feature_flags));
  ]

let config_of_json (j : Yojson.Safe.t) : (config, string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    let step = match List.assoc_opt "step_budget_limit" assoc with
               | Some (`Int n) -> n | _ -> 100_000 in
    let mem = match List.assoc_opt "memory_budget_mb" assoc with
              | Some (`Int n) -> n | _ -> 64 in
    let flags = match List.assoc_opt "feature_flags" assoc with
                | Some (`Assoc items) ->
                    List.filter_map (fun (k, v) ->
                      match v with `Bool b -> Some (k, b) | _ -> None) items
                | _ -> [] in
    Ok { step_budget_limit = step; memory_budget_mb = mem; feature_flags =
     flags }
  with Failure msg -> Error msg

type t = {
  user_id : string;        (* identity claim, asserted by supervisor (from
     auth) *)
  username : string;       (* for the REPL prompt and author tracking *)
  branch_id : string;      (* default branch UUID, usually user/main *)
  branch_name : string;    (* qualified: "alice/main" *)
  boot_mode : boot_mode;
```

```
  image_version : string;  (* for A/B tracking *)
  config : config;
}

let make
  ~user_id ~username ~branch_id ~branch_name ~boot_mode ~image_version
  ?(config = default_config) () : t =
  { user_id; username; branch_id; branch_name; boot_mode; image_version;
    config }

(* Accessors for cross-package callers that don't have the record type. *)
let user_id (m : t) : string = m.user_id
let username (m : t) : string = m.username
let branch_id (m : t) : string = m.branch_id
let branch_name (m : t) : string = m.branch_name

let to_json (m : t) : Yojson.Safe.t =
  `Assoc [
    ("user_id", `String m.user_id);
    ("username", `String m.username);
    ("branch_id", `String m.branch_id);
    ("branch_name", `String m.branch_name);
    ("boot_mode", `String (boot_mode_to_string m.boot_mode));
    ("image_version", `String m.image_version);
    ("config", config_to_json m.config);
  ]

let of_json (j : Yojson.Safe.t) : (t, string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    let user_id = match List.assoc "user_id" assoc with `String s -> s | _
     -> failwith "user_id" in
    let username = match List.assoc "username" assoc with `String s -> s |
     _ -> failwith "username" in
    let branch_id = match List.assoc "branch_id" assoc with `String s -> s
     | _ -> failwith "branch_id" in
    let branch_name = match List.assoc "branch_name" assoc with `String s
     -> s | _ -> failwith "branch_name" in
    let boot_mode_str = match List.assoc "boot_mode" assoc with `String s
     -> s | _ -> failwith "boot_mode" in
    let boot_mode = match boot_mode_of_string boot_mode_str with
                    | Some m -> m | None -> failwith "bad boot_mode" in
    let image_version = match List.assoc "image_version" assoc with `String
     s -> s | _ -> failwith "image_version" in
    let config = match List.assoc_opt "config" assoc with
                 | Some cj -> (match config_of_json cj with Ok c -> c |
    Error _ -> default_config)
                 | None -> default_config in
    Ok { user_id; username; branch_id; branch_name; boot_mode;
     image_version; config }
  with Failure msg -> Error msg
```

24 common/lib/protocol.ml

```
(** Pricklypear Common -- Health-check protocol.

    The supervisor pings each running image's HTTP /health endpoint
    periodically. Response carries the image's branch, uptime, and
    current status.

    Per spec [supervisor.health] / [image.boot]. *)
open Yojson.Safe

type health_status =
  | H_unknown
  | H_healthy
  | H_unhealthy
  | H_dead

let health_status_to_string = function
  | H_unknown -> "unknown"
  | H_healthy -> "healthy"
  | H_unhealthy -> "unhealthy"
  | H_dead -> "dead"

let health_status_of_string = function
  | "unknown" -> H_unknown
  | "healthy" -> H_healthy
  | "unhealthy" -> H_unhealthy
  | "dead" -> H_dead
  | _ -> H_unknown

type health_response = {
  ok : bool;
  branch : string;          (* qualified name like "alice/main" *)
  uptime_seconds : int;     (* seconds since image boot *)
  error : string option;    (* present if not ok *)
}

let health_response_to_json (h : health_response) : Yojson.Safe.t =
  `Assoc [
    ("ok", `Bool h.ok);
    ("branch", `String h.branch);
    ("uptime_seconds", `Int h.uptime_seconds);
    ("error", match h.error with Some e -> `String e | None -> `Null);
  ]

let health_response_of_json (j : Yojson.Safe.t) : (health_response, string)
    result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    let ok = match List.assoc "ok" assoc with `Bool b -> b | _ -> failwith
     "ok" in
    let branch = match List.assoc "branch" assoc with `String s -> s | _ ->
     failwith "branch" in
    let uptime = match List.assoc "uptime_seconds" assoc with `Int n -> n |
     _ -> failwith "uptime" in
    let error = match List.assoc "error" assoc with
                | `String s -> Some s | `Null -> None | _ -> failwith
     "error" in
    Ok { ok; branch; uptime_seconds = uptime; error }
  with Failure msg -> Error msg
```

25 spikes/awa_probe/probe.ml

```
let mk : 'a = Awa.Server.make
```

26 spikes/awa_probe_probe.ml

```
(* Inspect the Awa API surface (compile-time; no run). *)
module SSH = Awa_mirage.Make (struct
  type error = [ `Msg of string ]
  let pp_error fmt (`Msg s) = Fmt.string fmt s
  type write_error = [ `Closed | error ]
  let pp_write_error fmt = function
    | `Closed -> Fmt.string fmt "closed"
    | #error as e -> pp_error fmt e
  type flow = unit
  let read _ = assert false
  let write _ _ = assert false
  let writev _ _ = assert false
  let close _ = assert false
  let shutdown _ _ = assert false
end) (struct let sleep_ns _ = assert false end)
(struct
  let elapsed_ns () = 0L
  let period_ns () = None
end)

(* Trigger type errors that print the actual API signature. *)
let _ = (fun (s : Awa.Server.t) -> s) ()
let _ = (fun (m : Awa.Ssh.message list) -> m) ()
let _ = (fun (k : Awa.Hostkey.priv) -> k) ()
let _ = (fun (a : Awa.Keys.authenticator) -> a) ()
let _ = (fun (u : Awa.Server.user) -> u) ()
```

27 spikes/awa_ssh/spike.ml

```
(** Pricklypear awa-ssh feasibility probe (iteration 15 spike).

    Confirms that the awa-mirage SSH stack on the poohstack opam switch
    can have its three functor arguments satisfied from a plain
    [Lwt_unix.file_descr] flow, [Mtime]-based monotonic clock, and
    [Lwt_unix.sleep]-based sleep.

    The actual swap-in (replacing [supervisor/lib/ssh.ml]'s non-crypto
    line protocol with real SSH) is left for a dedicated future
    iteration -- this module only proves the type signature of
    `Awa_mirage.Make` accepts the three shim modules we'd ship with
    the swap.

    To probe the awa API surface, run: `dune build spikes/awa_ssh`. *)
open Printf
open Lwt.Syntax

(* ---- T : Mirage_time.S (sleep_ns over Lwt_unix.sleep) ---- *)
module Time = struct
  let sleep_ns (ns : int64) : unit Lwt.t =
    let secs = Int64.to_float ns /. 1e9 in
    Lwt_unix.sleep secs
end

(* ---- M : Mirage_clock.MCLOCK (elapsed_ns over Mtime_clock.elapsed) ---- *)
module Clock = struct
  let elapsed_ns () : int64 =
    let span = Mtime_clock.elapsed () in
    Mtime.Span.to_uint64_ns span
  let period_ns () : int64 option = None
end

(* ---- F : Mirage_flow.S (flow over a [Lwt_unix.file_descr]) ----
   This adapter is what the missing awa-lwt-unix package would have
   provided; on poohstack we supply it ourselves. The read/write/close
   implementation is complete enough for both directions of an SSH
   transport. *)
module Flow = struct
  type error = [ `Msg of string ]

  let pp_error fmt (`Msg s) = Fmt.string fmt s

  type write_error = [ `Closed | error ]

  let pp_write_error fmt = function
    | `Closed -> Fmt.string fmt "closed"
    | #error as e -> pp_error fmt e

  type flow = Lwt_unix.file_descr

  let read (fd : flow) : (Cstruct.t Mirage_flow.or_eof, error) result Lwt.t =
    let cap = 4096 in
    let buf = Bytes.create cap in
    Lwt.catch
      (fun () ->
        let* n = Lwt_unix.read fd buf 0 cap in
        if n = 0 then Lwt.return (Ok `Eof)
        else Lwt.return (Ok (`Data (Cstruct.of_bytes buf ~off:0 ~len:n))))
      (fun exn -> Lwt.return (Error (`Msg (Printexc.to_string exn))))

  let write (fd : flow) (cs : Cstruct.t)
    : (unit, write_error) result Lwt.t =
    Lwt.catch
      (fun () ->
        let bs = Cstruct.to_bytes cs in
        let* _ = Lwt_unix.write fd bs cs.Cstruct.off cs.Cstruct.len
        in
        Lwt.return (Ok ()))
      (fun exn -> Lwt.return (Error (`Msg (Printexc.to_string exn))))
```

```
70
71    let writev (fd : flow) (css : Cstruct.t list)
72      : (unit, write_error) result Lwt.t =
73      let rec go = function
74        | [] -> Lwt.return (Ok ())
75        | cs :: rest ->
76            let* r = write fd cs in
77            (match r with
78             | Ok () -> go rest
79             | Error e -> Lwt.return (Error e))
80      in
81      go css
82
83    let close (fd : flow) : unit Lwt.t =
84      Lwt.catch
85        (fun () -> Lwt_unix.close fd)
86        (fun _ -> Lwt.return_unit)
87
88    let shutdown (fd : flow) (_mode : [< `read | `write | `read_write ]) :
       unit Lwt.t =
89      Lwt.catch
90        (fun () ->
91          Unix.shutdown (Lwt_unix.unix_file_descr fd) Unix.SHUTDOWN_ALL;
92          Lwt.return_unit)
93        (fun _ -> Lwt.return_unit)
94  end
95
96  (* Link-check that Awa_mirage.Make compiles against these shims.
97     awa-mirage 0.6: Make takes only Mirage_flow.S (Time/Clock no longer
       params). *)
98  module SSH = Awa_mirage.Make (Flow)
99
100 (* A trivial entry point so dune has something to build. *)
101 let () = printf "awa-ssh spike compiles; SSH.spawn_server reachable\n%!"
```

28 spikes/mirage_solo5/config.ml

```
open Mirage

(* Minimal Phase 1 probe: prove mirage configure + the unix target
   round-trips, and that Mirage_main.Make can be invoked with the
   device graph mirage-runtime ships. This is the smallest "mirage
   unikernel" that can build from the config side -- two clocks only
   (ptime + mtime), no network yet (that's a later phase 2/3 concern).

   PROBE RESULT (iteration 27):
     *) `mirage configure -t unix` SUCCEEDS -- generates mirage/main.ml
        + dune.build.
     *) The generated `mirage/main.ml` correctly applies
        `Unikernel.Hello(Mirage_ptime)(Mirage_mtime)` and wires the
        Mirage runtime (sleep, logs, bootvar, RNG, etc).
     *) `pgx_lwt_mirage` 2.2 IS installed on poohstack. Its `Make`
        functor signature: `Make (RANDOM) (TIME) (MCLOCK) (PCLOCK)
        (STACK : Tcpip.Stack.V4V6) : val connect : STACK.t ->
        (module Pgx_lwt.S)` -- requires a real Tcpip.Stack.V4V6 device,
        only satisfiable inside a real mirage unikernel.
     *) `dune build` FAILS with 6 missing opam packages on poohstack:
          mirage-bootvar.unix, mirage-crypto-rng-mirage, mirage-logs,
          mirage-ptime.unix, mirage-sleep.unix, cmdliner-stdlib.
        These are Mirage 4.x runtime packages; installing them is a
        heavy switch modification (6+ new deps) and is OUT OF SCOPE
        for the spike per AGENTS.md "Do not add dependencies lightly".
        Such an install needs explicit user approval.
     *) `httpaf-mirage` does NOT exist as a package. The mirage http
        path is via `gluten-mirage` + `httpaf`, also not installed.
     *) Solo5 stack (mirage-solo5, ocaml-solo5, solo5-bindings-hvt,
        mirage-net-solo5) all absent from poohstack.

   To re-run the probe:
     cd spikes/mirage_solo5
     eval $(opam env --switch=poohstack --set-switch)
     mirage configure -t unix   # succeeds -- generates main.ml + dune.build
     dune build                 # FAILS mirage-sleep.unix not found

   See README.md in this folder + the iteration 27 entry of
   .ralph/build-pricklypear.md for the full findings + next steps. *)
let () =
  register "pp_mirage_spike"
    [
      main
        "Unikernel.Hello"
        (ptime @-> mtime @-> job)
        $ default_ptime $ default_mtime
    ]
```

29 spikes/mirage_solo5/unikernel.ml

```
open Lwt.Infix

module Hello
    (P : Mirage_ptime.S)
    (M : Mirage_mtime.S) = struct

  let start _pc _mc =
    Logs.info (fun m -> m "hello from mirage solo5 spike (phase 1 probe
     OK)");
    Lwt.return_unit

end
```

30 common/test/common_test.ml

```
(** Tests for the Pricklypear common library -- pure, no services. *)
open Pricklypear_common

let branch_tests = [
  ("branch: main is detected", `Quick, fun () ->
    let b = Branch.make
      ~id:"b1" ~parent_id:None ~owner:"system" ~name:"main"
      ~created_at:"2026-07-05T00:00:00Z" () in
    Alcotest.(check bool) "is_main" true (Branch.is_main b);
    Alcotest.(check string) "qualified" "main" (Branch.qualified_name b));
  ("branch: user branch qualified name", `Quick, fun () ->
    let b = Branch.make
      ~id:"b2" ~parent_id:(Some "b1") ~owner:"alice" ~name:"main"
      ~created_at:"2026-07-05T00:00:00Z" () in
    Alcotest.(check bool) "not main" false (Branch.is_main b);
    Alcotest.(check string) "qualified" "alice/main" (Branch.qualified_name
     b));
  ("branch: JSON round-trip", `Quick, fun () ->
    let b = Branch.make
      ~id:"b3" ~parent_id:(Some "b1") ~owner:"alice" ~name:"experiment"
      ~created_at:"2026-07-05T01:00:00Z" ~last_synced:(Some
     "2026-07-05T02:00:00Z") () in
    let j = Branch.to_json b in
    (match Branch.of_json j with
     | Ok b' ->
        Alcotest.(check string) "id" b.id b'.id;
        Alcotest.(check string) "owner" "alice" b'.owner;
        Alcotest.(check string) "name" "experiment" b'.name;
        Alcotest.(check string) "last_synced" "2026-07-05T02:00:00Z"
                                  (match b'.last_synced with Some s -> s |
     None -> "")
     | Error e -> Alcotest.failf "round-trip failed: %s" e));
]

let manifest_tests = [
  ("manifest: JSON round-trip with default config", `Quick, fun () ->
    let m = Manifest.make
      ~user_id:"u1" ~username:"alice" ~branch_id:"b2"
      ~branch_name:"alice/main" ~boot_mode:Manifest.Boot_on_connect
      ~image_version:"v1" () in
    let j = Manifest.to_json m in
    (match Manifest.of_json j with
     | Ok m' ->
        Alcotest.(check string) "user_id" "u1" m'.user_id;
        Alcotest.(check string) "username" "alice" m'.username;
        Alcotest.(check string) "branch_name" "alice/main" m'.branch_name;
        Alcotest.(check int) "step_budget" 100_000
     m'.config.step_budget_limit
     | Error e -> Alcotest.failf "round-trip failed: %s" e));
  ("manifest: boot_mode parse", `Quick, fun () ->
    Alcotest.(check bool) "on-connect parses"
      true (Manifest.Boot_on_connect =
              (match Manifest.boot_mode_of_string "on-connect" with
               | Some m -> m | None -> Manifest.Boot_always_on)));
]

let auth_types_tests = [
  ("auth: token string round-trip", `Quick, fun () ->
    let t = { Auth_types.opaque_id = "AAAA"; signature = "BBBB" } in
    let s = Auth_types.token_to_string t in
    (match Auth_types.token_of_string s with
     | Ok t' ->
        Alcotest.(check string) "opaque" "AAAA" t'.opaque_id;
        Alcotest.(check string) "sig" "BBBB" t'.signature
     | Error e -> Alcotest.failf "token parse failed: %s" e));
  ("auth: malformed token rejected", `Quick, fun () ->
    (match Auth_types.token_of_string "noseparator" with
     | Ok _ -> Alcotest.fail "should have failed"
     | Error _ -> ());
    Alcotest.(check bool) "rejected" true true);
  ("auth: verify_ssh_key_resp round-trip Ok", `Quick, fun () ->
```

```
    let r = Auth_types.Ok "u1" in
    let j = Auth_types.verify_ssh_key_resp_to_json r in
    (match Auth_types.verify_ssh_key_resp_of_json j with
     | Ok (Auth_types.Ok uid) -> Alcotest.(check string) "uid" "u1" uid
     | Ok Auth_types.Denied -> Alcotest.fail "expected Ok"
     | Error e -> Alcotest.failf "parse failed: %s" e));
]

let protocol_tests = [
  ("protocol: health_response round-trip ok", `Quick, fun () ->
    let h = { Protocol.ok = true; branch = "alice/main";
              uptime_seconds = 42; error = None } in
    let j = Protocol.health_response_to_json h in
    (match Protocol.health_response_of_json j with
     | Ok h' ->
        Alcotest.(check bool) "ok" true h'.ok;
        Alcotest.(check string) "branch" "alice/main" h'.branch;
        Alcotest.(check int) "uptime" 42 h'.uptime_seconds
     | Error e -> Alcotest.failf "parse failed: %s" e));
  ("protocol: health_status conversions", `Quick, fun () ->
    Alcotest.(check string) "healthy" "healthy"
      (Protocol.health_status_to_string Protocol.H_healthy);
    Alcotest.(check bool) "round-trip" true
      (Protocol.H_unhealthy = Protocol.health_status_of_string
     "unhealthy"));
]

let () =
  Alcotest.run "pricklypear-common" [
    "branch", branch_tests;
    "manifest", manifest_tests;
    "auth-types", auth_types_tests;
    "protocol", protocol_tests;
  ]
```

31 image/lib/agent_tools.ml

```
(** AI agent tools, propose/install, LLM rewrite/growth loops.
    Operates on [Session_types.t]; re-exported from [Session]. *)
open Lwt.Syntax
open Session_types

let ensure_callable_body (body_src : string) : string =
  try
    let e = Pricklypear_lisp.Reader.read_one body_src in
    match e with
    | Pricklypear_lisp.Ast.App (Pricklypear_lisp.Ast.Var "lambda", _) ->
     body_src
    | Pricklypear_lisp.Ast.Lambda _ -> body_src
    | _ -> Printf.sprintf "(lambda (x) %s)" body_src
  with _ -> body_src

(* Save/define a binding from source body text. Used by function-editor
   HTTP and AI propose. [author] is recorded on function_defs. *)
let define_binding (t : t) ~(name : string) ~(body_src : string)
    ~(author : string)
    : (Pricklypear_lisp.Ast.value, string) result Lwt.t =
  let body_src = ensure_callable_body body_src in
  let input = Printf.sprintf "(define %s %s)" name body_src in
  Lwt.catch
    (fun () ->
      let expr = Pricklypear_lisp.Reader.read_one input in
      let* v =
        Pricklypear_lisp.Eval.eval_string ~prims:t.prims t.env input
      in
      (match expr with
       | Pricklypear_lisp.Ast.App
           (Pricklypear_lisp.Ast.Var "define",
            [ Pricklypear_lisp.Ast.Var n; body_expr ]) ->
           let* () = Store.persist_function t.store n body_expr author in
           (* Stale chrome nav was the "bible only on home" bug: home
              live-calls ui/nav; other pages used a cached string. *)
           if n = "ui/nav" || n = "ui/home" || n = "ui/css" || n =
    "ui/theme"
              || String.starts_with ~prefix:"theme/" n then (
             t.ui_nav_html <- None;
             t.ui_css <- None);
           Lwt.return (Ok v)
       | _ ->
           (* Still bound in env if define special-form ran *)
           Lwt.return (Ok v)))
    (fun exn ->
      let msg =
        match exn with
        | Pricklypear_lisp.Ast.Interp_error e ->
            (match e with
             | Pricklypear_lisp.Ast.Parse_error (m, l, c) ->
                 Printf.sprintf "parse error: %s (%d:%d)" m l c
             | Pricklypear_lisp.Ast.User_error m -> m
             | Pricklypear_lisp.Ast.Unbound_var s -> "unbound: " ^ s
             | Pricklypear_lisp.Ast.Budget_exceeded -> "budget exceeded"
             | _ -> Printexc.to_string exn)
        | Pricklypear_lisp.Reader.Reader_error (m, l, c) ->
            Printf.sprintf "parse error: %s (%d:%d)" m l c
        | _ -> Printexc.to_string exn
      in
      Lwt.return (Error msg))

(* First-run recipe: hello page function + route. No Lisp knowledge
    required. *)
(* Optional elevate: PP_AI_ALLOW_WRITE=1 allows WriteDb/Publish/etc. *)
let ai_allow_set () =
  match Sys.getenv_opt "PP_AI_ALLOW_WRITE" with
  | Some ("1" | "true" | "TRUE" | "yes") ->
      Pricklypear_lisp.Effect.EffSet.normalize
        [ Pricklypear_lisp.Effect.ReadDb
        ; Pricklypear_lisp.Effect.WriteDb
        ; Pricklypear_lisp.Effect.Html
```

```
        ; Pricklypear_lisp.Effect.Publish
        ; Pricklypear_lisp.Effect.Ai
        ]
  | _ -> Pricklypear_lisp.Effect.ai_default_allow

let check_ai_body_effects (body_src : string) : (unit, string) result =
  try
    let expr = Pricklypear_lisp.Reader.read_one body_src in
    let fx = Pricklypear_lisp.Effect.infer expr in
    if Pricklypear_lisp.Effect.policy_allows ~allow:(ai_allow_set ()) fx
     then
      Ok ()
    else
      Error
        (Printf.sprintf
           "effect policy denied (%s); allow pure|read-db|write-db|html by
     default; use register_route tool for routes (not (route ...) in body);
     set PP_AI_ALLOW_WRITE=1 for branch/publish elevate"
           (Pricklypear_lisp.Effect.EffSet.pp fx))
  with _ -> Ok ()  (* parse errors handled by define_binding *)

(* [agent] temporary alias defined after agent_session_new / agent_say. *)

(** Install a Lisp body as [name] with AI provenance (parse + effect gate).
    *)
let install_ai_function (t : t) ~(name : string) ~(body_src : string)
    : Pricklypear_lisp.Ast.value Lwt.t =
  match Llm_client.extract_lisp_sexpr body_src with
  | Error e ->
      let preview =
        let s = String.trim body_src in
        if String.length s > 240 then String.sub s 0 240 ^ "?" else s
      in
      Lwt.return
        (Pricklypear_lisp.Ast.VStr
           (Printf.sprintf "ai body invalid: %s\n---\n%s" e preview))
  | Ok body_src ->
      (match check_ai_body_effects body_src with
       | Error pe ->
           Lwt.return
             (Pricklypear_lisp.Ast.VStr
                (Printf.sprintf "ai rejected by effects: %s\n---\n%s" pe
     body_src))
       | Ok () ->
           let author = "agent:" ^ Llm_client.model () in
           let* def = define_binding t ~name ~body_src ~author in
           (match def with
            | Ok _ ->
                let badge =
                  try
                    let e = Pricklypear_lisp.Reader.read_one body_src in
                    Pricklypear_lisp.Effect.EffSet.badge
                      (Pricklypear_lisp.Effect.infer e)
                  with _ -> "?"
                in
                Lwt.return
                  (Pricklypear_lisp.Ast.VStr
                     (Printf.sprintf "installed %s by %s [%s]\n%s"
                        name author badge body_src))
            | Error e ->
                Lwt.return
                  (Pricklypear_lisp.Ast.VStr
                     (Printf.sprintf "ai body invalid: %s\n---\n%s" e
     body_src))))

(** Pretty source body for a binding (env lambda or store). *)
let function_source (t : t) (name : string) : string option Lwt.t =
  let* found = Store.lookup_function t.store name in
  match found with
  | Some (expr, _) ->
      Lwt.return (Some (Pricklypear_lisp.Reader.pp_expr expr))
  | None ->
      (match Pricklypear_lisp.Env.lookup t.env name with
```

```
      | Some (Pricklypear_lisp.Ast.VLambda (params, body, _)) ->
          let e =
            Pricklypear_lisp.Ast.Lambda (params, body)
          in
          Lwt.return (Some (Pricklypear_lisp.Reader.pp_expr e))
      | _ -> Lwt.return_none)

let package_binding_names (t : t) (pkg : string) : string list Lwt.t =
  let* defs = Store.list_function_names t.store in
  let pkg =
    if pkg = "(root)" || pkg = "root" then "" else pkg
  in
  let names =
    List.filter_map
      (fun (name, _, _) ->
        let p = Packages.package_of_name name in
        if p = pkg then Some name else None)
      defs
    |> List.sort_uniq String.compare
  in
  Lwt.return names

(** Export package as lib.pp text: (define name body)? pretty-printed. *)
let export_package (t : t) (pkg : string) : string Lwt.t =
  let* names = package_binding_names t pkg in
  let label = if pkg = "" || pkg = "(root)" then "(root)" else pkg in
  let buf = Buffer.create 4096 in
  Buffer.add_string buf
    (Printf.sprintf
       ";; package %s -- exported from live branch %s\n;; Sync:
    scripts/pp_sync.py pull|push\n\n"
       label t.manifest.branch_name);
  let rec go = function
    | [] -> Lwt.return (Buffer.contents buf)
    | name :: rest ->
        let* src = function_source t name in
        (match src with
         | None -> go rest
         | Some body ->
             Buffer.add_string buf "(define ";
             Buffer.add_string buf name;
             Buffer.add_string buf "\n  ";
             (* indent body lines by 2 spaces after first *)
             let body_indented =
               body
               |> String.split_on_char '\n'
               |> (function
                    | [] -> ""
                    | h :: t ->
                        String.concat "\n"
                          (h :: List.map (fun line -> "  " ^ line) t))
             in
             Buffer.add_string buf body_indented;
             Buffer.add_string buf ")\n\n";
             go rest)
  in
  go names

(* AI propose: draft via agent portal, then install on current branch. *)
let propose_function (t : t) ~(name : string) ~(spec : string)
    : Pricklypear_lisp.Ast.value Lwt.t =
  let seed =
    "You write Pricklypear Lisp function bodies.\n" ^
    "OUTPUT RULES (strict):\n" ^
    "1. Reply with EXACTLY one s-expression and nothing else.\n" ^
    "2. The s-expression MUST be a lambda, e.g. (lambda (x) (* x 2)).\n" ^
    "3. Do NOT write analysis, markdown, numbered lists, or prose.\n" ^
    "4. Do NOT write (define ...).\n" ^
    "5. Do NOT wrap the answer in ``` fences.\n" ^
    "Prims: + - * = < > list car cdr cons null? string-append str " ^
    "dict-get dict-set get create update delete rows render-html."
  in
  let prompt =
```

```
    Printf.sprintf
      "Name: %s\nSpec: %s\n\nYour entire reply is one lambda s-expression only."
      name spec
  in
  (* Low temperature; authoring is not chat. *)
  let* res =
    Llm_client.agent_chat ~temperature:0.1 ~max_tokens:512
      ~seed ~prompt ()
  in
  match res with
  | Error e ->
      Lwt.return (Pricklypear_lisp.Ast.VStr ("agent error: " ^ e))
  | Ok body_src ->
      install_ai_function t ~name ~body_src

(** Pretty-print a stored function body, or default stub. *)
let current_fn_pretty (t : t) (name : string) : string Lwt.t =
  let* found = Store.lookup_function t.store name in
  match found with
  | Some (expr, _) ->
      Lwt.return (Pricklypear_lisp.Reader.pp_expr expr)
  | None ->
      (match Pricklypear_lisp.Env.lookup t.env name with
       | Some (Pricklypear_lisp.Ast.VLambda (params, body, _)) ->
           let p =
             "(lambda (" ^ String.concat " " params ^ ") " ^
             Pricklypear_lisp.Reader.pp_expr body ^ ")"
           in
           Lwt.return p
       | _ -> Lwt.return "(lambda () #f)")

(** Freeze AI rewrite into store (coding tool). mode: replace|wrap *)
(* ---- Tool-using agent loop (multi-function tasks) ---- *)

let json_string_field (j : Yojson.Safe.t) (k : string) : string option =
  match j with
  | `Assoc items ->
      (match List.assoc_opt k items with
       | Some (`String s) -> Some s
       | Some (`Int i) -> Some (string_of_int i)
       | Some (`Float f) -> Some (string_of_float f)
       | Some (`Bool b) -> Some (string_of_bool b)
       | Some other -> Some (Yojson.Safe.to_string other)
       | None -> None)
  | _ -> None

let pp_value_short (v : Pricklypear_lisp.Ast.value) : string =
  let s = Pricklypear_lisp.Reader.pp_value v in
  if String.length s > 4000 then String.sub s 0 4000 ^ "?" else s

let agent_tool_defs : Llm_client.tool_def list =
  let open Llm_client in
  let obj props required =
    `Assoc
      [ ("type", `String "object")
      ; ("properties", `Assoc props)
      ; ("required", `List (List.map (fun s -> `String s) required))
      ]
  in
  [ { name = "list_bindings"
    ; description = "List function/binding names visible in this image env"
    ; parameters = obj [] []
    }
  ; { name = "list_routes"
    ; description = "List HTTP routes and their handler function names"
    ; parameters = obj [] []
    }
  ; { name = "inspect_function"
    ; description = "Show the Lisp source body of a named function if present"
    ; parameters =
        obj
```

```
          [ ("name",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "function name, e.g. demo/double
    or todo/page")
               ])
          ]
          [ "name" ]
      }
    ; { name = "install_function"
      ; description =
          "Install a Lisp function on the current branch. body must be a
    lambda s-expression only, e.g. (lambda (x) (* x 2)) or (lambda ()
    (html/page ...)). Do not put (route ...) in the body -- use
    register_route. Author becomes agent:<model>."
      ; parameters =
          obj
            [ ("name",
               `Assoc
                 [ ("type", `String "string")
                 ; ("description", `String "binding name to define")
                 ])
            ; ("body",
               `Assoc
                 [ ("type", `String "string")
                 ; ("description", `String "lambda s-expression source")
                 ])
            ]
            [ "name"; "body" ]
      }
    ; { name = "register_route"
      ; description =
          "Bind an HTTP path to an already-installed handler function name on
    this branch. Use after install_function of the page handler. Example:
    path=/contacts handler=contacts/page. Do NOT put (route ...) inside
    install_function bodies."
      ; parameters =
          obj
            [ ("path",
               `Assoc
                 [ ("type", `String "string")
                 ; ("description", `String "URL path starting with /, e.g.
    /contacts")
                 ])
            ; ("handler",
               `Assoc
                 [ ("type", `String "string")
                 ; ("description", `String "function name, e.g.
    contacts/page")
                 ])
            ]
            [ "path"; "handler" ]
      }
    ; { name = "eval_expr"
      ; description =
          "Evaluate a short Lisp expression in this image (same env as the
    branch). Use for smoke checks like (count (contacts/all)) or
    (contacts/name (cdr (car (contacts/all)))). Returns the printed value
    or an error. Prefer small expressions; no multi-form progn dumps."
      ; parameters =
          obj
            [ ("expr",
               `Assoc
                 [ ("type", `String "string")
                 ; ("description", `String "one Lisp expression")
                 ])
            ]
            [ "expr" ]
      }
    ; { name = "smoke_route"
      ; description =
          "Call a registered user route handler in-process (no real HTTP).
    method GET|POST; optional form/query params as a JSON object of
```

```
    strings; optional contains substring that must appear in the HTML/text
    result (or redirect Location). Use after register_route to verify
    wiring: smoke_route path=/contacts method=GET contains=Contacts. On
    failure, inspect_function the handler, fix with install_function,
    re-smoke."
    ; parameters =
        obj
          [ ("path",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "route path e.g. /contacts")
               ])
          ; ("method",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "GET or POST (default GET)")
               ])
          ; ("params",
             `Assoc
               [ ("type", `String "object")
               ; ("description", `String "form/query params as string
    values, e.g. {\"action\":\"create\",\"name\":\"Ada\"}")
               ])
          ; ("contains",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "optional substring that must
    appear in body or redirect location")
               ])
          ]
          [ "path" ]
    }
  ; { name = "html_helpers"
    ; description =
        "Return the html/* arity cheatsheet and anti-patterns. Call this
    BEFORE writing page/form/list HTML with install_function. Required
    when smoke_route fails with unbound/arity/type errors in html/*."
    ; parameters = obj [] []
    }
  ; { name = "image_context"
    ; description = "Return user, branch, and a short summary of this image"
    ; parameters = obj [] []
    }
  ]

let json_object_string_pairs (j : Yojson.Safe.t) : (string * string) list =
  match j with
  | `Assoc items ->
      List.filter_map
        (fun (k, v) ->
          match v with
          | `String s -> Some (k, s)
          | `Int n -> Some (k, string_of_int n)
          | `Bool b -> Some (k, if b then "true" else "false")
          | `Null -> Some (k, "")
          | other -> Some (k, Yojson.Safe.to_string other))
        items
  | _ -> []

let value_smoke_kind (v : Pricklypear_lisp.Ast.value) : string * string =
  match Primitives.is_redirect v with
  | Some loc -> ("redirect", loc)
  | None ->
      (match Primitives.is_raw_html v with
       | Some html -> ("html", html)
       | None ->
           (match v with
            | Pricklypear_lisp.Ast.VStr s -> ("text", s)
            | _ -> ("value", Pricklypear_lisp.Reader.pp_value v)))

(** Human-readable message from evaluator/reader exceptions (for agent
    tools). *)
let pp_interp_exn (exn : exn) : string =
```

```
  match exn with
  | Pricklypear_lisp.Ast.Interp_error e ->
      (match e with
       | Pricklypear_lisp.Ast.Unbound_var s -> "unbound: " ^ s
       | Pricklypear_lisp.Ast.User_error m -> m
       | Pricklypear_lisp.Ast.Not_a_function _ -> "not a function"
       | Pricklypear_lisp.Ast.Arity_mismatch (name, exp, got) ->
           Printf.sprintf "arity mismatch: %s expected %d got %d" name exp got
       | Pricklypear_lisp.Ast.Wrong_arg_count (name, exp, got) ->
           Printf.sprintf "wrong arg count: %s expected %d got %d" name exp got
       | Pricklypear_lisp.Ast.Type_mismatch (expected, v) ->
           Printf.sprintf "type mismatch: expected %s got %s" expected
             (Pricklypear_lisp.Reader.pp_value v)
       | Pricklypear_lisp.Ast.Budget_exceeded -> "budget exceeded"
       | Pricklypear_lisp.Ast.Prim_error (prim, msg) ->
           Printf.sprintf "prim %s: %s" prim msg
       | Pricklypear_lisp.Ast.Parse_error (m, l, c) ->
           Printf.sprintf "parse error: %s (%d:%d)" m l c)
  | Pricklypear_lisp.Reader.Reader_error (m, l, c) ->
      Printf.sprintf "parse error: %s (%d:%d)" m l c
  | _ -> Printexc.to_string exn

let html_helpers_cheatsheet : string =
  "Pricklypear html/* cheatsheet (WRONG shapes are the #1 agent failure).\n" ^
  "\n" ^
  "CORE:\n" ^
  "  (html/attr \"class\" \"card\")           ; ONE key, ONE value ? string\n" ^
  "  (html/attrs (list a1 a2 a3))          ; join attr strings\n" ^
  "  (html/join (list s1 s2 s3))           ; concat HTML strings\n" ^
  "  (html/tag \"div\" attrs body)           ; attrs = string, body = string\n" ^
  "  (html/void \"input\" attrs)             ; void tag\n" ^
  "  (html/page (list part1 part2))        ; full page via render-html\n" ^
  "\n" ^
  "ELEMENTS (copy these arities exactly):\n" ^
  "  (html/h1 body)  (html/h2 body)  (html/h3 body)\n" ^
  "  (html/p body)                     ; escapes text\n" ^
  "  (html/p* attrs body)              ; raw body HTML allowed\n" ^
  "  (html/div attrs body)\n" ^
  "  (html/span attrs body)\n" ^
  "  (html/ul attrs body)  (html/li attrs body)\n" ^
  "  (html/strong body)  (html/code body)  (html/pre body)\n" ^
  "  (html/label attrs body)\n" ^
  "  (html/button attrs body)\n" ^
  "  (html/a href body)                ; NOT (html/a (list attrs) body)\n" ^
  "  (html/form method action attrs body)\n" ^
  "      e.g. (html/form \"POST\" \"/bookmarks\"\n" ^
  "             (html/attrs (list (html/attr \"class\" \"card\")))\n" ^
  "             (html/join (list ...fields...)))\n" ^
  "  (html/input attrs)\n" ^
  "  (html/hidden name value)\n" ^
  "  (html/text-input name attrs)      ; name is first arg STRING\n" ^
  "  (html/textarea name attrs body)\n" ^
  "  (html/void \"br\" \"\")                ; no html/br binding -- use void\n" ^
  "\n" ^
  "ANTI-PATTERNS (will crash smoke_route):\n" ^
  "  ? (html/form (list attrs...) (list kids...))\n" ^
  "  ? (html/a (list (html/attr ...)) title)\n" ^
  "  ? (html/text-input (list (html/attr \"name\" \"x\")))\n" ^
  "  ? (html/attr \"a\" \"b\" \"c\" \"d\")     ; only 2 args\n" ^
  "  ? (html/br)                        ; unbound\n" ^
  "  ? free variable `row` outside list/map\n" ^
  "\n" ^
  "GOOD list item:\n" ^
  "  (html/li (html/attr \"class\" \"card\")\n" ^
  "    (html/join (list\n" ^
  "      (html/a (dict-get fields \"url\") (html/text (dict-get fields \"title\")))\n" ^
```

```
  "      (html/form \"POST\" \"/bookmarks\" \"\"\n" ^
  "        (html/join (list (html/hidden \"action\" \"delete\")\n" ^
  "                         (html/hidden \"id\" id)\n" ^
  "                         (html/button (html/attr \"type\" \"submit\")
     \"Delete\")))))))\n" ^
  "\n" ^
  "Also: inspect_function html/form | html/a | contacts/form-html |
     todo/add-form\n"

let truncate_for_tool ~(limit : int) (s : string) : string =
  if String.length s <= limit then s
  else String.sub s 0 limit ^ "?(truncated)"

let run_smoke_route
    (t : t)
    ~(path : string)
    ~(meth : http_method)
    ~(params : (string * string) list)
    ~(contains : string option) : string Lwt.t =
  let path = String.trim path in
  let meth_s = match meth with `GET -> "GET" | `POST -> "POST" in
  match Hashtbl.find_opt t.routes path with
  | None ->
      Lwt.return
        (Printf.sprintf
           "ok: false\nerror: no route registered for %s\n(hint:
     register_route path=%s handler=...)"
           path path)
  | Some (handler, _vis) ->
      (match Pricklypear_lisp.Env.lookup t.env handler with
       | None ->
           Lwt.return
             (Printf.sprintf
                "ok: false\nroute: %s -> %s\nerror: handler unbound in env"
                path handler)
       | Some _ ->
           let req =
             { method_ = meth
             ; path
             ; params
             ; body = ""
             ; headers =
                 (match meth with
                  | `POST ->
                      [ ("content-type",
     "application/x-www-form-urlencoded") ]
                  | `GET -> [])
             }
           in
           t.current_request <- Some req;
           let* v_res =
             Lwt.finalize
               (fun () ->
                  Lwt.catch
                    (fun () ->
                       let* v =
                         Pricklypear_lisp.Eval.eval_string ~prims:t.prims
     t.env
                           ("(" ^ handler ^ ")")
                       in
                       Lwt.return (`Ok v))
                    (fun exn ->
                       Lwt.return
                         (`Err ("handler error: " ^ pp_interp_exn exn))))
               (fun () ->
                  t.current_request <- None;
                  Lwt.return_unit)
           in
           (match v_res with
            | `Err msg ->
                Lwt.return
                  (Printf.sprintf
                     "ok: false\nroute: %s -> %s\nmethod: %s\nerror: %s"
```

```
                    path handler meth_s msg)
          | `Ok v ->
              let kind, body = value_smoke_kind v in
              let contains_ok =
                match contains with
                | None | Some "" -> true
                | Some needle ->
                    let nlen = String.length needle in
                    let blen = String.length body in
                    let rec find i =
                      if i + nlen > blen then false
                      else if String.sub body i nlen = needle then true
                      else find (i + 1)
                    in
                    find 0
              in
              let ok = contains_ok in
              let snippet = truncate_for_tool ~limit:500 body in
              let lines =
                [ Printf.sprintf "ok: %s" (if ok then "true" else "false")
                ; Printf.sprintf "route: %s -> %s" path handler
                ; Printf.sprintf "method: %s" meth_s
                ; Printf.sprintf "kind: %s" kind
                ; Printf.sprintf "bytes: %d" (String.length body)
                ]
              in
              let lines =
                match contains with
                | None | Some "" -> lines
                | Some needle ->
                    lines
                    @ [ Printf.sprintf "contains %S: %s" needle
                          (if contains_ok then "true" else "false") ]
              in
              let lines =
                if not ok then
                  lines
                  @ [ "error: smoke failed -- inspect_function, fix,
    re-smoke" ]
                else lines
              in
              Lwt.return
                (String.concat "\n" lines ^ "\n--- snippet ---\n" ^
    snippet)))

let run_agent_tool (t : t) ~(name : string) ~(arguments : string)
    : string Lwt.t =
  let args_json =
    try Yojson.Safe.from_string arguments with _ -> `Assoc []
  in
  match name with
  | "list_bindings" ->
      let names =
        try
          match Primitives.prim_list_bindings_fn ~env:t.env [] with
          | Pricklypear_lisp.Ast.VList xs ->
              List.filter_map
                (function Pricklypear_lisp.Ast.VStr s -> Some s | _ ->
    None) xs
              |> List.sort String.compare
          | _ -> []
        with _ -> []
      in
      Lwt.return (if names = [] then "(no bindings)" else String.concat
    "\n" names)
  | "list_routes" ->
      let rs =
        Hashtbl.fold
          (fun route (handler, vis) acc ->
            (Printf.sprintf "%s -> %s [%s]" route handler vis) :: acc)
          t.routes []
        |> List.sort String.compare
      in
```

```
      Lwt.return (if rs = [] then "(no routes)" else String.concat "\n" rs)
  | "inspect_function" ->
      (match json_string_field args_json "name" with
       | None -> Lwt.return "error: need name"
       | Some n ->
           let* pretty = current_fn_pretty t n in
           Lwt.return pretty)
  | "install_function" ->
      (match json_string_field args_json "name", json_string_field
     args_json "body" with
       | Some n, Some body ->
           let* v = install_ai_function t ~name:n ~body_src:body in
           Lwt.return (pp_value_short v)
       | _ -> Lwt.return "error: need name and body")
  | "register_route" ->
      (match json_string_field args_json "path", json_string_field
     args_json "handler" with
       | Some path, Some handler ->
           let path = String.trim path in
           let handler = String.trim handler in
           let bad_prefix =
             List.exists
               (fun p ->
                 path = p
                 || (String.length path > String.length p
                     && String.sub path 0 (String.length p + 1) = p ^ "/"))
               [ "/api"; "/admin"; "/login"; "/logout"; "/health"; "/agent"
     ]
           in
           if path = "" || path.[0] <> '/' then
             Lwt.return "error: path must start with /"
           else if bad_prefix then
             Lwt.return ("error: path reserved: " ^ path)
           else if handler = "" then
             Lwt.return "error: need handler function name"
           else begin
             let vis =
               match json_string_field args_json "visibility" with
               | Some v -> Store.normalize_visibility v
               | None -> "private"
             in
             Hashtbl.replace t.routes path (handler, vis);
             let* _ =
               Store.register_route t.store ~route:path ~handler
                 ~visibility:vis ()
             in
             Lwt.return
               (Printf.sprintf "registered route %s -> %s (%s)"
                  path handler vis)
           end
       | _ -> Lwt.return "error: need path and handler")
  | "eval_expr" ->
      (match json_string_field args_json "expr" with
       | None -> Lwt.return "error: need expr"
       | Some expr ->
           let expr = String.trim expr in
           if expr = "" then Lwt.return "error: empty expr"
           else if String.length expr > 4000 then
             Lwt.return "error: expr too long (max 4000)"
           else
             Lwt.catch
               (fun () ->
                  let* v =
                    Pricklypear_lisp.Eval.eval_string ~prims:t.prims t.env
     expr
                  in
                  let s = Pricklypear_lisp.Reader.pp_value v in
                  Lwt.return
                    ("ok: true\nvalue: " ^ truncate_for_tool ~limit:1500 s))
               (fun exn ->
                  Lwt.return ("ok: false\nerror: " ^ pp_interp_exn exn)))
  | "smoke_route" ->
      (match json_string_field args_json "path" with
```

```
      | None -> Lwt.return "error: need path"
      | Some path ->
          let meth =
            match json_string_field args_json "method" with
            | Some m when String.uppercase_ascii (String.trim m) = "POST"
    ->
                `POST
            | _ -> `GET
          in
          let params =
            match args_json with
            | `Assoc items ->
                (match List.assoc_opt "params" items with
                 | Some p -> json_object_string_pairs p
                 | None -> [])
            | _ -> []
          in
          let contains = json_string_field args_json "contains" in
          run_smoke_route t ~path ~meth ~params ~contains)
  | "html_helpers" ->
      Lwt.return html_helpers_cheatsheet
  | "image_context" ->
      let names =
        try
          match Primitives.prim_list_bindings_fn ~env:t.env [] with
          | Pricklypear_lisp.Ast.VList xs ->
              List.filter_map
                (function Pricklypear_lisp.Ast.VStr s -> Some s | _ -> None)
                xs
          | _ -> []
        with _ -> []
      in
      let nshow = List.length names in
      let sample = String.concat ", " (List.filteri (fun i _ -> i < 40)
    names) in
      Lwt.return
        (Printf.sprintf
           "user=%s\nbranch=%s\nbinding_count=%d\nbindings_sample=%s"
           t.manifest.username t.manifest.branch_name nshow sample)
  | other ->
      Lwt.return ("error: unknown tool " ^ other)

(** One durable tool-capable loop over an ai_sessions row.
    Loads history from Postgres, resubmits full messages[] each turn,
    appends user/assistant/tool rows as it goes. *)
let run_loop
    (t : t)
    ~(sess : Ai_store.session)
    ?(max_steps = 16)
    ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
    () : string Lwt.t =
  t.agent_branch_lock <- true;
  let open Llm_client in
  let installs : string list ref = ref [] in
  let rec loop step =
    if step > max_steps then begin
      let summary =
        match !installs with
        | [] ->
            "agent: max steps exceeded (no installs recorded). " ^
            "Try a smaller task or raise max-steps."
        | xs ->
            "agent: max steps exceeded after installs:\n- " ^
            String.concat "\n- " (List.rev xs) ^
            "\n(Partial progress kept on this branch and in the session.)"
      in
      on_event (Fail summary);
      Lwt.return summary
    end else
      let* history = Ai_store.list_messages t.store ~session_id:sess.id in
      let msgs = Ai_store.messages_for_api sess history in
      (* Last step: no tools -- force a plain-text wrap-up. *)
      let tools_now =
```

```
      if (not sess.tools) || step = max_steps then [] else agent_tool_defs
    in
    let* turn =
      chat_turn ~temperature:0.2 ~max_tokens:2048
        ~tools:tools_now ~messages:msgs ()
    in
    match turn with
    | Turn_error e ->
        on_event (Fail e);
        Lwt.return ("agent error: " ^ e)
    | Final text ->
        let* r =
          Ai_store.append_messages t.store ~session_id:sess.id
            [ Msg { role = Assistant; content = text } ]
        in
        (match r with
         | Error e ->
             on_event (Fail e);
             Lwt.return ("agent error: " ^ e)
         | Ok () ->
             on_event (Token text);
             on_event Done;
             Lwt.return text)
    | Need_tools { content; tool_calls } ->
        if tool_calls = [] then begin
          let fallback =
            match content with
            | Some c when String.trim c <> "" -> c
            | _ ->
                "agent: model requested tools but sent none; stopping."
          in
          let* _ =
            Ai_store.append_messages t.store ~session_id:sess.id
              [ Msg { role = Assistant; content = fallback } ]
          in
          on_event (Token fallback);
          on_event Done;
          Lwt.return fallback
        end else begin
          (match content with
           | Some c when String.trim c <> "" -> on_event (Token (c ^
"\n"))
           | _ -> ());
          let* r1 =
            Ai_store.append_messages t.store ~session_id:sess.id
              [ Assistant_tools { content; tool_calls } ]
          in
          match r1 with
          | Error e ->
              on_event (Fail e);
              Lwt.return ("agent error: " ^ e)
          | Ok () ->
              let* tool_rows =
                Lwt_list.map_s
                  (fun (tc : tool_call) ->
                    on_event (Token (Printf.sprintf "[tool %s]\n"
tc.name));
                    let* result =
                      run_agent_tool t ~name:tc.name
~arguments:tc.arguments
                    in
                    if tc.name = "install_function" then begin
                      let n =
                        match
                          try
                            json_string_field
                              (Yojson.Safe.from_string tc.arguments)
"name"
                          with _ -> None
                        with
                        | Some name -> name
                        | None -> "?"
                      in
```

```
                        installs := n :: !installs
                      end;
                      on_event
                        (Token
                           (Printf.sprintf "? %s\n"
                              (if String.length result > 200 then
                                 String.sub result 0 200 ^ "?"
                               else result)));
                      Lwt.return
                        (Tool_result
                           { tool_call_id = tc.id; content = result }))
                  tool_calls
              in
              let* r2 =
                Ai_store.append_messages t.store ~session_id:sess.id
    tool_rows
              in
              (match r2 with
               | Error e ->
                   on_event (Fail e);
                   Lwt.return ("agent error: " ^ e)
               | Ok () -> loop (step + 1))
        end
  in
  Lwt.finalize
    (fun () -> loop 1)
    (fun () ->
      t.agent_branch_lock <- false;
      Lwt.return_unit)

let default_agent_seed (t : t) : string =
  "You are the agent for a running Pricklypear multi-user Lisp OS image " ^
  "(software, not a cactus).\n" ^
  Printf.sprintf
    "LOCKED BRANCH: %s -- you see only the effective bindings on this
     branch "
    t.manifest.branch_name ^
  "(personal overrides shadow main; one name = one function).\n" ^
  "All install_function writes go to THIS branch only. " ^
  "Do NOT attempt checkout, fork, or merge.\n" ^
  "Tools: image_context, list_bindings, list_routes, inspect_function, " ^
  "install_function, register_route, smoke_route, eval_expr,
     html_helpers.\n" ^
  "\n" ^
  "How to build a small app (dogfood pattern -- copy todo/notes style):\n" ^
  "1) Persist with create/rows/get/update/delete on data_rows. " ^
  "Build fields JSON with (dict-set* \"{}\" (list \"name\" n \"email\" e
     ...)). " ^
  "Optional init: (lambda () (deftype 'contact 'name 'email 'phone
     'notes)).\n" ^
  "2) install_function name=pkg/page body=(lambda () ...) using html/* " ^
  "helpers: html/page, html/form method action attrs body, html/join, " ^
  "html/text-input, html/hidden, html/button. Never invent html/nil or " ^
  "multi-arg html/attr bags like (html/attr \"a\" \"b\" \"c\" \"d\").\n" ^
  "3) Row shape from (rows 'type): each row is (id . fields-json). " ^
  "Use (car row) for id, (cdr row) for fields, (dict-get fields
     \"name\").\n" ^
  "4) POST handlers: branch on (request-method) and (request-param
     \"action\"); " ^
  "after mutate, (redirect \"/path\") or (render-html ...) for htmx.\n" ^
  "5) NEVER use outline/* as a database. outline/* is for notes trees
     only.\n" ^
  "6) Routes: call register_route path=/pkg handler=pkg/page. " ^
  "Do NOT (route ...) inside install_function (publish effect is blocked
     there).\n" ^
  "7) Inspect working apps first: inspect_function todo/page, notes/page, "
     ^
  "todo/add, html/form.\n" ^
  "\n" ^
  "Test loop (required for apps):\n" ^
  "8) After register_route, smoke_route path=/app method=GET
     contains=Title.\n" ^
```

```
  "9) Data: eval_expr (count (pkg/all)); POST via smoke_route params then
    GET.\n" ^
  "10) If smoke fails: call html_helpers, inspect_function, fix
    install_function, re-smoke until ok:true.\n" ^
  "\n" ^
  "HTML: call html_helpers before install_function of *page/*form/*list*
    bodies.\n" ^
  "Tool discipline:\n" ^
  "- Use ONLY the structured tool API. Never write <tool_call> XML in
    text.\n" ^
  "- install_function body = one lambda s-expression only, no
    prose/markdown.\n" ^
  "- Batch multiple tool calls in one turn when possible.\n" ^
  "- Prefer package/name names (contacts/page).\n" ^
  "- When finished, SHORT text summary and NO tool calls.\n" ^
  "- Do not invent bindings; use list_bindings / inspect_function."

let agent_session_new
    (t : t)
    ~(seed : string)
    ?(tools = true)
    ?title
    () : Pricklypear_lisp.Ast.value Lwt.t =
  let* r =
    Ai_store.create_session t.store
      ~owner:t.manifest.username
      ~seed
      ?title
      ~branch_ref:t.manifest.branch_name
      ~tools
      ~model:(Llm_client.model ())
      ()
  in
  match r with
  | Ok s -> Lwt.return (Pricklypear_lisp.Ast.VStr s.id)
  | Error e -> Lwt.return (Pricklypear_lisp.Ast.VStr ("agent-session-new
    error: " ^ e))

let agent_say
    (t : t)
    ~(session_id : string)
    ~(prompt : string)
    ?(max_steps = 16)
    ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
    () : Pricklypear_lisp.Ast.value Lwt.t =
  let* sess_opt =
    Ai_store.get_session t.store
      ~owner:t.manifest.username
      ~id:session_id
  in
  match sess_opt with
  | None ->
      Lwt.return
        (Pricklypear_lisp.Ast.VStr
           ("agent-say: no session " ^ session_id ^ " for " ^
    t.manifest.username))
  | Some sess ->
      let title =
        match sess.title with
        | Some _ -> None
        | None ->
            let p =
              let s = String.trim prompt in
              let key = "\nUser:\n" in
              let rec find i =
                if i + String.length key > String.length s then None
                else if String.sub s i (String.length key) = key then Some i
                else find (i + 1)
              in
              match find 0 with
              | Some i ->
                  String.trim
                    (String.sub s (i + String.length key)
```

```
                        (String.length s - i - String.length key))
              | None -> s
            in
            if String.trim p = "" then None
            else
              Some
                (if String.length p > 80 then String.sub p 0 80 ^ "?" else
    p)
      in
      let* () =
        Ai_store.touch_session t.store ~id:sess.id ?title
          ~branch_ref:t.manifest.branch_name ()
      in
      let* ar =
        Ai_store.append_messages t.store ~session_id:sess.id
          [ Llm_client.Msg { role = Llm_client.User; content = prompt } ]
      in
      match ar with
      | Error e ->
          Lwt.return (Pricklypear_lisp.Ast.VStr ("agent-say error: " ^ e))
      | Ok () ->
          let* text =
            run_loop t ~sess ~max_steps ~on_event ()
          in
          Lwt.return (Pricklypear_lisp.Ast.VStr text)

let agent_history (t : t) ~(session_id : string)
    : Pricklypear_lisp.Ast.value Lwt.t =
  let* sess_opt =
    Ai_store.get_session t.store
      ~owner:t.manifest.username
      ~id:session_id
  in
  match sess_opt with
  | None ->
      Lwt.return
        (Pricklypear_lisp.Ast.VStr
           ("agent-history: no session " ^ session_id))
  | Some _ ->
      let* pairs =
        Ai_store.history_summaries t.store ~session_id
      in
      let rows =
        List.map
          (fun (role, content) ->
            Pricklypear_lisp.Ast.VList
              [ Pricklypear_lisp.Ast.VStr role
              ; Pricklypear_lisp.Ast.VStr content
              ])
          pairs
      in
      Lwt.return (Pricklypear_lisp.Ast.VList rows)

let agent_sessions (t : t) ?(limit = 50) ()
    : Pricklypear_lisp.Ast.value Lwt.t =
  let* xs =
    Ai_store.list_sessions t.store
      ~owner:t.manifest.username
      ~limit
      ()
  in
  let rows =
    List.map
      (fun (s : Ai_store.session) ->
        let title = Option.value s.title ~default:"" in
        Pricklypear_lisp.Ast.VList
          [ Pricklypear_lisp.Ast.VStr s.id
          ; Pricklypear_lisp.Ast.VStr title
          ; Pricklypear_lisp.Ast.VStr s.updated_at
          ])
      xs
  in
  Lwt.return (Pricklypear_lisp.Ast.VList rows)
```

```
1013
1014 let agent_session_delete (t : t) ~(session_id : string)
1015     : Pricklypear_lisp.Ast.value Lwt.t =
1016   let* ok =
1017     Ai_store.delete_session t.store
1018       ~owner:t.manifest.username
1019       ~id:session_id
1020   in
1021   Lwt.return
1022     (Pricklypear_lisp.Ast.VStr
1023        (if ok then "deleted" else "not found"))
1024
1025 (** Temporary alias: new session (tools on) + one say.
1026     Prefer (agent-session-new) + (agent-say). [stream] ignored. *)
1027 let agent
1028     (t : t)
1029     ?(stream = false)
1030     ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
1031     ~(seed : string)
1032     ~(prompt : string)
1033     () : Pricklypear_lisp.Ast.value Lwt.t =
1034   let _ = stream in
1035   let title_opt =
1036     let p = String.trim prompt in
1037     if p = "" then None
1038     else Some (if String.length p > 80 then String.sub p 0 80 ^ "?" else p)
1039   in
1040   let* created =
1041     agent_session_new t ~seed ~tools:true ?title:title_opt ()
1042   in
1043   match created with
1044   | Pricklypear_lisp.Ast.VStr sid
1045     when not (String.starts_with ~prefix:"agent-session-new error" sid) ->
1046       agent_say t ~session_id:sid ~prompt ~on_event ()
1047   | other -> Lwt.return other
1048
1049 (** Temporary alias: new tools session + one say. *)
1050 let agent_task
1051     (t : t)
1052     ?(max_steps = 16)
1053     ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
1054     ~(prompt : string)
1055     () : Pricklypear_lisp.Ast.value Lwt.t =
1056   let seed = default_agent_seed t in
1057   let title_opt =
1058     let p = String.trim prompt in
1059     if p = "" then None
1060     else Some (if String.length p > 80 then String.sub p 0 80 ^ "?" else p)
1061   in
1062   let* created =
1063     agent_session_new t ~seed ~tools:true ?title:title_opt ()
1064   in
1065   match created with
1066   | Pricklypear_lisp.Ast.VStr sid when not (String.starts_with
       ~prefix:"agent-session-new error" sid) ->
1067       agent_say t ~session_id:sid ~prompt ~max_steps ~on_event ()
1068   | other -> Lwt.return other
1069
1070
1071 let llm_rewrite (t : t) ~(mode : string) ~(name : string) ~(concept :
       string)
1072     : Pricklypear_lisp.Ast.value Lwt.t =
1073   let* current = current_fn_pretty t name in
1074   let* res =
1075     Llm_client.rewrite_component_source ~mode ~name ~concept
1076       ~current_pretty:current ()
1077   in
1078   match res with
1079   | Error e -> Lwt.return (Pricklypear_lisp.Ast.VStr ("ai error: " ^ e))
1080   | Ok body_src ->
1081       (match check_ai_body_effects body_src with
1082        | Error pe ->
1083            Lwt.return
```

```
            (Pricklypear_lisp.Ast.VStr
               (Printf.sprintf "ai rejected by effects: %s\n---\n%s" pe
    body_src))
      | Ok () ->
          let author = "agent:" ^ Llm_client.model () in
          let* def = define_binding t ~name ~body_src ~author in
          (match def with
           | Ok _ ->
               Lwt.return
                 (Pricklypear_lisp.Ast.VStr
                    (Printf.sprintf "froze %s (%s) by %s\n%s"
                       name mode author body_src))
           | Error e ->
               Lwt.return
                 (Pricklypear_lisp.Ast.VStr
                    (Printf.sprintf "freeze failed: %s\n---\n%s" e
    body_src))))

let llm_inner t ~name ~concept =
  llm_rewrite t ~mode:"replace" ~name ~concept

let llm_outer t ~name ~concept =
  llm_rewrite t ~mode:"wrap" ~name ~concept

(** Growth-from-seed: smallest core then recursive outer wraps. *)
let llm_seed (t : t) ~(requirement : string) ?(max_steps = 5) ()
    : Pricklypear_lisp.Ast.value Lwt.t =
  let tip = "seed" in
  let* seed_res = Llm_client.seed_core_source ~requirement () in
  match seed_res with
  | Error e -> Lwt.return (Pricklypear_lisp.Ast.VStr ("seed error: " ^ e))
  | Ok body_src ->
      (match check_ai_body_effects body_src with
       | Error pe ->
           Lwt.return
             (Pricklypear_lisp.Ast.VStr ("seed rejected: " ^ pe ^ "\n" ^
    body_src))
       | Ok () ->
           let author = "agent:" ^ Llm_client.model () in
           let* def = define_binding t ~name:tip ~body_src ~author in
           (match def with
            | Error e ->
                Lwt.return (Pricklypear_lisp.Ast.VStr ("seed freeze failed:
    " ^ e))
            | Ok _ ->
                let log = Buffer.create 256 in
                Buffer.add_string log (Printf.sprintf "step0 seed %s\n%s\n"
    tip body_src);
                let rec grow step =
                  if step > max_steps then
                    Lwt.return
                      (Pricklypear_lisp.Ast.VStr
                         (Buffer.contents log ^ "stopped: max-steps\n"))
                  else
                    let* pretty = current_fn_pretty t tip in
                    let* judge =
                      Llm_client.growth_judge ~requirement
    ~current_pretty:pretty
                        ~step ()
                    in
                    match judge with
                    | Error e ->
                        Lwt.return
                          (Pricklypear_lisp.Ast.VStr
                             (Buffer.contents log ^ "judge error: " ^ e))
                    | Ok line ->
                        let upper = String.uppercase_ascii (String.trim
    line) in
                        if String.length upper >= 4 && String.sub upper 0 4
    = "DONE"
                        then
                          Lwt.return
                            (Pricklypear_lisp.Ast.VStr
```

```
                              (Buffer.contents log ^ "DONE at step " ^
                               string_of_int step ^ "\n" ^ pretty))
                        else
                          let concept =
                            if String.length line >= 5
                               && String.sub (String.lowercase_ascii line)
    0 5 = "next:"
                            then String.trim (String.sub line 5
    (String.length line - 5))
                            else String.trim line
                          in
                          Buffer.add_string log
                            (Printf.sprintf "step%d NEXT %s\n" step
    concept);
                          let* wr =
                            llm_outer t ~name:tip ~concept
                          in
                          Buffer.add_string log
                            (Pricklypear_lisp.Reader.pp_value wr ^ "\n");
                          grow (step + 1)
                in
                grow 1))

(* Register the 5 CRUD primitives (get/create/update/delete/list) over
   the shared `data_rows` table. These are NORMAL prims on the Lwt-typed
   ABI (post-iteration-12): prim_fn = value list -> value Lwt.t. They
   return their real DB result through the promise chain -- e.g.
   (get 'user 42) returns the user's JSON fields as a VStr in `(str (get
   'user 42))`; `(mapcar (lambda (id) (get 'user id)) ids)` works.

   Per spec [storage.types] / migration 003: data_rows is shared across
   branches (no branch_id column), so these prims don't walk the
   recursive CTE -- they hit the table directly. Author is the session's
   current username (recorded on every create). *)
```

32 image/lib/ai_store.ml

```
(** Durable agent conversation store.

    Spec [ai-authoring.agent-portal]: ai_sessions + ai_messages.
    No LLM calls here -- maps rows <-> Llm_client.chat_message. *)
open Lwt.Syntax

type session = {
  id : string;
  owner : string;
  title : string option;
  seed : string;
  branch_ref : string option;
  tools : bool;
  model : string option;
  created_at : string;
  updated_at : string;
}

let pg_conn (store : Store.t) : Store.Pg.t option = Store.current_conn store

let cell_string row i =
  match List.nth_opt row i with
  | Some v -> Pgx.Value.to_string v
  | None -> None

let cell_bool row i =
  match cell_string row i with
  | Some ("t" | "true" | "TRUE" | "1") -> true
  | Some ("f" | "false" | "FALSE" | "0") -> false
  | _ -> false

let row_to_session row : session option =
  match
    ( cell_string row 0
    , cell_string row 1
    , cell_string row 2
    , cell_string row 3
    , cell_string row 4
    , cell_string row 6
    , cell_string row 7
    , cell_string row 8 )
  with
  | ( Some id
    , Some owner
    , title
    , Some seed
    , branch_ref
    , model
    , Some created_at
    , Some updated_at ) ->
      Some
        { id
        ; owner
        ; title
        ; seed
        ; branch_ref
        ; tools = cell_bool row 5
        ; model
        ; created_at
        ; updated_at
        }
  | _ -> None

let session_select_cols =
  "id::text, owner, title, seed, branch_ref, tools::text, model, \
   created_at::text, updated_at::text"

let create_session
    (store : Store.t)
    ~(owner : string)
    ~(seed : string)
```

```
    ?title
    ?branch_ref
    ?(tools = true)
    ?model
    () : (session, string) result Lwt.t =
  match pg_conn store with
  | None -> Lwt.return (Error "ai_store: no database")
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "INSERT INTO ai_sessions \
             (owner, title, seed, branch_ref, tools, model) \
             VALUES ($1, $2, $3, $4, $5, $6) \
             RETURNING " ^ session_select_cols
          in
          let* rows =
            Store.Pg.execute conn
              ~params:
                [ Pgx.Value.of_string owner
                ; (match title with
                   | Some t -> Pgx.Value.of_string t
                   | None -> Pgx.Value.null)
                ; Pgx.Value.of_string seed
                ; (match branch_ref with
                   | Some b -> Pgx.Value.of_string b
                   | None -> Pgx.Value.null)
                ; Pgx.Value.of_bool tools
                ; (match model with
                   | Some m -> Pgx.Value.of_string m
                   | None -> Pgx.Value.null)
                ]
              sql
          in
          match rows with
          | row :: _ ->
              (match row_to_session row with
               | Some s -> Lwt.return (Ok s)
               | None -> Lwt.return (Error "ai_store: create parse failed"))
          | [] -> Lwt.return (Error "ai_store: create returned no row"))
        (fun exn ->
          Lwt.return
            (Error ("ai_store create: " ^ Printexc.to_string exn)))

let get_session (store : Store.t) ~(owner : string) ~(id : string)
    : session option Lwt.t =
  match pg_conn store with
  | None -> Lwt.return_none
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT " ^ session_select_cols ^
            " FROM ai_sessions WHERE id = $1::uuid AND owner = $2 LIMIT 1"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:
                [ Pgx.Value.of_string id; Pgx.Value.of_string owner ]
              sql
          in
          match rows with
          | row :: _ -> Lwt.return (row_to_session row)
          | [] -> Lwt.return_none)
        (fun exn ->
          Printf.eprintf "[ai_store] get_session: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return_none)

let list_sessions (store : Store.t) ~(owner : string) ?(limit = 50) ()
    : session list Lwt.t =
  match pg_conn store with
  | None -> Lwt.return []
```

```
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT " ^ session_select_cols ^
            " FROM ai_sessions WHERE owner = $1 \
             ORDER BY updated_at DESC LIMIT $2"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:
                [ Pgx.Value.of_string owner
                ; Pgx.Value.of_string (string_of_int limit)
                ]
              sql
          in
          Lwt.return (List.filter_map row_to_session rows))
        (fun exn ->
          Printf.eprintf "[ai_store] list_sessions: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return [])

let delete_session (store : Store.t) ~(owner : string) ~(id : string)
    : bool Lwt.t =
  match pg_conn store with
  | None -> Lwt.return false
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "DELETE FROM ai_sessions \
             WHERE id = $1::uuid AND owner = $2 \
             RETURNING id::text"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:
                [ Pgx.Value.of_string id; Pgx.Value.of_string owner ]
              sql
          in
          Lwt.return (rows <> []))
        (fun exn ->
          Printf.eprintf "[ai_store] delete_session: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return false)

let touch_session
    (store : Store.t)
    ~(id : string)
    ?title
    ?branch_ref
    () : unit Lwt.t =
  match pg_conn store with
  | None -> Lwt.return_unit
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "UPDATE ai_sessions SET \
             updated_at = now(), \
             title = COALESCE($2, title), \
             branch_ref = COALESCE($3, branch_ref) \
             WHERE id = $1::uuid"
          in
          let* _ =
            Store.Pg.execute conn
              ~params:
                [ Pgx.Value.of_string id
                ; (match title with
                   | Some t -> Pgx.Value.of_string t
                   | None -> Pgx.Value.null)
                ; (match branch_ref with
                   | Some b -> Pgx.Value.of_string b
```

```
                  | None -> Pgx.Value.null)
               ]
             sql
         in
         Lwt.return_unit)
       (fun exn ->
         Printf.eprintf "[ai_store] touch_session: %s\n%!"
           (Printexc.to_string exn);
         Lwt.return_unit)

(** Encode tool_calls list as JSON for jsonb column. *)
let tool_calls_to_json (tcs : Llm_client.tool_call list) : string =
  Yojson.Safe.to_string
    (`List
       (List.map
          (fun (tc : Llm_client.tool_call) ->
            `Assoc
              [ ("id", `String tc.id)
              ; ("name", `String tc.name)
              ; ("arguments", `String tc.arguments)
              ])
          tcs))

let tool_calls_of_json_string (s : string) : Llm_client.tool_call list =
  try
    match Yojson.Safe.from_string s with
    | `List xs ->
        List.filter_map
          (function
            | `Assoc items ->
                let id =
                  match List.assoc_opt "id" items with
                  | Some (`String x) -> Some x
                  | _ -> None
                in
                let name =
                  match List.assoc_opt "name" items with
                  | Some (`String x) -> Some x
                  | _ -> None
                in
                let arguments =
                  match List.assoc_opt "arguments" items with
                  | Some (`String x) -> x
                  | Some other -> Yojson.Safe.to_string other
                  | None -> "{}"
                in
                (match id, name with
                 | Some id, Some name ->
                     Some ({ id; name; arguments } : Llm_client.tool_call)
                 | _ -> None)
            | _ -> None)
          xs
    | _ -> []
  with _ -> []

let chat_message_to_row (m : Llm_client.chat_message)
    : (string * string option * string option * string option) option =
  match m with
  | Llm_client.Msg { role = Llm_client.System; _ } ->
      (* System is session.seed only -- never stored. *)
      None
  | Llm_client.Msg { role = Llm_client.User; content } ->
      Some ("user", Some content, None, None)
  | Llm_client.Msg { role = Llm_client.Assistant; content } ->
      Some ("assistant", Some content, None, None)
  | Llm_client.Msg { role = Llm_client.Tool; content } ->
      (* Bare tool role without tool_call_id is incomplete; still store. *)
      Some ("tool", Some content, None, None)
  | Llm_client.Assistant_tools { content; tool_calls } ->
      Some
        ( "assistant"
        , content
        , Some (tool_calls_to_json tool_calls)
```

```
        , None )
  | Llm_client.Tool_result { tool_call_id; content } ->
      Some ("tool", Some content, None, Some tool_call_id)

let row_to_chat_message
    ~(role : string)
    ~(content : string option)
    ~(tool_calls : string option)
    ~(tool_call_id : string option) : Llm_client.chat_message option =
  match role with
  | "user" ->
      Some
        (Llm_client.Msg
           { role = Llm_client.User
           ; content = Option.value content ~default:""
           })
  | "assistant" ->
      (match tool_calls with
       | Some tc when String.trim tc <> "" && tc <> "null" ->
           let tcs = tool_calls_of_json_string tc in
           if tcs = [] then
             Some
               (Llm_client.Msg
                  { role = Llm_client.Assistant
                  ; content = Option.value content ~default:""
                  })
           else
             Some (Llm_client.Assistant_tools { content; tool_calls = tcs })
       | _ ->
           Some
             (Llm_client.Msg
                { role = Llm_client.Assistant
                ; content = Option.value content ~default:""
                }))
  | "tool" ->
      (match tool_call_id with
       | Some tid ->
           Some
             (Llm_client.Tool_result
                { tool_call_id = tid
                ; content = Option.value content ~default:""
                })
       | None ->
           Some
             (Llm_client.Msg
                { role = Llm_client.Tool
                ; content = Option.value content ~default:""
                }))
  | _ -> None

let next_seq (conn : Store.Pg.t) ~(session_id : string) : int Lwt.t =
  let sql =
    "SELECT COALESCE(MAX(seq), -1)::text FROM ai_messages \
     WHERE session_id = $1::uuid"
  in
  let* rows =
    Store.Pg.execute conn ~params:[ Pgx.Value.of_string session_id ] sql
  in
  match rows with
  | row :: _ ->
      (match cell_string row 0 with
       | Some s ->
           Lwt.return ((int_of_string_opt s |> Option.value ~default:(-1))
     + 1)
       | None -> Lwt.return 0)
  | [] -> Lwt.return 0

let list_messages (store : Store.t) ~(session_id : string)
    : Llm_client.chat_message list Lwt.t =
  match pg_conn store with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
```

```
        (fun () ->
          let sql =
            "SELECT role, content, tool_calls::text, tool_call_id \
             FROM ai_messages \
             WHERE session_id = $1::uuid \
             ORDER BY seq ASC"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:[ Pgx.Value.of_string session_id ]
              sql
          in
          let msgs =
            List.filter_map
              (fun row ->
                match cell_string row 0 with
                | None -> None
                | Some role ->
                    row_to_chat_message
                      ~role
                      ~content:(cell_string row 1)
                      ~tool_calls:(cell_string row 2)
                      ~tool_call_id:(cell_string row 3))
              rows
          in
          Lwt.return msgs)
        (fun exn ->
          Printf.eprintf "[ai_store] list_messages: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return [])

let append_messages
    (store : Store.t)
    ~(session_id : string)
    (msgs : Llm_client.chat_message list) : (unit, string) result Lwt.t =
  if msgs = [] then Lwt.return (Ok ())
  else
    match pg_conn store with
    | None -> Lwt.return (Error "ai_store: no database")
    | Some conn ->
        Lwt.catch
          (fun () ->
            let* start = next_seq conn ~session_id in
            let rec go i = function
              | [] -> Lwt.return (Ok ())
              | m :: rest ->
                  (match chat_message_to_row m with
                   | None -> go i rest (* skip system *)
                   | Some (role, content, tool_calls, tool_call_id) ->
                       let sql =
                         "INSERT INTO ai_messages \
                          (session_id, seq, role, content, tool_calls,
    tool_call_id) \
                          VALUES ($1::uuid, $2, $3, $4, $5::jsonb, $6)"
                       in
                       let* _ =
                         Store.Pg.execute conn
                           ~params:
                             [ Pgx.Value.of_string session_id
                             ; Pgx.Value.of_string (string_of_int i)
                             ; Pgx.Value.of_string role
                             ; (match content with
                                | Some c -> Pgx.Value.of_string c
                                | None -> Pgx.Value.null)
                             ; (match tool_calls with
                                | Some t -> Pgx.Value.of_string t
                                | None -> Pgx.Value.null)
                             ; (match tool_call_id with
                                | Some t -> Pgx.Value.of_string t
                                | None -> Pgx.Value.null)
                             ]
                           sql
                       in
```

```
                        go (i + 1) rest)
            in
            let* r = go start msgs in
            (match r with
             | Error _ as e -> Lwt.return e
             | Ok () ->
                 let* () = touch_session store ~id:session_id () in
                 Lwt.return (Ok ()))
          (fun exn ->
            Lwt.return
              (Error ("ai_store append: " ^ Printexc.to_string exn)))

(** Build the full messages list for a provider call:
    system seed (not stored) + durable history. *)
let messages_for_api (sess : session) (history : Llm_client.chat_message
     list)
    : Llm_client.chat_message list =
  Llm_client.Msg { role = Llm_client.System; content = sess.seed }
  :: history

(** Pretty history for Lisp/HTTP (no tool_calls detail required). *)
let history_summaries (store : Store.t) ~(session_id : string)
    : (string * string) list Lwt.t =
  let* msgs = list_messages store ~session_id in
  let rec go acc = function
    | [] -> Lwt.return (List.rev acc)
    | Llm_client.Msg { role; content } :: rest ->
        let r =
          match role with
          | Llm_client.User -> "user"
          | Llm_client.Assistant -> "assistant"
          | Llm_client.System -> "system"
          | Llm_client.Tool -> "tool"
        in
        go ((r, content) :: acc) rest
    | Llm_client.Assistant_tools { content; tool_calls } :: rest ->
        let names =
          String.concat ","
            (List.map (fun (t : Llm_client.tool_call) -> t.name) tool_calls)
        in
        let c =
          match content with
          | Some s when String.trim s <> "" -> s ^ " [tools: " ^ names ^ "]"
          | _ -> "[tools: " ^ names ^ "]"
        in
        go (("assistant", c) :: acc) rest
    | Llm_client.Tool_result { tool_call_id; content } :: rest ->
        go
          (("tool", tool_call_id ^ " ? " ^ content) :: acc)
          rest
  in
  go [] msgs
```

33 image/lib/auth_password.ml

```
(** Pricklypear Image -- interim password hashing.

    Complete system wants argon2id (auth.borg). Until argon2 is on the
    poohstack switch we use salted SHA-256:

      sha256$<salt_hex>$<digest_hex>

    Compatible with the dream-product-cut interim format. Upgrade path:
    rehash on successful login once argon2 is available. *)

let hex_encode (b : string) : string =
  let len = String.length b in
  let buf = Bytes.create (len * 2) in
  let hex = "0123456789abcdef" in
  for i = 0 to len - 1 do
    let c = Char.code b.[i] in
    Bytes.set buf (i * 2) hex.[c lsr 4];
    Bytes.set buf (i * 2 + 1) hex.[c land 0xf]
  done;
  Bytes.unsafe_to_string buf

let random_bytes (n : int) : string =
  Mirage_crypto_rng.generate n

let sha256_hex (s : string) : string =
  Digestif.SHA256.(digest_string s |> to_hex)

(** Constant-time string compare to avoid password/hash timing leaks. *)
let secure_equal (a : string) (b : string) : bool =
  let la = String.length a and lb = String.length b in
  let len = max la lb in
  let acc = ref (la lxor lb) in
  for i = 0 to len - 1 do
    let ca = if i < la then Char.code a.[i] else 0 in
    let cb = if i < lb then Char.code b.[i] else 0 in
    acc := !acc lor (ca lxor cb)
  done;
  !acc = 0

let hash_password ?(salt_len = 16) (password : string) : string =
  let salt = random_bytes salt_len in
  let salt_hex = hex_encode salt in
  let digest = sha256_hex (salt_hex ^ ":" ^ password) in
  Printf.sprintf "sha256$%s$%s" salt_hex digest

let verify_password ~(stored : string) (password : string) : bool =
  match String.split_on_char '$' stored with
  | ["sha256"; salt_hex; digest] when salt_hex <> "" && digest <> "" ->
      let candidate = sha256_hex (salt_hex ^ ":" ^ password) in
      secure_equal candidate digest
  | _ -> false
```

34 image/lib/auth_store.ml

```
(** Pricklypear Image -- in-process auth store (single-image Option B).

    Operates on migrations/002_auth.sql tables from the image process.
    No separate authd. Auth holds verify + mint only; session identity
    is attached per HTTP request (auth.multi-user-product / web-login). *)
open Lwt.Syntax

type user = {
  id : string;
  username : string;
  is_admin : bool;
}

type session_row = {
  id : string;
  user_id : string;
  username : string;
  is_admin : bool;
  expires_at : string;
}

let env_or name dft =
  match Sys.getenv_opt name with Some s -> s | None -> dft

let pg_conn (store : Store.t) : Store.Pg.t option = Store.current_conn store

let cell_string row i =
  match List.nth_opt row i with
  | Some v -> Pgx.Value.to_string v
  | None -> None

let cell_bool row i =
  match cell_string row i with
  | Some ("t" | "true" | "TRUE" | "1") -> true
  | Some ("f" | "false" | "FALSE" | "0") -> false
  | _ -> false

let log_auth (store : Store.t) ?user_id ~kind ~success ?source_ip ()
  : unit Lwt.t =
  match pg_conn store with
  | None -> Lwt.return_unit
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "INSERT INTO auth_log (user_id, kind, success, source_ip) " ^
            "VALUES ($1, $2, $3, $4)"
          in
          let* _ =
            Store.Pg.execute conn
              ~params:
                [ (match user_id with
                   | Some u -> Pgx.Value.of_string u
                   | None -> Pgx.Value.null);
                  Pgx.Value.of_string kind;
                  Pgx.Value.of_bool success;
                  (match source_ip with
                   | Some ip -> Pgx.Value.of_string ip
                   | None -> Pgx.Value.null) ]
              sql
          in
          Lwt.return_unit)
        (fun exn ->
          Printf.eprintf "[auth_store] log_auth: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return_unit)

let count_users (store : Store.t) : int Lwt.t =
  match pg_conn store with
  | None -> Lwt.return 0
  | Some conn ->
```

```
      Lwt.catch
        (fun () ->
          let* rows =
            Store.Pg.execute conn "SELECT count(*)::text FROM users"
          in
          match rows with
          | row :: _ ->
              (match cell_string row 0 with
               | Some s ->
                   Lwt.return
                     (int_of_string_opt s |> Option.value ~default:0)
               | None -> Lwt.return 0)
          | [] -> Lwt.return 0)
        (fun exn ->
          Printf.eprintf "[auth_store] count_users: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return 0)

let get_user_by_username (store : Store.t) (username : string)
  : user option Lwt.t =
  match pg_conn store with
  | None -> Lwt.return_none
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT id::text, username, is_admin::text FROM users " ^
            "WHERE username = $1 LIMIT 1"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:[ Pgx.Value.of_string username ] sql
          in
          match rows with
          | row :: _ ->
              (match cell_string row 0, cell_string row 1 with
               | Some id, Some uname ->
                   Lwt.return_some
                     { id; username = uname; is_admin = cell_bool row 2 }
               | _ -> Lwt.return_none)
          | [] -> Lwt.return_none)
        (fun exn ->
          Printf.eprintf "[auth_store] get_user_by_username: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return_none)

let get_user_by_id (store : Store.t) (user_id : string) : user option Lwt.t
    =
  match pg_conn store with
  | None -> Lwt.return_none
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT id::text, username, is_admin::text FROM users " ^
            "WHERE id = $1::uuid LIMIT 1"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
          match rows with
          | row :: _ ->
              (match cell_string row 0, cell_string row 1 with
               | Some id, Some uname ->
                   Lwt.return_some
                     { id; username = uname; is_admin = cell_bool row 2 }
               | _ -> Lwt.return_none)
          | [] -> Lwt.return_none)
        (fun exn ->
          Printf.eprintf "[auth_store] get_user_by_id: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return_none)
```

```
144
145 let create_user (store : Store.t) ~(username : string) ~(is_admin : bool)
146   : (user, string) result Lwt.t =
147   match pg_conn store with
148   | None -> Lwt.return (Error "no database")
149   | Some conn ->
150       Lwt.catch
151         (fun () ->
152           let sql =
153             "INSERT INTO users (username, is_admin) VALUES ($1, $2) " ^
154             "RETURNING id::text, username, is_admin::text"
155           in
156           let* rows =
157             Store.Pg.execute conn
158               ~params:
159                 [ Pgx.Value.of_string username;
160                   Pgx.Value.of_bool is_admin ]
161               sql
162           in
163           match rows with
164           | row :: _ ->
165               (match cell_string row 0, cell_string row 1 with
166                | Some id, Some uname ->
167                    Lwt.return
168                      (Ok
169                         { id; username = uname;
170                           is_admin = cell_bool row 2 })
171                | _ -> Lwt.return (Error "create_user: bad row"))
172           | [] -> Lwt.return (Error "create_user: no row"))
173         (fun exn ->
174           let msg = Printexc.to_string exn in
175           Printf.eprintf "[auth_store] create_user: %s\n%!" msg;
176           Lwt.return (Error msg))
177
178 let set_password (store : Store.t) ~(user_id : string) ~(password : string)
179   : (unit, string) result Lwt.t =
180   match pg_conn store with
181   | None -> Lwt.return (Error "no database")
182   | Some conn ->
183       Lwt.catch
184         (fun () ->
185           let hash = Auth_password.hash_password password in
186           let secret =
187             Yojson.Safe.to_string (`Assoc [ ("hash", `String hash) ])
188           in
189           (* One password credential per user: replace existing. *)
190           let* _ =
191             Store.Pg.execute conn
192               ~params:[ Pgx.Value.of_string user_id ]
193               "DELETE FROM credentials WHERE user_id = $1::uuid AND kind =
    'password'"
194           in
195           let sql =
196             "INSERT INTO credentials (user_id, kind, public_data,
    secret_data) " ^
197               "VALUES ($1::uuid, 'password', NULL, $2::jsonb)"
198           in
199           let* _ =
200             Store.Pg.execute conn
201               ~params:
202                 [ Pgx.Value.of_string user_id;
203                   Pgx.Value.of_string secret ]
204               sql
205           in
206           Lwt.return (Ok ()))
207         (fun exn ->
208           let msg = Printexc.to_string exn in
209           Printf.eprintf "[auth_store] set_password: %s\n%!" msg;
210           Lwt.return (Error msg))
211
212 let password_hash_for_user (store : Store.t) ~(username : string)
213   : (user * string) option Lwt.t =
214   match pg_conn store with
```

```
  | None -> Lwt.return_none
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT u.id::text, u.username, u.is_admin::text, " ^
            "       c.secret_data::text " ^
            "FROM users u " ^
            "JOIN credentials c ON c.user_id = u.id " ^
            "WHERE u.username = $1 AND c.kind = 'password' " ^
            "ORDER BY c.created_at DESC LIMIT 1"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:[ Pgx.Value.of_string username ] sql
          in
          match rows with
          | row :: _ ->
              (match
                 cell_string row 0, cell_string row 1, cell_string row 3
               with
               | Some id, Some uname, Some secret_json ->
                   let hash_opt =
                     try
                       match Yojson.Safe.from_string secret_json with
                       | `Assoc a ->
                           (match List.assoc_opt "hash" a with
                            | Some (`String h) -> Some h
                            | _ -> None)
                       | `String h -> Some h
                       | _ -> None
                     with _ -> None
                   in
                   (match hash_opt with
                    | Some h ->
                        Lwt.return_some
                          ( { id; username = uname;
                              is_admin = cell_bool row 2 },
                            h )
                    | None -> Lwt.return_none)
               | _ -> Lwt.return_none)
          | [] -> Lwt.return_none)
        (fun exn ->
          Printf.eprintf "[auth_store] password_hash_for_user: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return_none)

let verify_password (store : Store.t) ~(username : string)
    ~(password : string) ?source_ip () : user option Lwt.t =
  let* found = password_hash_for_user store ~username in
  match found with
  | None ->
      let* () =
        log_auth store ~kind:"password" ~success:false ?source_ip ()
      in
      Lwt.return_none
  | Some (user, stored) ->
      if Auth_password.verify_password ~stored password then
        let* () =
          log_auth store ~user_id:user.id ~kind:"password" ~success:true
            ?source_ip ()
        in
        (* Touch last_used_at *)
        (match pg_conn store with
         | None -> Lwt.return_some user
         | Some conn ->
             let* _ =
               Lwt.catch
                 (fun () ->
                   Store.Pg.execute conn
                     ~params:[ Pgx.Value.of_string user.id ]
                     "UPDATE credentials SET last_used_at = now() \
                      WHERE user_id = $1::uuid AND kind = 'password'")
```

```
                (fun _ -> Lwt.return [])
            in
            Lwt.return_some user)
      else
        let* () =
          log_auth store ~user_id:user.id ~kind:"password" ~success:false
            ?source_ip ()
        in
        Lwt.return_none

(** Session token: opaque random id. Cookie carries raw token; DB stores
    only SHA-256 hex of the token (auth.web-login). *)
let token_hash (raw_token : string) : string =
  Auth_password.sha256_hex raw_token

let mint_session
    (store : Store.t)
    ~(user_id : string)
    ?(ttl_seconds = 60 * 60 * 24 * 14)
    ?user_agent
    ?source_ip
    () : (string, string) result Lwt.t =
  match pg_conn store with
  | None -> Lwt.return (Error "no database")
  | Some conn ->
      Lwt.catch
        (fun () ->
          let raw =
            Base64.encode_string ~pad:false
              (Auth_password.random_bytes 32)
          in
          let th = token_hash raw in
          let sql =
            "INSERT INTO auth_sessions \
             (user_id, token_hash, expires_at) \
             VALUES ($1::uuid, $2, now() + ($3::text || '
    seconds')::interval) \
             RETURNING id::text"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:
                [ Pgx.Value.of_string user_id;
                  Pgx.Value.of_string th;
                  Pgx.Value.of_string (string_of_int ttl_seconds) ]
              sql
          in
          let* () =
            log_auth store ~user_id ~kind:"mint" ~success:true ?source_ip ()
          in
          ignore user_agent;
          match rows with
          | _ :: _ -> Lwt.return (Ok raw)
          | [] -> Lwt.return (Error "mint_session: no row"))
        (fun exn ->
          let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] mint_session: %s\n%!" msg;
          Lwt.return (Error msg))

let lookup_session (store : Store.t) ~(raw_token : string)
  : session_row option Lwt.t =
  if raw_token = "" then Lwt.return_none
  else
    match pg_conn store with
    | None -> Lwt.return_none
    | Some conn ->
        Lwt.catch
          (fun () ->
            let th = token_hash raw_token in
            let sql =
              "SELECT s.id::text, s.user_id::text, u.username, \
                      u.is_admin::text, s.expires_at::text \
               FROM auth_sessions s \
```

```
               JOIN users u ON u.id = s.user_id \
               WHERE s.token_hash = $1 \
                 AND s.revoked_at IS NULL \
                 AND s.expires_at > now() \
               LIMIT 1"
           in
           let* rows =
             Store.Pg.execute conn
               ~params:[ Pgx.Value.of_string th ] sql
           in
           match rows with
           | row :: _ ->
               (match
                  cell_string row 0,
                  cell_string row 1,
                  cell_string row 2,
                  cell_string row 4
                with
                | Some id, Some uid, Some uname, Some exp ->
                    Lwt.return_some
                      { id; user_id = uid; username = uname;
                        is_admin = cell_bool row 3;
                        expires_at = exp }
                | _ -> Lwt.return_none)
           | [] -> Lwt.return_none)
         (fun exn ->
           Printf.eprintf "[auth_store] lookup_session: %s\n%!"
             (Printexc.to_string exn);
           Lwt.return_none)

let revoke_session (store : Store.t) ~(raw_token : string) ?source_ip ()
  : unit Lwt.t =
  match pg_conn store with
  | None -> Lwt.return_unit
  | Some conn ->
      Lwt.catch
        (fun () ->
          let th = token_hash raw_token in
          let sql =
            "UPDATE auth_sessions SET revoked_at = now() \
             WHERE token_hash = $1 AND revoked_at IS NULL \
             RETURNING user_id::text"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:[ Pgx.Value.of_string th ] sql
          in
          match rows with
          | row :: _ ->
              let uid = cell_string row 0 in
              log_auth store ?user_id:uid ~kind:"logout" ~success:true
                ?source_ip ()
          | [] -> Lwt.return_unit)
        (fun exn ->
          Printf.eprintf "[auth_store] revoke_session: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return_unit)

let revoke_all_sessions (store : Store.t) ~(user_id : string) : int Lwt.t =
  match pg_conn store with
  | None -> Lwt.return 0
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "UPDATE auth_sessions SET revoked_at = now() \
             WHERE user_id = $1::uuid AND revoked_at IS NULL \
             RETURNING id"
          in
          let* rows =
            Store.Pg.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
```

```
          let* () =
            log_auth store ~user_id ~kind:"revoke_all" ~success:true ()
          in
          Lwt.return (List.length rows))
        (fun exn ->
          Printf.eprintf "[auth_store] revoke_all_sessions: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return 0)

let list_users (store : Store.t) : user list Lwt.t =
  match pg_conn store with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let* rows =
            Store.Pg.execute conn
              "SELECT id::text, username, is_admin::text \
               FROM users ORDER BY username"
          in
          let users =
            List.filter_map
              (fun row ->
                match cell_string row 0, cell_string row 1 with
                | Some id, Some uname ->
                    Some
                      { id; username = uname;
                        is_admin = cell_bool row 2 }
                | _ -> None)
              rows
          in
          Lwt.return users)
        (fun exn ->
          Printf.eprintf "[auth_store] list_users: %s\n%!"
            (Printexc.to_string exn);
          Lwt.return [])

(** First boot: if users table empty, create admin from env.
    PP_BOOTSTRAP_USER (default admin), PP_BOOTSTRAP_PASSWORD (default
     admin).
    If admin already exists but has no password credential, set one from env
    (dev recovery for pre-auth DBs that only had ssh_key). *)
let bootstrap_admin_if_empty (store : Store.t) : user option Lwt.t =
  let username = env_or "PP_BOOTSTRAP_USER" "admin" in
  let password = env_or "PP_BOOTSTRAP_PASSWORD" "admin" in
  let* n = count_users store in
  if n > 0 then
    let* admin = get_user_by_username store username in
    match admin with
    | None -> Lwt.return_none
    | Some user ->
        let* has_pw = password_hash_for_user store ~username in
        (match has_pw with
         | Some _ -> Lwt.return_none
         | None ->
             let* pw = set_password store ~user_id:user.id ~password in
             (match pw with
              | Error e ->
                  Printf.eprintf
                    "[auth_store] bootstrap password for existing %s:
    %s\n%!"
                    username e;
                  Lwt.return_none
              | Ok () ->
                  Printf.eprintf
                    "[auth_store] set password for existing admin %s\n%!"
                    username;
                  Lwt.return_some user))
  else
    let* created = create_user store ~username ~is_admin:true in
    match created with
    | Error e ->
        Printf.eprintf "[auth_store] bootstrap failed: %s\n%!" e;
```

```
      Lwt.return_none
  | Ok user ->
      let* pw = set_password store ~user_id:user.id ~password in
      (match pw with
       | Error e ->
           Printf.eprintf "[auth_store] bootstrap password: %s\n%!" e;
           Lwt.return_none
       | Ok () ->
           Printf.eprintf
             "[auth_store] bootstrapped admin user %s (set
   PP_BOOTSTRAP_PASSWORD)\n%!"
             username;
           let* () =
             log_auth store ~user_id:user.id ~kind:"bootstrap"
               ~success:true ()
           in
           Lwt.return_some user)
```

35 image/lib/boot.ml

```
(** Pricklypear Image -- Boot.

    Per spec [image.boot]:
      1. Read boot manifest.
      2. Loads function_defs and type_defs for branch_id from Postgres
         (TODO -- for minimal image, the env starts empty; lookups still
         hit the store on demand).
      3. Start the REPL endpoint (internal line protocol).
      4. Start the htmx server (httpaf-lwt-unix).
      5. Register health (handled by Http.serve's /health route).
      6. Mark self healthy, signal supervisor (TODO: signal via sessions
         table; minimal image just runs).

    Reads env vars PP_USER, PP_USERNAME, PP_BRANCH_ID, PP_BRANCH_NAME,
    PP_BOOT_MODE, PP_IMAGE_VERSION for the manifest. These are set by
    the supervisor when it spawns the image (or by [scripts/dev.sh] for
    standalone dev runs). *)
open Lwt.Syntax

let env_or name dft = match Sys.getenv_opt name with Some s -> s | None ->
     dft

let manifest_from_env () : Pricklypear_common.Manifest.t =
  let boot_mode =
    match env_or "PP_BOOT_MODE" "on-connect" with
    | "always-on" -> Pricklypear_common.Manifest.Boot_always_on
    | _ -> Pricklypear_common.Manifest.Boot_on_connect
  in
  Pricklypear_common.Manifest.make
    ~user_id:(env_or "PP_USER_ID" "dev-user")
    ~username:(env_or "PP_USERNAME" "dev")
    ~branch_id:(env_or "PP_BRANCH_ID"
                  "00000000-0000-0000-0000-000000000001")
    ~branch_name:(env_or "PP_BRANCH_NAME" "main")
    ~boot_mode
    ~image_version:(env_or "PP_IMAGE_VERSION" "0.1.0-dev")
    ()

let start () : unit Lwt.t =
  Printf.eprintf "[boot] Pricklypear image starting up...\n";
  (* RNG required for password salt + session tokens (auth). *)
  Mirage_crypto_rng_unix.use_default ();
  let manifest = manifest_from_env () in
  let cfg = Store.config_from_env () in
  let store = Store.make ~cfg ~branch_id:manifest.branch_id () in
  (* Try to connect to PG but don't fail boot if it's down -- the
     minimal image serves system pages with no DB. *)
  let* () = Store.connect store in
  (* First-boot admin when users table is empty (auth.bootstrap-admin). *)
  let* _ = Auth_store.bootstrap_admin_if_empty store in
  let session = Session.make ~manifest ~store () in
  let http_port = int_of_string (env_or "PP_HTTP_PORT" "8080") in
  let repl_port = int_of_string (env_or "PP_REPL_PORT" "7878") in
  let _ = Lwt.async (fun () -> Repl_endpoint.serve ~session ~port:repl_port
     ()) in
  Printf.eprintf "[boot] serving http://0.0.0.0:%d/  (htmx)\n" http_port;
  Printf.eprintf "[boot] REPL endpoint 127.0.0.1:%d (internal)\n" repl_port;
  Printf.eprintf "[boot] user=%s branch=%s version=%s\n"
    manifest.username manifest.branch_name manifest.image_version;
  (* Block on the HTTP server promise (never resolves until shutdown).
     The REPL endpoint runs in async. *)
  Http.serve ~session ~port:http_port ()
```

36 image/lib/crud.ml

```
(** Pricklypear Image -- CRUD primitives over the `data_rows` table.

    Each of get/create/update/delete/list is a NORMAL PRIMITIVE on the
    Lwt-typed evaluator ABI (post-iteration-12 refactor): prim_fn is
    `value list -> value Lwt.t`, so these prims return their real DB
    result as a promise. They compose normally:
      (define u (get 'user 42))
      (str u)
    `(get 'note id)` returning the note as its value is what unblocked
    CRUD on the new ABI.

    Per spec [storage.types] / migration 003:
      - `data_rows(id uuid PK, type_name text, fields jsonb, author text,
     ...)`
      - Data rows are SHARED across branches (no branch_id column);
        fork/merge only copies code, not data. All branches see the
        same data_rows.
      - The `fields` payload is a JSON document; the Lisp layer sees
        it as a string for now. A future iteration can add a parser
        that converts to/from Lisp assoc lists or a dict value type.
      - No ORM; plain parameterized SQL via pgx. SERIALIZABLE
        transactions by default for mutations -- once our pgx wrapper
        exposes BEGIN/COMMIT, wrap create/update/delete in a
        transaction. For now, each op is one statement (atomic by
        itself, multi-step ops would need wrapping).

    The store handle is shared with [Store.t]; we reuse its [Pg.t]
    via [Store.conn_of]. Returns VNil with no DB or no rows so the
    REPL prints `()` and Lisp code can do `(if (get 'user id) ...)`. *)
open Lwt.Syntax
open Pricklypear_lisp.Ast

module Pg = Store.Pg

let value_of_id cell : value =
  (* Pgx.Value.to_string returns string option; normalize to VStr/VNil *)
  match Pgx.Value.to_string cell with
  | Some s -> VStr s
  | None -> VNil

(** (deftype 'name field1 field2 ...) -> VStr "defined type <name>".
    INSERTs a row into type_defs with branch_id = session's branch and
    fields = JSON array ["field1","field2",...] (the field names in
    declared order). Per spec [storage.types]: type declarations are
    metadata; the actual data lives in the shared data_rows table
    (migration 003) keyed by (type_name, id). Type declarations may
    eventually drive field-level validation; for now they're documentation.

    The first arg is the type name, then zero or more field names --
    each as a quoted symbol (e.g. (deftype 'user 'name 'email 'age))
    so they arrive as VStr (quote of a bare symbol evaluates to VStr).
    Bare symbols would be looked up as variables and fail. *)
let deftype (store : Store.t) ~(args : value list)
    ~(author : string) : value Lwt.t =
  let field_strs =
    List.filter_map
      (fun v -> match v with VStr s -> Some s | _ -> None)
      args
  in
  (match field_strs with
   | [] -> Lwt.return (VStr "deftype: need at least a type name")
   | name :: fields ->
       let* _ = Store.persist_type store ~name ~fields ~author in
       Lwt.return (VStr (Printf.sprintf "defined type %s" name)))

(** (get 'type id) -> VStr fields-json | VNil.
    Reads ONE row by (type_name, id). The id is a UUID string.
    Returns the row's `fields` JSON as a VStr so Lisp can pass it
    onward; or VNil if not found. *)
let get (store : Store.t) ~(type_name : string) ~(id : string) : value
    Lwt.t =
```

```
  match Store.current_conn store with
  | None -> Lwt.return VNil
  | Some conn ->
      let sql =
        "SELECT fields::text FROM data_rows " ^
        "WHERE type_name = $1 AND id = $2::uuid" in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name;
                    Pgx.Value.of_string id ]
          sql
      in
      (match rows with
       | [] -> Lwt.return VNil
       | row :: _ ->
           (match row with
            | [ fields_cell ] ->
                (match Pgx.Value.to_string fields_cell with
                 | Some s -> Lwt.return (VStr s)
                 | None -> Lwt.return VNil)
            | _ -> Lwt.return VNil))

(** (select-for-update 'type id) -> VStr fields-json | VNil.
    Reads ONE row by (type_name, id) AND locks it with SELECT ... FOR
    UPDATE for the duration of the surrounding transaction.

    Per spec [storage.transactions]: "Explicit row locks:
    (select-for-update 'type id) Exposed as a primitive for finer
    control."

    IMPORTANT: This MUST be called inside a (with-transaction ...)
    block (or the prim-level equivalent) for the lock to have
    transactional scope. If called outside any transaction, pgx will
    wrap the single statement in an implicit transaction; the lock
    is released as soon as the statement returns (i.e. useless).
    Usage pattern (v1, pre-with-transaction prim -- i.e. today):
      (define u (select-for-update 'user id))  ; lock acquired
      (define u2 (dict-set u "active" "false"))
      (update 'user id u2)                      ; update within tx
    When the transaction commits/rolls back, the lock is released.

    In v1 (no with-transaction prim YET), this is essentially a
    hinted-get. Improper use outside a transaction is non-fatal but
    provides no concurrency protection. A future (with-transaction ...)
    prim will give this real meaning. *)
let select_for_update
    (store : Store.t) ~(type_name : string) ~(id : string) : value Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return VNil
  | Some conn ->
      let sql =
        "SELECT fields::text FROM data_rows " ^
        "WHERE type_name = $1 AND id = $2::uuid " ^
        "FOR UPDATE" in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name;
                    Pgx.Value.of_string id ]
          sql
      in
      (match rows with
       | [] -> Lwt.return VNil
       | row :: _ ->
           (match row with
            | [ fields_cell ] ->
                (match Pgx.Value.to_string fields_cell with
                 | Some s -> Lwt.return (VStr s)
                 | None -> Lwt.return VNil)
            | _ -> Lwt.return VNil))

(** (create 'type "{fields-json}") -> VStr new-id | VStr "create failed".
    INSERTs a row with the given JSON fields, returning the new UUID.
    The author is the session's current username.
```

```
143
144     Why take fields as a JSON STRING (not a Lisp value):
145     - The Lisp layer doesn't yet have a dict/assoc value type.
146     - Caller can build the JSON via (string-append ...) or a future
147       (json ...) prim; for now both caller and DB speak JSON strings.
148     - The DB validates it's well-formed JSON (jsonb column). *)
149 let create
150     (store : Store.t) ~(type_name : string) ~(fields_json : string)
151     ~(author : string)
152     : value Lwt.t =
153   match Store.current_conn store with
154   | None -> Lwt.return (VStr "create failed: no DB")
155   | Some conn ->
156       let sql =
157         "INSERT INTO data_rows (type_name, fields, author) " ^
158         "VALUES ($1, $2::jsonb, $3) RETURNING id::text" in
159       let* rows =
160         Pg.execute conn
161           ~params:[ Pgx.Value.of_string type_name;
162                     Pgx.Value.of_string fields_json;
163                     Pgx.Value.of_string author ]
164           sql
165       in
166       (match rows with
167        | [] -> Lwt.return (VStr "create failed")
168        | row :: _ ->
169            (match row with
170             | [ id_cell ] -> Lwt.return (value_of_id id_cell)
171             | _ -> Lwt.return (VStr "create failed")))
172
173 (** (update 'type id "{fields-json}") -> VStr id | VNil.
174     UPDATE fields for ONE row by (type_name, id). Returns the id
175     (echo of the row updated) on success, or VNil if no row matched. *)
176 let update
177     (store : Store.t) ~(type_name : string) ~(id : string)
178     ~(fields_json : string)
179     : value Lwt.t =
180   match Store.current_conn store with
181   | None -> Lwt.return VNil
182   | Some conn ->
183       let sql =
184         "UPDATE data_rows SET fields = $3::jsonb, updated_at = now() " ^
185         "WHERE type_name = $1 AND id = $2::uuid " ^
186         "RETURNING id::text" in
187       let* rows =
188         Pg.execute conn
189           ~params:[ Pgx.Value.of_string type_name;
190                     Pgx.Value.of_string id;
191                     Pgx.Value.of_string fields_json ]
192           sql
193       in
194       (match rows with
195        | [] -> Lwt.return VNil
196        | row :: _ ->
197            (match row with
198             | [ id_cell ] -> Lwt.return (value_of_id id_cell)
199             | _ -> Lwt.return VNil))
200
201 (** (update-field 'type id "field" "value") -> VStr id | VNil.
202     Ergonomic shorthand for the v1 dict-set + update round-trip:
203       (define u (get 'type id))
204       (update 'type id (dict-set u "field" "value"))
205     -> (update-field 'type id "field" "value")
206     Composition is pure-OCaml (one get, one dict-set, one update inside
207     this prim). Lwt-typed; returns VStr id on success, VNil if no row
208     matched or no DB. Value is JSON-coerced (numeric strings emit as JSON
209     numbers; non-parseable values emit as JSON strings -- same rules as
210     dict-set). NO transactional scope beyond the implicit per-statement
211     pgx tx -- see ``select-for-update`` caveat for why that matters for
212     concurrent writers; the row is fetched and updated in two separate
213     pgx statements. *)
214 let update_field
215     (store : Store.t) ~(type_name : string) ~(id : string)
```

```
    ~(field : string) ~(value : string)
    : value Lwt.t =
  let* current = get store ~type_name ~id in
  (match current with
   | VNil -> Lwt.return VNil  (* no such row *)
   | _ ->
       let json_str =
         match current with VStr s -> s | _ -> "{}"
       in
       (* Compose with the SAME logic as Primitives.prim_dict_set so
          numeric-auto-detect and JSON passthrough behave identically. *)
       let base =
         try
           (match Yojson.Safe.from_string json_str with
            | `Assoc kvs -> kvs
            | _ -> [])
         with _ -> []
       in
       let v_json =
         try Yojson.Safe.from_string value
         with _ -> `String value
       in
       let updated =
         (field, v_json) ::
         (List.filter (fun (k, _) -> k <> field) base)
       in
       let updated_json = Yojson.Safe.to_string (`Assoc updated) in
       update store ~type_name ~id ~fields_json:updated_json)

(** (delete 'type id) -> VStr id | VNil.
    DELETEs one row by (type_name, id). Returns the id on success
    (echo), VNil if no row matched. *)
let delete (store : Store.t) ~(type_name : string) ~(id : string) : value
    Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return VNil
  | Some conn ->
      let sql =
        "DELETE FROM data_rows " ^
        "WHERE type_name = $1 AND id = $2::uuid " ^
        "RETURNING id::text" in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name;
                    Pgx.Value.of_string id ]
          sql
      in
      (match rows with
       | [] -> Lwt.return VNil
       | row :: _ ->
           (match row with
            | [ id_cell ] -> Lwt.return (value_of_id id_cell)
            | _ -> Lwt.return VNil))

(** (list 'type [limit]) -> VList of (VStr id, VStr fields) pairs.
    With no limit, defaults to 100 (avoid OOM on a huge table).
    Each row is a 2-element VList `[id fields]` so callers can
    `(mapcar (lambda (row) (car row)) rows)` to get ids, etc. *)
let list
    (store : Store.t) ~(type_name : string) ?(limit : int = 100) ()
    : value Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return (VList [])
  | Some conn ->
      let sql =
        "SELECT id::text, fields::text FROM data_rows " ^
        "WHERE type_name = $1 ORDER BY created_at ASC LIMIT $2" in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name;
                    Pgx.Value.of_int limit ]
          sql
      in
```

```
      let lisp_rows =
        List.map
          (fun row ->
            match row with
            | [ id_cell; fields_cell ] ->
                VPair (value_of_id id_cell, value_of_id fields_cell)
            | _ -> VNil)
          rows
      in
      Lwt.return (VList lisp_rows)

(** (find-row 'type "field" "value") -> VStr fields-json | VNil.
    First row where fields->>field = value (text equality). *)
let find_row
    (store : Store.t) ~(type_name : string) ~(field : string) ~(value :
     string)
    : value Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return VNil
  | Some conn ->
      let sql =
        "SELECT fields::text FROM data_rows " ^
        "WHERE type_name = $1 AND fields->>$2 = $3 " ^
        "ORDER BY created_at ASC LIMIT 1"
      in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name
                  ; Pgx.Value.of_string field
                  ; Pgx.Value.of_string value ]
          sql
      in
      (match rows with
       | [] -> Lwt.return VNil
       | row :: _ ->
           (match row with
            | [ fields_cell ] -> Lwt.return (value_of_id fields_cell)
            | _ -> Lwt.return VNil))

(** (find-rows 'type "f1" "v1" "f2" "v2" ... [limit])
    -> VList of (id . fields) pairs.
    All field filters AND together (fields->>k = v text). Optional
    trailing VInt limit (default 500). Orders by created_at ASC. *)
let find_rows
    (store : Store.t) ~(type_name : string)
    ~(filters : (string * string) list) ?(limit : int = 500) ()
    : value Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return (VList [])
  | Some conn ->
      if filters = [] then list store ~type_name ~limit ()
      else
        let buf = Buffer.create 128 in
        Buffer.add_string buf
          "SELECT id::text, fields::text FROM data_rows WHERE type_name =
    $1";
        let params = ref [ Pgx.Value.of_string type_name ] in
        let i = ref 2 in
        List.iter
          (fun (field, value) ->
            Buffer.add_string buf
              (Printf.sprintf " AND fields->>$%d = $%d" !i (!i + 1));
            params := !params @ [ Pgx.Value.of_string field
                                ; Pgx.Value.of_string value ];
            i := !i + 2)
          filters;
        Buffer.add_string buf
          (Printf.sprintf
             " ORDER BY COALESCE((fields->>'verse')::int, 0) ASC,
    created_at ASC LIMIT $%d"
             !i);
        params := !params @ [ Pgx.Value.of_int limit ];
        let* rows = Pg.execute conn ~params:!params (Buffer.contents buf) in
```

```
        let lisp_rows =
          List.map
            (fun row ->
              match row with
              | [ id_cell; fields_cell ] ->
                  VPair (value_of_id id_cell, value_of_id fields_cell)
              | _ -> VNil)
            rows
        in
        Lwt.return (VList lisp_rows)

(** (find-random 'type) -> (id . fields) pair | VNil.
    One random row of the given type (for bible/random etc.). *)
let find_random (store : Store.t) ~(type_name : string) : value Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return VNil
  | Some conn ->
      let sql =
        "SELECT id::text, fields::text FROM data_rows " ^
        "WHERE type_name = $1 ORDER BY random() LIMIT 1"
      in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name ]
          sql
      in
      (match rows with
       | [] -> Lwt.return VNil
       | row :: _ ->
           (match row with
            | [ id_cell; fields_cell ] ->
                Lwt.return
                  (VPair (value_of_id id_cell, value_of_id fields_cell))
            | _ -> Lwt.return VNil))

(** (find-text 'type "field" "needle" [limit]) -> VList of (id . fields).
    Case-insensitive substring match on fields->>field. *)
let find_text
    (store : Store.t) ~(type_name : string) ~(field : string)
    ~(needle : string) ?(limit : int = 50) ()
    : value Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return (VList [])
  | Some conn ->
      let sql =
        "SELECT id::text, fields::text FROM data_rows " ^
        "WHERE type_name = $1 AND fields->>$2 ILIKE $3 " ^
        "ORDER BY created_at ASC LIMIT $4"
      in
      let pat = "%" ^ needle ^ "%" in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name
                  ; Pgx.Value.of_string field
                  ; Pgx.Value.of_string pat
                  ; Pgx.Value.of_int limit ]
          sql
      in
      let lisp_rows =
        List.map
          (fun row ->
            match row with
            | [ id_cell; fields_cell ] ->
                VPair (value_of_id id_cell, value_of_id fields_cell)
            | _ -> VNil)
          rows
      in
      Lwt.return (VList lisp_rows)

(* ---- OCaml UI helpers (image.data-browser) -- not Lisp prims ---- *)

type row_record = {
  id : string;
```

```
  fields_json : string;
  author : string option;
  created_at : string option;
  updated_at : string option;
}

let count_by_type (store : Store.t) ~(type_name : string) : int Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return 0
  | Some conn ->
      let sql =
        "SELECT COUNT(*)::int FROM data_rows WHERE type_name = $1"
      in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name ]
          sql
      in
      (match rows with
       | [ cell :: _ ] ->
           (match Pgx.Value.to_int cell with
            | Some n -> Lwt.return n
            | None -> Lwt.return 0)
       | _ -> Lwt.return 0)

let list_data_type_names (store : Store.t) : string list Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return []
  | Some conn ->
      let sql =
        "SELECT DISTINCT type_name FROM data_rows ORDER BY type_name ASC"
      in
      let* rows = Pg.execute conn sql in
      Lwt.return
        (List.filter_map
           (fun row ->
             match row with
             | [ cell ] -> Pgx.Value.to_string cell
             | _ -> None)
           rows)

let list_page
    (store : Store.t) ~(type_name : string)
    ?(limit : int = 50) ?(offset : int = 0) ()
    : row_record list Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return []
  | Some conn ->
      let lim = max 1 (min limit 200) in
      let off = max 0 offset in
      let sql =
        "SELECT id::text, fields::text, author, " ^
        "created_at::text, updated_at::text " ^
        "FROM data_rows WHERE type_name = $1 " ^
        "ORDER BY created_at ASC LIMIT $2 OFFSET $3"
      in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name
                  ; Pgx.Value.of_int lim
                  ; Pgx.Value.of_int off ]
          sql
      in
      Lwt.return
        (List.filter_map
           (fun row ->
             match row with
             | [ id_c; fields_c; author_c; created_c; updated_c ] ->
                 (match Pgx.Value.to_string id_c,
                        Pgx.Value.to_string fields_c with
                  | Some id, Some fields_json ->
                      Some {
                        id;
```

```
                        fields_json;
                        author = Pgx.Value.to_string author_c;
                        created_at = Pgx.Value.to_string created_c;
                        updated_at = Pgx.Value.to_string updated_c;
                      }
                | _ -> None)
            | _ -> None)
          rows)

let get_row
    (store : Store.t) ~(type_name : string) ~(id : string)
    : row_record option Lwt.t =
  match Store.current_conn store with
  | None -> Lwt.return None
  | Some conn ->
      let sql =
        "SELECT id::text, fields::text, author, " ^
        "created_at::text, updated_at::text " ^
        "FROM data_rows WHERE type_name = $1 AND id = $2::uuid"
      in
      let* rows =
        Pg.execute conn
          ~params:[ Pgx.Value.of_string type_name
                  ; Pgx.Value.of_string id ]
          sql
      in
      (match rows with
       | [ id_c; fields_c; author_c; created_c; updated_c ] :: _ ->
           (match Pgx.Value.to_string id_c, Pgx.Value.to_string fields_c
    with
            | Some id, Some fields_json ->
                Lwt.return
                  (Some {
                     id;
                     fields_json;
                     author = Pgx.Value.to_string author_c;
                     created_at = Pgx.Value.to_string created_c;
                     updated_at = Pgx.Value.to_string updated_c;
                   })
            | _ -> Lwt.return None)
       | _ -> Lwt.return None)
```

37 image/lib/http.ml

```
(** Pricklypear Image -- HTTP server entry: route dispatch + serve.

    System pages + browser product surfaces:
      /  /health  /branches  /bindings  /types  /data ?
      /repl  /api/eval  /edit  /inspect  /agent
      user routes via (route ...)
*)
open Lwt.Syntax
open Pure_html
open Http_core
open Http_data
open Http_branches
open Http_agent
open Http_pages

let request_handler (session : Session.t) (_sockaddr : Unix.sockaddr)
    (reqd : Httpaf.Reqd.t) : unit =
  let req = Httpaf.Reqd.request reqd in
  let target = req.Httpaf.Request.target in
  let path =
    match String.index_opt target '?' with
    | Some i -> String.sub target 0 i
    | None -> target
  in
  let meth = req.Httpaf.Request.meth in
  let headers = header_list req in
  (* Thunk page: Lwt promises are eager -- must not start page work
     before auth middleware decides proceed (auth.route-visibility). *)
  let launch (page_fn : unit -> unit Lwt.t) : unit =
    Lwt.async (fun () ->
      Lwt_mutex.with_lock request_mutex (fun () ->
        Lwt.catch
          (fun () ->
            let* proceed =
              if auth_disabled () then begin
                (* Dev escape: still try attach so whoami is real if
    cookie. *)
                let* user_opt = resolve_identity session ~headers in
                let* () =
                  match user_opt with
                  | Some user ->
                      let tok = cookie_value headers cookie_name in
                      attach_from_user session user ~auth_token:tok
                        ~checkout_personal_now:false ()
                  | None -> Session.attach_anonymous session
                in
                Lwt.return true
              end else begin
                (* Attach identity BEFORE classify: route table is
                   branch-scoped (inheritance CTE). Anonymous uses
                   system/main so package public routes resolve. *)
                let* user_opt = resolve_identity session ~headers in
                let* () =
                  match user_opt with
                  | Some user ->
                      let tok = cookie_value headers cookie_name in
                      attach_from_user session user ~auth_token:tok
                        ~checkout_personal_now:false ()
                  | None -> Session.attach_anonymous session
                in
                let classif = classify_path session path in
                let needs_auth =
                  match classif with
                  | `Hard_public -> false
                  | `Hard_private | `Unknown -> true
                  | `User (_, vis) -> vis <> "public"
                in
                match user_opt, needs_auth with
                | Some _, _ -> Lwt.return true
                | None, false -> Lwt.return true
                | None, true ->
```

```
                    let apiish =
                      String.length path >= 4
                      && String.sub path 0 4 = "/api"
                    in
                    if apiish then unauthorized_api reqd
                    else require_login_redirect reqd ~path;
                    Lwt.return false
              end
            in
            if not proceed then Lwt.return_unit
            else
              (* Refresh ui/nav cache every request so Lisp chrome is shared
                 by home, system pages, and user routes -- not only /. *)
              let* () = Session.refresh_ui_shell session in
              page_fn ())
          (fun exn ->
            let body =
              chrome ~session ~title:"Error" [
                HTML.p [HTML.class_ "err"] [txt "%s" (Printexc.to_string
   exn)];
              ]
            in
            html_response reqd ~session ~status:`Internal_server_error body;
            Lwt.return_unit)))
in
(** Lisp user-route if bound; else kernel [fallback]. *)
let launch_user_route_or (fallback : unit -> unit) : unit =
  match Session.lookup_route session path with
  | None ->
      launch (fun () ->
        fallback ();
        Lwt.return_unit)
  | Some handler_name ->
      let page () : unit Lwt.t =
        let* body_str =
          match meth with
          | `POST -> read_body reqd
          | _ -> Lwt.return ""
        in
        if String.length body_str > max_user_body_bytes then begin
          text_response reqd ~status:`Payload_too_large
            ~content_type:"text/plain"
            "request body too large (max 64KiB)";
          Lwt.return_unit
        end else begin
          let query = parse_query target in
          let form =
            match meth with
            | `POST ->
                let ct =
                  match Httpaf.Headers.get req.headers "content-type" with
                  | Some s -> s
                  | None -> ""
                in
                if String.length ct >= 33
                   && String.sub ct 0 33
                      = "application/x-www-form-urlencoded"
                   || ct = "" && String.contains body_str '='
                then parse_form_body body_str
                else []
            | _ -> []
          in
          let params = merge_params ~query ~form in
          let fragment = wants_fragment ~path ~headers in
          let method_ : Session.http_method =
            match meth with `POST -> `POST | _ -> `GET
          in
          let request : Session.request =
            { method_; path; params; body = body_str; headers }
          in
          let* result =
            Session.call_handler session ~request handler_name
          in
```

```
            (match result with
             | Some value ->
                 (* Public front-door pages: no product chrome (private
    nav). *)
                 if path = "/welcome" || path = "/book" then begin
                   let title =
                     if path = "/book" then "Pricklypear book"
                     else "Pricklypear"
                   in
                   let body =
                     match Primitives.is_raw_html value with
                     | Some raw -> raw
                     | None ->
                         (match value with
                          | Pricklypear_lisp.Ast.VStr s -> s
                          | _ -> Pricklypear_lisp.Reader.pp_value value)
                   in
                   Http_code.serve_public_body reqd ~title body;
                   Lwt.return_unit
                 end else begin
                   respond_handler_value ~session ~reqd ~path ~fragment
    value;
                   Lwt.return_unit
                 end
             | None ->
                 fallback ();
                 Lwt.return_unit)
          end
        in
        launch (fun () -> page ())
  in
  match meth, path with
  | `GET, "/login" ->
      launch (fun () ->
        let* () = Lwt.return_unit in
         login_form_page reqd;
         Lwt.return_unit)
  | `POST, "/login" -> launch (fun () -> post_login session reqd)
  | `POST, "/logout" -> launch (fun () -> post_logout session reqd)
  | `GET, "/admin/users" -> launch (fun () -> admin_users_page session reqd)
  | `POST, "/admin/users" -> launch (fun () -> post_admin_users session
    reqd)
  | `GET, "/" -> launch (fun () -> home_page session reqd)
  | `GET, "/welcome" ->
      launch_user_route_or (fun () -> Http_code.fallback_welcome reqd)
  | `GET, "/book" ->
      launch_user_route_or (fun () -> Http_code.fallback_book reqd)
  | `GET, "/book.pdf" ->
      launch (fun () ->
        Http_code.handle_public_artifact reqd ~name:"book.pdf";
        Lwt.return_unit)
  | `GET, "/src.tgz" ->
      launch (fun () ->
        Http_code.handle_public_artifact reqd ~name:"src.tgz";
        Lwt.return_unit)
  | `GET, "/code" ->
      launch (fun () ->
        Http_code.handle_code_page reqd ~target;
        Lwt.return_unit)
  | `GET, path when String.length path >= 10
                 && String.sub path 0 10 = "/code/raw/" ->
      let rel = String.sub path 10 (String.length path - 10) in
      launch (fun () ->
        Http_code.handle_raw reqd ~rel;
        Lwt.return_unit)
  | `GET, path when String.length path >= 11
                 && String.sub path 0 11 = "/assets/cm/" ->
      (* Back-compat alias: /assets/cm/* ? static/codemirror/* *)
      let rel =
        "codemirror/" ^ String.sub path 11 (String.length path - 11)
      in
      launch (fun () ->
        Http_code.handle_asset reqd ~rel;
```

```
      Lwt.return_unit)
| `GET, path when String.length path >= 8
              && String.sub path 0 8 = "/assets/" ->
    let rel = String.sub path 8 (String.length path - 8) in
    launch (fun () ->
      Http_code.handle_asset reqd ~rel;
      Lwt.return_unit)
| `GET, "/about" ->
    launch (fun () ->
      let* () = Lwt.return_unit in
       about_page session reqd;
       Lwt.return_unit)
| `GET, "/pages" ->
    launch (fun () ->
      let* () = Lwt.return_unit in
       pages_list_page session reqd;
       Lwt.return_unit)
| `POST, "/api/recipe/hello" ->
    launch (fun () ->
      let* res = Session.recipe_hello session in
       match res with
       | Ok msg ->
           let body =
             chrome ~session ~title:"Hello recipe" [
               HTML.div [HTML.class_ "card"] [
                 HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                 HTML.p [] [
                   HTML.a [HTML.href "/hello"] [txt "Open /hello"];
                   txt " ? ";
                   HTML.a [HTML.href "/edit/hi-page"] [txt "Edit
  hi-page"];
                   txt " ? ";
                   HTML.a [HTML.href "/pages"] [txt "Pages"];
                   txt " ? ";
                   HTML.a [HTML.href "/"] [txt "Home"];
                 ];
               ];
             ]
           in
           html_response reqd ~session body;
           Lwt.return_unit
       | Error e ->
           let body =
             chrome ~session ~title:"Hello recipe failed" [
               HTML.p [HTML.class_ "err"] [txt "%s" e];
               HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
             ]
           in
           html_response reqd ~session ~status:`Internal_server_error
  body;
           Lwt.return_unit)
| `POST, "/api/recipe/todo" ->
    launch (fun () ->
      let* res = Session.recipe_todo session in
       match res with
       | Ok msg ->
           let body =
             chrome ~session ~title:"Todo library" [
               HTML.div [HTML.class_ "card"] [
                 HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                 HTML.p [] [
                   HTML.a [HTML.href "/todos"] [txt "Open /todos"];
                   txt " ? ";
                   HTML.a [HTML.href "/bindings?pkg=todo"] [txt "Package
  todo"];
                   txt " ? ";
                   HTML.a [HTML.href "/pages"] [txt "Pages"];
                 ];
               ];
             ]
           in
           html_response reqd ~session body;
           Lwt.return_unit
```

```
        | Error e ->
            let body =
              chrome ~session ~title:"Todo recipe failed" [
                HTML.p [HTML.class_ "err"] [txt "%s" e];
                HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
              ]
            in
            html_response reqd ~session ~status:`Internal_server_error body;
            Lwt.return_unit)
  | `POST, "/api/recipe/notes" ->
      launch (fun () ->
        let* res = Session.recipe_notes session in
        match res with
        | Ok msg ->
            let body =
              chrome ~session ~title:"Notes library" [
                HTML.div [HTML.class_ "card"] [
                  HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                  HTML.p [] [
                    HTML.a [HTML.href "/notes"] [txt "Open /notes"];
                    txt " ? ";
                    HTML.a [HTML.href "/bindings?pkg=notes"] [txt "Package notes"];
                    txt " ? ";
                    HTML.a [HTML.href "/bindings?pkg=outline"] [txt "outline"];
                  ];
                  HTML.p [HTML.class_ "muted"] [
                    txt "Outline-first notes (org-shaped). Content lives in data_rows.";
                  ];
                ];
              ]
            in
            html_response reqd ~session body;
            Lwt.return_unit
        | Error e ->
            let body =
              chrome ~session ~title:"Notes recipe failed" [
                HTML.p [HTML.class_ "err"] [txt "%s" e];
                HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
              ]
            in
            html_response reqd ~session ~status:`Internal_server_error body;
            Lwt.return_unit)
  | `POST, "/api/recipe/dabar" ->
      launch (fun () ->
        let* res = Session.recipe_dabar session in
        match res with
        | Ok msg ->
            let body =
              chrome ~session ~title:"Dabar library" [
                HTML.div [HTML.class_ "card"] [
                  HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                  HTML.p [] [
                    HTML.a [HTML.href "/dabar?book=John&chapter=3"]
                      [txt "Open /dabar"];
                    txt " ? ";
                    HTML.a [HTML.href "/dabar?book=John&chapter=3&verse=16"]
                      [txt "Study John 3:16"];
                    txt " ? ";
                    HTML.a [HTML.href "/strongs?n=G2316"] [txt "Strong's G2316"];
                  ];
                  HTML.p [HTML.class_ "muted"] [
                    txt "Seeds: scripts/seed_bible_bsb.py ? seed_strongs.py ? seed_interlinear_dogfood.py";
                  ];
                ];
              ]
```

```
            in
            html_response reqd ~session body;
            Lwt.return_unit
        | Error e ->
            let body =
              chrome ~session ~title:"Dabar recipe failed" [
                HTML.p [HTML.class_ "err"] [txt "%s" e];
                HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
              ]
            in
            html_response reqd ~session ~status:`Internal_server_error
  body;
            Lwt.return_unit)
| `GET, "/health" ->
    let h =
      { Pricklypear_common.Protocol.ok = true;
        branch = Session.branch_name session;
        uptime_seconds = Session.uptime_seconds session;
        error = None }
    in
    let j = Pricklypear_common.Protocol.health_response_to_json h in
    text_response reqd ~content_type:"application/json"
      (Yojson.Safe.pretty_to_string j)
| `GET, "/branches" -> launch (fun () -> branches_page session reqd)
| `POST, "/branches/checkout" -> launch (fun () -> branches_checkout
  session reqd)
| `POST, "/branches/fork" -> launch (fun () -> branches_fork session reqd)
| `POST, "/branches/unoverride" -> launch (fun () -> branches_unoverride
  session reqd)
| `POST, "/branches/promote" -> launch (fun () -> branches_promote
  session reqd)
| `GET, "/bindings" -> launch (fun () -> list_bindings_page session reqd)
| `GET, "/types" -> launch (fun () -> list_types_page session reqd)
| `GET, "/data" | `GET, "/data/" ->
    launch (fun () -> data_index_page session reqd)
| `GET, path when String.length path >= 6
                && String.sub path 0 6 = "/data/" ->
    let rest = String.sub path 6 (String.length path - 6) in
    let segs = String.split_on_char '/' rest |> List.filter ((<>) "") in
    (match segs with
     | [ty; "new"] ->
         launch (fun () -> data_new_page session ty reqd)
     | [ty; id; "delete"] ->
         (* GET delete not supported -- point at detail *)
         launch (fun () -> data_detail_page session ty id reqd)
     | [ty; id] ->
         launch (fun () -> data_detail_page session ty id reqd)
     | [ty] ->
         launch (fun () -> data_list_page session ty reqd)
     | _ ->
         html_response reqd ~session ~status:`Bad_request
           (chrome ~session ~title:"Bad data path" [
              HTML.p [] [txt "use /data, /data/<type>, /data/<type>/new,
  /data/<type>/<id>"];
            ]))
| `POST, path when String.length path >= 6
                 && String.sub path 0 6 = "/data/" ->
    let rest = String.sub path 6 (String.length path - 6) in
    let segs = String.split_on_char '/' rest |> List.filter ((<>) "") in
    (match segs with
     | [ty; id; "delete"] ->
         launch (fun () -> data_delete session ty id reqd)
     | [ty; id] ->
         launch (fun () -> data_update session ty id reqd)
     | [ty] ->
         launch (fun () -> data_create session ty reqd)
     | _ ->
         html_response reqd ~session ~status:`Bad_request
           (chrome ~session ~title:"Bad data path" [
              HTML.p [] [txt "POST /data/<type> | /data/<type>/<id> |
  ?/delete"];
            ]))
| `GET, "/repl" ->
```

```
      launch (fun () ->
        let* () = Lwt.return_unit in
         repl_page session reqd;
         Lwt.return_unit)
  | `POST, "/api/eval" -> launch (fun () -> api_eval session reqd)
  | `POST, "/api/agent" -> launch (fun () -> api_agent session reqd)
  | `POST, "/api/agent-task" -> launch (fun () -> api_agent_task session
    reqd)
  | `POST, "/api/ai/propose" -> launch (fun () -> ai_propose session reqd)
  | `GET, "/agent" -> launch (fun () -> agent_page session reqd)
  | `POST, "/api/edit/create" -> launch (fun () -> api_edit_create session
    reqd)
  | `GET, path when String.length path >= 6
                && String.sub path 0 6 = "/page/" ->
      let seg = String.sub path 6 (String.length path - 6) in
      launch (fun () -> page_inspector session seg reqd)
  | `GET, "/page" | `GET, "/page/" ->
      launch (fun () -> page_inspector session "" reqd)
  | `GET, path when String.length path >= 17
                && String.sub path 0 17 = "/inspect/binding/" ->
      let name =
        Packages.name_decode (String.sub path 17 (String.length path - 17))
      in
      launch (fun () -> inspect_binding_page session name reqd)
  | `GET, path when String.length path >= 13
                && String.sub path 0 13 = "/inspect/row/" ->
      let rest = String.sub path 13 (String.length path - 13) in
      (match String.split_on_char '/' rest with
       | [ty; id] -> launch (fun () -> inspect_row_page session ty id reqd)
       | _ ->
           html_response reqd ~session ~status:`Bad_request
             (chrome ~session ~title:"Bad inspect path" [
                HTML.p [] [txt "use /inspect/row/<type>/<id>"];
              ]))
  | `GET, path when String.length path >= 9
                && String.sub path 0 9 = "/binding/" ->
      let name =
        Packages.name_decode (String.sub path 9 (String.length path - 9))
      in
      launch (fun () -> show_binding_page session name reqd)
  | `GET, "/edit" | `GET, "/edit/" ->
      (* package + local query form from Functions browser *)
      let pkg = Option.value (query_param target "package") ~default:"" in
      let local = Option.value (query_param target "local") ~default:"" in
      let name = Packages.join_name ~package:pkg ~local in
      if name = "" then
        html_response reqd ~session
          (chrome ~session ~title:"New function" [
             HTML.p [] [txt "Provide package and/or name (see Functions)."];
             HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "Functions"]];
           ])
      else launch (fun () -> editor_page session name reqd)
  | `GET, path when String.length path >= 6
                && String.sub path 0 6 = "/edit/" ->
      let name =
        Packages.name_decode (String.sub path 6 (String.length path - 6))
      in
      launch (fun () -> editor_page session name reqd)
  | `POST, path when String.length path >= 6
                 && String.sub path 0 6 = "/edit/" ->
      let name =
        Packages.name_decode (String.sub path 6 (String.length path - 6))
      in
      launch (fun () -> save_edit session name reqd)
  | ((`GET | `POST) as umeth), _ ->
      (* User routes: GET + POST (it33). Same path ? same handler;
         request-* prims expose method/params for branching. *)
      (match Session.lookup_route session path with
       | Some handler_name ->
           let page () : unit Lwt.t =
             let* body_str =
               match umeth with
               | `POST -> read_body reqd
```

```
              | `GET -> Lwt.return ""
            in
            if String.length body_str > max_user_body_bytes then begin
              text_response reqd ~status:`Payload_too_large
                ~content_type:"text/plain"
                "request body too large (max 64KiB)";
              Lwt.return_unit
            end else begin
              let query = parse_query target in
              let form =
                match umeth with
                | `GET -> []
                | `POST ->
                    let ct =
                      match
                        Httpaf.Headers.get req.headers "content-type"
                      with
                      | Some s -> s
                      | None -> ""
                    in
                    if String.length ct >= 33
                       && String.sub ct 0 33
                          = "application/x-www-form-urlencoded"
                       || ct = ""
                          && String.contains body_str '='
                    then parse_form_body body_str
                    else []
              in
              let params = merge_params ~query ~form in
              let headers = header_list req in
              let fragment = wants_fragment ~path ~headers in
              let method_ : Session.http_method =
                match umeth with `GET -> `GET | `POST -> `POST
              in
              let request : Session.request =
                { method_; path; params; body = body_str; headers }
              in
              let* result =
                Session.call_handler session ~request handler_name
              in
              (match result with
               | Some value ->
                   respond_handler_value ~session ~reqd ~path ~fragment
    value;
                   Lwt.return_unit
               | None ->
                   if fragment then
                     fragment_response reqd ~status:`Internal_server_error
                       ("handler " ^ handler_name ^ " returned no value")
                   else
                     html_response reqd ~session
    ~status:`Internal_server_error
                       (chrome ~session ~title:"Route handler error" [
                          HTML.p []
                            [txt "handler %s returned no value"
    handler_name];
                        ]);
                   Lwt.return_unit)
            end
          in
          launch (fun () -> page ())
        | None ->
            let headers = header_list req in
            if wants_fragment ~path ~headers then
              fragment_response reqd ~status:`Not_found
                ("no partial for " ^ path)
            else
              html_response reqd ~session ~status:`Not_found
                (chrome ~session ~title:"Not found" [
                   HTML.p [] [txt "no route for %s" path];
                 ]))
    | _, _ ->
        let headers = header_list req in
```

```
      if wants_fragment ~path ~headers then
        fragment_response reqd ~status:`Not_found ("no partial for " ^ path)
      else
        html_response reqd ~session ~status:`Not_found
          (chrome ~session ~title:"Not found" [
             HTML.p [] [txt "no route for %s" path];
           ])

let error_handler (_sockaddr : Unix.sockaddr)
    ?(request : Httpaf.Request.t option)
    (err : Httpaf.Server_connection.error)
    (respond : Httpaf.Headers.t -> [> `write ] Httpaf.Body.t) : unit =
  let status, msg =
    match err with
    | `Bad_request -> `Bad_request, "bad request"
    | `Bad_gateway -> `Bad_gateway, "bad gateway"
    | `Internal_server_error -> `Internal_server_error, "internal server
     error"
    | `Exn exn -> `Internal_server_error, Printexc.to_string exn
  in
  let _ = request in
  let body_str =
    Pure_html.to_string
      (HTML.html [HTML.lang "en"] [
         HTML.head [] [HTML.title [] "Pricklypear Error"];
         HTML.body [] [
           HTML.h1 [] [txt "Error"];
           HTML.p [] [txt "%s" msg];
         ];
       ])
  in
  let headers =
    Httpaf.Headers.of_list
      [("content-type", "text/html");
       ("content-length", string_of_int (String.length body_str))]
  in
  let body_w = respond headers in
  Httpaf.Body.write_string body_w body_str;
  Httpaf.Body.close_writer body_w

let serve ~(session : Session.t) ?(port = default_port) () : unit Lwt.t =
  let handler = request_handler session in
  let establisher =
    Httpaf_lwt_unix.Server.create_connection_handler
      ~request_handler:handler
      ~error_handler
  in
  let addr = Lwt_unix.ADDR_INET (Unix.inet_addr_any, port) in
  Printf.eprintf "[http] serving htmx on 0.0.0.0:%d\n" port;
  let* _server = Lwt_io.establish_server_with_client_socket addr
     establisher in
  let waiter, _waker = Lwt.wait () in
  waiter
```

38 image/lib/http_agent.ml

```
(** Agent portal + AI propose HTTP handlers. *)
open Lwt.Syntax
open Pure_html
open Http_core

let sse_write (body : [ `write ] Httpaf.Body.t) (event_json : string) :
    unit Lwt.t =
  let chunk = "data: " ^ event_json ^ "\n\n" in
  Httpaf.Body.write_string body chunk;
  let promise, wakeup = Lwt.task () in
  Httpaf.Body.flush body (fun () ->
    if Lwt.state promise = Lwt.Sleep then Lwt.wakeup_later wakeup ());
  (* Don't hang forever if the peer never acks the flush. *)
  Lwt.pick
    [ promise
    ; (let* () = Lwt_unix.sleep 2.0 in Lwt.return_unit)
    ]

let default_http_agent_seed =
  (* Short fallback when no session seed; tools sessions should prefer
     Session.default_agent_seed (dogfood + register_route rules). *)
  "You are the co-pilot for a Pricklypear multi-user Lisp OS image " ^
  "(software, not a cactus). Prefer CONTEXT; if unknown say so. " ^
  "Use structured tools only (never <tool_call> XML). " ^
  "install_function for lambdas; register_route for HTTP paths; " ^
  "data_rows via create/rows (not outline/*); inspect todo/page first."

(** POST /api/agent -- durable sessions.
    Body JSON:
      {prompt, seed?, session_id?, tools?, max_steps?}
    Creates a session when session_id omitted; always returns session_id.
    Progress SSE when Accept prefers event-stream or stream:true; default
    is non-stream JSON (token streaming deferred / flaky on Synthetic). *)
(** Pull the human line out of the HTTP agent prompt wrapper for titles/UI.
    *)
let agent_user_visible_text (s : string) : string =
  let s = String.trim s in
  let key = "\nUser:\n" in
  let rec find_sub hay needle i =
    let nlen = String.length needle in
    let hlen = String.length hay in
    if i + nlen > hlen then None
    else if String.sub hay i nlen = needle then Some i
    else find_sub hay needle (i + 1)
  in
  match find_sub s key 0 with
  | Some i ->
      String.trim
        (String.sub s (i + String.length key)
           (String.length s - i - String.length key))
  | None ->
      if String.length s >= 20
         && String.sub s 0 20 = "=== IMAGE CONTEXT ===" then
        let n = min 80 (String.length s) in
        String.sub s 0 n ^ (if String.length s > 80 then "?" else "")
      else s

let agent_session_label (s : Ai_store.session) : string =
  let short = String.sub s.id 0 (min 8 (String.length s.id)) in
  match s.title with
  | Some t when String.trim t <> "" ->
      let t = agent_user_visible_text t in
      (* Old rows stored the full IMAGE CONTEXT wrapper as title (truncated
         before User:). Don't show that dump in the chat list. *)
      if String.trim t = ""
         || (String.length t >= 20
             && String.sub t 0 20 = "=== IMAGE CONTEXT ===")
         || String.starts_with ~prefix:"=== IMAGE CONTEXT" t then
        "chat " ^ short
      else if String.length t > 72 then String.sub t 0 72 ^ "?"
      else t
```

```
  | _ -> "chat " ^ short

let api_agent (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let* body = read_body reqd in
  let seed, prompt, session_id_opt, tools, max_steps, want_stream =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          let seed =
            match List.assoc_opt "seed" items with
            | Some (`String s) -> s
            | _ -> default_http_agent_seed
          in
          let prompt =
            match List.assoc_opt "prompt" items with
            | Some (`String s) -> s
            | _ ->
                (match List.assoc_opt "q" items with
                 | Some (`String s) -> s
                 | _ -> "")
          in
          let sid =
            match List.assoc_opt "session_id" items with
            | Some (`String s) when String.trim s <> "" -> Some (String.trim s)
            | _ -> None
          in
          let tools =
            match List.assoc_opt "tools" items with
            | Some (`Bool b) -> b
            | Some (`String "0") | Some (`String "false") -> false
            | _ -> true
          in
          let max_steps =
            match List.assoc_opt "max_steps" items with
            | Some (`Int n) -> n
            | Some (`String s) ->
                (match int_of_string_opt s with Some n -> n | None -> 16)
            | _ -> 16
          in
          let stream =
            match List.assoc_opt "stream" items with
            | Some (`Bool b) -> b
            | Some (`String "1") | Some (`String "true") -> true
            | Some (`String "0") | Some (`String "false") -> false
            | _ -> false
          in
          (seed, prompt, sid, tools, max_steps, stream)
      | _ -> (default_http_agent_seed, "", None, true, 16, false)
    with _ ->
      let seed =
        Option.value (form_field body "seed") ~default:default_http_agent_seed
      in
      let prompt =
        match form_field body "prompt" with
        | Some p -> p
        | None -> Option.value (form_field body "q") ~default:""
      in
      let sid = form_field body "session_id" in
      (seed, prompt, sid, true, 16, false)
  in
  if String.trim prompt = "" then begin
    json_response reqd
      (`Assoc [ ("ok", `Bool false); ("error", `String "need prompt") ]);
    Lwt.return_unit
  end else
    let* sid_r =
      match session_id_opt with
      | Some sid -> Lwt.return (Ok sid)
      | None ->
          let title_opt =
            let p = agent_user_visible_text prompt in
```

```
          if String.trim p = "" then None
          else
            Some
              (if String.length p > 80 then String.sub p 0 80 ^ "?" else p)
      in
      let* v =
        Session.agent_session_new session ~seed ~tools ?title:title_opt ()
      in
      (match v with
       | Pricklypear_lisp.Ast.VStr s
         when not (String.starts_with ~prefix:"agent-session-new error" s) ->
           Lwt.return (Ok s)
       | Pricklypear_lisp.Ast.VStr e -> Lwt.return (Error e)
       | other ->
           Lwt.return
             (Error (Pricklypear_lisp.Reader.pp_value other)))
  in
  match sid_r with
  | Error e ->
      json_response reqd
        (`Assoc [ ("ok", `Bool false); ("error", `String e) ]);
      Lwt.return_unit
  | Ok sid ->
      if not want_stream then begin
        let* v =
          Session.agent_say session ~session_id:sid ~prompt ~max_steps ()
        in
        let text =
          match v with
          | Pricklypear_lisp.Ast.VStr s -> s
          | _ -> Pricklypear_lisp.Reader.pp_value v
        in
        json_response reqd
          (`Assoc
             [ ("ok", `Bool true)
             ; ("session_id", `String sid)
             ; ("text", `String text)
             ]);
        Lwt.return_unit
      end else begin
        (* Progress SSE (tool/token events), not provider token stream. *)
        let headers =
          Httpaf.Headers.of_list
            [ ("content-type", "text/event-stream; charset=utf-8")
            ; ("cache-control", "no-cache")
            ; ("connection", "keep-alive")
            ; ("x-accel-buffering", "no")
            ]
        in
        let resp = Httpaf.Response.create ~headers `OK in
        let body_w =
          Httpaf.Reqd.respond_with_streaming
            ~flush_headers_immediately:true
            reqd resp
        in
        let queue : string Queue.t = Queue.create () in
        let cond = Lwt_condition.create () in
        let finished = ref false in
        let push_json (j : string) : unit =
          Queue.add j queue;
          Lwt_condition.signal cond ()
        in
        push_json
          (Yojson.Safe.to_string
             (`Assoc
                [ ("t", `String "session")
                ; ("session_id", `String sid)
                ]));
        let on_event = function
          | Llm_client.Token tok ->
```

```
                push_json
                  (Yojson.Safe.to_string
                     (`Assoc [ ("t", `String "token"); ("s", `String tok)
    ]))
            | Llm_client.Done ->
                push_json
                  (Yojson.Safe.to_string (`Assoc [ ("t", `String "done")
    ]));
                finished := true;
                Lwt_condition.signal cond ()
            | Llm_client.Fail e ->
                push_json
                  (Yojson.Safe.to_string
                     (`Assoc [ ("t", `String "error"); ("s", `String e) ]));
                finished := true;
                Lwt_condition.signal cond ()
          in
          Lwt.async (fun () ->
            let* v =
              Session.agent_say session ~session_id:sid ~prompt ~max_steps
                ~on_event ()
            in
            if not !finished then begin
              (match v with
               | Pricklypear_lisp.Ast.VStr s ->
                   push_json
                     (Yojson.Safe.to_string
                        (`Assoc
                           [ ("t", `String "done")
                           ; ("text", `String s)
                           ; ("session_id", `String sid)
                           ]))
               | _ ->
                   push_json
                     (Yojson.Safe.to_string
                        (`Assoc [ ("t", `String "done") ])));
              finished := true;
              Lwt_condition.signal cond ()
            end;
            Lwt.return_unit);
          let rec drain () =
            if not (Queue.is_empty queue) then
              let j = Queue.take queue in
              let* () = sse_write body_w j in
              drain ()
            else if !finished then begin
              Httpaf.Body.close_writer body_w;
              Lwt.return_unit
            end else
              let* () = Lwt_condition.wait cond in
              drain ()
          in
          drain ()
        end

let api_agent_task (session : Session.t) (reqd : Httpaf.Reqd.t) : unit
    Lwt.t =
  (* Alias: create tools session + say (same as /api/agent with tools). *)
  let* body = read_body reqd in
  let prompt =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          (match List.assoc_opt "prompt" items with
           | Some (`String s) -> s
           | _ ->
               (match List.assoc_opt "task" items with
                | Some (`String s) -> s
                | _ -> ""))
      | _ -> ""
    with _ ->
      Option.value (form_field body "prompt")
        ~default:(Option.value (form_field body "task") ~default:"")
```

```
  in
  if String.trim prompt = "" then begin
    json_response reqd
      (`Assoc [ ("ok", `Bool false); ("error", `String "need prompt/task")
    ]);
    Lwt.return_unit
  end else
    let* v = Session.agent_task session ~prompt () in
    let text =
      match v with
      | Pricklypear_lisp.Ast.VStr s -> s
      | _ -> Pricklypear_lisp.Reader.pp_value v
    in
    json_response reqd
      (`Assoc [ ("ok", `Bool true); ("text", `String text) ]);
    Lwt.return_unit

let agent_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  (* Durable chats: list + open with ?session=id (agent-portal UI). *)
  let target = (Httpaf.Reqd.request reqd).Httpaf.Request.target in
  let open_sid =
    match query_param target "session" with
    | Some s when String.trim s <> "" -> Some (String.trim s)
    | _ -> None
  in
  let ctx =
    Printf.sprintf "user=%s branch=%s"
      (Session.username session) (Session.branch_name session)
  in
  let default_seed = Session.default_agent_seed session in
  let* sessions =
    Ai_store.list_sessions (Session.store session)
      ~owner:(Session.username session) ~limit:40 ()
  in
  let* open_sess, history =
    match open_sid with
    | None -> Lwt.return (None, [])
    | Some id ->
        let* s_opt =
          Ai_store.get_session (Session.store session)
            ~owner:(Session.username session) ~id
        in
        (match s_opt with
         | None -> Lwt.return (None, [])
         | Some s ->
             let* pairs =
               Ai_store.history_summaries (Session.store session)
    ~session_id:id
             in
             Lwt.return (Some s, pairs))
  in
  let seed_value =
    match open_sess with
    | Some s when String.trim s.seed <> "" -> s.seed
    | _ -> default_seed
  in
  let tools_checked =
    match open_sess with Some s -> s.tools | None -> true
  in
  let sid_display =
    match open_sess with Some s -> s.id | None -> "(new on send)"
  in
  let history_nodes =
    List.map
      (fun (role, content) ->
        let role_label =
          match role with
          | "user" -> "you"
          | "assistant" -> "agent"
          | other -> other
        in
        let shown =
          if role = "user" then agent_user_visible_text content else content
```

```
        in
        let shown =
          if String.length shown > 4000 then
            String.sub shown 0 4000 ^ "\n?(truncated)"
          else shown
        in
        HTML.div
          [ HTML.class_ "card"; HTML.style_ "margin:.4rem 0" ]
          [ HTML.b [] [txt "%s" role_label]
          ; HTML.pre
              [ HTML.style_ "white-space:pre-wrap;margin:.35rem 0 0" ]
              [txt "%s" shown]
          ])
      history
  in
  let session_list_nodes =
    if sessions = [] then
      [ HTML.p [HTML.class_ "muted"] [txt "No chats yet -- send a
     message."] ]
    else
      List.map
        (fun (s : Ai_store.session) ->
          let active =
            match open_sess with
            | Some o when o.id = s.id -> true
            | _ -> false
          in
          let label = agent_session_label s in
          let href = "/agent?session=" ^ s.id in
          HTML.li
            [ HTML.style_
                (if active then "font-weight:700;margin:.25rem 0"
                 else "margin:.25rem 0")
            ]
            [ HTML.a [HTML.href "%s" href] [txt "%s" label]
            ; HTML.span [HTML.class_ "muted"]
                [txt "%s" (" ? " ^ s.updated_at)]
            ])
        sessions
  in
  let initial_sid_js =
    match open_sess with
    | Some s -> Yojson.Safe.to_string (`String s.id)
    | None -> "null"
  in
  let tools_input =
    if tools_checked then
      HTML.input
        [ HTML.type_ "checkbox"
        ; HTML.name "tools"
        ; HTML.id "tools"
        ; HTML.value "1"
        ; HTML.checked
        ]
    else
      HTML.input
        [ HTML.type_ "checkbox"
        ; HTML.name "tools"
        ; HTML.id "tools"
        ; HTML.value "1"
        ]
  in
  let js =
    "const log=document.getElementById('log');" ^
    "const msg=document.getElementById('msg');" ^
    "const send=document.getElementById('send');" ^
    "const neu=document.getElementById('newchat');" ^
    "const sidEl=document.getElementById('sid');" ^
    "const toolsEl=document.getElementById('tools');" ^
    "const seedEl=document.getElementById('seed');" ^
    "const baseCtx=" ^ Yojson.Safe.to_string (`String ctx) ^ ";" ^
    "let sid=" ^ initial_sid_js ^ ";" ^
    "function add(role,text){" ^
```

```
    "const d=document.createElement('div');" ^
    "d.className='card';" ^
    "d.style.margin='.4rem 0';" ^
    "d.innerHTML='<b>'+role+'</b><pre
     style=\"white-space:pre-wrap;margin:.35rem 0 0\">'+" ^
     "String(text).replace(/[&<>]/g,c=>({\"&\":\"&amp;\",\"<\":\"&lt;\",\">\":\"&gt;\"}[c]))+"
     ^
    "'</pre>';log.appendChild(d);log.scrollTop=log.scrollHeight;}" ^
    "function setSid(s){sid=s||null;sidEl.textContent=sid?sid:'(new on
     send)';" ^
    "if(sid){const u=new
     URL(window.location.href);u.searchParams.set('session',sid);" ^
    "history.replaceState({},'',u.pathname+u.search);}" ^
    "else{history.replaceState({},'','/agent');}}" ^
    "setSid(sid);" ^
    "neu.onclick=()=>{window.location.href='/agent';};" ^
    "send.onclick=async()=>{" ^
    "const q=msg.value.trim();if(!q)return;" ^
    "const seed=seedEl.value;" ^
    "const tools=toolsEl.checked;" ^
    "const prompt='=== IMAGE CONTEXT ===\\n'+" ^
    "'system: Pricklypear multi-user Lisp OS image (software)\\n'+" ^
    "baseCtx+'\\n=== END CONTEXT ===\\n\\nUser:\\n'+q;" ^
    "add('you',q);msg.value='';send.disabled=true;" ^
    "try{" ^
    "const body={seed,prompt,tools,stream:false};" ^
    "if(sid)body.session_id=sid;" ^
    "const r=await fetch('/api/agent',{method:'POST'," ^
    "headers:{'content-type':'application/json'}," ^
    "body:JSON.stringify(body)});" ^
    "const j=await r.json();" ^
    "if(j.session_id)setSid(j.session_id);" ^
    "add('agent',j.text||j.error||JSON.stringify(j));" ^
    "if(j.session_id){" ^
    "const href='/agent?session='+encodeURIComponent(j.session_id);" ^
    "if(!document.querySelector('#chat-list a[href=\"'+href+'\"]'))" ^
    " setTimeout(function(){window.location.href=href;},350);}" ^
    "}catch(e){add('error',String(e));}" ^
    "finally{send.disabled=false;msg.focus();}" ^
    "};" ^

    "msg.addEventListener('keydown',e=>{if(e.key==='Enter'&&!e.shiftKey){e.preventDefault();send.click();}});"
in
let body =
  chrome ~session ~title:"Agent" [
    HTML.div [HTML.class_ "card"] [
      HTML.h2 [] [txt "Chats"];
      HTML.p [HTML.class_ "muted"] [txt
        "Durable threads in Postgres. Open one to resume; New chat starts
   empty."];
      HTML.p [] [
        HTML.a [HTML.href "/agent"; HTML.class_ "btn"] [txt "New chat"];
      ];
      HTML.ul [HTML.id "chat-list"] session_list_nodes;
    ];
    HTML.div [HTML.class_ "card"] [
      HTML.h2 [] [txt "Conversation"];
      HTML.p [] [
        txt "session: ";
        HTML.code [HTML.id "sid"] [txt "%s" sid_display];
      ];
      HTML.p [] [
        HTML.label [HTML.for_ "tools"] [
          tools_input;
          txt " tools (inspect / install)"
        ];
      ];
      HTML.details [] [
        HTML.summary [] [txt "seed (system persona)"];
        HTML.textarea [HTML.name "seed"; HTML.id "seed"] "%s" seed_value;
      ];
      HTML.div
```

```
          [ HTML.id "log"
          ; HTML.style_
              "max-height:28rem;overflow:auto;border:1px solid #ccc;padding:.5rem;margin:.5rem 0"
          ]
          history_nodes;
        HTML.p [] [
          HTML.label [HTML.for_ "msg"] [txt "message"];
        ];
        HTML.textarea [HTML.name "msg"; HTML.id "msg"]
          "%s" "What branch am I on?";
        HTML.p [] [
          HTML.button [HTML.type_ "button"; HTML.id "send"] [txt "Send"];
          txt " ";
          HTML.button [HTML.type_ "button"; HTML.id "newchat"] [txt "New chat"];
        ];
       ];
       HTML.script [] "%s" js;
       HTML.div [HTML.class_ "card"] [
         HTML.h2 [] [txt "Write a function"];
         HTML.p [HTML.class_ "muted"] [txt
           "One-shot authoring (not chat). Same as (ai/write name spec)."];
         HTML.p [] [
           HTML.label [HTML.for_ "fn_name"] [txt "name "];
           HTML.input [HTML.type_ "text"; HTML.name "fn_name"; HTML.id "fn_name";
                       HTML.value "demo/double"; HTML.placeholder "pkg/name"];
         ];
         HTML.p [] [
           HTML.label [HTML.for_ "fn_spec"] [txt "spec"];
         ];
         HTML.textarea [HTML.name "fn_spec"; HTML.id "fn_spec"]
           "%s" "A pure lambda of one int x that returns (* x 2).";
         HTML.p [] [
           HTML.button [HTML.type_ "button"; HTML.id "write_fn"] [txt "Generate & install"];
         ];
         HTML.pre [HTML.id "write_out"; HTML.class_ "card"] [txt ""];
       ];
       HTML.script [] "%s" (
         "const wout=document.getElementById('write_out');" ^
         "const wbtn=document.getElementById('write_fn');" ^
         "wbtn.onclick=async()=>{" ^
         "const name=document.getElementById('fn_name').value.trim();" ^
         "const spec=document.getElementById('fn_spec').value;" ^
         "wout.textContent='?'; wbtn.disabled=true;" ^
         "try{" ^
         "const r=await fetch('/api/ai/propose',{method:'POST'," ^
         "headers:{'content-type':'application/json'}," ^
         "body:JSON.stringify({name,spec})});" ^
         "const j=await r.json();" ^
         "wout.textContent=j.result||j.error||JSON.stringify(j);" ^
         "}catch(e){wout.textContent=String(e);}" ^
         "finally{wbtn.disabled=false;}" ^
         "};"
       );
     ]
   in
   html_response reqd ~session body;
   Lwt.return_unit

let ai_propose (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
   let* body = read_body reqd in
   let name_opt, spec_opt =
     try
       match Yojson.Safe.from_string body with
       | `Assoc items ->
           let name =
             match List.assoc_opt "name" items with
             | Some (`String s) -> Some s
             | _ -> None
```

```
          in
          let spec =
            match List.assoc_opt "spec" items with
            | Some (`String s) -> Some s
            | _ -> None
          in
          (name, spec)
      | _ -> (None, None)
    with _ ->
      let pkg = Option.value (form_field body "package") ~default:"" in
      let local = Option.value (form_field body "local") ~default:"" in
      let name =
        match form_field body "name" with
        | Some n when String.trim n <> "" -> Some (String.trim n)
        | _ ->
            let j = Packages.join_name ~package:pkg ~local in
            if j = "" then None else Some j
      in
      (name, form_field body "spec")
  in
  match name_opt, spec_opt with
  | Some name, Some spec when name <> "" && spec <> "" ->
      let* v = Session.propose_function session ~name ~spec in
      let rendered = Pricklypear_lisp.Reader.pp_value v in
      let wants_html =
        match Httpaf.Headers.get (Httpaf.Reqd.request reqd).headers
     "content-type" with
        | Some ct when String.length ct >= 33
            && String.sub ct 0 33 = "application/x-www-form-urlencoded" ->
     true
        | _ ->
            (match form_field body "name" with Some _ -> true | None ->
     false)
      in
      if wants_html then
        let body_node =
          chrome ~session ~title:("AI propose " ^ name) [
            HTML.div [HTML.class_ "card"] [
              HTML.pre [] [txt "%s" rendered];
              HTML.p [] [
                HTML.a [HTML.href "%s" ("/edit/" ^ name)] [txt "open
     editor"];
                txt " ? ";
                HTML.a [HTML.href "%s" (Packages.inspect_href name)] [txt
     "inspect"];
                txt " ? ";
                HTML.a [HTML.href "/bindings"] [txt "functions"];
              ];
            ];
          ]
        in
        html_response reqd ~session body_node;
        Lwt.return_unit
      else begin
        json_response reqd
          (`Assoc [ ("ok", `Bool true)
                  ; ("result", `String rendered)
                  ; ("name", `String name) ]);
        Lwt.return_unit
      end
  | _ ->
      json_response reqd ~status:`Bad_request
        (`Assoc [("ok", `Bool false); ("error", `String "name and spec
     required")]);
      Lwt.return_unit
```

39 image/lib/http_branches.ml

```
(** Branch browser UI: list, fork, checkout, promote. *)
open Lwt.Syntax
open Pure_html
open Http_core

let branch_ref_of (b : Pricklypear_common.Branch.t) : string =
  b.Pricklypear_common.Branch.owner ^ "/" ^ b.Pricklypear_common.Branch.name

let branch_label (b : Pricklypear_common.Branch.t) : string =
  let r = branch_ref_of b in
  if b.owner = "system" && b.name = "main" then r ^ " (defaults)"
  else if b.name = "personal" then r ^ " (garden)"
  else r

let id_to_ref_map (branches : Pricklypear_common.Branch.t list)
  : (string, string) Hashtbl.t =
  let h = Hashtbl.create 16 in
  List.iter
    (fun (b : Pricklypear_common.Branch.t) ->
      Hashtbl.replace h b.id (branch_ref_of b))
    branches;
  h

let parent_ref_of (idmap : (string, string) Hashtbl.t)
    (b : Pricklypear_common.Branch.t) : string =
  match b.parent_id with
  | None -> "--"
  | Some pid ->
      (match Hashtbl.find_opt idmap pid with
       | Some r -> r
       | None -> String.sub pid 0 (min 8 (String.length pid)))

let short_ts (s : string) : string =
  (* "2026-07-12 03:04:05.123+00" ? first 16 chars-ish *)
  if String.length s <= 16 then s else String.sub s 0 16

let branches_flash (msg : string) : Pure_html.node list =
  if String.trim msg = "" then []
  else [ HTML.p [HTML.class_ "ok"] [txt "%s" msg] ]

let branches_err (msg : string) : Pure_html.node list =
  if String.trim msg = "" then []
  else [ HTML.p [HTML.class_ "err"] [txt "%s" msg] ]

let checkout_form ~(ref_ : string) ~(current : bool) : Pure_html.node =
  if current then
    HTML.span [HTML.class_ "muted"] [txt "current"]
  else
    HTML.form
      [ HTML.method_ `POST
      ; HTML.action "/branches/checkout"
      ; HTML.enctype `urlencoded
      ; HTML.class_ "inline-form"
      ]
      [ HTML.input [HTML.type_ "hidden"; HTML.name "ref"; HTML.value "%s"
    ref_]
      ; HTML.button [HTML.type_ "submit"; HTML.class_ "ol-btn"] [txt
    "Checkout"]
      ]

let list_branches_page (session : Session.t) (reqd : Httpaf.Reqd.t)
    ?(msg = "") ?(err = "") () : unit Lwt.t =
  let store = Session.store session in
  let current = Session.branch_name session in
  let* branches = Store.list_branches store in
  let idmap = id_to_ref_map branches in
  let* rows =
    Lwt_list.map_s
      (fun (b : Pricklypear_common.Branch.t) ->
        let ref_ = branch_ref_of b in
```

```
      let is_cur = ref_ = current || (current = "main" && ref_ =
  "system/main") in
      let* n_fn = Store.count_functions_on_branch store ~branch_id:b.id in
      let* n_rt = Store.count_routes_on_branch store ~branch_id:b.id in
      let* ov =
        Store.list_overrides store ~branch_id:b.id ~parent_id:b.parent_id
      in
      let n_ov = List.length ov in
      let star = if is_cur then "?" else "" in
      Lwt.return
        (HTML.tr
           (if is_cur then [HTML.class_ "br-current"] else [])
           [ HTML.td [] [txt "%s" star]
           ; HTML.td []
               [ HTML.a [HTML.href "%s" ("/branches?ref=" ^ ref_)]
                   [txt "%s" (branch_label b)] ]
           ; HTML.td [] [txt "%s" (parent_ref_of idmap b)]
           ; HTML.td [] [txt "%d" n_fn]
           ; HTML.td [] [txt "%d" n_ov]
           ; HTML.td [] [txt "%d" n_rt]
           ; HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts
  b.created_at)]
           ; HTML.td []
               [ HTML.a
                   [ HTML.href "%s" ("/branches?ref=" ^ ref_)
                   ; HTML.class_ "btn secondary" ]
                   [txt "Open"]
               ; txt " "
               ; checkout_form ~ref_ ~current:is_cur
               ]
           ]))
    branches
in
let fork_options =
  List.map
    (fun (b : Pricklypear_common.Branch.t) ->
      let ref_ = branch_ref_of b in
      HTML.option [HTML.value "%s" ref_] "%s" ref_)
    branches
in
let body =
  chrome ~session ~title:"Branches"
    (branches_flash msg @ branches_err err @
     [ HTML.p [HTML.class_ "muted"]
         [ txt "On: "
         ; HTML.strong [] [txt "%s" current]
         ; txt " -- code branches; data_rows are shared."
         ]
     ; (match branches with
        | [] -> HTML.p [] [txt "(no branches in store)"]
        | _ ->
            HTML.div [HTML.class_ "br-table-wrap"]
              [ HTML.table [HTML.class_ "br-table"]
                  [ HTML.thead []
                      [ HTML.tr []
                          [ HTML.th [] [txt ""]
                          ; HTML.th [] [txt "branch"]
                          ; HTML.th [] [txt "parent"]
                          ; HTML.th [] [txt "own code"]
                          ; HTML.th [] [txt "overrides"]
                          ; HTML.th [] [txt "routes"]
                          ; HTML.th [] [txt "created"]
                          ; HTML.th [] [txt "actions"]
                          ]
                      ]
                  ; HTML.tbody [] rows
                  ]
              ])
     ; HTML.div [HTML.class_ "card"]
         [ HTML.h2 [] [txt "Create branch (full fork)"]
         ; HTML.p [HTML.class_ "muted"]
             [txt "Copies code from parent. Garden (personal) stays an
  empty inherit child."]
```

```
          ; HTML.form
              [ HTML.method_ `POST
              ; HTML.action "/branches/fork"
              ; HTML.enctype `urlencoded
              ]
              [ HTML.p []
                  [ HTML.label [] [txt "From "]
                  ; HTML.select [HTML.name "from"] fork_options
                  ]
              ; HTML.p []
                  [ HTML.label [] [txt "New name "]
                  ; HTML.input
                      [ HTML.type_ "text"
                      ; HTML.name "name"
                      ; HTML.placeholder "experiment"
                      ; HTML.required
                      ]
                  ]
              ; HTML.p []
                  [ HTML.button [HTML.type_ "submit"] [txt "Fork"] ]
              ]
          ]
      ])
  in
  html_response reqd ~session body;
  Lwt.return_unit

let branch_detail_page (session : Session.t) (reqd : Httpaf.Reqd.t)
    ~(ref_ : string) ?(msg = "") ?(err = "") () : unit Lwt.t =
  let store = Session.store session in
  let current = Session.branch_name session in
  let owner, name =
    match String.split_on_char '/' ref_ with
    | [o; n] -> o, n
    | [n] -> "system", n
    | _ -> "system", ref_
  in
  let* found = Store.lookup_branch_id_by_name store ~owner ~name in
  match found with
  | None ->
      list_branches_page session reqd ~err:("no such branch: " ^ ref_) ()
  | Some bid ->
      let* branches = Store.list_branches store in
      let idmap = id_to_ref_map branches in
      let b_opt =
        List.find_opt
          (fun (b : Pricklypear_common.Branch.t) -> b.id = bid)
          branches
      in
      let parent_id =
        match b_opt with Some b -> b.parent_id | None -> None
      in
      let parent_ref =
        match b_opt with
        | Some b -> parent_ref_of idmap b
        | None -> "--"
      in
      let is_cur =
        ref_ = current
        || (owner ^ "/" ^ name) = current
      in
      let* n_fn = Store.count_functions_on_branch store ~branch_id:bid in
      let* n_rt = Store.count_routes_on_branch store ~branch_id:bid in
      let* ov = Store.list_overrides store ~branch_id:bid ~parent_id in
      let* own = Store.list_own_functions store ~branch_id:bid in
      let* routes = Store.list_own_routes store ~branch_id:bid in
      let n_ov = List.length ov in
      let override_rows =
        List.map
          (fun (fname, child_u, parent_u) ->
            let kind =
              match parent_u with
              | None -> "new"
```

```
            | Some pu when pu > child_u -> "main newer"
            | Some _ -> "override"
          in
          let kind_cls =
            match kind with
            | "main newer" -> "err"
            | "new" -> "ok"
            | _ -> "muted"
          in
          HTML.tr []
            [ HTML.td []
                [ HTML.a [HTML.href "%s" ("/binding/" ^ fname)] [txt "%s" fname] ]
            ; HTML.td [HTML.class_ "%s" kind_cls] [txt "%s" kind]
            ; HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts child_u)]
            ; HTML.td []
                [ HTML.a
                    [ HTML.href "%s"
                        ("/branches?ref=" ^ ref_ ^ "&diff=" ^ fname)
                    ]
                    [txt "diff"]
                ; txt " "
                ; HTML.form
                    [ HTML.method_ 'POST
                    ; HTML.action "/branches/unoverride"
                    ; HTML.enctype 'urlencoded
                    ; HTML.class_ "inline-form"
                    ]
                    [ HTML.input
                        [ HTML.type_ "hidden"
                        ; HTML.name "ref"
                        ; HTML.value "%s" ref_
                        ]
                    ; HTML.input
                        [ HTML.type_ "hidden"
                        ; HTML.name "name"
                        ; HTML.value "%s" fname
                        ]
                    ; HTML.button
                        [ HTML.type_ "submit"
                        ; HTML.class_ "ol-btn"
                        ; HTML.title_ "Delete pin; inherit parent"
                        ]
                        [txt "use parent"]
                    ]
                ]
            ])
        ov
    in
    let own_rows =
      List.map
        (fun (fname, author, upd) ->
          HTML.tr []
            [ HTML.td []
                [ HTML.a [HTML.href "%s" ("/binding/" ^ fname)] [txt "%s" fname] ]
            ; HTML.td [] [txt "%s" author]
            ; HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts upd)]
            ])
        own
    in
    let route_rows =
      List.map
        (fun (route, handler) ->
          HTML.tr []
            [ HTML.td [] [HTML.code [] [txt "%s" route]]
            ; HTML.td []
                [ HTML.a [HTML.href "%s" ("/binding/" ^ handler)]
                    [txt "%s" handler] ]
            ])
        routes
    in
    let promote_checks =
```

```
283         List.map
284           (fun (fname, _, _) ->
285             HTML.p []
286               [ HTML.label []
287                   [ HTML.input
288                       [ HTML.type_ "checkbox"
289                       ; HTML.name "name"
290                       ; HTML.value "%s" fname
291                       ]
292                   ; txt " %s" fname
293                   ]
294               ])
295           ov
296       in
297       (* optional diff *)
298       let target = reqd |> Httpaf.Reqd.request |> fun r ->
      r.Httpaf.Request.target in
299       let diff_name = query_param target "diff" in
300       let* diff_block =
301         match diff_name, parent_id with
302         | None, _ | Some "", _ -> Lwt.return []
303         | Some dname, None ->
304             let* child =
305               Store.lookup_function_on_branch store ~branch_id:bid
      ~name:dname
306             in
307             Lwt.return
308               [ HTML.div [HTML.class_ "card"]
309                   [ HTML.h2 [] [txt "Diff: %s (no parent)" dname]
310                   ; (match child with
311                      | None -> HTML.p [] [txt "(missing)"]
312                      | Some (e, _, _) ->
313                          HTML.pre []
314                            [txt "%s" (Pricklypear_lisp.Reader.pp_expr e)])
315                   ]
316               ]
317         | Some dname, Some pid ->
318             let* child =
319               Store.lookup_function_on_branch store ~branch_id:bid
      ~name:dname
320             in
321             let* parent =
322               Store.lookup_function_on_branch store ~branch_id:pid
      ~name:dname
323             in
324             let show = function
325               | None -> "(absent)"
326               | Some (e, _, _) -> Pricklypear_lisp.Reader.pp_expr e
327             in
328             Lwt.return
329               [ HTML.div [HTML.class_ "card"]
330                   [ HTML.h2 [] [txt "Diff: %s" dname]
331                   ; HTML.div [HTML.class_ "br-diff"]
332                       [ HTML.div []
333                           [ HTML.h3 [] [txt "parent"]
334                           ; HTML.pre [] [txt "%s" (show parent)]
335                           ]
336                       ; HTML.div []
337                           [ HTML.h3 [] [txt "this branch"]
338                           ; HTML.pre [] [txt "%s" (show child)]
339                           ]
340                       ]
341                   ]
342               ]
343       in
344       let body =
345         chrome ~session ~title:("Branch " ^ ref_)
346           (branches_flash msg @ branches_err err @
347            [ HTML.p []
348                [ HTML.a [HTML.href "/branches"] [txt "? all branches"]
349                ; txt " ? parent: "
350                ; (if parent_ref = "--" then txt "--"
351                   else
```

```
                  HTML.a [HTML.href "%s" ("/branches?ref=" ^ parent_ref)]
                    [txt "%s" parent_ref])
            ; txt " ? "
            ; checkout_form ~ref_ ~current:is_cur
            ]
      ; HTML.div [HTML.class_ "br-stats"]
          [ HTML.span [HTML.class_ "card br-stat"]
              [txt "own functions: %d" n_fn]
          ; HTML.span [HTML.class_ "card br-stat"]
              [txt "overrides: %d" n_ov]
          ; HTML.span [HTML.class_ "card br-stat"]
              [txt "own routes: %d" n_rt]
          ]
      ; HTML.div [HTML.class_ "card"]
          [ HTML.h2 [] [txt "Overrides / own pins"]
          ; HTML.p [HTML.class_ "muted"]
              [txt "Rows on this branch. ?use parent? deletes the pin
(unoverride)."]
          ; (if override_rows = [] then
               HTML.p [HTML.class_ "muted"] [txt "(none -- fully
inherited)"]
             else
               HTML.table [HTML.class_ "br-table"]
                 [ HTML.thead []
                     [ HTML.tr []
                         [ HTML.th [] [txt "name"]
                         ; HTML.th [] [txt "kind"]
                         ; HTML.th [] [txt "updated"]
                         ; HTML.th [] [txt "actions"]
                         ]
                     ]
                 ; HTML.tbody [] override_rows
                 ])
          ]
      ; HTML.div [HTML.class_ "card"]
          [ HTML.h2 [] [txt "All own code"]
          ; (if own_rows = [] then
               HTML.p [HTML.class_ "muted"] [txt "(empty branch)"]
             else
               HTML.table [HTML.class_ "br-table"]
                 [ HTML.thead []
                     [ HTML.tr []
                         [ HTML.th [] [txt "name"]
                         ; HTML.th [] [txt "author"]
                         ; HTML.th [] [txt "updated"]
                         ]
                     ]
                 ; HTML.tbody [] own_rows
                 ])
          ]
      ; HTML.div [HTML.class_ "card"]
          [ HTML.h2 [] [txt "Own routes"]
          ; (if route_rows = [] then
               HTML.p [HTML.class_ "muted"] [txt "(none on this
branch)"]
             else
               HTML.table [HTML.class_ "br-table"]
                 [ HTML.thead []
                     [ HTML.tr []
                         [ HTML.th [] [txt "route"]
                         ; HTML.th [] [txt "handler"]
                         ]
                     ]
                 ; HTML.tbody [] route_rows
                 ])
          ]
      ; HTML.div [HTML.class_ "card"]
          [ HTML.h2 [] [txt "Promote to system/main"]
          ; HTML.p [HTML.class_ "muted"]
              [txt "Select pins to merge into live defaults. Also
pp_sync pull + git push for durable libs/*.pp."]
          ; (if promote_checks = [] then
```

```
                    HTML.p [HTML.class_ "muted"] [txt "(nothing to
    promote)"]
                else
                  HTML.form
                    [ HTML.method_ `POST
                    ; HTML.action "/branches/promote"
                    ; HTML.enctype `urlencoded
                    ]
                    (HTML.input
                       [ HTML.type_ "hidden"
                       ; HTML.name "ref"
                       ; HTML.value "%s" ref_
                       ]
                     :: promote_checks
                     @ [ HTML.p []
                           [ HTML.button [HTML.type_ "submit"]
                               [txt "Merge selected ? system/main"] ]
                       ]))
            ]
          ]
          @ diff_block)
      in
      html_response reqd ~session body;
      Lwt.return_unit

let branches_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t
    =
  let target = (Httpaf.Reqd.request reqd).target in
  match query_param target "ref" with
  | Some ref_ when String.trim ref_ <> "" ->
      branch_detail_page session reqd ~ref_:(String.trim ref_) ()
  | _ -> list_branches_page session reqd ()

let branches_checkout (session : Session.t) (reqd : Httpaf.Reqd.t) : unit
    Lwt.t =
  let* body = read_body reqd in
  let ref_ = Option.value (form_field body "ref") ~default:"" |>
    String.trim in
  if ref_ = "" then list_branches_page session reqd ~err:"ref required" ()
  else
    let* r = Session.checkout_ref session ref_ in
    match r with
    | Error e -> list_branches_page session reqd ~err:e ()
    | Ok () ->
        redirect_response reqd ("/branches?ref=" ^ ref_);
        Lwt.return_unit

let branches_fork (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t
    =
  let* body = read_body reqd in
  let from_ = Option.value (form_field body "from") ~default:"system/main"
    in
  let name = Option.value (form_field body "name") ~default:"" |>
    String.trim in
  if name = "" then list_branches_page session reqd ~err:"name required" ()
  else
    let owner = Session.username session in
    let child_ref =
      if String.contains name '/' then name else owner ^ "/" ^ name
    in
    let* r = Session.fork_ref session ~parent_ref:from_ ~child_ref in
    match r with
    | Error e -> list_branches_page session reqd ~err:e ()
    | Ok _ ->
        list_branches_page session reqd
          ~msg:("forked " ^ child_ref ^ " from " ^ from_) ()

let branches_unoverride (session : Session.t) (reqd : Httpaf.Reqd.t)
  : unit Lwt.t =
  let* body = read_body reqd in
  let ref_ = Option.value (form_field body "ref") ~default:"" |>
    String.trim in
```

```
  let name = Option.value (form_field body "name") ~default:"" |>
     String.trim in
  (* Checkout target branch first if needed so delete hits the right id. *)
  let* _ =
    if ref_ <> "" && Session.branch_name session <> ref_ then
      Session.checkout_ref session ref_
    else Lwt.return (Ok ())
  in
  let* r = Session.unoverride session name in
  match r with
  | Error e ->
      branch_detail_page session reqd ~ref_ ~err:e ()
  | Ok true ->
      branch_detail_page session reqd ~ref_
        ~msg:("unoverrode " ^ name ^ " -- now inherits parent") ()
  | Ok false ->
      branch_detail_page session reqd ~ref_
        ~err:("no pin named " ^ name ^ " on this branch") ()

let branches_promote (session : Session.t) (reqd : Httpaf.Reqd.t) : unit
    Lwt.t =
  let* body = read_body reqd in
  let ref_ = Option.value (form_field body "ref") ~default:"" |>
     String.trim in
  let form = parse_form_body body in
  let names =
    List.filter_map
      (fun (k, v) -> if k = "name" && String.trim v <> "" then Some v else
     None)
      form
  in
  let* r = Session.promote_to_main session ~source_ref:ref_ ~names in
  match r with
  | Error e -> branch_detail_page session reqd ~ref_ ~err:e ()
  | Ok n ->
      branch_detail_page session reqd ~ref_
        ~msg:(Printf.sprintf "promoted %d function(s) ? system/main" n) ()
```

40 image/lib/http_code.ml

```
(** Public release: read-only source browser, CodeMirror assets,
    book.pdf + src.tgz from PP_PUBLIC_DIR.

    See product-experience.public-release. *)
let env_dir (name : string) (default : string) : string =
  match Sys.getenv_opt name with
  | Some d when String.trim d <> "" -> d
  | _ -> default

let source_root () : string =
  match Sys.getenv_opt "PP_SOURCE_ROOT" with
  | Some d when String.trim d <> "" -> d
  | _ ->
      (match Sys.getenv_opt "PP_ROOT" with
       | Some d when String.trim d <> "" -> d
       | _ ->
           if Sys.file_exists "pricklypear.borg" then "."
           else if Sys.file_exists "../pricklypear.borg" then ".."
           else ".")

let public_dir () : string =
  match Sys.getenv_opt "PP_PUBLIC_DIR" with
  | Some d when String.trim d <> "" -> d
  | _ ->
      let cand = Filename.concat (source_root ()) "public-artifacts" in
      if Sys.file_exists cand then cand else "/tmp/pp-public"

let static_root () : string =
  match Sys.getenv_opt "PP_STATIC_DIR" with
  | Some d when String.trim d <> "" -> d
  | _ ->
      let root = source_root () in
      let a = Filename.concat root "image/static" in
      if Sys.file_exists a then a
      else if Sys.file_exists "image/static" then "image/static"
      else "image/static"

let skip_dirs : string list =
  [ "_build"
  ; ".git"
  ; ".pi"
  ; ".ralph"
  ; "node_modules"
  ; "_opam"
  ; ".opamswitch"
  ; "spikes"
  ; "supervisor"
  ; ".borge-bugs"
  ; "public-artifacts"
  ; "_build_test"
  ]

let allowed_ext (name : string) : bool =
  let lower = String.lowercase_ascii name in
  if lower = "dune" || lower = "dune-project" then true
  else
    List.exists
      (fun suf -> Filename.check_suffix lower suf)
      [ ".borg"
      ; ".ml"
      ; ".mli"
      ; ".pp"
      ; ".md"
      ; ".sql"
      ; ".sh"
      ; ".ksh"
      ; ".dune"
      ; ".txt"
      ; ".opam"
      ]

```

```
72  let max_file_bytes = 2 * 1024 * 1024
73
74  let html_escape (s : string) : string =
75    let buf = Buffer.create (String.length s) in
76    String.iter
77      (function
78        | '&' -> Buffer.add_string buf "&amp;"
79        | '<' -> Buffer.add_string buf "&lt;"
80        | '>' -> Buffer.add_string buf "&gt;"
81        | '"' -> Buffer.add_string buf "&quot;"
82        | '\'' -> Buffer.add_string buf "&#39;"
83        | c -> Buffer.add_char buf c)
84      s;
85    Buffer.contents buf
86
87  let pct_encode (s : string) : string =
88    let buf = Buffer.create (String.length s * 2) in
89    String.iter
90      (function
91        | ('A' .. 'Z' | 'a' .. 'z' | '0' .. '9' | '-' | '_' | '.' | '~' |
       '/') as c ->
92            Buffer.add_char buf c
93        | c -> Buffer.add_string buf (Printf.sprintf "%%%02X" (Char.code c)))
94      s;
95    Buffer.contents buf
96
97  let hex_nibble (c : char) : int option =
98    match c with
99    | '0' .. '9' -> Some (Char.code c - Char.code '0')
100   | 'a' .. 'f' -> Some (Char.code c - Char.code 'a' + 10)
101   | 'A' .. 'F' -> Some (Char.code c - Char.code 'A' + 10)
102   | _ -> None
103
104 let pct_decode (s : string) : string =
105   let buf = Buffer.create (String.length s) in
106   let len = String.length s in
107   let rec loop i =
108     if i >= len then Buffer.contents buf
109     else if s.[i] = '%' && i + 2 < len then (
110       match hex_nibble s.[i + 1], hex_nibble s.[i + 2] with
111       | Some a, Some b ->
112           Buffer.add_char buf (Char.chr ((a lsl 4) lor b));
113           loop (i + 3)
114       | _ ->
115           Buffer.add_char buf s.[i];
116           loop (i + 1))
117     else if s.[i] = '+' then (
118       Buffer.add_char buf ' ';
119       loop (i + 1))
120     else (
121       Buffer.add_char buf s.[i];
122       loop (i + 1))
123   in
124   loop 0
125
126 let realpath_opt (path : string) : string option =
127   try Some (Unix.realpath path) with
128   | Unix.Unix_error _ | Sys_error _ -> None
129
130 (** Strip leading slashes; reject .. and empty segments.
131     Callers should pct-decode first so %2e%2e cannot sneak through. *)
132 let normalize_rel (rel : string) : string option =
133   let rel =
134     let len = String.length rel in
135     if len > 0 && rel.[0] = '/' then String.sub rel 1 (len - 1) else rel
136   in
137   if rel = "" then Some ""
138   else
139     let parts = String.split_on_char '/' rel in
140     let ok =
141       List.for_all
142         (fun p ->
```

```
          p <> "" && p <> "." && p <> ".." && not (String.contains p
     '\000'))
        parts
    in
    if not ok then None else Some (String.concat "/" parts)

(** Resolve [rel] under [root]. Does not require the target to exist
    (realpath on missing files fails); safety is normalize_rel forbidding
    ".." plus realpath on the root only. *)
let resolve_under ~(root : string) (rel : string) : string option =
  match realpath_opt root with
  | None -> None
  | Some root_real ->
      (match normalize_rel rel with
       | None -> None
       | Some "" -> Some root_real
       | Some rel -> Some (Filename.concat root_real rel))

let read_file_limited (path : string) : (string, string) result =
  try
    let st = Unix.stat path in
    if st.Unix.st_kind <> Unix.S_REG then Error "not a file"
    else if st.Unix.st_size > max_file_bytes then Error "file too large"
    else
      let ic = open_in_bin path in
      let len = in_channel_length ic in
      let s = really_input_string ic len in
      close_in ic;
      Ok s
  with exn -> Error (Printexc.to_string exn)

let mime_for_name (name : string) : string =
  let lower = String.lowercase_ascii name in
  if Filename.check_suffix lower ".css" then "text/css; charset=utf-8"
  else if Filename.check_suffix lower ".js" then
    "application/javascript; charset=utf-8"
  else if Filename.check_suffix lower ".pdf" then "application/pdf"
  else if Filename.check_suffix lower ".tgz"
       || Filename.check_suffix lower ".gz" then "application/gzip"
  else if Filename.check_suffix lower ".svg" then "image/svg+xml"
  else if Filename.check_suffix lower ".png" then "image/png"
  else if Filename.check_suffix lower ".jpg"
       || Filename.check_suffix lower ".jpeg" then "image/jpeg"
  else if Filename.check_suffix lower ".gif" then "image/gif"
  else if Filename.check_suffix lower ".webp" then "image/webp"
  else if Filename.check_suffix lower ".ico" then "image/x-icon"
  else "application/octet-stream"

let cm_mode_for (name : string) : string =
  let lower = String.lowercase_ascii name in
  if Filename.check_suffix lower ".ml"
     || Filename.check_suffix lower ".mli" then "text/x-ocaml"
  else if Filename.check_suffix lower ".md" then "text/x-markdown"
  else "text/plain"

type tree_node =
  | Dir of string * tree_node list
  | File of string * string (* name, rel_path *)

let rec walk_tree ~(root : string) ~(rel : string) : tree_node list =
  let full = if rel = "" then root else Filename.concat root rel in
  let entries =
    try Sys.readdir full |> Array.to_list with Sys_error _ -> []
  in
  let entries = List.sort String.compare entries in
  List.filter_map
    (fun name ->
      if List.mem name skip_dirs then None
      else if String.length name > 0 && name.[0] = '.' then None
      else
        let child_rel = if rel = "" then name else rel ^ "/" ^ name in
        let child_full = Filename.concat full name in
        try
```

```
          if Sys.is_directory child_full then
            let kids = walk_tree ~root ~rel:child_rel in
            if kids = [] then None else Some (Dir (name, kids))
          else if allowed_ext name then Some (File (name, child_rel))
          else None
        with Sys_error _ -> None)
    entries

let rec tree_to_html (nodes : tree_node list) ~(active : string) : string =
  let buf = Buffer.create 256 in
  Buffer.add_string buf "<ul class=\"code-tree\">";
  List.iter
    (function
      | Dir (name, kids) ->
          Buffer.add_string buf "<li class=\"code-dir\"><details open>";
          Buffer.add_string buf "<summary>";
          Buffer.add_string buf (html_escape name);
          Buffer.add_string buf "</summary>";
          Buffer.add_string buf (tree_to_html kids ~active);
          Buffer.add_string buf "</details></li>"
      | File (name, rel) ->
          let cls =
            if rel = active then "code-file active" else "code-file"
          in
          let href = "/code?f=" ^ pct_encode rel in
          Buffer.add_string buf (Printf.sprintf "<li class=\"%s\">" cls);
          Buffer.add_string buf
            (Printf.sprintf "<a href=\"%s\">%s</a>" href (html_escape
    name));
          Buffer.add_string buf "</li>")
    nodes;
  Buffer.add_string buf "</ul>";
  Buffer.contents buf

let code_css : string =

    "body{margin:0;font-family:ui-monospace,Menlo,monospace;background:#faf7f4;color:#1a1a1a}\n\

    header.code-hdr{display:flex;flex-wrap:wrap;gap:.75rem;align-items:center;\n\
   justify-content:space-between;padding:.5rem .75rem;border-bottom:2px
    solid #000;\n\
   background:#fff}\n\
   header.code-hdr a{color:#000080;font-weight:700}\n\
   .code-wrap{display:flex;min-height:calc(100vh - 3rem)}\n\
   .code-side{width:16rem;min-width:12rem;max-width:40vw;overflow:auto;\n\
   border-right:2px solid
     #000;background:#e8e2db;padding:.5rem;font-size:.85rem}\n\
   .code-main{flex:1;display:flex;flex-direction:column;min-width:0}\n\
   .code-meta{padding:.35rem .75rem;border-bottom:1px solid
     #808080;background:#fff;\n\
   font-size:.85rem}\n\
   .code-editor{flex:1;min-height:0}\n\
   .CodeMirror{height:100%;font-size:13px;line-height:1.4}\n\
   ul.code-tree{list-style:none;margin:0;padding-left:.85rem}\n\
   li.code-file a{text-decoration:none;color:#000}\n\
   li.code-file.active a{font-weight:700;background:#fff;padding:0 .2rem}\n\
   li.code-dir>details>summary{cursor:pointer;font-weight:700}\n\
   .code-err{color:#8b0000;padding:1rem}\n\
   @media(max-width:40rem){.code-wrap{flex-direction:column}\n\

     .code-side{width:auto;max-width:none;max-height:30vh;border-right:none;\n\
   border-bottom:2px solid #000}}\n"

let serve_bytes (reqd : Httpaf.Reqd.t) ?(status = `OK) ~content_type
    (body : string) : unit =
  let headers =
    Httpaf.Headers.of_list
      [ ("content-type", content_type)
      ; ("content-length", string_of_int (String.length body))
      ; ("cache-control", "public, max-age=3600")
      ]
  in
  let resp = Httpaf.Response.create ~headers status in
```

```
  Httpaf.Reqd.respond_with_string reqd resp body

let serve_file_path (reqd : Httpaf.Reqd.t) ~(path : string) : unit =
  match read_file_limited path with
  | Error msg ->
      serve_bytes reqd ~status:`Not_found ~content_type:"text/plain; charset=utf-8"
        ("not found: " ^ msg)
  | Ok body ->
      let ct = mime_for_name (Filename.basename path) in
      serve_bytes reqd ~content_type:ct body

let handle_asset (reqd : Httpaf.Reqd.t) ~(rel : string) : unit =
  let rel = pct_decode rel in
  match resolve_under ~root:(static_root ()) rel with
  | None ->
      serve_bytes reqd ~status:`Forbidden ~content_type:"text/plain; charset=utf-8"
        "forbidden"
  | Some path ->
      if Sys.file_exists path && not (Sys.is_directory path) then
        serve_file_path reqd ~path
      else
        serve_bytes reqd ~status:`Not_found ~content_type:"text/plain; charset=utf-8"
          "not found"

let handle_public_artifact (reqd : Httpaf.Reqd.t) ~(name : string) : unit =
  let root = public_dir () in
  match resolve_under ~root name with
  | None ->
      serve_bytes reqd ~status:`Not_found ~content_type:"text/html; charset=utf-8"
        ("<!doctype html><title>missing</title><p>Artifact <code>"
         ^ name
         ^ "</code> not installed. Run <code>scripts/public_artifacts.sh</code> \
            and set <code>PP_PUBLIC_DIR</code>.</p><p><a href=\"/welcome\">welcome</a></p>")
  | Some path ->
      if Sys.file_exists path && not (Sys.is_directory path) then
        serve_file_path reqd ~path
      else
        serve_bytes reqd ~status:`Not_found ~content_type:"text/html; charset=utf-8"
          ("<!doctype html><title>missing</title><p>No <code>"
           ^ name
           ^ "</code> in <code>"
           ^ root
           ^ "</code>. Run <code>scripts/public_artifacts.sh</code>.</p>\
              <p><a href=\"/welcome\">welcome</a> ? <a href=\"/code\">code</a></p>")

let handle_raw (reqd : Httpaf.Reqd.t) ~(rel : string) : unit =
  let root = source_root () in
  let rel = pct_decode rel in
  match resolve_under ~root rel with
  | None ->
      serve_bytes reqd ~status:`Forbidden ~content_type:"text/plain; charset=utf-8"
        "forbidden path"
  | Some path ->
      if not (Sys.file_exists path) then
        serve_bytes reqd ~status:`Not_found ~content_type:"text/plain; charset=utf-8"
          "not found"
      else if Sys.is_directory path then
        serve_bytes reqd ~status:`Bad_request ~content_type:"text/plain; charset=utf-8"
          "is a directory"
      else if not (allowed_ext (Filename.basename path)) then
        serve_bytes reqd ~status:`Forbidden ~content_type:"text/plain; charset=utf-8"
```

```
          "extension not allowed"
      else (
        match read_file_limited path with
        | Error msg ->
            serve_bytes reqd ~status:`Bad_request
              ~content_type:"text/plain; charset=utf-8" msg
        | Ok body ->
            serve_bytes reqd ~content_type:"text/plain; charset=utf-8" body)

let query_param (target : string) (key : string) : string option =
  match String.index_opt target '?' with
  | None -> None
  | Some i ->
      let q = String.sub target (i + 1) (String.length target - i - 1) in
      let pairs = String.split_on_char '&' q in
      List.find_map
        (fun pair ->
          match String.split_on_char '=' pair with
          | k :: rest when k = key ->
              let v = String.concat "=" rest in
              Some (pct_decode v)
          | _ -> None)
        pairs

let handle_code_page (reqd : Httpaf.Reqd.t) ~(target : string) : unit =
  let root = source_root () in
  let active =
    match query_param target "f" with
    | Some f when f <> "" -> f
    | _ ->
        if Sys.file_exists (Filename.concat root "pricklypear.borg") then
          "pricklypear.borg"
        else ""
  in
  let tree = walk_tree ~root ~rel:"" in
  let tree_html = tree_to_html tree ~active in
  let mode = cm_mode_for active in
  let title =
    if active = "" then "Source" else active
  in
  let boot_js =
    Printf.sprintf
      {|
var MODE=%s;
var FILE=%s;
var ta=document.getElementById('src');
var cm=CodeMirror.fromTextArea(ta,{
  lineNumbers:true,
  readOnly:true,
  mode:MODE,
  lineWrapping:true,
  viewportMargin:40
});
function loadFile(path){
  if(!path){cm.setValue('// pick a file from the tree');return;}
  fetch('/code/raw/'+path.split('/').map(encodeURIComponent).join('/'))
    .then(function(r){if(!r.ok)throw new Error(r.statusText);return
     r.text();})
    .then(function(t){
      var m='text/plain';
      if(/\.ml$|\.mli$/i.test(path))m='text/x-ocaml';
      else if(/\.md$/i.test(path))m='text/x-markdown';
      cm.setOption('mode',m);
      cm.setValue(t);
      document.getElementById('code-path').textContent=path;
    })
    .catch(function(e){cm.setValue('// error: '+e);});
}
loadFile(FILE);
|}
      (Yojson.Safe.to_string (`String mode))
      (Yojson.Safe.to_string (`String active))
  in
```

```
  let html =
    Printf.sprintf
      {|<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<title>Pricklypear -- %s</title>
<link rel="stylesheet" href="/assets/cm/codemirror.min.css"/>
<style>%s</style>
</head>
<body>
<header class="code-hdr">
  <div><strong>Pricklypear source</strong> ? read-only</div>
  <nav>
    <a href="/welcome">Welcome</a> ?
    <a href="/book">Book</a> ?
    <a href="/book.pdf">PDF</a> ?
    <a href="/src.tgz">.tgz</a> ?
    <a href="/login">Log in</a>
  </nav>
</header>
<div class="code-wrap">
  <aside class="code-side">%s</aside>
  <div class="code-main">
    <div class="code-meta">file: <span id="code-path">%s</span> ? root:
     <code>%s</code></div>
    <div class="code-editor"><textarea id="src"></textarea></div>
  </div>
</div>
<script src="/assets/cm/codemirror.min.js"></script>
<script src="/assets/cm/mode/mllike/mllike.min.js"></script>
<script src="/assets/cm/mode/markdown/markdown.min.js"></script>
<script>%s</script>
</body>
</html>|}
      (html_escape title) code_css tree_html (html_escape active)
      (html_escape root) boot_js
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" html

(** Public Lisp pages: body HTML only (no product chrome / private nav). *)
let serve_public_body (reqd : Httpaf.Reqd.t) ?(title = "Pricklypear")
    (body_html : string) : unit =
  let doc =
    Printf.sprintf
      {|<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<title>%s</title>
<style>
body{margin:0;font-family:system-ui,-apple-system,sans-serif;background:#faf7f4;color:#1a1a1a;line-height:1.5}
a{color:#000080}
</style>
</head>
<body>
%s
</body>
</html>|}
      (html_escape title) body_html
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" doc

let fallback_welcome (reqd : Httpaf.Reqd.t) : unit =
  let body =
    {|<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"/>
<title>Pricklypear</title>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<style>
body{font-family:system-ui,sans-serif;max-width:40rem;margin:2rem
     auto;padding:0 1rem;line-height:1.5;background:#faf7f4}
```

```
a{color:#000080;font-weight:700} h1{letter-spacing:-.02em}
.actions a{display:inline-block;margin:.25rem .35rem .25rem 0;padding:.4rem
    .75rem;border:2px solid
    #000;text-decoration:none;color:#000;background:#fff}
</style></head><body>
<h1>Pricklypear</h1>
<p>A multi-user Lisp OS -- live image, Postgres object store, htmx
    interface.</p>
<p class="actions">
<a href="/code">Explore the source</a>
<a href="/book.pdf">Book (PDF)</a>
<a href="/src.tgz">Source tarball</a>
<a href="/login">Log in</a>
</p>
<p>Lisp welcome page not loaded yet (seed layout + welcome packages).</p>
</body></html>|}
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" body

let fallback_book (reqd : Httpaf.Reqd.t) : unit =
  let body =
    {|<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"/>
<title>Pricklypear book</title>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<style>body{font-family:system-ui,sans-serif;max-width:40rem;margin:2rem
    auto;padding:0 1rem;line-height:1.5}
a{font-weight:700}</style></head><body>
<h1>The Pricklypear book</h1>
<p>Paginated literate rendering of the codebase via <code>borge book
    print</code>.</p>
<p><a href="/book.pdf">Download PDF</a> ? <a href="/src.tgz">tarball</a> ?
<a href="/code">browse</a> ? <a href="/welcome">welcome</a></p>
</body></html>|}
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" body
```

41 image/lib/http_core.ml

```
(** Pricklypear Image -- HTTP shared infrastructure.
    Responses, chrome, forms, auth gate helpers, login/admin.
    Page handlers live in [Http_pages]; dispatch in [Http]. *)
open Lwt.Syntax
open Pure_html

let default_port = 8080

let emergency_css : string =
  (* Labeled recovery only -- NOT the product design system.
     Product CSS comes from Lisp (ui/css) via Session.ui_css. *)
  "/* emergency-css */\n\
   body{font-family:ui-monospace,Menlo,monospace;margin:1rem;\n\
   background:#fff3cd;color:#1a1a1a;line-height:1.4}\n\
   .emergency-banner{display:block;width:100%;padding:.4rem .55rem;\n\
   border:2px solid #8b0000;background:#fff3cd;color:#1a1a1a;\n\
   font-size:.85rem;font-family:Courier,monospace;margin:.35rem 0}\n\
   .emergency-banner a{color:#000080}\n\
   nav.emergency-nav{width:100%;margin-top:.35rem}\n\
   .emergency-home{border:2px solid
    #8b0000;background:#fff3cd;padding:.75rem}\n\
   a{color:#000080}\n\
   header{border-bottom:2px solid #000;padding:.5rem;background:#fff}\n\
   main{padding:.75rem}\n\
   .card{border:2px solid #000;background:#fff;padding:.75rem;margin:.5rem
    0}\n\
   .err{color:#8b0000;font-weight:700}\n\
   .ok{color:#006600;font-weight:700}\n\
   button,.btn,input[type=submit]{font:inherit;padding:.45rem .75rem;\n\
   border:2px solid #000;background:#eee;cursor:pointer}\n\
   table{border-collapse:collapse;width:100%}\n\
   th,td{border:1px solid #808080;padding:.35rem}\n\
   pre,textarea,input[type=text],input[type=password]{\n\
   font-family:ui-monospace,Menlo,monospace;width:100%;\n\
   box-sizing:border-box;border:1px solid #000;padding:.4rem}\n\
   "

let html_response (reqd : Httpaf.Reqd.t) ?(status = `OK)
    ?session ?(css : string option) (body : Pure_html.node) : unit =
  let css =
    match css with
    | Some c when String.trim c <> "" -> c
    | _ ->
        (match session with
         | Some s ->
             (match Session.ui_css s with
              | Some c when String.trim c <> "" -> c
              | _ -> emergency_css)
         | None -> emergency_css)
  in
  let html_doc = HTML.html [HTML.lang "en"] [
    HTML.head [] [
      HTML.title [] "Pricklypear";
      HTML.meta [HTML.charset "utf-8"];
      (* Mobile-first: prevent tiny text, enable proper layout *)
      HTML.meta [HTML.name "viewport";
                 HTML.content "width=device-width, initial-scale=1"];
      HTML.style [] "%s" css;

      (* htmx for create-in-place editor empty state *)
      HTML.script [HTML.src "https://unpkg.com/htmx.org@1.9.12"] "";
      (* Close nav dropdowns on outside click / when another opens *)
      HTML.script [] "%s"
        ("document.addEventListener('click',function(e){" ^
         (* Outline: only .ol-fold on the summary toggles collapse. *)
         "var
    sum=e.target&&e.target.closest?e.target.closest('summary.ol-sum'):null;"
    ^

    "if(sum&&!(e.target.closest&&e.target.closest('.ol-fold'))){e.preventDefault();}"
    ^
```

```
66
     "document.querySelectorAll('details.menu-drop[open],details.ol-actions[open]').forEach(function(d){"
     ^
67           "if(!d.contains(e.target))d.removeAttribute('open');});});" ^
68           "document.addEventListener('toggle',function(e){" ^
69           "var d=e.target;if(!d||!d.classList)return;" ^
70           "var cls=d.classList.contains('menu-drop')?'menu-drop':" ^
71           "(d.classList.contains('ol-actions')?'ol-actions':null);" ^
72           "if(!cls||!d.open)return;" ^
73
     "document.querySelectorAll('details.'+cls+'[open]').forEach(function(o){"
     ^
74           "if(o!==d)o.removeAttribute('open');});},true);");
75       ];
76       HTML.body [] [body];
77     ] in
78     let s = Pure_html.to_string html_doc in
79     let headers =
80       Httpaf.Headers.of_list
81         [("content-type", "text/html; charset=utf-8");
82          ("content-length", string_of_int (String.length s))]
83     in
84     let resp = Httpaf.Response.create ~headers status in
85     Httpaf.Reqd.respond_with_string reqd resp s
86
87   let text_response (reqd : Httpaf.Reqd.t) ?(status = `OK) ~content_type (s :
       string) : unit =
88     let headers =
89       Httpaf.Headers.of_list
90         [("content-type", content_type);
91          ("content-length", string_of_int (String.length s))]
92     in
93     let resp = Httpaf.Response.create ~headers status in
94     Httpaf.Reqd.respond_with_string reqd resp s
95
96   let json_response (reqd : Httpaf.Reqd.t) ?(status = `OK) (j :
       Yojson.Safe.t) : unit =
97     let s = Yojson.Safe.to_string j in
98     text_response reqd ~status ~content_type:"application/json" s
99
100  (** Labeled emergency nav only -- NOT a product menu twin.
101      Product chrome is Lisp (ui/nav). See image.system-browser. *)
102  let emergency_nav_nodes () : Pure_html.node list =
103    [ HTML.span [HTML.class_ "emergency-banner"]
104        [ txt "EMERGENCY NAV -- ui/nav missing or failed ? "
105        ; HTML.a [HTML.href "/bindings?pkg=ui"] [txt "fix ui shell"]
106        ; txt " ? "
107        ; HTML.a [HTML.href "/"] [txt "home"]
108        ; txt " ? "
109        ; HTML.a [HTML.href "/login"] [txt "login"]
110        ; txt " ? "
111        ; HTML.a [HTML.href "/health"] [txt "health"]
112        ]
113    ]
114
115  let value_to_html_string (v : Pricklypear_lisp.Ast.value) : string =
116    match Primitives.is_raw_html v with
117    | Some s -> s
118    | None ->
119        (match v with
120         | Pricklypear_lisp.Ast.VStr s -> s
121         | _ -> Pricklypear_lisp.Reader.pp_value v)
122
123  let try_ui_fn (session : Session.t) (name : string) : string option Lwt.t =
124    Lwt.catch
125      (fun () ->
126        let* r = Session.call_handler session name in
127        match r with
128        | Some v -> Lwt.return (Some (value_to_html_string v))
129        | None ->
130            Printf.eprintf "[http] try_ui_fn %s: unbound or no value\n%!"
       name;
131            Lwt.return None)
```

```
    (fun exn ->
      Printf.eprintf "[http] try_ui_fn %s failed: %s\n%!"
        name (Printexc.to_string exn);
      Lwt.return None)

(** Fragment / partial: no document chrome (htmx swap target). *)
let is_partials_path (path : string) : bool =
  String.length path >= 10 && String.sub path 0 10 = "/partials/"

let header_list (req : Httpaf.Request.t) : (string * string) list =
  Httpaf.Headers.to_list req.headers
  |> List.map (fun (k, v) -> (String.lowercase_ascii k, v))

let wants_fragment ~(path : string) ~(headers : (string * string) list) :
    bool =
  if is_partials_path path then true
  else
    match List.assoc_opt "hx-request" headers with
    | Some v when v <> "" && String.lowercase_ascii v <> "false" -> true
    | _ -> false

(* ---------- Auth (single-image Option B: in-process sessions) ----------
    *)

let cookie_name = "pp_session"

(** Serialize request identity attach -- cotenant multi-user shares one
    Session.t; concurrent Alice/Bob would otherwise race branch/env. *)
let request_mutex = Lwt_mutex.create ()

let auth_disabled () : bool =
  match Sys.getenv_opt "PP_AUTH_DISABLED" with
  | Some ("1" | "true" | "TRUE" | "yes" | "YES") -> true
  | _ -> false

let has_prefix (path : string) (prefix : string) : bool =
  path = prefix
  || (String.length path > String.length prefix
      && String.sub path 0 (String.length prefix) = prefix
      && path.[String.length prefix] = '/')

let is_system_hard_public (path : string) : bool =
  path = "/login"
  || path = "/health"
  || path = "/logout"
  || path = "/welcome"
  || path = "/book"
  || path = "/book.pdf"
  || path = "/src.tgz"
  || path = "/code"
  || has_prefix path "/code"
  || path = "/assets"
  || has_prefix path "/assets"

(** Kernel paths Lisp cannot open to the world (auth.route-visibility). *)
let is_system_hard_private (path : string) : bool =
  let prefixes =
    [ "/admin"
    ; "/data"
    ; "/api"
    ; "/bindings"
    ; "/binding"
    ; "/edit"
    ; "/repl"
    ; "/branches"
    ; "/types"
    ; "/pages"
    ; "/page"
    ; "/inspect"
    ; "/agent"
    ]
  in
  List.exists
```

```
    (fun p ->
      path = p
      || (String.length path > String.length p
          && String.sub path 0 (String.length p) = p
          && path.[String.length p] = '/'))
    prefixes
  || path = "/"

(* Back-compat name used only for hard public. *)
let is_public_path (path : string) : bool = is_system_hard_public path

(** Classify path for the request gate.
    Returns [`Hard_public | `Hard_private | `User of string * string
    | `Unknown] where User is (handler, visibility). *)
let classify_path (session : Session.t) (path : string) =
  if is_system_hard_public path then `Hard_public
  else if is_system_hard_private path then `Hard_private
  else
    match Session.lookup_route_meta session path with
    | Some (handler, vis) -> `User (handler, vis)
    | None -> `Unknown

let parse_cookies (headers : (string * string) list) : (string * string)
     list =
  match List.assoc_opt "cookie" headers with
  | None -> []
  | Some raw ->
      String.split_on_char ';' raw
      |> List.filter_map (fun part ->
             let part = String.trim part in
             match String.index_opt part '=' with
             | None -> None
             | Some i ->
                 let k = String.trim (String.sub part 0 i) in
                 let v =
                   String.trim
                     (String.sub part (i + 1) (String.length part - i - 1))
                 in
                 if k = "" then None else Some (k, v))

let cookie_value (headers : (string * string) list) (name : string)
  : string option =
  List.assoc_opt name (parse_cookies headers)

let b64_decode_opt (s : string) : string option =
  match Base64.decode ~pad:true s with
  | Ok raw -> Some raw
  | Error _ ->
      (match Base64.decode ~pad:false s with
       | Ok raw -> Some raw
       | Error _ -> None)

(** Parse "Basic base64(user:pass)" Authorization header. *)
let parse_basic_auth (headers : (string * string) list)
  : (string * string) option =
  match List.assoc_opt "authorization" headers with
  | None -> None
  | Some v ->
      let v = String.trim v in
      if String.length v < 7 then None
      else
        let scheme = String.lowercase_ascii (String.sub v 0 5) in
        if scheme <> "basic" then None
        else
          let b64 = String.trim (String.sub v 5 (String.length v - 5)) in
          match b64_decode_opt b64 with
          | None -> None
          | Some decoded ->
              (match String.index_opt decoded ':' with
               | None -> None
               | Some i ->
                   let u = String.sub decoded 0 i in
                   let p =
```

```
                      String.sub decoded (i + 1)
                        (String.length decoded - i - 1)
                    in
                    Some (u, p))

let set_cookie_header ~(token : string) : string =
  (* Dev is plain HTTP -- omit Secure. Path=/; HttpOnly; SameSite=Lax. *)
  Printf.sprintf "%s=%s; Path=/; HttpOnly; SameSite=Lax; Max-Age=%d"
    cookie_name token (60 * 60 * 24 * 14)

let clear_cookie_header () : string =
  Printf.sprintf
    "%s=; Path=/; HttpOnly; SameSite=Lax; Max-Age=0" cookie_name

let redirect_with_cookie (reqd : Httpaf.Reqd.t) ~(location : string)
    ~(set_cookie : string) : unit =
  let headers =
    Httpaf.Headers.of_list
      [ ("location", location)
      ; ("set-cookie", set_cookie)
      ; ("content-length", "0") ]
  in
  let resp = Httpaf.Response.create ~headers `See_other in
  Httpaf.Reqd.respond_with_string reqd resp ""

let unauthorized_api (reqd : Httpaf.Reqd.t) : unit =
  json_response reqd ~status:`Unauthorized
    (`Assoc
       [ ("ok", `Bool false)
       ; ("error", `String "authentication required") ])

(** Resolve identity for this request: cookie session, then Basic auth.
    Returns None if unauthenticated. *)
let resolve_identity (session : Session.t)
    ~(headers : (string * string) list)
  : Auth_store.user option Lwt.t =
  let store = Session.store session in
  match cookie_value headers cookie_name with
  | Some token when token <> "" ->
      let* row = Auth_store.lookup_session store ~raw_token:token in
      (match row with
       | Some s ->
           Lwt.return_some
             { Auth_store.id = s.user_id;
               username = s.username;
               is_admin = s.is_admin }
       | None ->
           (* Fall through to Basic if cookie stale. *)
           (match parse_basic_auth headers with
            | None -> Lwt.return_none
            | Some (u, p) ->
                Auth_store.verify_password store ~username:u ~password:p
    ()))
  | _ ->
      (match parse_basic_auth headers with
       | None -> Lwt.return_none
       | Some (u, p) ->
           Auth_store.verify_password store ~username:u ~password:p ())

let attach_from_user (session : Session.t) (user : Auth_store.user)
    ?(auth_token : string option = None)
    ?(checkout_personal_now = false) () : unit Lwt.t =
  Session.attach_identity session
    ~user_id:user.Auth_store.id
    ~username:user.Auth_store.username
    ~is_admin:user.Auth_store.is_admin
    ~auth_token
    ~checkout_personal_now
    ()

let fragment_response (reqd : Httpaf.Reqd.t) ?(status = `OK) (html : string)
    : unit =
  text_response reqd ~status ~content_type:"text/html; charset=utf-8" html
```

```
347
348 let chrome ?nav_html ~(session : Session.t) ~(title : string)
349     (content : Pure_html.node list) : Pure_html.node =
350   let nav_html =
351     match nav_html with
352     | Some _ as s -> s
353     | None -> Session.ui_nav_html session
354   in
355   let nav =
356     match nav_html with
357     | Some html when String.trim html <> "" ->
358         HTML.nav [] [txt ~raw:true "%s" html]
359     | _ -> HTML.nav [HTML.class_ "emergency-nav"] (emergency_nav_nodes ())
360   in
361   HTML.div [] [
362     HTML.header [] [
363       HTML.span [HTML.class_ "brand"] [txt "Pricklypear"];
364       HTML.span [HTML.class_ "meta"] [
365         txt "user=%s  branch=%s  up=%ds"
366           (Session.username session)
367           (Session.branch_name session)
368           (Session.uptime_seconds session);
369       ];
370       nav;
371     ];
372     HTML.main [] (
373       (if String.trim title = "" then content
374        else HTML.h1 [] [txt "%s" title] :: content)
375     );
376   ]
377
378 let chrome_lwt ~(session : Session.t) ~(title : string)
379     (content : Pure_html.node list) : Pure_html.node Lwt.t =
380   let* nav_html = try_ui_fn session "ui/nav" in
381   Lwt.return (chrome ?nav_html ~session ~title content)
382
383 let read_body (reqd : Httpaf.Reqd.t) : string Lwt.t =
384   let body = Httpaf.Reqd.request_body reqd in
385   let promise, wakeup = Lwt.task () in
386   let buf = Buffer.create 1024 in
387   let rec sched () =
388     Httpaf.Body.schedule_read body
389       ~on_eof:(fun () -> Lwt.wakeup_later wakeup (Buffer.contents buf))
390       ~on_read:(fun bs ~off ~len ->
391         Buffer.add_string buf (Bigstringaf.substring bs ~off ~len);
392         sched ())
393   in
394   sched ();
395   promise
396
397 let url_decode (s : string) : string =
398   let buf = Buffer.create (String.length s) in
399   let len = String.length s in
400   let rec loop i =
401     if i >= len then Buffer.contents buf
402     else match s.[i] with
403     | '+' -> Buffer.add_char buf ' '; loop (i + 1)
404     | '%' when i + 2 < len ->
405         let hex = String.sub s (i + 1) 2 in
406         (try
407            let code = int_of_string ("0x" ^ hex) in
408            Buffer.add_char buf (Char.chr code)
409          with _ -> Buffer.add_char buf '%');
410         loop (i + 3)
411     | c -> Buffer.add_char buf c; loop (i + 1)
412   in
413   loop 0
414
415 let form_field (body : string) (name : string) : string option =
416   let pairs = String.split_on_char '&' body in
417   let rec find = function
418     | [] -> None
419     | p :: rest ->
```

```
        (match String.split_on_char '=' p with
         | k :: vs ->
             if url_decode k = name then
               Some (url_decode (String.concat "=" vs))
             else find rest
         | _ -> find rest)
  in
  find pairs

let parse_form_body (body : string) : (string * string) list =
  if body = "" then []
  else
    String.split_on_char '&' body
    |> List.filter_map (fun p ->
           if p = "" then None
           else
             match String.split_on_char '=' p with
             | k :: vs ->
                 Some (url_decode k, url_decode (String.concat "=" vs))
             | _ -> None)

(** Merge query + form params; form fields win on key conflict (it33). *)
let merge_params ~(query : (string * string) list)
    ~(form : (string * string) list) : (string * string) list =
  let tbl = Hashtbl.create 16 in
  List.iter (fun (k, v) -> Hashtbl.replace tbl k v) query;
  List.iter (fun (k, v) -> Hashtbl.replace tbl k v) form;
  Hashtbl.fold (fun k v acc -> (k, v) :: acc) tbl []

let parse_query (target : string) : (string * string) list =
  match String.index_opt target '?' with
  | None -> []
  | Some i ->
      let q = String.sub target (i + 1) (String.length target - i - 1) in
      String.split_on_char '&' q
      |> List.filter_map (fun p ->
             match String.split_on_char '=' p with
             | k :: vs ->
                 Some (url_decode k, url_decode (String.concat "=" vs))
             | _ -> None)

let query_param (target : string) (key : string) : string option =
  List.assoc_opt key (parse_query target)

let max_user_body_bytes = 64 * 1024

let redirect_response (reqd : Httpaf.Reqd.t) (location : string) : unit =
  let headers =
    Httpaf.Headers.of_list
      [ ("location", location)
      ; ("content-length", "0")
      ]
  in
  let resp = Httpaf.Response.create ~headers `See_other in
  Httpaf.Reqd.respond_with_string reqd resp ""

let require_login_redirect (reqd : Httpaf.Reqd.t) ~(path : string) : unit =
  if String.length path >= 5 && String.sub path 0 5 = "/api/" then
    unauthorized_api reqd
  else if path = "/" then
    (* Public host: anonymous workspace root ? welcome landing. *)
    redirect_response reqd "/welcome"
  else
    redirect_response reqd "/login"

let login_form_page ?(error = "") (reqd : Httpaf.Reqd.t) : unit =
  let err =
    if error = "" then HTML.div [] []
    else HTML.p [HTML.class_ "err"] [txt "%s" error]
  in
  let body =
    HTML.html [HTML.lang "en"] [
      HTML.head [] [
```

```
        HTML.title [] "Pricklypear -- Sign in";
        HTML.meta [HTML.charset "utf-8"];
        HTML.meta [HTML.name "viewport";
                   HTML.content "width=device-width, initial-scale=1"];
        HTML.style [] "%s"
          "body{font-family:ui-monospace,Menlo,monospace;max-width:28rem;
           margin:4rem auto;padding:1rem;background:#f4f1ea;color:#1a1a1a}
           .card{border:2px solid #1a1a1a;background:#fff;padding:1.25rem}
           h1{font-size:1.25rem;margin:0 0 1rem}
           label{display:block;margin:.75rem 0 .25rem;font-size:.85rem}

     input[type=text],input[type=password]{width:100%;box-sizing:border-box;
           border:1px solid #1a1a1a;padding:.4rem;font:inherit}
           button{margin-top:1rem;border:2px solid
     #1a1a1a;background:#1a1a1a;
           color:#fff;padding:.45rem .9rem;font:inherit;cursor:pointer}
           .err{color:#8b0000;margin:.5rem 0}
           .muted{color:#555;font-size:.85rem;margin-top:1rem}";
       ];
       HTML.body [] [
         HTML.div [HTML.class_ "card"] [
           HTML.h1 [] [txt "Sign in"];
           err;
           HTML.form [HTML.method_ `POST; HTML.action "/login"] [
             HTML.label [HTML.for_ "username"] [txt "Username"];
             HTML.input [HTML.type_ "text"; HTML.name "username";
                         HTML.id "username"; HTML.required;
                         HTML.autocomplete `username];
             HTML.label [HTML.for_ "password"] [txt "Password"];
             HTML.input [HTML.type_ "password"; HTML.name "password";
                         HTML.id "password"; HTML.required;
                         HTML.autocomplete `current_password];
             HTML.button [HTML.type_ "submit"] [txt "Log in"];
           ];
           HTML.p [HTML.class_ "muted"] [
             txt "Your garden: username/personal after login.";
           ];
         ];
       ];
     ]
  in
  html_response reqd body

let post_login (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let* body = read_body reqd in
  let username =
    Option.value (form_field body "username") ~default:"" |> String.trim
  in
  let password = Option.value (form_field body "password") ~default:"" in
  if username = "" || password = "" then begin
    login_form_page ~error:"Invalid username or password" reqd;
    Lwt.return_unit
  end else
    let store = Session.store session in
    let* user_opt =
      Auth_store.verify_password store ~username ~password ()
    in
    match user_opt with
    | None ->
        login_form_page ~error:"Invalid username or password" reqd;
        Lwt.return_unit
    | Some user ->
        let* minted =
          Auth_store.mint_session store ~user_id:user.Auth_store.id ()
        in
        (match minted with
         | Error e ->
             login_form_page ~error:("Login failed: " ^ e) reqd;
             Lwt.return_unit
         | Ok token ->
             let* () =
               attach_from_user session user ~auth_token:(Some token)
                 ~checkout_personal_now:true ()
```

```
            in
            redirect_with_cookie reqd ~location:"/"
              ~set_cookie:(set_cookie_header ~token);
            Lwt.return_unit

let post_logout (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let headers = header_list (Httpaf.Reqd.request reqd) in
  let token = cookie_value headers cookie_name in
  let store = Session.store session in
  let* () =
    match token with
    | Some t when t <> "" -> Auth_store.revoke_session store ~raw_token:t ()
    | _ -> Lwt.return_unit
  in
  Session.clear_auth_token session;
  redirect_with_cookie reqd ~location:"/login"
    ~set_cookie:(clear_cookie_header ());
  Lwt.return_unit

let admin_users_page (session : Session.t) (reqd : Httpaf.Reqd.t)
  : unit Lwt.t =
  if not (Session.is_admin session) then begin
    html_response reqd ~session ~status:`Forbidden
      (chrome ~session ~title:"Forbidden" [
         HTML.p [HTML.class_ "err"] [txt "admin only"];
       ]);
    Lwt.return_unit
  end else
    let store = Session.store session in
    let* users = Auth_store.list_users store in
    let rows =
      List.map
        (fun (u : Auth_store.user) ->
          HTML.tr [] [
            HTML.td [] [txt "%s" u.Auth_store.username];
            HTML.td [] [txt "%s" (if u.Auth_store.is_admin then "yes" else
    "no")];
            HTML.td [] [HTML.code [] [txt "%s" u.Auth_store.id]];
            HTML.td [] [
              HTML.form
                [ HTML.method_ `POST; HTML.action "/admin/users" ]
                [ HTML.input
                    [ HTML.type_ "hidden"; HTML.name "action";
                      HTML.value "revoke_sessions" ]
                ; HTML.input
                    [ HTML.type_ "hidden"; HTML.name "user_id";
                      HTML.value "%s" u.Auth_store.id ]
                ; HTML.button [HTML.type_ "submit"] [txt "Revoke sessions"]
                ]
            ];
          ])
        users
    in
    let body =
      chrome ~session ~title:"Users" [
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "Users"];
          HTML.table [] [
            HTML.thead [] [
              HTML.tr [] [
                HTML.th [] [txt "username"];
                HTML.th [] [txt "admin"];
                HTML.th [] [txt "id"];
                HTML.th [] [txt "actions"];
              ];
            ];
            HTML.tbody [] rows;
          ];
        ];
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "Create user"];
          HTML.form [HTML.method_ `POST; HTML.action "/admin/users"] [
            HTML.input
```

```
              [ HTML.type_ "hidden"; HTML.name "action";
                HTML.value "create" ];
            HTML.p [] [
              HTML.label [] [txt "username "];
              HTML.input [HTML.type_ "text"; HTML.name "username";
    HTML.required];
            ];
            HTML.p [] [
              HTML.label [] [txt "password "];
              HTML.input [HTML.type_ "password"; HTML.name "password";
    HTML.required];
            ];
            HTML.p [] [
              HTML.label [] [
                HTML.input [HTML.type_ "checkbox"; HTML.name "is_admin";
    HTML.value "1"];
                txt " is_admin";
              ];
            ];
            HTML.button [HTML.type_ "submit"] [txt "Create"];
          ];
        ];
        HTML.p [] [
          HTML.form [HTML.method_ `POST; HTML.action "/logout"] [
            HTML.button [HTML.type_ "submit"] [txt "Log out"];
          ];
        ];
      ]
    in
    html_response reqd ~session body;
    Lwt.return_unit

let post_admin_users (session : Session.t) (reqd : Httpaf.Reqd.t)
  : unit Lwt.t =
  if not (Session.is_admin session) then begin
    html_response reqd ~session ~status:`Forbidden
      (chrome ~session ~title:"Forbidden" [
         HTML.p [HTML.class_ "err"] [txt "admin only"];
       ]);
    Lwt.return_unit
  end else
    let* body = read_body reqd in
    let action =
      Option.value (form_field body "action") ~default:"create"
      |> String.trim
    in
    if action = "revoke_sessions" then
      let user_id =
        Option.value (form_field body "user_id") ~default:"" |> String.trim
      in
      if user_id = "" then begin
        html_response reqd ~session ~status:`Bad_request
          (chrome ~session ~title:"Revoke sessions" [
             HTML.p [HTML.class_ "err"] [txt "user_id required"];
             HTML.p [] [HTML.a [HTML.href "/admin/users"] [txt "Back"]];
           ]);
        Lwt.return_unit
      end else
        let store = Session.store session in
        let* n = Auth_store.revoke_all_sessions store ~user_id in
        let* () =
          Auth_store.log_auth store ~user_id ~kind:"admin_revoke_sessions"
            ~success:true ()
        in
        ignore n;
        redirect_response reqd "/admin/users";
        Lwt.return_unit
    else
      let username =
        Option.value (form_field body "username") ~default:"" |> String.trim
      in
      let password = Option.value (form_field body "password") ~default:""
    in
```

```
      let is_admin =
        match form_field body "is_admin" with
        | Some ("1" | "on" | "true") -> true
        | _ -> false
      in
      if username = "" || password = "" then begin
        html_response reqd ~session ~status:`Bad_request
          (chrome ~session ~title:"Create user" [
             HTML.p [HTML.class_ "err"] [txt "username and password
    required"];
             HTML.p [] [HTML.a [HTML.href "/admin/users"] [txt "Back"]];
           ]);
        Lwt.return_unit
      end else
        let store = Session.store session in
        let* created = Auth_store.create_user store ~username ~is_admin in
        match created with
        | Error e ->
            html_response reqd ~session ~status:`Bad_request
              (chrome ~session ~title:"Create user" [
                 HTML.p [HTML.class_ "err"] [txt "%s" e];
                 HTML.p [] [HTML.a [HTML.href "/admin/users"] [txt "Back"]];
               ]);
            Lwt.return_unit
        | Ok user ->
            let* pw =
              Auth_store.set_password store ~user_id:user.Auth_store.id
                ~password
            in
            (match pw with
             | Error e ->
                 html_response reqd ~session ~status:`Internal_server_error
                   (chrome ~session ~title:"Create user" [
                      HTML.p [HTML.class_ "err"] [txt "password: %s" e];
                    ]);
                 Lwt.return_unit
             | Ok () ->
                 let* () =
                   Auth_store.log_auth store ~user_id:user.Auth_store.id
                     ~kind:"admin_create_user" ~success:true ()
                 in
                 redirect_response reqd "/admin/users";
                 Lwt.return_unit)

let respond_handler_value
    ~(session : Session.t)
    ~(reqd : Httpaf.Reqd.t)
    ~(path : string)
    ~(fragment : bool)
    (value : Pricklypear_lisp.Ast.value) : unit =
  match Primitives.is_redirect value with
  | Some loc -> redirect_response reqd loc
  | None ->
      if fragment then
        let html =
          match Primitives.is_raw_html value with
          | Some raw -> raw
          | None ->
              (match value with
               | Pricklypear_lisp.Ast.VStr s -> s
               | _ -> Pricklypear_lisp.Reader.pp_value value)
        in
        fragment_response reqd html
      else
        let body =
          match Primitives.is_raw_html value with
          | Some raw_html ->
              chrome ~session ~title:path [ txt ~raw:true "%s" raw_html ]
          | None ->
              (match value with
               | Pricklypear_lisp.Ast.VStr s ->
                   chrome ~session ~title:path [ HTML.p [] [txt "%s" s] ]
               | _ ->
```

```
                  let rendered =
                    Pricklypear_lisp.Reader.pp_value value
                  in
                  chrome ~session ~title:path [
                    HTML.pre [] [txt "%s" rendered];
                  ])
        in
        html_response reqd ~session body
```

42 image/lib/http_data.ml

```
(** Pricklypear Image -- /types + /data CRUD browser pages. *)
open Lwt.Syntax
open Pure_html
open Http_core

let list_types_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit
    Lwt.t =
  let* types = Store.list_types (Session.store session) in
  let rows =
    List.map
      (fun (name, fields) ->
        let field_list = String.concat ", " fields in
        HTML.tr [] [
          HTML.td [] [
            HTML.a [HTML.href "/data/%s" name] [txt "%s" name];
          ];
          HTML.td [] [txt "%s" field_list];
          HTML.td [] [
            HTML.a [HTML.href "/data/%s" name] [txt "rows"];
          ];
        ])
      types
  in
  let body =
    chrome ~session ~title:("Data types (" ^ Session.branch_name session ^
    ")") [
      HTML.p [HTML.class_ "meta"] [
        txt "Declared shapes for Data rows. Browse/edit instances at ";
        HTML.a [HTML.href "/data"] [txt "/data"];
        txt " or via (create 'type \"{...}\") in the REPL.";
      ];
      (match types with
       | [] ->
           HTML.p [] [txt "(no types yet; declare via (deftype 'name
    'field1 ...))"]
       | _ ->
           HTML.table [] [
             HTML.thead [] [
               HTML.tr [] [
                 HTML.th [] [txt "type"];
                 HTML.th [] [txt "fields"];
                 HTML.th [] [txt ""];
               ]
             ];
             HTML.tbody [] rows;
           ]);
    ]
  in
  html_response reqd ~session body;
  Lwt.return_unit

(* ---- data browser (image.data-browser) ---- *)

let data_type_name_ok (s : string) : bool =
  let n = String.length s in
  n > 0 && n <= 128 &&
  (try
     for i = 0 to n - 1 do
       match s.[i] with
       | 'a' .. 'z' | 'A' .. 'Z' | '0' .. '9' | '-' | '_' | '/' | '.' -> ()
       | _ -> raise Exit
     done;
     true
   with Exit -> false)

let json_object_assoc (s : string) : ((string * Yojson.Safe.t) list,
    string) result =
  try
    match Yojson.Safe.from_string s with
    | `Assoc kvs -> Ok kvs
    | _ -> Error "fields JSON must be an object"
```

```
  with exn -> Error ("invalid JSON: " ^ Printexc.to_string exn)

let fields_assoc_of_json (s : string) : (string * Yojson.Safe.t) list =
  match json_object_assoc s with
  | Ok kvs -> kvs
  | Error _ -> []

let string_of_json_field (v : Yojson.Safe.t) : string =
  match v with
  | `String s -> s
  | `Int i -> string_of_int i
  | `Intlit s -> s
  | `Float f -> string_of_float f
  | `Bool b -> if b then "true" else "false"
  | `Null -> ""
  | other -> Yojson.Safe.to_string other

let truncate_cell (s : string) ~(max_len : int) : string =
  if String.length s <= max_len then s
  else String.sub s 0 max_len ^ "?"

let short_id (id : string) : string =
  if String.length id <= 8 then id else String.sub id 0 8

(** Build fields JSON for create/update.
    Prefer non-empty fields_json (must be object). Else declared form
    fields as string values. On update, [base] preserves unknown keys. *)
let build_fields_json
    ~(declared : string list)
    ~(form : (string * string) list)
    ~(fields_json_opt : string option)
    ~(base : (string * Yojson.Safe.t) list)
    : (string, string) result =
  match fields_json_opt with
  | Some raw when String.trim raw <> "" ->
      (match json_object_assoc (String.trim raw) with
       | Ok kvs -> Ok (Yojson.Safe.to_string (`Assoc kvs))
       | Error e -> Error e)
  | _ ->
      let form_tbl = Hashtbl.create 16 in
      List.iter (fun (k, v) -> Hashtbl.replace form_tbl k v) form;
      let declared_keys =
        if declared = [] then
          (* no type_defs: take every form key except control fields *)
          Hashtbl.fold
            (fun k _ acc ->
              if k = "fields_json" || k = "action" then acc else k :: acc)
            form_tbl []
          |> List.sort_uniq String.compare
        else declared
      in
      let updated =
        List.fold_left
          (fun acc key ->
            match Hashtbl.find_opt form_tbl key with
            | None -> acc
            | Some v ->
                (key, `String v) :: List.filter (fun (k, _) -> k <> key)
    acc)
          base declared_keys
      in
      Ok (Yojson.Safe.to_string (`Assoc updated))

let shared_data_banner () =
  HTML.p [HTML.class_ "meta"] [
    txt "Data rows are shared across branches (not versioned by branch). ";
    HTML.a [HTML.href "/types"] [txt "Type defs"];
    txt " ? ";
    HTML.a [HTML.href "/data"] [txt "All data"];
  ]

let data_index_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit
    Lwt.t =
```

```
  let store = Session.store session in
  let* types = Store.list_types store in
  let* orphan_names = Crud.list_data_type_names store in
  let declared = List.map fst types in
  let declared_set =
    List.fold_left (fun acc n -> n :: acc) [] declared
  in
  let orphans =
    List.filter (fun n -> not (List.mem n declared_set)) orphan_names
  in
  let* rows =
    Lwt_list.map_s
      (fun (name, fields) ->
        let* n = Crud.count_by_type store ~type_name:name in
        Lwt.return
          (HTML.tr [] [
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "%s" name];
             ];
             HTML.td [] [txt "%s" (String.concat ", " fields)];
             HTML.td [] [txt "%d" n];
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "browse"];
               txt " ? ";
               HTML.a [HTML.href "/data/%s/new" name] [txt "new"];
             ];
           ]))
      types
  in
  let* orphan_rows =
    Lwt_list.map_s
      (fun name ->
        let* n = Crud.count_by_type store ~type_name:name in
        Lwt.return
          (HTML.tr [] [
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "%s" name];
             ];
             HTML.td [] [txt "(no type_def)"];
             HTML.td [] [txt "%d" n];
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "browse"];
             ];
           ]))
      orphans
  in
  let body =
    chrome ~session ~title:"Data" [
      shared_data_banner ();
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Types with rows"];
        (match rows @ orphan_rows with
         | [] ->
             HTML.p [] [
               txt "(no data_rows yet; declare a type via (deftype ?) or ";
               HTML.a [HTML.href "/types"] [txt "/types"];
               txt ")";
             ]
         | all ->
             HTML.table [] [
               HTML.thead [] [
                 HTML.tr [] [
                   HTML.th [] [txt "type"];
                   HTML.th [] [txt "fields"];
                   HTML.th [] [txt "rows"];
                   HTML.th [] [txt ""];
                 ];
               ];
               HTML.tbody [] all;
             ]);
      ];
    ]
  in
```

```
  html_response reqd ~session body;
  Lwt.return_unit

let data_list_page (session : Session.t) (type_name : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
         HTML.p [] [HTML.a [HTML.href "/data"] [txt "Back to /data"]];
       ]);
    Lwt.return_unit
  end else
    let store = Session.store session in
    let target = (Httpaf.Reqd.request reqd).Httpaf.Request.target in
    let limit =
      match query_param target "limit" with
      | Some s ->
          (try max 1 (min 200 (int_of_string s)) with _ -> 50)
      | None -> 50
    in
    let offset =
      match query_param target "offset" with
      | Some s -> (try max 0 (int_of_string s) with _ -> 0)
      | None -> 0
    in
    let* types = Store.list_types store in
    let declared =
      match List.assoc_opt type_name types with
      | Some fs -> fs
      | None -> []
    in
    let* total = Crud.count_by_type store ~type_name in
    let* rows = Crud.list_page store ~type_name ~limit ~offset () in
    let col_fields =
      if declared = [] then
        (* infer columns from first row keys *)
        match rows with
        | r :: _ -> List.map fst (fields_assoc_of_json r.Crud.fields_json)
        | [] -> []
      else declared
    in
    let header_cells =
      HTML.th [] [txt "id"]
      :: List.map (fun f -> HTML.th [] [txt "%s" f]) col_fields
    in
    let data_rows =
      List.map
        (fun (r : Crud.row_record) ->
          let assoc = fields_assoc_of_json r.Crud.fields_json in
          let cells =
            List.map
              (fun f ->
                let v =
                  match List.assoc_opt f assoc with
                  | Some j -> string_of_json_field j
                  | None -> ""
                in
                HTML.td [] [txt "%s" (truncate_cell v ~max_len:48)])
              col_fields
          in
          HTML.tr []
            (HTML.td [] [
               HTML.a
                 [HTML.href "/data/%s/%s" type_name r.Crud.id]
                 [txt "%s" (short_id r.Crud.id)];
             ]
             :: cells))
        rows
    in
    let prev_off = max 0 (offset - limit) in
    let next_off = offset + limit in
    let has_prev = offset > 0 in
```

```
285     let has_next = next_off < total in
286     let body =
287       chrome ~session ~title:("Data ? " ^ type_name) [
288         shared_data_banner ();
289         HTML.div [HTML.class_ "card"] [
290           HTML.h2 [] [txt "%s" type_name];
291           HTML.p [HTML.class_ "meta"] [
292             txt "%d row(s) ? showing %d-%d ? limit %d"
293               total
294               (if total = 0 then 0 else offset + 1)
295               (min total (offset + List.length rows))
296               limit;
297           ];
298           HTML.p [] [
299             HTML.a [HTML.href "/data/%s/new" type_name] [txt "New row"];
300             txt " ? ";
301             HTML.a [HTML.href "/data"] [txt "All types"];
302             txt " ? ";
303             HTML.a [HTML.href "/types"] [txt "Type defs"];
304           ];
305           (match data_rows with
306            | [] ->
307                HTML.p [] [
308                  txt "(no rows yet) ";
309                  HTML.a [HTML.href "/data/%s/new" type_name] [txt "Create
    one"];
310                ]
311             | _ ->
312                HTML.table [] [
313                  HTML.thead [] [HTML.tr [] header_cells];
314                  HTML.tbody [] data_rows;
315                ]);
316           HTML.p []
317             ((if has_prev then
318                 [ HTML.a
319                     [ HTML.href "/data/%s?limit=%d&offset=%d"
320                         type_name limit prev_off ]
321                     [txt "? Prev"] ]
322               else [])
323              @ (if has_prev && has_next then [txt " ? "] else [])
324              @ (if has_next then
325                   [ HTML.a
326                       [ HTML.href "/data/%s?limit=%d&offset=%d"
327                           type_name limit next_off ]
328                       [txt "Next ?"] ]
329                 else []));
330         ];
331       ]
332     in
333     html_response reqd ~session body;
334     Lwt.return_unit
335
336 let field_inputs ~(declared : string list)
337     ~(values : (string * string) list) : Pure_html.node list =
338   let val_of k = Option.value (List.assoc_opt k values) ~default:"" in
339   if declared = [] then
340     [ HTML.p [HTML.class_ "meta"] [
341         txt "No declared fields -- use raw JSON below (or deftype first).";
342       ] ]
343   else
344     List.map
345       (fun f ->
346         HTML.p [] [
347           HTML.label [] [txt "%s " f];
348           HTML.input
349             [ HTML.type_ "text"
350             ; HTML.name "%s" f
351             ; HTML.value "%s" (val_of f)
352             ];
353         ])
354       declared
355
356 let data_new_page (session : Session.t) (type_name : string)
```

```
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    Lwt.return_unit
  end else
    let* types = Store.list_types (Session.store session) in
    let declared =
      match List.assoc_opt type_name types with Some fs -> fs | None -> []
    in
    let body =
      chrome ~session ~title:("New ? " ^ type_name) [
        shared_data_banner ();
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "Create %s" type_name];
          HTML.form
            [ HTML.method_ `POST; HTML.action "/data/%s" type_name ]
            (field_inputs ~declared ~values:[]
             @ [ HTML.p [] [
                   HTML.label [] [txt "fields_json (optional; wins if set)
    "];
                   HTML.textarea
                     [ HTML.name "fields_json" ]
                     "%s" "";
                 ]
               ; HTML.button [HTML.type_ "submit"] [txt "Create"]
               ; HTML.p [] [
                   HTML.a [HTML.href "/data/%s" type_name] [txt "Cancel"];
                 ]
               ]);
        ];
      ]
    in
    html_response reqd ~session body;
    Lwt.return_unit

let data_create (session : Session.t) (type_name : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    Lwt.return_unit
  end else
    let store = Session.store session in
    let* body = read_body reqd in
    let form = parse_form_body body in
    let* types = Store.list_types store in
    let declared =
      match List.assoc_opt type_name types with Some fs -> fs | None -> []
    in
    let fields_json_opt = List.assoc_opt "fields_json" form in
    match
      build_fields_json ~declared ~form ~fields_json_opt ~base:[]
    with
    | Error e ->
        html_response reqd ~session ~status:`Bad_request
          (chrome ~session ~title:"Create failed" [
             HTML.p [HTML.class_ "err"] [txt "%s" e];
             HTML.p [] [
               HTML.a [HTML.href "/data/%s/new" type_name] [txt "Back"];
             ];
           ]);
        Lwt.return_unit
    | Ok fields_json ->
        let author = Session.username session in
        let* v =
          Crud.create store ~type_name ~fields_json ~author
        in
        (match v with
```

```
429           | Pricklypear_lisp.Ast.VStr id
430             when String.length id = 36 && not (String.contains id ' ') ->
431               redirect_response reqd
432                 (Printf.sprintf "/data/%s/%s" type_name id);
433               Lwt.return_unit
434           | Pricklypear_lisp.Ast.VStr msg ->
435               html_response reqd ~session ~status:`Internal_server_error
436                 (chrome ~session ~title:"Create failed" [
437                    HTML.p [HTML.class_ "err"] [txt "%s" msg];
438                    HTML.p [] [
439                      HTML.a [HTML.href "/data/%s/new" type_name] [txt
    "Back"];
440                    ];
441                  ]);
442               Lwt.return_unit
443           | _ ->
444               html_response reqd ~session ~status:`Internal_server_error
445                 (chrome ~session ~title:"Create failed" [
446                    HTML.p [HTML.class_ "err"] [txt "create failed"];
447                  ]);
448               Lwt.return_unit)
449
450 let data_detail_page (session : Session.t) (type_name : string)
451     (id : string) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
452   if not (data_type_name_ok type_name) then begin
453     html_response reqd ~session ~status:`Bad_request
454       (chrome ~session ~title:"Bad type" [
455          HTML.p [HTML.class_ "err"] [txt "invalid type name"];
456        ]);
457     Lwt.return_unit
458   end else
459     let store = Session.store session in
460     let* row_opt = Crud.get_row store ~type_name ~id in
461     match row_opt with
462     | None ->
463         html_response reqd ~session ~status:`Not_found
464           (chrome ~session ~title:"Row not found" [
465              HTML.p [HTML.class_ "err"] [txt "row not found"];
466              HTML.p [] [
467                HTML.a [HTML.href "/data/%s" type_name] [txt "Back to list"];
468              ];
469            ]);
470         Lwt.return_unit
471     | Some row ->
472         let* types = Store.list_types store in
473         let declared =
474           match List.assoc_opt type_name types with
475           | Some fs -> fs
476           | None -> []
477         in
478         let assoc = fields_assoc_of_json row.Crud.fields_json in
479         let declared_set = declared in
480         let extra_keys =
481           List.filter_map
482             (fun (k, _) ->
483               if List.mem k declared_set then None else Some k)
484             assoc
485         in
486         let values =
487           List.map (fun (k, v) -> (k, string_of_json_field v)) assoc
488         in
489         (* Include extra keys so edit form can preserve/update unknowns. *)
490         let form_fields =
491           declared
492           @ List.map (fun k -> k) extra_keys
493         in
494         let meta_bits =
495           [ HTML.p [HTML.class_ "meta"] [txt "id: %s" row.Crud.id] ]
496           @ (match row.Crud.author with
497              | Some a -> [HTML.p [HTML.class_ "meta"] [txt "author: %s" a]]
498              | None -> [])
499           @ (match row.Crud.created_at with
500              | Some t -> [HTML.p [HTML.class_ "meta"] [txt "created: %s" t]]
```

```
              | None -> [])
          @ (match row.Crud.updated_at with
             | Some t -> [HTML.p [HTML.class_ "meta"] [txt "updated: %s" t]]
             | None -> [])
        in
        let pretty =
          try Yojson.Safe.pretty_to_string
                (Yojson.Safe.from_string row.Crud.fields_json)
          with _ -> row.Crud.fields_json
        in
        let body =
          chrome ~session
            ~title:(Printf.sprintf "Data ? %s ? %s" type_name (short_id id))
            [ shared_data_banner ()
            ; HTML.div [HTML.class_ "card"] (
                [ HTML.h2 [] [txt "%s" type_name] ]
                @ meta_bits
                @ [ HTML.p [] [
                      HTML.a [HTML.href "/data/%s" type_name] [txt "? List"];
                      txt " ? ";
                      HTML.a [HTML.href "/data"] [txt "All data"];
                    ]
                  ; HTML.h3 [] [txt "Fields"]
                  ; HTML.pre [] [txt "%s" pretty]
                  ]
              )
            ; HTML.div [HTML.class_ "card"] [
                HTML.h3 [] [txt "Edit"];
                HTML.form
                  [ HTML.method_ `POST
                  ; HTML.action "/data/%s/%s" type_name id
                  ]
                  (field_inputs ~declared:form_fields ~values
                   @ [ HTML.p [] [
                         HTML.label []
                           [txt "fields_json (optional; full replace if set) "];
                         HTML.textarea
                           [ HTML.name "fields_json" ]
                           "%s" "";
                       ]
                     ; HTML.button [HTML.type_ "submit"] [txt "Save"]
                     ]);
              ]
            ; HTML.div [HTML.class_ "card"] [
                HTML.h3 [] [txt "Delete"];
                HTML.form
                  [ HTML.method_ `POST
                  ; HTML.action "/data/%s/%s/delete" type_name id
                  ]
                  [ HTML.button [HTML.type_ "submit"] [txt "Delete row"] ];
              ]
            ]
        in
        html_response reqd ~session body;
        Lwt.return_unit

let data_update (session : Session.t) (type_name : string) (id : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    Lwt.return_unit
  end else
    let store = Session.store session in
    let* body = read_body reqd in
    let form = parse_form_body body in
    let* row_opt = Crud.get_row store ~type_name ~id in
    match row_opt with
    | None ->
```

```
        html_response reqd ~session ~status:`Not_found
          (chrome ~session ~title:"Row not found" [
             HTML.p [HTML.class_ "err"] [txt "row not found"];
           ]);
        Lwt.return_unit
    | Some row ->
        let* types = Store.list_types store in
        let declared =
          match List.assoc_opt type_name types with
          | Some fs -> fs
          | None -> []
        in
        let base = fields_assoc_of_json row.Crud.fields_json in
        let extra_keys =
          List.filter_map
            (fun (k, _) -> if List.mem k declared then None else Some k)
            base
        in
        let form_fields = declared @ extra_keys in
        let fields_json_opt = List.assoc_opt "fields_json" form in
        (match
           build_fields_json ~declared:form_fields ~form ~fields_json_opt
             ~base
         with
         | Error e ->
             html_response reqd ~session ~status:`Bad_request
               (chrome ~session ~title:"Update failed" [
                  HTML.p [HTML.class_ "err"] [txt "%s" e];
                  HTML.p [] [
                    HTML.a
                      [HTML.href "/data/%s/%s" type_name id]
                      [txt "Back"];
                  ];
                ]);
             Lwt.return_unit
         | Ok fields_json ->
             let* v = Crud.update store ~type_name ~id ~fields_json in
             (match v with
              | Pricklypear_lisp.Ast.VStr _ ->
                  redirect_response reqd
                    (Printf.sprintf "/data/%s/%s" type_name id);
                  Lwt.return_unit
              | _ ->
                  html_response reqd ~session ~status:`Not_found
                    (chrome ~session ~title:"Update failed" [
                       HTML.p [HTML.class_ "err"] [txt "row not found"];
                     ]);
                  Lwt.return_unit))

let data_delete (session : Session.t) (type_name : string) (id : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    Lwt.return_unit
  end else
    let store = Session.store session in
    let* v = Crud.delete store ~type_name ~id in
    match v with
    | Pricklypear_lisp.Ast.VStr _ ->
        redirect_response reqd (Printf.sprintf "/data/%s" type_name);
        Lwt.return_unit
    | _ ->
        html_response reqd ~session ~status:`Not_found
          (chrome ~session ~title:"Delete failed" [
             HTML.p [HTML.class_ "err"] [txt "row not found"];
             HTML.p [] [
               HTML.a [HTML.href "/data/%s" type_name] [txt "Back to list"];
             ];
           ]);
        Lwt.return_unit
```

43 image/lib/http_pages.ml

```
(** Pricklypear Image -- HTTP system & product page handlers. *)
open Lwt.Syntax
open Pure_html
open Http_core
open Http_data
open Http_branches
open Http_agent

let show_binding_page (session : Session.t) (name : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let* found = Store.lookup_function (Session.store session) name in
  let body =
    match found with
    | None ->
        chrome ~session ~title:name [
          HTML.p [] [txt "(no such binding on this branch)"];
          HTML.p [] [HTML.a [HTML.href "%s" ("/edit/" ^ name)] [txt "create it"]];
        ]
    | Some (expr, author) ->
        let pp = Pricklypear_lisp.Reader.pp_expr expr in
        chrome ~session ~title:name [
          HTML.p [] [txt "author: %s" author];
          HTML.pre [] [txt "%s" pp];
          HTML.p [] [
            HTML.a [HTML.href "%s" (Packages.edit_href name)] [txt "edit"];
            txt " ? ";
            HTML.a [HTML.href "%s" (Packages.inspect_href name)] [txt "inspect"];
          ];
        ]
  in
  html_response reqd ~session body;
  Lwt.return_unit

open Http_data

let about_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit =
  let body =
    chrome ~session ~title:"About Pricklypear" [
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "What is this?"];
        HTML.p [] [txt
          "Pricklypear is a multi-user live computing environment. You log into your running image, write Lisp functions that are stored as durable data in Postgres, branch code like git while sharing data, bind functions to URLs, and optionally let AI propose functions onto your branch."];
        HTML.p [] [txt
          "It is Smalltalk-like (live, inspectable, editable) without being Smalltalk (no single-user OOP image dump). The image is the CPU; Postgres is the disk."];
      ];
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Core concepts"];
        HTML.ul [] [
          HTML.li [] [txt "Image -- your running personal process (REPL + HTTP). Disposable; rebuildable from the store."];
          HTML.li [] [txt "Store -- Postgres holds functions, types, routes, data rows, auth, branches."];
          HTML.li [] [txt "Function / binding -- a named live value, usually a lambda. Top-level (define ...) persists it."];
          HTML.li [] [txt "Branch -- versioned code world (system/main, alice/experiment). Code branches; data is shared."];
          HTML.li [] [txt "Type + row -- (deftype ...) then (create/get/update/rows ...) over shared data_rows."];
          HTML.li [] [txt "Route / page -- (route \"/path\" \"handler\") makes a URL call a function."];
          HTML.li [] [txt "AI propose -- model writes a function on the current branch as agent:<model>; you keep or discard."];
        ];
```

```
      ];
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "How to access"];
        HTML.ul [] [
          HTML.li [] [txt "Start stack: scripts/dev.sh start"];
          HTML.li [] [txt "Text login/REPL: scripts/repl.sh  (SSH public key line protocol on :17822)"];
          HTML.li [] [txt "This browser UI: usually http://127.0.0.1:18080/ after an image boots"];
          HTML.li [] [txt "Browser: /  Workspace home ? /about ? /bindings Functions ? /pages Pages ? /types Data ? /repl ? /edit/<name>"];
          HTML.li [] [txt "Auth admin API: http://127.0.0.1:17879  (create users, tokens)"];
          HTML.li [] [txt "Full docs: docs/concepts-and-horizon.md in the repo"];
        ];
      ];
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "5-minute first tour"];
        HTML.pre [] [txt "%s"
          "1. scripts/repl.sh\n\
2. (define hi-page (lambda () (render-html \"<h1>Hello</h1>\")))\n\
3. (route \"/hello\" \"hi-page\")\n\
4. open /hello in the browser\n\
5. edit at /edit/hi-page\n\
6. optional: (propose-function \"double\" \"lambda of int x -> (* x 2)\")"];
      ];
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Long-term possibilities"];
        HTML.ul [] [
          HTML.li [] [txt "Personal OS -- always-on image; notes, CRM, automations as live functions."];
          HTML.li [] [txt "Community commons -- system/main as shared tools; personal gardens as branches."];
          HTML.li [] [txt "AI workshop -- AI cohabits the image; propose apps (types+fns+routes); merge selectively."];
          HTML.li [] [txt "Internal tools platform -- inspectable live ops/admin tools without redeploying for every change."];
          HTML.li [] [txt "Education -- legible multi-user systems/PL lab with forks and merges."];
          HTML.li [] [txt "End-user products -- friendlier skin (Functions/Pages/Data) on the same substrate."];
          HTML.li [] [txt "Hardened service -- Solo5, real SSH/web login, stronger auth and permissions."];
        ];
        HTML.p [] [
          HTML.a [HTML.href "/"] [txt "back home"];
          txt " ? ";
          HTML.a [HTML.href "/repl"] [txt "open REPL"];
        ];
      ];
    ]
  in
  html_response reqd ~session body

(* Page path for inspector: "/hello" -> "hello" for URL segment. *)
let page_inspect_path (route : string) : string =
  if String.length route > 0 && route.[0] = '/' then
    String.sub route 1 (String.length route - 1)
  else route

let pages_list_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit =
  let routes = Session.list_pages_meta session in
  let rows =
    List.map
      (fun (route, handler, vis) ->
        let inspect = "/page/" ^ page_inspect_path route in
        HTML.tr [] [
          HTML.td [] [HTML.a [HTML.href "%s" route] [txt "%s" route]];
          HTML.td [] [
            HTML.a [HTML.href "%s" (Packages.edit_href handler)] [txt "%s" handler];
          ];
```

```
          HTML.td [] [txt "%s" vis];
          HTML.td [] [
            HTML.a [HTML.href "%s" route] [txt "open"];
            txt " ? ";
            HTML.a [HTML.href "%s" (Packages.edit_href handler)] [txt "edit function"];
            txt " ? ";
            HTML.a [HTML.href "%s" inspect] [txt "inspect page"];
          ];
        ])
      routes
  in
  let body =
    chrome ~session ~title:"Pages" [
      HTML.p [HTML.class_ "meta"] [
        txt "Page = URL endpoint. Function = code it runs. Visibility: private (login) or public (anonymous GET). ";
        txt "(route path handler 'public) to open a page.";
      ];
      (match routes with
       | [] ->
           HTML.p [] [
             txt "(no pages yet -- use the Hello recipe on Home, or (route \"/x\" \"fn\") in the REPL)";
           ]
       | _ ->
           HTML.table [] [
             HTML.thead [] [
               HTML.tr [] [
                 HTML.th [] [txt "page (URL)"];
                 HTML.th [] [txt "function (handler)"];
                 HTML.th [] [txt "visibility"];
                 HTML.th [] [txt "actions"];
               ]
             ];
             HTML.tbody [] rows;
           ]);
    ]
  in
  html_response reqd ~session body

(** Labeled emergency home -- NOT a second product workspace.
    Real home is Lisp ui/home (libs/ui). See image.system-browser. *)
let emergency_home_content (session : Session.t) : Pure_html.node list =
  let who =
    Printf.sprintf
      "You are %s on branch %s. Product home is Lisp (ui/home); this page is host recovery only."
      (Session.username session) (Session.branch_name session)
  in
  [ HTML.div [HTML.class_ "card emergency-home"] [
      HTML.h2 [] [txt "EMERGENCY HOME -- ui/home missing or failed"];
      HTML.p [] [txt "%s" who];
      HTML.p [] [
        HTML.a [HTML.href "/bindings?pkg=ui"] [txt "Open ui package"];
        txt " ? ";
        HTML.a [HTML.href "/bindings"] [txt "Functions"];
        txt " ? ";
        HTML.a [HTML.href "/repl"] [txt "REPL"];
        txt " ? ";
        HTML.a [HTML.href "/about"] [txt "About"];
        txt " ? ";
        HTML.a [HTML.href "/login"] [txt "Login"];
      ];
      HTML.p [HTML.class_ "muted"] [
        txt "%s"
          "Fix: ensure system/main has ui/* from seed, or delete a broken personal pin of ui/home / ui/menu / ui/nav.";
      ];
    ]
  ]

let home_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
```

```
  (* Nav comes from Session.ui_nav_html (refreshed in request launch). *)
  let* home_html = try_ui_fn session "ui/home" in
  let content =
    match home_html with
    | Some html when String.trim html <> "" ->
        [ txt ~raw:true "%s" html ]
    | _ -> emergency_home_content session
  in
  let body = chrome ~session ~title:"" content in
  html_response reqd ~session body;
  Lwt.return_unit

let repl_page (session : Session.t) (reqd : Httpaf.Reqd.t) : unit =
  let js =
    "const out=document.getElementById('out');" ^
    "document.getElementById('run').onclick=async()=>{" ^
    "const expr=document.getElementById('expr').value;" ^
    "out.textContent='...';" ^
    "try{" ^
    "const r=await fetch('/api/eval',{method:'POST'," ^
    "headers:{'content-type':'application/json'}," ^
    "body:JSON.stringify({expr})});" ^
    "const j=await r.json();" ^
    "out.textContent=j.ok?j.value:('error: '+j.error);" ^
    "out.className=j.ok?'ok':'err';" ^
    "}catch(e){out.textContent=String(e);out.className='err';}" ^
    "};" ^
    "document.getElementById('clear').onclick=()=>{out.textContent='';};"
  in
  let body =
    chrome ~session ~title:"Browser REPL" [
      HTML.div [HTML.class_ "card"] [
        HTML.p [] [txt "Eval against the live session env. (define ...)
    persists."];
        HTML.textarea [HTML.id "expr"] "%s" "(+ 1 2)";
        HTML.p [] [
          HTML.button [HTML.id "run"; HTML.type_ "button"] [txt "Eval"];
          txt " ";
          HTML.button [HTML.id "clear"; HTML.type_ "button"; HTML.class_
    "secondary"]
            [txt "Clear out"];
        ];
        HTML.pre [HTML.id "out"] [txt ""];
      ];
      HTML.script [] "%s" js;
    ]
  in
  html_response reqd ~session body

(* Default stub body for a brand-new function. *)
let stub_body (name : string) : string =
  Printf.sprintf "(lambda ()\n  \"TODO: implement %s\")" name

(* Static user-function callees: walk AST App heads that are Vars. *)
let rec collect_var_apps (acc : string list) (e : Pricklypear_lisp.Ast.expr)
    : string list =
  match e with
  | Pricklypear_lisp.Ast.Var _ | Pricklypear_lisp.Ast.Lit _ -> acc
  | Pricklypear_lisp.Ast.App (Pricklypear_lisp.Ast.Var f, args) ->
      List.fold_left collect_var_apps (f :: acc) args
  | Pricklypear_lisp.Ast.App (f, args) ->
      List.fold_left collect_var_apps (collect_var_apps acc f) args
  | Pricklypear_lisp.Ast.Let (_, e1, e2) ->
      collect_var_apps (collect_var_apps acc e1) e2
  | Pricklypear_lisp.Ast.Letrec (binds, body) ->
      let acc =
        List.fold_left (fun a (_, e) -> collect_var_apps a e) acc binds
      in
      collect_var_apps acc body
  | Pricklypear_lisp.Ast.If (c, t, e) ->
      collect_var_apps (collect_var_apps (collect_var_apps acc c) t) e
  | Pricklypear_lisp.Ast.Lambda (_, body) -> collect_var_apps acc body
  | Pricklypear_lisp.Ast.Prim (_, args) ->
```

```
      List.fold_left collect_var_apps acc args
  | Pricklypear_lisp.Ast.Seq (a, b) ->
      collect_var_apps (collect_var_apps acc a) b
  | Pricklypear_lisp.Ast.Quote _ -> acc
  | Pricklypear_lisp.Ast.Define (_, body) -> collect_var_apps acc body

let page_function_graph (session : Session.t) ~(handler : string)
    : (string * string) list Lwt.t =
  let max_depth = 8 in
  let rec walk depth seen name role acc =
    if depth > max_depth || List.mem name seen then Lwt.return acc
    else
      let* found = Store.lookup_function (Session.store session) name in
      match found with
      | None -> Lwt.return ((name, role ^ " (missing)") :: acc)
      | Some (expr, _) ->
          let acc = (name, role) :: acc in
          let seen = name :: seen in
          let candidates = collect_var_apps [] expr in
          let rec filter_bind names out =
            match names with
            | [] -> Lwt.return (List.rev out)
            | n :: rest ->
                let* f = Store.lookup_function (Session.store session) n in
                (match f with
                 | Some _ when n <> name && not (List.mem n out) ->
                     filter_bind rest (n :: out)
                 | _ -> filter_bind rest out)
          in
          let* callees = filter_bind candidates [] in
          let rec fold_c cs acc =
            match cs with
            | [] -> Lwt.return acc
            | c :: rest ->
                let* acc = walk (depth + 1) seen c "callee" acc in
                fold_c rest acc
          in
          fold_c callees acc
  in
  let* rows = walk 0 [] handler "handler" [] in
  Lwt.return (List.rev rows)

let contains_ci (hay : string) (needle : string) : bool =
  if needle = "" then true
  else
    let h = String.lowercase_ascii hay in
    let n = String.lowercase_ascii needle in
    let rec loop i =
      if i + String.length n > String.length h then false
      else if String.sub h i (String.length n) = n then true
      else loop (i + 1)
    in
    loop 0

(** function-name ? routes that use it as the *direct* HTTP handler.

    Previously this ran [page_function_graph] for every route (recursive
    AST walk + Store.lookup_function per callee, depth 8) -- hundreds of
    PG round-trips on every /bindings load. "Used by" on the package
    list only needs the direct handler mapping; deep graphs stay on
    /page/<route> inspect. *)
let used_by_pages_map (session : Session.t)
    : (string, string list) Hashtbl.t Lwt.t =
  let routes = Session.list_pages session in
  let tbl : (string, string list) Hashtbl.t = Hashtbl.create 32 in
  List.iter
    (fun (route, handler) ->
      let prev = try Hashtbl.find tbl handler with Not_found -> [] in
      if List.mem route prev then ()
      else Hashtbl.replace tbl handler (route :: prev))
    routes;
  Lwt.return tbl

```

```
let list_bindings_page (session : Session.t) (reqd : Httpaf.Reqd.t)
    : unit Lwt.t =
  let target = (Httpaf.Reqd.request reqd).Httpaf.Request.target in
  let q = Option.value (query_param target "q") ~default:"" |> String.trim
     in
  let pkg_q = query_param target "pkg" in
  let fn_q = query_param target "fn" in
  let* defs_meta =
    Store.list_function_names_meta (Session.store session)
  in
  (* Effective view only: one row per name (closest branch wins). *)
  let defs =
    List.map (fun (n, e, a, d) -> (n, e, a, d)) defs_meta
  in
  let* used_by = used_by_pages_map session in
  let kernel = Session.kernel_catalog session in
  let matches_ws (name, _body, author, _depth) =
    q = ""
    || contains_ci name q
    || contains_ci (Packages.package_of_name name) q
    || contains_ci author q
  in
  let defs_f = List.filter matches_ws defs in
  (* group_by_package expects 3-tuples; strip depth for grouping. *)
  let defs_for_group =
    List.map (fun (n, e, a, _) -> (n, e, a)) defs_f
  in
  let groups = Packages.group_by_package defs_for_group in
  let depth_of name =
    match List.find_opt (fun (n, _, _, _) -> n = name) defs_f with
    | Some (_, _, _, d) -> d
    | None -> 0
  in
  let origin_badge depth author =
    if depth = 0 then
      HTML.span [HTML.class_ "ok"] [txt "own"]
    else
      HTML.span [HTML.class_ "muted"]
        [txt "inherited ? %s" author]
  in
  let used_links name =
    let routes = try Hashtbl.find used_by name with Not_found -> [] in
    match List.rev routes with
    | [] -> HTML.span [HTML.class_ "muted"] [txt "--"]
    | rs ->
        HTML.span []
          (List.concat
             (List.mapi
                (fun i route ->
                  let link =
                    HTML.a
                      [HTML.href "%s" ("/page/" ^ page_inspect_path route)]
                      [txt "%s" route]
                  in
                  if i = 0 then [link] else [txt " "; link])
                rs))
  in
  let pkg_href pkg =
    "/bindings?pkg=" ^ Packages.name_encode pkg
  in
  let fn_href pkg fn =
    "/bindings?pkg=" ^ Packages.name_encode pkg ^ "&fn="
    ^ Packages.name_encode fn
  in
  match pkg_q, fn_q with
  | Some "kernel", Some fname ->
      let summary =
        match List.assoc_opt fname kernel with
        | Some s -> s
        | None -> "unknown primitive"
      in
      let body =
        chrome ~session ~title:("Primitive " ^ fname) [
```

```
          HTML.p [] [
            HTML.a [HTML.href "/bindings"] [txt "Functions"];
            txt " ? ";
            HTML.a [HTML.href "/bindings?pkg=kernel"] [txt "(kernel)"];
            txt " ? %s" fname;
          ];
          HTML.div [HTML.class_ "card"] [
            HTML.p [] [
              HTML.span [HTML.class_ "muted"] [txt "kind: "];
              txt "primitive (OCaml)";
            ];
            HTML.p [] [
              HTML.span [HTML.class_ "muted"] [txt "name: "];
              txt "%s" fname;
            ];
            HTML.p [] [txt "%s" summary];
            HTML.p [HTML.class_ "muted"] [
              txt "Built into the image kernel -- not stored as Lisp. App logic should usually be Lisp functions you can edit.";
            ];
          ];
        ]
      in
      html_response reqd ~session body;
      Lwt.return_unit
  | Some pkg, Some local ->
      let full =
        if String.contains local '/' then local
        else if pkg = "" || pkg = "(root)" || pkg = "root" then local
        else Packages.join_name ~package:pkg ~local
      in
      let* found = Store.lookup_function (Session.store session) full in
      let crumb_pkg = if pkg = "(root)" then "" else pkg in
      let body =
        match found with
        | None ->
            chrome ~session ~title:full [
              HTML.p [] [
                HTML.a [HTML.href "/bindings"] [txt "Functions"];
                txt " ? ";
                HTML.a [HTML.href "%s" (pkg_href pkg)]
                  [txt "%s" (Packages.package_label crumb_pkg)];
              ];
              HTML.p [HTML.class_ "err"] [txt "no function %s" full];
              HTML.p [] [
                HTML.a [HTML.href "%s" (Packages.edit_href full)] [txt "create it"];
              ];
            ]
        | Some (expr, author) ->
            let badge =
              Pricklypear_lisp.Effect.EffSet.badge
                (Pricklypear_lisp.Effect.infer expr)
            in
            let pp = Pricklypear_lisp.Reader.pp_expr expr in
            let depth = depth_of full in
            chrome ~session ~title:full [
              HTML.p [] [
                HTML.a [HTML.href "/bindings"] [txt "Functions"];
                txt " ? ";
                HTML.a [HTML.href "%s" (pkg_href pkg)]
                  [txt "%s" (Packages.package_label crumb_pkg)];
                txt " ? %s" (Packages.local_name full);
              ];
              HTML.div [HTML.class_ "card"] [
                HTML.p [] [
                  HTML.span [HTML.class_ "muted"] [txt "kind: "];
                  txt "function (Lisp)";
                ];
                HTML.p [] [
                  HTML.span [HTML.class_ "muted"] [txt "origin: "];
                  origin_badge depth author;
                ];
```

```
              HTML.p [] [
                HTML.span [HTML.class_ "muted"] [txt "author: "];
                txt "%s" author;
              ];
              HTML.p [] [
                HTML.span [HTML.class_ "muted"] [txt "effects: "];
                txt "%s" badge;
              ];
              HTML.p [] [
                HTML.span [HTML.class_ "muted"] [txt "used by: "];
                used_links full;
              ];
              HTML.h2 [] [txt "definition"];
              HTML.pre [] [txt "%s" pp];
              HTML.p [] [
                HTML.a
                  [ HTML.href "%s" (Packages.edit_href full)
                  ; HTML.class_ "btn" ]
                  [txt "Edit"];
                txt " ";
                HTML.a
                  [ HTML.href "%s" (Packages.inspect_href full)
                  ; HTML.class_ "btn secondary" ]
                  [txt "Inspect AST"];
              ];
            ];
          ]
    in
    html_response reqd ~session body;
    Lwt.return_unit
| Some "kernel", None ->
    let rows =
      if q = "" then kernel
      else List.filter (fun (n, s) -> contains_ci n q || contains_ci s q)
    kernel
    in
    let items =
      List.map
        (fun (n, summary) ->
          HTML.tr [] [
            HTML.td [] [
              HTML.a [HTML.href "%s" (fn_href "kernel" n)] [txt "%s" n];
            ];
            HTML.td [] [txt "primitive"];
            HTML.td [] [txt "%s" summary];
          ])
        rows
    in
    let body =
      chrome ~session ~title:"Package (kernel)" [
        HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "? all packages"]];
        HTML.p [HTML.class_ "meta"] [
          txt "Built-in OCaml primitives. Read-only. Prefer app logic as
    Lisp.";
        ];
        HTML.table [] [
          HTML.thead [] [
            HTML.tr [] [
              HTML.th [] [txt "name"];
              HTML.th [] [txt "kind"];
              HTML.th [] [txt "summary"];
            ];
          ];
          HTML.tbody [] items;
        ];
      ]
    in
    html_response reqd ~session body;
    Lwt.return_unit
| Some pkg, None ->
    let pkg_key = if pkg = "(root)" || pkg = "root" then "" else pkg in
    let rows =
      List.filter
```

```
543           (fun (name, _, _, _) -> Packages.package_of_name name = pkg_key)
544           defs_f
545       in
546       let items =
547         List.map
548           (fun (name, body, author, depth) ->
549             let badge =
550               Pricklypear_lisp.Effect.EffSet.badge
551                 (Pricklypear_lisp.Effect.infer body)
552             in
553             let local = Packages.local_name name in
554             HTML.tr [] [
555               HTML.td [] [
556                 HTML.a [HTML.href "%s" (fn_href pkg local)] [txt "%s" name];
557               ];
558               HTML.td [] [txt "%s" badge];
559               HTML.td [] [origin_badge depth author];
560               HTML.td [] [used_links name];
561             ])
562           rows
563       in
564       let body =
565         chrome ~session ~title:("Package " ^ Packages.package_label
    pkg_key) [
566           HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "? all packages"]];
567           HTML.p [HTML.class_ "muted"] [
568             txt "Effective view for branch %s -- one row per name (own or
    inherited)."
569               (Session.branch_name session);
570           ];
571           (match items with
572            | [] -> HTML.p [] [txt "(no functions in this package)"]
573            | _ ->
574                HTML.table [] [
575                  HTML.thead [] [
576                    HTML.tr [] [
577                      HTML.th [] [txt "function"];
578                      HTML.th [] [txt "effects"];
579                      HTML.th [] [txt "origin"];
580                      HTML.th [] [txt "used by"];
581                    ];
582                  ];
583                  HTML.tbody [] items;
584                ]);
585           HTML.p [] [
586             HTML.a
587               [ HTML.href "%s"
588                   (if pkg_key = "" then "/edit/?local="
589                    else
590                      "/edit/?package=" ^ Packages.name_encode pkg_key
591                      ^ "&local=") ]
592               [txt "new in this package?"];
593           ];
594         ]
595       in
596       html_response reqd ~session body;
597       Lwt.return_unit
598   | None, _ ->
599       let ws_pkgs =
600         List.map
601           (fun (pkg, rows) ->
602             let label = Packages.package_label pkg in
603             let key = if pkg = "" then "root" else pkg in
604             (label, pkg_href key, List.length rows, "workspace"))
605           groups
606       in
607       let all_pkgs =
608         ( "(kernel)"
609         , "/bindings?pkg=kernel"
610         , List.length kernel
611         , "kernel" )
612         :: ws_pkgs
613       in
```

```
      let filtered =
        if q = "" then all_pkgs
        else List.filter (fun (label, _, _, _) -> contains_ci label q) all_pkgs
      in
      let items =
        List.map
          (fun (label, href, n, kind) ->
            HTML.tr [] [
              HTML.td [] [HTML.a [HTML.href "%s" href] [txt "%s" label]];
              HTML.td [] [txt "%s" kind];
              HTML.td [] [txt "%d" n];
            ])
          filtered
      in
      let body =
        chrome ~session
          ~title:("Functions (" ^ Session.branch_name session ^ ")")
          [ HTML.p [HTML.class_ "meta"] [
              txt "Drill: package ? function ? definition. Kernel = OCaml prims (read-only). Workspace = Lisp you can edit.";
            ]
          ; HTML.div [HTML.class_ "card"] [
              HTML.form [HTML.method_ `GET; HTML.action "/bindings"] [
                HTML.p [] [
                  HTML.label [HTML.for_ "q"] [txt "search "];
                  HTML.input
                    [ HTML.type_ "text"
                    ; HTML.name "q"
                    ; HTML.id "q"
                    ; HTML.value "%s" q
                    ; HTML.placeholder "package or name" ];
                  txt " ";
                  HTML.button [HTML.type_ "submit"] [txt "Filter"];
                ];
              ];
            ]
          ; HTML.div [HTML.class_ "card"] [
              HTML.h2 [] [txt "New function"];
              HTML.form [HTML.method_ `GET; HTML.action "/edit/"] [
                HTML.p [] [
                  HTML.label [] [txt "package "];
                  HTML.input
                    [ HTML.type_ "text"
                    ; HTML.name "package"
                    ; HTML.placeholder "todo" ];
                  txt " ";
                  HTML.label [] [txt "name "];
                  HTML.input
                    [ HTML.type_ "text"
                    ; HTML.name "local"
                    ; HTML.placeholder "list" ];
                  txt " ";
                  HTML.button [HTML.type_ "submit"] [txt "Create / edit"];
                ];
              ];
            ]
          ; HTML.table [] [
              HTML.thead [] [
                HTML.tr [] [
                  HTML.th [] [txt "package"];
                  HTML.th [] [txt "kind"];
                  HTML.th [] [txt "count"];
                ];
              ];
              HTML.tbody [] items;
            ]
          ; HTML.p [HTML.class_ "muted"] [
              txt "Tip: open (kernel) to browse create, +, render-html, propose-function, ?";
            ]
          ]
      in
```

```
      html_response reqd ~session body;
      Lwt.return_unit

let editor_form_partial ~(name : string) ~(body_src : string) ?(msg = "") ()
    : Pure_html.node =
  let flash =
    if msg = "" then HTML.div [] []
    else
      HTML.p
        [ HTML.class_
            (if String.length msg >= 5 && String.sub msg 0 5 = "error" then
    "err"
             else "ok") ]
        [ txt "%s" msg ]
  in
  HTML.div [HTML.id "editor-root"] [
    flash;
    HTML.div [HTML.class_ "card"] [
      HTML.form
        [ HTML.method_ `POST
        ; HTML.action "%s" ("/edit/" ^ name)
        ; HTML.enctype `urlencoded ]
        [ HTML.p [] [
            HTML.label [HTML.for_ "name"] [txt "name "];
            HTML.input
              [ HTML.type_ "text"
              ; HTML.name "name"
              ; HTML.id "name"
              ; HTML.value "%s" name ];
          ]
        ; HTML.p [] [HTML.label [HTML.for_ "body"] [txt "body
    (s-expression)"]]
        ; (HTML.textarea [HTML.name "body"; HTML.id "body"] "%s" body_src)
        ; HTML.p [] [
            HTML.button [HTML.type_ "submit"] [txt "Save (define +
    persist)"];
            txt " ";
            HTML.a
              [ HTML.href "%s" ("/binding/" ^ name)
              ; HTML.class_ "btn secondary" ]
              [ txt "View" ];
          ];
        ];
    ];
  ]

let route_for_edit_name (session : Session.t) (name : string)
    : (string * string) option =
  let route = "/" ^ name in
  match Session.lookup_route session route with
  | Some handler -> Some (route, handler)
  | None ->
      (match Session.lookup_route session name with
       | Some handler -> Some (name, handler)
       | None -> None)

let missing_editor_card (session : Session.t) (name : string) :
    Pure_html.node =
  let route_opt = route_for_edit_name session name in
  let route_block =
    match route_opt with
    | None -> HTML.div [] []
    | Some (route, handler) ->
        HTML.div [HTML.class_ "card"] [
          HTML.p [] [
            txt "There is a page ";
            HTML.a [HTML.href "%s" route] [txt "%s" route];
            txt " that currently runs function ";
            HTML.span [HTML.class_ "ok"] [txt "%s" handler];
            txt ".";
          ];
          HTML.p [] [
```

```
752             HTML.a [HTML.href "%s" (Packages.edit_href handler);
    HTML.class_ "btn"]
753               [txt "Edit %s" handler];
754             txt " ";
755             HTML.a
756               [ HTML.href "%s" ("/page/" ^ page_inspect_path route)
757               ; HTML.class_ "btn secondary" ]
758               [ txt "Inspect page" ];
759           ];
760         ]
761   in
762   let create_form =
763     HTML.div [HTML.class_ "card"] [
764       HTML.p [] [txt "Function \"%s\" does not exist." name];
765       HTML.p [HTML.class_ "muted"] [
766         txt "Create it here -- the editor loads in place (htmx).";
767       ];
768       HTML.form
769         [ HTML.method_ `POST
770         ; HTML.action "/api/edit/create"
771         ; HTML.enctype `urlencoded
772         ; Hx.post "/api/edit/create"
773         ; Hx.target "#editor-root"
774         ; Hx.swap "outerHTML" ]
775         [ HTML.input [HTML.type_ "hidden"; HTML.name "name"; HTML.value
    "%s" name]
776         ; (match route_opt with
777            | Some (route, _) ->
778                HTML.p [] [
779                  HTML.label [] [
780                    HTML.input
781                      [ HTML.type_ "checkbox"
782                      ; HTML.name "rebind_route"
783                      ; HTML.value "1" ];
784                    txt " also point page %s at this new function" route;
785                  ];
786                ]
787            | None -> HTML.div [] [])
788         ; HTML.p [] [
789             HTML.button [HTML.type_ "submit"] [txt "Create function %s"
    name];
790             txt " ";
791             HTML.a [HTML.href "/bindings"; HTML.class_ "btn secondary"]
792               [txt "Functions list"];
793           ];
794         ];
795     ]
796   in
797   HTML.div [HTML.id "editor-root"] [route_block; create_form]
798
799 let editor_page (session : Session.t) (name : string) ?(msg = "")
800     ?(body_src = "") (reqd : Httpaf.Reqd.t) : unit Lwt.t =
801   let* found = Store.lookup_function (Session.store session) name in
802   let body =
803     match found, body_src with
804     | _, src when src <> "" ->
805         chrome ~session ~title:("Edit " ^ name)
806           [ editor_form_partial ~name ~body_src:src ~msg () ]
807     | Some (expr, _), _ ->
808         let initial = Pricklypear_lisp.Reader.pp_expr expr in
809         chrome ~session ~title:("Edit " ^ name)
810           [ editor_form_partial ~name ~body_src:initial ~msg () ]
811     | None, _ ->
812         chrome ~session ~title:("Edit " ^ name)
813           [ missing_editor_card session name ]
814   in
815   html_response reqd ~session body;
816   Lwt.return_unit
817
818 let api_edit_create (session : Session.t) (reqd : Httpaf.Reqd.t) : unit
    Lwt.t =
819   let* body = read_body reqd in
820   let name =
```

```
    match form_field body "name" with
    | Some n when String.trim n <> "" -> String.trim n
    | _ -> ""
  in
  if name = "" then begin
    html_response reqd ~session ~status:`Bad_request
      (HTML.div [HTML.class_ "err"] [txt "name required"]);
    Lwt.return_unit
  end else
    let rebind =
      match form_field body "rebind_route" with
      | Some ("1" | "on" | "true") -> true
      | _ -> false
    in
    let body_src = stub_body name in
    let* res =
      Session.define_binding session ~name ~body_src
        ~author:(Session.username session)
    in
    match res with
    | Error e ->
        html_response reqd ~session ~status:`Bad_request
          (HTML.div [HTML.id "editor-root"; HTML.class_ "err"]
             [txt "create failed: %s" e]);
        Lwt.return_unit
    | Ok _ ->
        let* () =
          if rebind then
            Session.set_route session ~route:("/" ^ name) ~handler:name ()
          else Lwt.return_unit
        in
        let msg =
          if rebind then
            Printf.sprintf "created %s and pointed /%s at it" name name
          else Printf.sprintf "created %s -- edit and save" name
        in
        let node = editor_form_partial ~name ~body_src ~msg () in
        (* Partial only -- htmx swaps into #editor-root *)
        let s = Pure_html.to_string node in
        let headers =
          Httpaf.Headers.of_list
            [ ("content-type", "text/html; charset=utf-8")
            ; ("content-length", string_of_int (String.length s)) ]
        in
        let resp = Httpaf.Response.create ~headers `OK in
        Httpaf.Reqd.respond_with_string reqd resp s;
        Lwt.return_unit

let page_inspector (session : Session.t) (path_seg : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let route =
    if path_seg = "" then "/"
    else if path_seg.[0] = '/' then path_seg
    else "/" ^ path_seg
  in
  match Session.lookup_route session route with
  | None ->
      html_response reqd ~session ~status:`Not_found
        (chrome ~session ~title:("Page " ^ route) [
           HTML.p [HTML.class_ "err"] [txt "no page registered for %s"
    route];
           HTML.p [] [HTML.a [HTML.href "/pages"] [txt "all pages"]];
         ]);
      Lwt.return_unit
  | Some handler ->
      let* graph = page_function_graph session ~handler in
      let* found = Store.lookup_function (Session.store session) handler in
      let body_block =
        match found with
        | None ->
            HTML.p [HTML.class_ "err"] [txt "handler function missing in
    store"]
        | Some (expr, author) ->
```

```
            HTML.div [] [
              HTML.p [HTML.class_ "muted"] [txt "author: %s" author];
              HTML.pre [] [txt "%s" (Pricklypear_lisp.Reader.pp_expr expr)];
            ]
      in
      let rows =
        List.map
          (fun (n, role) ->
            HTML.tr [] [
              HTML.td [] [txt "%s" n];
              HTML.td [] [txt "%s" role];
              HTML.td [] [
                HTML.a [HTML.href "%s" (Packages.edit_href n)] [txt "edit"];
                txt " ? ";
                HTML.a [HTML.href "%s" (Packages.inspect_href n)] [txt
    "inspect"];
              ];
            ])
          graph
      in
      let body =
        chrome ~session ~title:("Page " ^ route) [
          HTML.div [HTML.class_ "card"] [
            HTML.h2 [] [txt "Routing"];
            HTML.p [] [
              txt "Page URL: ";
              HTML.a [HTML.href "%s" route] [txt "%s" route];
            ];
            HTML.p [] [
              txt "Handler function: ";
              HTML.a [HTML.href "%s" (Packages.edit_href handler)] [txt
    "%s" handler];
            ];
            HTML.p [] [
              HTML.a [HTML.href "%s" route; HTML.class_ "btn"] [txt "Open
    page"];
              txt " ";
              HTML.a [HTML.href "%s" (Packages.edit_href handler);
    HTML.class_ "btn secondary"]
                [txt "Edit handler"];
            ];
          ];
          HTML.div [HTML.class_ "card"] [
            HTML.h2 [] [txt "Functions this page uses"];
            HTML.p [HTML.class_ "muted"] [
              txt "Static call graph from the handler body (user functions
    only).";
            ];
            HTML.table [] [
              HTML.thead [] [
                HTML.tr [] [
                  HTML.th [] [txt "function"];
                  HTML.th [] [txt "role"];
                  HTML.th [] [txt "actions"];
                ];
              ];
              HTML.tbody [] rows;
            ];
          ];
          HTML.div [HTML.class_ "card"] [
            HTML.h2 [] [txt "Entry body (handler)"];
            body_block;
          ];
        ]
      in
      html_response reqd ~session body;
      Lwt.return_unit

let inspect_binding_page (session : Session.t) (name : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let* found = Store.lookup_function (Session.store session) name in
  let body =
    match found with
```

```
    | None ->
        chrome ~session ~title:("Inspect " ^ name) [
          HTML.p [HTML.class_ "err"] [txt "not found"];
        ]
    | Some (expr, author) ->
        let pp = Pricklypear_lisp.Reader.pp_expr expr in
        let j =
          try Yojson.Safe.pretty_to_string
                (Pricklypear_lisp.Ast.expr_to_json expr)
          with _ -> "(json unavailable)"
        in
        chrome ~session ~title:("Inspect binding " ^ name) [
          HTML.div [HTML.class_ "card"] [
            HTML.p [] [txt "author: %s" author];
            HTML.p [] [txt "branch: %s" (Session.branch_name session)];
            HTML.h2 [] [txt "pretty"];
            HTML.pre [] [txt "%s" pp];
            HTML.h2 [] [txt "AST JSON"];
            HTML.pre [] [txt "%s" j];
            HTML.p [] [
              HTML.a [HTML.href "%s" (Packages.edit_href name)] [txt
    "edit"];
            ];
          ];
        ]
  in
  html_response reqd ~session body;
  Lwt.return_unit

let inspect_row_page (session : Session.t) (type_name : string) (id :
    string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  (* Canonical row URL is /data/<type>/<id> (image.data-browser). *)
  ignore session;
  redirect_response reqd
    (Printf.sprintf "/data/%s/%s" type_name id);
  Lwt.return_unit

let api_eval (session : Session.t) (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let* body = read_body reqd in
  let expr_opt =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          (match List.assoc_opt "expr" items with
           | Some (`String s) -> Some s
           | _ -> None)
      | `String s -> Some s
      | _ -> None
    with _ ->
      (* also accept raw text body *)
      if String.trim body = "" then None else Some body
  in
  match expr_opt with
  | None ->
      json_response reqd ~status:`Bad_request
        (`Assoc [("ok", `Bool false); ("error", `String "expr required")]);
      Lwt.return_unit
  | Some expr ->
      Lwt.catch
        (fun () ->
          let* v = Session.eval_one session expr in
          let rendered = Pricklypear_lisp.Reader.pp_value v in
          json_response reqd
            (`Assoc [("ok", `Bool true); ("value", `String rendered)]);
          Lwt.return_unit)
        (fun exn ->
          let msg =
            match exn with
            | Pricklypear_lisp.Ast.Interp_error e ->
                (match e with
                 | Pricklypear_lisp.Ast.Parse_error (m, l, c) ->
                     Printf.sprintf "parse error: %s (%d:%d)" m l c
```

```
                  | Pricklypear_lisp.Ast.User_error m -> m
                  | Pricklypear_lisp.Ast.Unbound_var s -> "unbound: " ^ s
                  | Pricklypear_lisp.Ast.Budget_exceeded -> "budget exceeded"
                  | _ -> Printexc.to_string exn)
            | _ -> Printexc.to_string exn
          in
          json_response reqd
            (`Assoc [("ok", `Bool false); ("error", `String msg)]);
          Lwt.return_unit)

let save_edit (session : Session.t) (path_name : string)
    (reqd : Httpaf.Reqd.t) : unit Lwt.t =
  let* body = read_body reqd in
  let name =
    match form_field body "name" with
    | Some n when String.trim n <> "" -> String.trim n
    | _ -> path_name
  in
  let body_src = Option.value (form_field body "body") ~default:"" in
  let* res =
    Session.define_binding session ~name ~body_src
      ~author:(Session.username session)
  in
  match res with
  | Ok v ->
      let msg =
        Printf.sprintf "saved %s => %s" name
          (Pricklypear_lisp.Reader.pp_value v)
      in
      editor_page session name ~msg ~body_src reqd
  | Error e ->
      editor_page session name ~msg:("error: " ^ e) ~body_src reqd
```

44 image/lib/lineedit.ml

```
(** Pricklypear Image -- Minimal line editor.

    For the minimal image: read a single line of input from a channel and
    return it. No cursor movement, no history, no reverse-search.

    Per spec [image.repl-experience]: real line editing should use
    ocaml-linenoise (small, Mirage-compatible-ish). The minimal stub is
    acceptable for the boot-checkpoint smoke test. TODO: linenoise. *)
open Lwt.Syntax

(* Read a line, returning [None] at EOF. *)
let read_line (ic : Lwt_io.input_channel) : string option Lwt.t =
  let* line = Lwt_io.read_line_opt ic in
  Lwt.return line
```

45 image/lib/llm_client.ml

```
(** OpenAI-compatible LLM client for Pricklypear AI authoring.

    Uses curl for HTTPS (cohttp-lwt-unix not on poohstack). Config via env:
      LLM_API_BASE  default https://api.synthetic.new/openai/v1
      LLM_API_KEY   required for live calls
      LLM_MODEL     default hf:zai-org/GLM-4.7-Flash

    Spec [ai-authoring.tools]: propose functions on the current branch
    with agent provenance. This module only does HTTP; session/http
    layers own store writes. *)
open Lwt.Syntax

let env_or name dft =
  match Sys.getenv_opt name with
  | Some s when s <> "" -> s
  | _ -> dft

let api_base () = env_or "LLM_API_BASE"
     "https://api.synthetic.new/openai/v1"
let api_key () = env_or "LLM_API_KEY" ""
let model () = env_or "LLM_MODEL" "hf:zai-org/GLM-4.7-Flash"

type call_result = (string, string) result

(** Progress events for live sinks (SSE / REPL). Not a capability effect. *)
type agent_event =
  | Token of string
  | Done
  | Fail of string

let shell_quote (s : string) : string =
  (* single-quote for POSIX sh, escape embedded single quotes *)
  let buf = Buffer.create (String.length s + 8) in
  Buffer.add_char buf '\'';
  String.iter
    (function
      | '\'' -> Buffer.add_string buf "'\\''"
      | c -> Buffer.add_char buf c)
    s;
  Buffer.add_char buf '\'';
  Buffer.contents buf

let extract_content (body : string) : call_result =
  try
    let json = Yojson.Safe.from_string body in
    let open Yojson.Safe.Util in
    let choices = json |> member "choices" |> to_list in
    match choices with
    | [] -> Error "LLM: empty choices"
    | choice :: _ ->
        let msg = choice |> member "message" in
        let finish =
          match choice |> member "finish_reason" |> to_string_option with
          | Some f -> f
          | None -> "?"
        in
        let content =
          match msg |> member "content" |> to_string_option with
          | Some c when c <> "" -> c
          | _ ->
              (match msg |> member "reasoning_content" |> to_string_option
    with
               | Some r when r <> "" -> r
               | _ ->
                   match msg |> member "reasoning" |> to_string_option with
                   | Some r when r <> "" -> r
                   | _ -> "")
        in
        if content = "" then
          Error (Printf.sprintf "LLM: empty content (finish=%s)" finish)
        else Ok content
```

```
  with e ->
    let trunc =
      if String.length body > 300 then String.sub body 0 300 ^ "..." else
     body
    in
    Error ("LLM parse error: " ^ Printexc.to_string e ^ " body=" ^ trunc)

let call ?(temperature = 0.2) ?(max_tokens = 4096)
    ~(system : string) ~(user : string) () : call_result Lwt.t =
  let key = api_key () in
  if key = "" then
    Lwt.return (Error "LLM_API_KEY not set")
  else
    let body_json =
      `Assoc
        [ ("model", `String (model ()))
        ; ("messages",
           `List
             [ `Assoc [("role", `String "system"); ("content", `String
    system)]
             ; `Assoc [("role", `String "user"); ("content", `String user)]
             ])
        ; ("temperature", `Float temperature)
        ; ("max_tokens", `Int max_tokens)
        ]
    in
    let body_str = Yojson.Safe.to_string body_json in
    let url = api_base () ^ "/chat/completions" in
    (* Write body to a temp file to avoid shell length limits. *)
    let tmp = Filename.temp_file "pp_llm_" ".json" in
    let oc = open_out tmp in
    output_string oc body_str;
    close_out oc;
    let cmd =
      Printf.sprintf
        "curl -sS --max-time 180 -X POST %s \
         -H %s -H %s --data-binary @%s 2>&1"
        (shell_quote url)
        (shell_quote "Content-Type: application/json")
        (shell_quote ("Authorization: Bearer " ^ key))
        (shell_quote tmp)
    in
    Printf.eprintf "[llm] POST %s model=%s\n" url (model ());
    Lwt.catch
      (fun () ->
        let* stdout =
          Lwt_process.pread ~timeout:200. (Lwt_process.shell cmd)
        in
        (try Sys.remove tmp with _ -> ());
        if String.trim stdout = "" then
          Lwt.return (Error "LLM: empty curl response")
        else if String.length stdout > 12
                && (String.sub stdout 0 6 = "curl: "
                    || String.sub stdout 0 5 = "curl ") then
          Lwt.return (Error ("curl: " ^ String.trim stdout))
        else
          Lwt.return (extract_content stdout))
      (fun exn ->
        (try Sys.remove tmp with _ -> ());
        Lwt.return (Error ("LLM transport: " ^ Printexc.to_string exn)))

(** Strip optional markdown fences from model output. *)
let strip_fences (s : string) : string =
  let s = String.trim s in
  if String.length s >= 3 && String.sub s 0 3 = "```" then
    let start =
      match String.index_opt s '\n' with
      | Some i -> i + 1
      | None -> 3
    in
    let rest = String.sub s start (String.length s - start) in
    match String.rindex_opt rest '`' with
    | Some i when i >= 3 && String.sub rest (i - 2) 3 = "```" ->
```

```
        String.trim (String.sub rest 0 (i - 2))
    | _ ->
        (match String.rindex_opt rest '\n' with
         | Some i -> String.trim (String.sub rest 0 i)
         | None -> String.trim rest)
  else s

(** Pull the first balanced s-expression from model noise
    (reasoning essays, markdown, numbered lists). *)
let extract_lisp_sexpr (s : string) : (string, string) result =
  let s = strip_fences s in
  let len = String.length s in
  let rec find_open i =
    if i >= len then None
    else if s.[i] = '(' then Some i
    else find_open (i + 1)
  in
  match find_open 0 with
  | None ->
      Error "no Lisp s-expression found in model output (expected a (lambda
     ...) form)"
  | Some start ->
      let rec scan i depth in_str =
        if i >= len then Error "unbalanced parentheses in model output"
        else
          let c = s.[i] in
          if in_str then
            if c = '\\' && i + 1 < len then scan (i + 2) depth true
            else if c = '"' then scan (i + 1) depth false
            else scan (i + 1) depth true
          else
            match c with
            | '"' -> scan (i + 1) depth true
            | '(' -> scan (i + 1) (depth + 1) false
            | ')' ->
                if depth <= 1 then
                  Ok (String.trim (String.sub s start (i - start + 1)))
                else scan (i + 1) (depth - 1) false
            | ';' ->
                (* skip line comment *)
                let rec skip j =
                  if j >= len || s.[j] = '\n' then j else skip (j + 1)
                in
                scan (skip i) depth false
            | _ -> scan (i + 1) depth false
      in
      (match scan (start + 1) 1 false with
       | Error e -> Error e
       | Ok sexpr ->
           (* Prefer a lambda/list form; reject tiny garbage. *)
           if String.length sexpr < 3 then
             Error "extracted s-expression too short"
           else Ok sexpr)

(** Extract delta content from one streaming SSE JSON payload. *)
let extract_stream_delta (data : string) : string option =
  if data = "[DONE]" then None
  else
    try
      let json = Yojson.Safe.from_string data in
      let open Yojson.Safe.Util in
      match json |> member "choices" |> to_list with
      | [] -> None
      | choice :: _ ->
          let delta = choice |> member "delta" in
          (match delta |> member "content" |> to_string_option with
           | Some s when s <> "" -> Some s
           | _ -> None)
    with _ -> None

(** Unified agent chat: system=seed, user=prompt.
    When [stream] is true, uses OpenAI stream:true and fires [on_event]
    for each token. Always returns the final concatenated text. *)
```

```
let agent_chat
    ?(temperature = 0.3)
    ?(max_tokens = 4096)
    ?(stream = false)
    ?(on_event : agent_event -> unit = fun _ -> ())
    ~(seed : string)
    ~(prompt : string)
    () : call_result Lwt.t =
  let key = api_key () in
  if key = "" then begin
    on_event (Fail "LLM_API_KEY not set");
    Lwt.return (Error "LLM_API_KEY not set")
  end else if not stream then begin
    let* res = call ~temperature ~max_tokens ~system:seed ~user:prompt () in
    (match res with
     | Ok text -> on_event (Token text); on_event Done; Lwt.return (Ok text)
     | Error e -> on_event (Fail e); Lwt.return (Error e))
  end else
    let body_json =
      `Assoc
        [ ("model", `String (model ()))
        ; ("messages",
           `List
             [ `Assoc [("role", `String "system"); ("content", `String seed)]
             ; `Assoc [("role", `String "user"); ("content", `String prompt)]
             ])
        ; ("temperature", `Float temperature)
        ; ("max_tokens", `Int max_tokens)
        ; ("stream", `Bool true)
        ]
    in
    let body_str = Yojson.Safe.to_string body_json in
    let url = api_base () ^ "/chat/completions" in
    let tmp = Filename.temp_file "pp_llm_stream_" ".json" in
    let oc = open_out tmp in
    output_string oc body_str;
    close_out oc;
    let cmd =
      Printf.sprintf
        "curl -sS -N --max-time 180 -X POST %s \
         -H %s -H %s --data-binary @%s 2>&1"
        (shell_quote url)
        (shell_quote "Content-Type: application/json")
        (shell_quote ("Authorization: Bearer " ^ key))
        (shell_quote tmp)
    in
    Printf.eprintf "[llm] STREAM %s model=%s\n" url (model ());
    let buf = Buffer.create 4096 in
    Lwt.catch
      (fun () ->
        let proc = Lwt_process.open_process_in (Lwt_process.shell cmd) in
        let rec loop () =
          let* line_opt = Lwt_io.read_line_opt proc#stdout in
          match line_opt with
          | None ->
              let* st = proc#close in
              (try Sys.remove tmp with _ -> ());
              ignore st;
              let text = Buffer.contents buf in
              if String.trim text = "" then begin
                on_event (Fail "LLM stream: empty content");
                Lwt.return (Error "LLM stream: empty content")
              end else begin
                on_event Done;
                Lwt.return (Ok text)
              end
          | Some line ->
              let line = String.trim line in
              if line = "" then loop ()
              else if String.length line >= 6
                      && String.sub line 0 6 = "data: " then
```

```
                let data = String.sub line 6 (String.length line - 6) |>
    String.trim in
                if data = "[DONE]" then loop ()
                else begin
                  (match extract_stream_delta data with
                   | Some tok ->
                       Buffer.add_string buf tok;
                       on_event (Token tok)
                   | None -> ());
                  loop ()
                end
              else if String.length line >= 5
                      && (String.sub line 0 5 = "curl "
                          || String.sub line 0 5 = "curl:") then begin
                (try Sys.remove tmp with _ -> ());
                on_event (Fail line);
                let* _ = proc#close in
                Lwt.return (Error line)
              end else
                loop ()
        in
        loop ())
      (fun exn ->
        (try Sys.remove tmp with _ -> ());
        let e = "LLM stream transport: " ^ Printexc.to_string exn in
        on_event (Fail e);
        Lwt.return (Error e))


(** ---- Multi-turn chat + OpenAI tools (agent loop) ---- *)

type role = System | User | Assistant | Tool

type tool_call = {
  id : string;
  name : string;
  arguments : string;  (* JSON object string *)
}

type chat_message =
  | Msg of { role : role; content : string }
  | Assistant_tools of { content : string option; tool_calls : tool_call
    list }
  | Tool_result of { tool_call_id : string; content : string }

type tool_def = {
  name : string;
  description : string;
  parameters : Yojson.Safe.t;  (* JSON Schema object *)
}

type turn_result =
  | Final of string
  | Need_tools of { content : string option; tool_calls : tool_call list }
  | Turn_error of string

let role_to_string = function
  | System -> "system"
  | User -> "user"
  | Assistant -> "assistant"
  | Tool -> "tool"

let message_to_json = function
  | Msg { role; content } ->
      `Assoc
        [ ("role", `String (role_to_string role))
        ; ("content", `String content)
        ]
  | Assistant_tools { content; tool_calls } ->
      let tcs =
        List.map
          (fun (tc : tool_call) ->
            `Assoc
```

```
              [ ("id", `String tc.id)
              ; ("type", `String "function")
              ; ("function",
                 `Assoc
                   [ ("name", `String tc.name)
                   ; ("arguments", `String tc.arguments)
                   ])
              ])
          tool_calls
      in
      let base =
        [ ("role", `String "assistant")
        ; ("tool_calls", `List tcs)
        ]
      in
      let base =
        match content with
        | Some c when c <> "" -> ("content", `String c) :: base
        | _ -> ("content", `Null) :: base
      in
      `Assoc base
  | Tool_result { tool_call_id; content } ->
      `Assoc
        [ ("role", `String "tool")
        ; ("tool_call_id", `String tool_call_id)
        ; ("content", `String content)
        ]

let tool_def_to_json (t : tool_def) : Yojson.Safe.t =
  `Assoc
    [ ("type", `String "function")
    ; ("function",
       `Assoc
         [ ("name", `String t.name)
         ; ("description", `String t.description)
         ; ("parameters", t.parameters)
         ])
    ]

(** Recover when models dump tools as XML in content instead of API
    tool_calls.
    Format seen in the wild (GLM etc.):
      <tool_call>install_function
      <arg_key>name</arg_key><arg_value>foo</arg_value>
      <arg_key>body</arg_key><arg_value>(lambda () 1)</arg_value>
      </tool_call> *)
let find_substring ~(hay : string) ~(needle : string) ~(from : int) : int
    option =
  let nlen = String.length needle in
  let hlen = String.length hay in
  let rec go i =
    if i + nlen > hlen then None
    else if String.sub hay i nlen = needle then Some i
    else go (i + 1)
  in
  go from

let parse_xml_tool_calls (text : string) : tool_call list =
  let open_tag = "<tool_call>" in
  let close_tag = "</tool_call>" in
  let rec find_blocks acc from =
    match find_substring ~hay:text ~needle:open_tag ~from with
    | None -> List.rev acc
    | Some start ->
        let body_start = start + String.length open_tag in
        (match find_substring ~hay:text ~needle:close_tag ~from:body_start
    with
         | None -> List.rev acc
         | Some stop ->
             let block = String.sub text body_start (stop - body_start) in
             find_blocks (block :: acc) (stop + String.length close_tag))
  in
  let parse_args block =
```

```
    let rec collect acc from =
      match find_substring ~hay:block ~needle:"<arg_key>" ~from with
      | None -> List.rev acc
      | Some k0 ->
          let k1 = k0 + String.length "<arg_key>" in
          (match find_substring ~hay:block ~needle:"</arg_key>" ~from:k1
     with
           | None -> List.rev acc
           | Some k2 ->
               let key = String.trim (String.sub block k1 (k2 - k1)) in
               let after_key = k2 + String.length "</arg_key>" in
               (match
                  find_substring ~hay:block ~needle:"<arg_value>"
     ~from:after_key
                with
                | None -> List.rev acc
                | Some v0 ->
                    let v1 = v0 + String.length "<arg_value>" in
                    (match
                       find_substring ~hay:block ~needle:"</arg_value>"
     ~from:v1
                     with
                     | None -> List.rev acc
                     | Some v2 ->
                         let value = String.sub block v1 (v2 - v1) in
                         collect ((key, value) :: acc)
                           (v2 + String.length "</arg_value>"))))
    in
    collect [] 0
  in
  let parse_one idx block =
    let block = String.trim block in
    let name =
      match find_substring ~hay:block ~needle:"<arg_key>" ~from:0 with
      | Some i -> String.trim (String.sub block 0 i)
      | None ->
          (match String.split_on_char '\n' block with
           | n :: _ -> String.trim n
           | [] -> "")
    in
    let name =
      match String.split_on_char ' ' (String.trim name) with
      | x :: _ -> String.trim x
      | [] -> String.trim name
    in
    if name = "" then None
    else
      let args = parse_args block in
      let arguments =
        Yojson.Safe.to_string
          (`Assoc (List.map (fun (k, v) -> (k, `String v)) args))
      in
      Some
        { id = Printf.sprintf "xml-tool-%d" (idx + 1)
        ; name
        ; arguments
        }
  in
  find_blocks [] 0
  |> List.mapi parse_one
  |> List.filter_map (fun x -> x)

let parse_tool_calls (msg : Yojson.Safe.t) : tool_call list =
  let open Yojson.Safe.Util in
  match msg |> member "tool_calls" with
  | `Null -> []
  | `List xs ->
      List.filter_map
        (fun tc ->
          try
            let id = tc |> member "id" |> to_string in
            let fn = tc |> member "function" in
            let name = fn |> member "name" |> to_string in
```

```
            let arguments =
              match fn |> member "arguments" with
              | `String s -> s
              | other -> Yojson.Safe.to_string other
            in
            Some { id; name; arguments }
          with _ -> None)
        xs
  | _ -> []

let parse_turn_response (body : string) : turn_result =
  try
    let json = Yojson.Safe.from_string body in
    let open Yojson.Safe.Util in
    match json |> member "choices" |> to_list with
    | [] -> Turn_error "LLM: empty choices"
    | choice :: _ ->
        let msg = choice |> member "message" in
        let finish =
          match choice |> member "finish_reason" |> to_string_option with
          | Some f -> f
          | None -> ""
        in
        let content_opt =
          match msg |> member "content" |> to_string_option with
          | Some c when String.trim c <> "" -> Some c
          | _ -> None
        in
        let tcs = parse_tool_calls msg in
        if tcs <> [] then
          Need_tools { content = content_opt; tool_calls = tcs }
        else
          (* Fallback: XML tool_call blocks in content (common model
     failure). *)
          let xml_tcs =
            match content_opt with
            | Some c -> parse_xml_tool_calls c
            | None -> []
          in
          if xml_tcs <> [] then
            Need_tools { content = None; tool_calls = xml_tcs }
          else if finish = "tool_calls" then
            Turn_error "LLM: finish_reason=tool_calls but no tool_calls
     payload"
          else
            (match content_opt with
             | Some c -> Final c
             | None ->
                 (match msg |> member "reasoning_content" |>
     to_string_option with
                  | Some r when String.trim r <> "" -> Final r
                  | _ ->
                      Turn_error
                        (Printf.sprintf "LLM: empty content (finish=%s)"
     finish)))
  with e ->
    let trunc =
      if String.length body > 300 then String.sub body 0 300 ^ "..." else
     body
    in
    Turn_error ("LLM parse error: " ^ Printexc.to_string e ^ " body=" ^
     trunc)

(** One non-streaming chat completions call over a full message list. *)
let chat_turn
    ?(temperature = 0.2)
    ?(max_tokens = 4096)
    ?(tools : tool_def list = [])
    ~(messages : chat_message list)
    () : turn_result Lwt.t =
  let key = api_key () in
  if key = "" then
    Lwt.return (Turn_error "LLM_API_KEY not set")
```

```
  else
    let msgs = `List (List.map message_to_json messages) in
    let fields =
      [ ("model", `String (model ()))
      ; ("messages", msgs)
      ; ("temperature", `Float temperature)
      ; ("max_tokens", `Int max_tokens)
      ]
    in
    let fields =
      if tools = [] then fields
      else
        ("tools", `List (List.map tool_def_to_json tools))
        :: ("tool_choice", `String "auto")
        :: fields
    in
    let body_str = Yojson.Safe.to_string (`Assoc fields) in
    let url = api_base () ^ "/chat/completions" in
    let tmp = Filename.temp_file "pp_llm_turn_" ".json" in
    let oc = open_out tmp in
    output_string oc body_str;
    close_out oc;
    let cmd =
      Printf.sprintf
        "curl -sS --max-time 180 -X POST %s \
         -H %s -H %s --data-binary @%s 2>&1"
        (shell_quote url)
        (shell_quote "Content-Type: application/json")
        (shell_quote ("Authorization: Bearer " ^ key))
        (shell_quote tmp)
    in
    Printf.eprintf "[llm] TURN %s model=%s tools=%d msgs=%d\n%!"
      url (model ()) (List.length tools) (List.length messages);
    Lwt.catch
      (fun () ->
        let* stdout =
          Lwt_process.pread ~timeout:200. (Lwt_process.shell cmd)
        in
        (try Sys.remove tmp with _ -> ());
        if String.trim stdout = "" then
          Lwt.return (Turn_error "LLM: empty curl response")
        else if String.length stdout > 6
                && String.sub stdout 0 5 = "curl " then
          Lwt.return (Turn_error ("curl: " ^ String.trim stdout))
        else
          Lwt.return (parse_turn_response stdout))
      (fun exn ->
        (try Sys.remove tmp with _ -> ());
        Lwt.return (Turn_error ("LLM transport: " ^ Printexc.to_string
    exn)))

(** Ask the model to write one Lisp function body (s-expr).
    Returns source text for (define name <body>). *)
let propose_function_source
    ~(name : string)
    ~(spec : string)
    ?(context = "")
    () : call_result Lwt.t =
  let system =
    "You are a Pricklypear Lisp author. Respond ONLY with a single Lisp \
     s-expression that is the BODY of a function (typically a lambda). \
     No markdown fences, no explanation, no (define ...) wrapper. \
     Available prims include: + - * = < > <= >= not list car cdr cons \
     null? pair? count string-append str type-of whoami list-bindings \
     dict-get dict-set dict-keys get create update delete rows deftype \
     fork merge route checkout render-html. Use quote for symbols/data. \
     Prefer small pure functions unless the task needs DB/IO prims."
  in
  let user =
    Printf.sprintf
      "Function name: %s\nRequirement:\n%s\n%s\nWrite only the body
    s-expression."
      name spec
```

```
      (if context = "" then "" else "Context:\n" ^ context ^ "\n")
  in
  let* res = call ~system ~user () in
  match res with
  | Error e -> Lwt.return (Error e)
  | Ok text ->
      let cleaned = strip_fences text in
      Lwt.return (Ok cleaned)

(** Surgical AST author: rewrite ONE component (coding tool). *)
let rewrite_component_source
    ~(mode : string)  (* "replace" | "wrap" *)
    ~(name : string)
    ~(concept : string)
    ~(current_pretty : string)
    () : call_result Lwt.t =
  let system =
    "You are editing a Pricklypear Lisp AST as a coding tool. " ^
    "Return ONLY one replacement s-expression (typically a lambda). " ^
    "No markdown fences, no commentary, no (define ...) wrapper. " ^
    "You write ONE component of a larger program tree. " ^
    "Available prims: + - * = < > <= >= not list car cdr cons null? pair? "
    ^
    "count string-append str type-of dict-get dict-set dict-keys " ^
    "get create update delete rows deftype render-html whoami. " ^
    "Prefer pure functions when possible."
  in
  let user =
    if mode = "wrap" then
      Printf.sprintf
        "Mode: WRAP (llm-outer)\nFunction name: %s\n\
         User concept: %s\n\
         Current function (pretty):\n%s\n\
         Write a new lambda that wraps this behavior (call-through or \
         carefully inline). Preserve the essential behavior and add \
         ONLY the requested responsibility. Return one lambda only."
        name concept current_pretty
    else
      Printf.sprintf
        "Mode: REPLACE body (llm-inner)\nFunction name: %s\n\
         User concept: %s\n\
         Current function (pretty):\n%s\n\
         Write a replacement function body (usually a lambda) that \
         satisfies the concept. Return one s-expression only."
        name concept current_pretty
  in
  let* res = call ~system ~user () in
  match res with
  | Error e -> Lwt.return (Error e)
  | Ok text -> Lwt.return (Ok (strip_fences text))

(** Seed: smallest clean core for growth. *)
let seed_core_source ~(requirement : string) () : call_result Lwt.t =
  let system =
    "You write the SMALLEST and CLEANEST Pricklypear Lisp nucleus function.
     " ^
    "Return ONLY one lambda s-expression. No markdown, no define wrapper. "
    ^
    "Prefer pure computation. No wrapping, no IO unless absolutely
    required."
  in
  let user =
    Printf.sprintf
      "Requirement:\n%s\nWrite the smallest pure-ish lambda that captures
     the essential core."
      requirement
  in
  let* res = call ~system ~user () in
  match res with
  | Error e -> Lwt.return (Error e)
  | Ok text -> Lwt.return (Ok (strip_fences text))

(** Judge growth: returns Ok "done" or Ok "next: <responsibility>" *)
```

```
let growth_judge
    ~(requirement : string)
    ~(current_pretty : string)
    ~(step : int)
    () : call_result Lwt.t =
  let system =
    "You judge whether a growing Lisp function set satisfies a requirement.
     " ^
    "Respond with EXACTLY one line: either DONE or NEXT: <single short
     responsibility>. " ^
    "No markdown. NEXT must be one outer wrap responsibility only."
  in
  let user =
    Printf.sprintf
      "Requirement:\n%s\nStep: %d\nCurrent tip function:\n%s\nJudge."
      requirement step current_pretty
  in
  let* res = call ~system ~user ~temperature:0.1 () in
  match res with
  | Error e -> Lwt.return (Error e)
  | Ok text -> Lwt.return (Ok (String.trim (strip_fences text)))
```

46 image/lib/loader.ml

```
(** Pure library loader: Lisp source text ? define / route / deftype.

    Same source files live in git (libs/<name>/lib.pp) and load into the
    live branch. Evaluates AST forms directly (no pp_expr round-trip). *)
open Lwt.Syntax
open Pricklypear_lisp.Ast

let libs_dir () : string =
  match Sys.getenv_opt "PP_LIBS_DIR" with
  | Some d when d <> "" -> d
  | _ ->
      if Sys.file_exists "libs" then "libs"
      else if Sys.file_exists "../libs" then "../libs"
      else "libs"

let library_path (name : string) : string =
  let root = libs_dir () in
  let a = Filename.concat root (Filename.concat name "lib.pp") in
  let b = Filename.concat root (name ^ ".pp") in
  if Sys.file_exists a then a
  else if Sys.file_exists b then b
  else a

let read_file (path : string) : (string, string) result =
  try
    let ic = open_in_bin path in
    let len = in_channel_length ic in
    let s = really_input_string ic len in
    close_in ic;
    Ok s
  with exn -> Error (path ^ ": " ^ Printexc.to_string exn)

let strip_line_comments (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i in_str =
    if i >= len then Buffer.contents buf
    else
      let c = s.[i] in
      if in_str then begin
        Buffer.add_char buf c;
        if c = '\\' && i + 1 < len then begin
          Buffer.add_char buf s.[i + 1];
          loop (i + 2) true
        end else if c = '"' then loop (i + 1) false
        else loop (i + 1) true
      end else if c = '"' then begin
        Buffer.add_char buf c;
        loop (i + 1) true
      end else if c = ';' then
        let rec skip j =
          if j >= len || s.[j] = '\n' then j else skip (j + 1)
        in
        let j = skip i in
        if j < len then Buffer.add_char buf '\n';
        loop (if j < len then j + 1 else j) false
      else begin
        Buffer.add_char buf c;
        loop (i + 1) false
      end
  in
  loop 0 false

let load_string
    ~(eval_form : expr -> value Lwt.t)
    (src : string)
    : (string, string) result Lwt.t =
  try
    let src = strip_line_comments src in
    let forms = Pricklypear_lisp.Reader.read_all src in
    let rec go i = function
```

```
      | [] ->
          Lwt.return (Ok (Printf.sprintf "loaded %d form(s)" i))
      | e :: rest ->
          Lwt.catch
            (fun () ->
              let* _ = eval_form e in
              go (i + 1) rest)
            (fun exn ->
              let msg =
                match exn with
                | Interp_error err ->
                    (match err with
                     | User_error m -> m
                     | Parse_error (m, l, c) ->
                         Printf.sprintf "parse: %s (%d:%d)" m l c
                     | Unbound_var s -> "unbound: " ^ s
                     | _ -> Printexc.to_string exn)
                | _ -> Printexc.to_string exn
              in
              let snippet =
                try String.sub (Pricklypear_lisp.Reader.pp_expr e) 0
                      (min 80 (String.length
    (Pricklypear_lisp.Reader.pp_expr e)))
                with _ -> "?"
              in
              Lwt.return
                (Error
                   (Printf.sprintf "form %d failed: %s\n  in: %s" (i + 1)
    msg
                      snippet)))
    in
    go 0 forms
  with
  | Pricklypear_lisp.Reader.Reader_error (m, l, c) ->
      Lwt.return (Error (Printf.sprintf "reader: %s (%d:%d)" m l c))
  | exn -> Lwt.return (Error (Printexc.to_string exn))

let load_library
    ~(eval_form : expr -> value Lwt.t)
    (name : string)
    : (string, string) result Lwt.t =
  let path = library_path name in
  match read_file path with
  | Error e -> Lwt.return (Error e)
  | Ok src ->
      let* r = load_string ~eval_form src in
      (match r with
       | Ok msg -> Lwt.return (Ok (Printf.sprintf "%s from %s" msg path))
       | Error e -> Lwt.return (Error e))
```

47 image/lib/packages.ml

```
(** In-branch prefix packages: organize function names with '/'.

    Orthogonal to branch ownership. Full binding key remains the
    entire name string (e.g. "todo/list"). See image.packages. *)

let package_of_name (name : string) : string =
  match String.rindex_opt name '/' with
  | None -> ""
  | Some i -> String.sub name 0 i

let local_name (name : string) : string =
  match String.rindex_opt name '/' with
  | None -> name
  | Some i -> String.sub name (i + 1) (String.length name - i - 1)

let join_name ~(package : string) ~(local : string) : string =
  let package = String.trim package in
  let local = String.trim local in
  if package = "" then local
  else if local = "" then package
  else
    let pkg =
      if String.length package > 0 && package.[String.length package - 1] =
     '/'
      then String.sub package 0 (String.length package - 1)
      else package
    in
    pkg ^ "/" ^ local

(** Group (name, body, author) defs by package. Sorted package keys;
    root "" first. Within package, sort by full name. *)
let group_by_package
    (defs : (string * 'a * string) list)
    : (string * (string * 'a * string) list) list =
  let tbl : (string, (string * 'a * string) list) Hashtbl.t =
    Hashtbl.create 16
  in
  List.iter
    (fun ((name, _, _) as row) ->
      let pkg = package_of_name name in
      let prev = try Hashtbl.find tbl pkg with Not_found -> [] in
      Hashtbl.replace tbl pkg (row :: prev))
    defs;
  let pkgs =
    Hashtbl.fold (fun k _ acc -> k :: acc) tbl []
    |> List.sort (fun a b ->
           match a, b with
           | "", _ -> -1
           | _, "" -> 1
           | _ -> String.compare a b)
  in
  List.map
    (fun pkg ->
      let rows =
        Hashtbl.find tbl pkg
        |> List.sort (fun (a, _, _) (b, _, _) -> String.compare a b)
      in
      (pkg, rows))
    pkgs

let package_label (pkg : string) : string =
  if pkg = "" then "(root)" else pkg

(** URL-encode a function name for path use (encode / as %2F). *)
let name_encode (name : string) : string =
  let buf = Buffer.create (String.length name + 8) in
  String.iter
    (function
      | '/' -> Buffer.add_string buf "%2F"
      | ' ' -> Buffer.add_string buf "%20"
      | '+' -> Buffer.add_string buf "%2B"
```

```
      | '(' -> Buffer.add_string buf "%28"
      | ')' -> Buffer.add_string buf "%29"
      | '&' -> Buffer.add_string buf "%26"
      | '=' -> Buffer.add_string buf "%3D"
      | c -> Buffer.add_char buf c)
    name;
  Buffer.contents buf

(** Decode path segment(s) after /edit/ back to a function name. *)
let name_decode (s : string) : string =
  (* reuse minimal decode for %2F and + *)
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i =
    if i >= len then Buffer.contents buf
    else
      match s.[i] with
      | '+' ->
          Buffer.add_char buf ' ';
          loop (i + 1)
      | '%' when i + 2 < len ->
          let hex = String.sub s (i + 1) 2 in
          (try
             let code = int_of_string ("0x" ^ hex) in
             Buffer.add_char buf (Char.chr code)
           with _ -> Buffer.add_char buf '%');
          loop (i + 3)
      | c ->
          Buffer.add_char buf c;
          loop (i + 1)
  in
  loop 0

let edit_href (name : string) : string = "/edit/" ^ name_encode name
let binding_href (name : string) : string = "/binding/" ^ name_encode name
let inspect_href (name : string) : string =
  "/inspect/binding/" ^ name_encode name
```

48 image/lib/primitives.ml

```
(** Pricklypear Image -- Primitive registry.

    Primitives are OCaml functions exposed to Lisp. They live in the image
    (not the lisp/ package) because they're I/O-shaped: read from store,
    emit htmx, persist, etc. The lisp/ package is pure I/O-free.

    Spec [lisp-interpreter.primitives] lists the full ~20; for the minimal
    image, ~5 are sufficient to demo the loop. *)
open Lwt
open Pricklypear_lisp.Ast
(* A prim fn takes evaluated args, returns a value (or raises) wrapped in
   a promise. Pure prims register via [register_sync] which wraps the sync
   prim in [Lwt.return]; DB-touching prims register via [register] and
   return their promise directly. *)
type prim_fn = value list -> value Lwt.t

(* The sync form: pure prims return [value] immediately. They get wrapped
   in [Lwt.return] when registered via [register_sync]. *)
type prim_fn_sync = value list -> value

type t = (string, prim_fn) Hashtbl.t

let make () : t = Hashtbl.create 16

let register (t : t) (name : string) (fn : prim_fn) : unit =
  Hashtbl.replace t name fn

let register_sync (t : t) (name : string) (fn : prim_fn_sync) : unit =
  Hashtbl.replace t name (fun args -> Lwt.return (fn args))

let lookup (t : t) (name : string) : prim_fn option =
  Hashtbl.find_opt t name

(* Numeric helpers: VInt | VFloat with promotion (int+float ? float). *)
(* --- Session-aware primitives ---

   These need access to the running session (env, manifest). They're
   registered as closures at session-make time via
   [register_session_prims]. The signature stays prim_fn (value list
   -> value) -- the closure captures the session. *)

(* (whoami) -> VStr. Current session username. [username_fn] is
   re-read on each call so cotenant multi-user can switch identity
   per HTTP request without re-registering prims. *)
let prim_whoami_fn ~(username_fn : unit -> string) (_ : value list) : value
    =
  VStr (username_fn ())

(* (current-author) -> VStr. Same as whoami for now; reserved for
   the multi-user permission model where (current-author) consults
   the binding being executed rather than the session. *)
let prim_current_author_fn ~(username_fn : unit -> string)
    (_ : value list) : value =
  VStr (username_fn ())

(* (list-bindings) -> VList of binding-name VStrs. Lists all names
   currently visible in the session env. *)
let prim_list_bindings_fn ~(env : Pricklypear_lisp.Ast.env) (_ : value
    list) : value =
  let names =
    Pricklypear_lisp.Env.fold env
      (fun name _ acc -> name :: acc) []
  in
  VList (List.map (fun s -> VStr s) names)

(* Register session-aware primitives into [t]. Called at session-make
   time with a dynamic username getter + env. *)
let register_session_prims
    (t : t)
    ~(username_fn : unit -> string)
    ~(env : Pricklypear_lisp.Ast.env)
```

```
    : unit =
  register_sync t "whoami" (prim_whoami_fn ~username_fn);
  register_sync t "current-author" (prim_current_author_fn ~username_fn);
  register_sync t "list-bindings" (prim_list_bindings_fn ~env)


(* (date-today) -> VStr "YYYY-MM-DD" in local time. *)

open Prims_pure

(* Re-export pure helpers used by Session / Http. *)
let is_raw_html = Prims_pure.is_raw_html
let is_redirect = Prims_pure.is_redirect
let redirect_value = Prims_pure.redirect_value
let prim_dict_set = Prims_pure.prim_dict_set

(* Register the minimal set. Returns the populated table. *)
let make_with_defaults () : t =
  let t = make () in
  register_sync t "+" prim_add;
  register_sync t "-" prim_sub;
  register_sync t "*" prim_mul;
  register_sync t "div" prim_div;
  register_sync t "mod" prim_mod;
  register_sync t "abs" prim_abs;
  register_sync t "min" prim_min;
  register_sync t "max" prim_max;
  register_sync t "=" prim_num_eq;
  register_sync t "<" prim_lt;
  register_sync t ">" prim_gt;
  register_sync t "<=" prim_le;
  register_sync t ">=" prim_ge;
  register_sync t "not" prim_not;
  register_sync t "list" prim_list;
  register_sync t "car" prim_car;
  register_sync t "cdr" prim_cdr;
  register_sync t "cons" prim_cons;
  register_sync t "null?" prim_null_p;
  register_sync t "pair?" prim_pair_p;
  register_sync t "string-append" prim_string_append;
  register_sync t "string-length" prim_string_length;
  register_sync t "substring" prim_substring;
  register_sync t "string-contains?" prim_string_contains;
  register_sync t "string-eq" prim_string_eq;
  register_sync t "string-trim" prim_string_trim;
  register_sync t "html-escape" prim_html_escape;
  register_sync t "json-parse" prim_json_parse;
  register_sync t "json-emit" prim_json_emit;
  register_sync t "type-of" prim_type_of;
  register_sync t "str" prim_str;
  register_sync t "as-str" prim_as_str;
  register_sync t "append" prim_append;
  register_sync t "reverse" prim_reverse;
  register_sync t "nth" prim_nth;
  (* list-branches is registered as a session DB prim in session.ml *)
  register_sync t "hx-get" prim_hx_get;
  register_sync t "hx-post" prim_hx_post;
  register_sync t "count" prim_count;
  register_sync t "dict-get" prim_dict_get;
  register_sync t "dict-set" prim_dict_set;
  register_sync t "date-today" prim_date_today;
  register_sync t "parse-int" prim_parse_int;
  register_sync t "parse-float" prim_parse_float;
  register_sync t "floor" prim_floor;
  register_sync t "round" prim_round;
  register_sync t "date-weekday" prim_date_weekday;
  register_sync t "date-format" prim_date_format;
  register_sync t "date-add-days" prim_date_add_days;
  register_sync t "date-diff-days" prim_date_diff_days;
  register_sync t "fsrs-schedule" prim_fsrs_schedule;
  register_sync t "dict-keys" prim_dict_keys;
  register_sync t "render-html" prim_render_html;
  t
```

```
143
144 (* Short docs for the Functions browser (kernel package). *)
145 let prim_summary (name : string) : string =
146   match name with
147   | "+" -> "add numbers"
148   | "-" -> "subtract / negate"
149   | "*" -> "multiply"
150   | "div" -> "integer divide"
151   | "mod" -> "integer remainder"
152   | "abs" -> "absolute value"
153   | "min" -> "minimum of VInts"
154   | "max" -> "maximum of VInts"
155   | "=" -> "numeric equality"
156   | "<" | ">" | "<=" | ">=" -> "numeric compare"
157   | "not" -> "boolean not"
158   | "list" -> "build a list"
159   | "car" -> "first of pair/list"
160   | "cdr" -> "rest of pair/list"
161   | "cons" -> "cons cell"
162   | "null?" -> "empty list?"
163   | "pair?" -> "is pair?"
164   | "count" -> "length of list"
165   | "date-today" -> "local date YYYY-MM-DD"
166   | "parse-int" -> "string or int to int"
167   | "parse-float" -> "string or number to float"
168   | "floor" | "round" -> "float to int"
169   | "date-weekday" -> "weekday 0=Sun for y m d"
170   | "date-format" -> "format y m d as YYYY-MM-DD"
171   | "date-add-days" -> "add n days to YYYY-MM-DD (or today)"
172   | "date-diff-days" -> "days between two YYYY-MM-DD strings"
173   | "fsrs-schedule" -> "FSRS-4.5: (S D elapsed rating) -> (S' D' interval)"
174
175   | "string-append" -> "concat strings"
176   | "string-length" -> "length of string"
177   | "substring" -> "substring start [len]"
178   | "string-contains?" -> "haystack contains needle?"
179   | "string-eq" -> "string equality"
180   | "string-trim" -> "trim whitespace from string"
181   | "html-escape" -> "escape text for HTML"
182   | "json-parse" -> "parse JSON string to Lisp value"
183   | "json-emit" -> "emit Lisp value as JSON string"
184   | "request-method" -> "HTTP method of active request"
185   | "request-path" -> "path of active request"
186   | "request-param" -> "form/query param of active request"
187   | "request-header" -> "HTTP header of active request"
188   | "hx-request?" -> "true if HX-Request header present"
189   | "request-params" -> "all params as JSON object"
190   | "request-body" -> "raw body of active request"
191   | "redirect" -> "HTTP 303 redirect response"
192   | "append" -> "concatenate lists"
193   | "reverse" -> "reverse list"
194   | "nth" -> "nth element of list (0-based)"
195   | "str" -> "value to string"
196   | "as-str" -> "plain string coerce (no quotes)"
197   | "type-of" -> "runtime type tag"
198   | "dict-get" -> "JSON object field get"
199   | "dict-set" -> "JSON object field set"
200   | "dict-keys" -> "JSON object keys"
201   | "render-html" -> "mark string as raw HTML"
202   | "hx-get" | "hx-post" -> "htmx attribute helper"
203   | "whoami" | "current-author" -> "session identity"
204   | "list-bindings" -> "names in env"
205   | "list-branches" -> "list branch refs"
206   | "current-branch" -> "current branch name"
207   | "fork" -> "fork a branch"
208   | "merge" -> "merge branches"
209   | "checkout" -> "switch branch"
210   | "route" -> "bind URL to function"
211   | "deftype" -> "declare a data type"
212   | "get" -> "read one data row"
213   | "create" -> "insert data row ? id"
214   | "update" -> "replace row fields JSON"
215   | "update-field" -> "set one JSON field"
```

```
  | "delete" -> "delete data row"
  | "rows" -> "list data rows"
  | "find-row" -> "first row by fields->>key"
  | "find-rows" -> "rows filtered by fields"
  | "find-text" -> "substring search on a field"
  | "find-random" -> "one random data row of a type"
  | "select-for-update" -> "lock row in transaction"
  | "agent-session-new" -> "create durable AI session: (agent-session-new
     seed [tools?])"
  | "agent-say" -> "append user msg + run tool loop: (agent-say sid prompt)"
  | "agent-history" -> "list messages in AI session"
  | "agent-sessions" -> "list recent AI sessions for whoami"
  | "agent-session-delete" -> "delete AI session and messages"
  | "agent" -> "alias: new session + one say (prefer agent-session-new/say)"
  | "agent-task" -> "alias: new tools session + one say"
  | "propose-function" -> "AI draft+install function on branch"
  | "install-function" -> "install AI body string as named function"
  | "function-source" -> "pretty-print stored function body"
  | "package-bindings" -> "list binding names in a package"
  | "export-package" -> "export package as lib.pp text"
  | "llm-inner" | "llm-outer" | "llm-rewrite" | "llm-seed" ->
      "AI rewrite / growth tool"
  | "__with_transaction__" -> "internal transaction helper"
  | "load-string" -> "load Lisp source text (define/route/deftype)"
  | "load-library" -> "load libs/<name>/lib.pp into current branch"
  | _ -> "built-in primitive"

(** All registered prim names with one-line summaries, sorted. *)
let catalog (t : t) : (string * string) list =
  Hashtbl.fold
    (fun name _ acc -> (name, prim_summary name) :: acc)
    t []
  |> List.sort (fun (a, _) (b, _) -> String.compare a b)

let list_names (t : t) : string list =
  List.map fst (catalog t)

(* Convert the table into the shape Pricklypear_lisp.Eval.prim_table
   expects. Delegating to a separate function so the integration with
   the interpreter is explicit. *)
let to_eval_table (t : t) : Pricklypear_lisp.Eval.prim_table =
  let module E = Pricklypear_lisp.Eval in
  let tbl = E.make_prim_table () in
  Hashtbl.iter (fun name fn -> E.register_prim tbl name fn) t;
  tbl
```

49 image/lib/prims_pure.ml

```
(** Pure Lisp primitives (no DB, no Session.t).
    Registered by [Primitives.make_with_defaults]. *)
open Pricklypear_lisp.Ast

type num = NInt of int64 | NFloat of float

let num_of_value = function
  | VInt n -> NInt n
  | VFloat f -> NFloat f
  | v ->
      raise
        (Interp_error
           (User_error ("numeric op needs int or float, got " ^
                        (match v with
                         | VStr _ -> "string"
                         | VBool _ -> "bool"
                         | VNil -> "nil"
                         | _ -> "other"))))

let float_of_num = function
  | NInt n -> Int64.to_float n
  | NFloat f -> f

let value_of_num = function
  | NInt n -> VInt n
  | NFloat f -> VFloat f

let num_add a b =
  match a, b with
  | NInt x, NInt y -> NInt (Int64.add x y)
  | _ -> NFloat (float_of_num a +. float_of_num b)

let num_sub a b =
  match a, b with
  | NInt x, NInt y -> NInt (Int64.sub x y)
  | _ -> NFloat (float_of_num a -. float_of_num b)

let num_mul a b =
  match a, b with
  | NInt x, NInt y -> NInt (Int64.mul x y)
  | _ -> NFloat (float_of_num a *. float_of_num b)

let num_div a b =
  let bf = float_of_num b in
  if bf = 0. then
    raise (Interp_error (User_error "div by zero"))
  else
    match a, b with
    | NInt x, NInt y -> NInt (Int64.div x y)
    | _ -> NFloat (float_of_num a /. bf)

let num_neg = function
  | NInt n -> NInt (Int64.neg n)
  | NFloat f -> NFloat (~-. f)

let num_compare a b =
  match a, b with
  | NInt x, NInt y -> Int64.compare x y
  | _ -> compare (float_of_num a) (float_of_num b)

(* "+" sums two-or-more numeric args. Variadic. *)
let prim_add (args : value list) : value =
  if args = [] then
    raise (Interp_error (User_error "+ needs at least one arg"))
  else
    value_of_num
      (List.fold_left
         (fun acc v -> num_add acc (num_of_value v))
         (num_of_value (List.hd args))
         (List.tl args))

```

```
(* "-" binary: (- a b) = a - b; unary: (- a) = -a. *)
let prim_sub args =
  match args with
  | [ a ] -> value_of_num (num_neg (num_of_value a))
  | [ a; b ] -> value_of_num (num_sub (num_of_value a) (num_of_value b))
  | _ -> raise (Interp_error (User_error "- needs one or two numeric args"))

(* "*". Two-or-more numeric args. *)
let prim_mul (args : value list) : value =
  if args = [] then
    raise (Interp_error (User_error "* needs at least one arg"))
  else
    value_of_num
      (List.fold_left
         (fun acc v -> num_mul acc (num_of_value v))
         (num_of_value (List.hd args))
         (List.tl args))

(* "=" numeric equality (int/float). *)
let prim_num_eq args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) = 0)
  | _ -> raise (Interp_error (User_error "= needs two numeric args"))

let prim_lt args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) < 0)
  | _ -> raise (Interp_error (User_error "< needs two numeric args"))

let prim_gt args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) > 0)
  | _ -> raise (Interp_error (User_error "> needs two numeric args"))

let prim_le args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) <= 0)
  | _ -> raise (Interp_error (User_error "<= needs two numeric args"))

let prim_ge args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) >= 0)
  | _ -> raise (Interp_error (User_error ">= needs two numeric args"))

(* "not" boolean negation *)
let prim_not args =
  match args with
  | [ VBool b ] -> VBool (not b)
  | [ VNil ] -> VBool true
  | [ _ ] -> VBool false   (* any non-nil non-false is truthy *)
  | _ -> raise (Interp_error (User_error "not needs one arg"))

(* (list a b c ...) -> VList. Variadic. *)
let prim_list args = VList args

(* (car xs): head of a VList or VPair. Errors on empty / atoms. *)
let prim_car args =
  match args with
  | [ VList (h :: _) ] -> h
  | [ VList [] ] -> VNil
  | [ VPair (a, _) ] -> a
  | _ -> raise (Interp_error (User_error "car needs a list or pair"))

(* (cdr xs): tail of a VList or VPair. *)
let prim_cdr args =
  match args with
  | [ VList (_ :: rest) ] -> VList rest
  | [ VList [] ] -> VNil
  | [ VPair (_, b) ] -> b
  | _ -> raise (Interp_error (User_error "cdr needs a list or pair"))

(* (cons a b) -> VPair. Classic Lisp cons cell. *)
let prim_cons args =
```

```
  match args with
  | [ a; b ] -> VPair (a, b)
  | _ -> raise (Interp_error (User_error "cons needs two args"))

(* (null? x) -> VBool. True if VNil. *)
let prim_null_p args =
  match args with
  | [ VNil ] -> VBool true
  | [ VList [] ] -> VBool true
  | [ _ ] -> VBool false
  | _ -> raise (Interp_error (User_error "null? needs one arg"))

(* (pair? x) -> VBool. True if VPair or non-empty VList. *)
let prim_pair_p args =
  match args with
  | [ VPair _ ] -> VBool true
  | [ VList (_ :: _) ] -> VBool true
  | _ -> VBool false

(* (string-append s1 s2 ...) -> VStr. Variadic over VStr. *)
let prim_string_append args =
  let buf = Buffer.create 32 in
  List.iter (fun v ->
    match v with
    | VStr s -> Buffer.add_string buf s
    | _ -> raise (Interp_error (User_error "string-append needs VStr
     args")))
    args;
  VStr (Buffer.contents buf)

(* (type-of x) -> VStr. Returns a type tag for runtime reflection. *)
let prim_type_of args =
  match args with
  | [ VInt _ ] -> VStr "int"
  | [ VFloat _ ] -> VStr "float"
  | [ VStr _ ] -> VStr "str"
  | [ VBool _ ] -> VStr "bool"
  | [ VNil ] -> VStr "nil"
  | [ VPair _ ] -> VStr "pair"
  | [ VList _ ] -> VStr "list"
  | [ VLambda _ ] -> VStr "lambda"
  | [ VPrim name ] -> VStr ("prim:" ^ name)
  | [ VRef (t, _) ] -> VStr ("ref:" ^ t)
  | _ -> raise (Interp_error (User_error "type-of needs one arg"))

(* (str x) -> VStr. Coerces any value to its pretty-printed form. *)
let prim_str args =
  match args with
  | [ v ] -> VStr (Pricklypear_lisp.Reader.pp_value v)
  | _ -> raise (Interp_error (User_error "str needs one arg"))

(* (as-str x) -> VStr without Lisp quotes (for path/ref building). *)
let prim_as_str args =
  match args with
  | [ VStr s ] -> VStr s
  | [ VInt n ] -> VStr (Int64.to_string n)
  | [ VFloat f ] -> VStr (Printf.sprintf "%.12g" f)
  | [ VBool true ] -> VStr "true"
  | [ VBool false ] -> VStr "false"
  | [ VNil ] -> VStr ""
  | [ v ] -> VStr (Pricklypear_lisp.Reader.pp_value v)
  | _ -> raise (Interp_error (User_error "as-str needs one arg"))

(* (parse-float s|n) -> VFloat *)
let prim_parse_float args =
  match args with
  | [ VFloat f ] -> VFloat f
  | [ VInt n ] -> VFloat (Int64.to_float n)
  | [ VStr s ] ->
      (try VFloat (float_of_string (String.trim s))
       with _ -> VFloat 0.)
  | _ -> raise (Interp_error (User_error "parse-float needs string or
     number"))
```

```
216
217 (* (floor x) -> VInt *)
218 let prim_floor args =
219   match args with
220   | [ VInt n ] -> VInt n
221   | [ VFloat f ] -> VInt (Int64.of_float (floor f))
222   | _ -> raise (Interp_error (User_error "floor needs one numeric arg"))
223
224 (* (round x) -> VInt *)
225 let prim_round args =
226   match args with
227   | [ VInt n ] -> VInt n
228   | [ VFloat f ] -> VInt (Int64.of_float (Float.round f))
229   | _ -> raise (Interp_error (User_error "round needs one numeric arg"))
230
231 (* (div a b) -- integer div when both int; else float. *)
232 let prim_div args =
233   match args with
234   | [ a; b ] -> value_of_num (num_div (num_of_value a) (num_of_value b))
235   | _ -> raise (Interp_error (User_error "div needs two numeric args"))
236
237 let prim_mod args =
238   match args with
239   | [ VInt a; VInt b ] when b <> 0L -> VInt (Int64.rem a b)
240   | [ VInt _; VInt 0L ] ->
241       raise (Interp_error (User_error "mod by zero"))
242   | [ a; b ] ->
243       let bf = float_of_num (num_of_value b) in
244       if bf = 0. then raise (Interp_error (User_error "mod by zero"))
245       else VFloat (mod_float (float_of_num (num_of_value a)) bf)
246   | _ -> raise (Interp_error (User_error "mod needs two numeric args"))
247
248 let prim_abs args =
249   match args with
250   | [ VInt a ] -> VInt (Int64.abs a)
251   | [ VFloat f ] -> VFloat (abs_float f)
252   | _ -> raise (Interp_error (User_error "abs needs one numeric arg"))
253
254 let prim_min args =
255   match args with
256   | [] -> raise (Interp_error (User_error "min needs at least one numeric
      arg"))
257   | h :: rest ->
258       value_of_num
259         (List.fold_left
260            (fun acc v ->
261              let n = num_of_value v in
262              if num_compare n acc < 0 then n else acc)
263            (num_of_value h) rest)
264
265 let prim_max args =
266   match args with
267   | [] -> raise (Interp_error (User_error "max needs at least one numeric
      arg"))
268   | h :: rest ->
269       value_of_num
270         (List.fold_left
271            (fun acc v ->
272              let n = num_of_value v in
273              if num_compare n acc > 0 then n else acc)
274            (num_of_value h) rest)
275
276 let prim_string_length args =
277   match args with
278   | [ VStr s ] -> VInt (Int64.of_int (String.length s))
279   | _ -> raise (Interp_error (User_error "string-length needs one VStr"))
280
281 let prim_substring args =
282   match args with
283   | [ VStr s; VInt start; VInt len ] ->
284       let st = Int64.to_int start in
285       let n = Int64.to_int len in
286       let sl = String.length s in
```

```
      if st < 0 || n < 0 || st > sl then
        raise (Interp_error (User_error "substring out of range"))
      else
        let n = min n (sl - st) in
        VStr (String.sub s st n)
  | [ VStr s; VInt start ] ->
      let st = Int64.to_int start in
      let sl = String.length s in
      if st < 0 || st > sl then
        raise (Interp_error (User_error "substring out of range"))
      else VStr (String.sub s st (sl - st))
  | _ ->
      raise
        (Interp_error
           (User_error "substring needs string start [len]"))

let prim_string_contains args =
  match args with
  | [ VStr hay; VStr needle ] ->
      let rec loop i =
        if needle = "" then true
        else if i + String.length needle > String.length hay then false
        else if String.sub hay i (String.length needle) = needle then true
        else loop (i + 1)
      in
      VBool (loop 0)
  | _ ->
      raise
        (Interp_error (User_error "string-contains? needs two VStr args"))

let list_of_value = function
  | VList xs -> xs
  | VNil -> []
  | VPair (a, b) ->
      let rec unwind p acc =
        match p with
        | VPair (x, y) -> unwind y (x :: acc)
        | VNil | VList [] -> List.rev acc
        | VList xs -> List.rev_append acc xs
        | other -> List.rev (other :: acc)
      in
      unwind (VPair (a, b)) []
  | _ -> raise (Interp_error (User_error "expected a list"))

let prim_append args =
  match args with
  | [ a; b ] -> VList (list_of_value a @ list_of_value b)
  | xs ->
      VList
        (List.concat_map
           (function
             | VList l -> l
             | VNil -> []
             | _ -> raise (Interp_error (User_error "append needs lists")))
           xs)

let prim_reverse args =
  match args with
  | [ xs ] -> VList (List.rev (list_of_value xs))
  | _ -> raise (Interp_error (User_error "reverse needs one list"))

let prim_nth args =
  match args with
  | [ xs; VInt i ] ->
      let n = Int64.to_int i in
      let lst = list_of_value xs in
      if n < 0 || n >= List.length lst then VNil
      else List.nth lst n
  | _ -> raise (Interp_error (User_error "nth needs list and VInt"))

(* ---- dict primitives over VStr JSON -----------------------------

   Iteration-14 decision: don't introduce a new VDict value type yet
```

```
   (that would ripple through ast.ml / reader.ml / eval.ml / value
   JSON round-trip). Instead, (get 'type id) returns a VStr holding
   the fields JSON; these prims operate on VStr JSON directly so Lisp
   code can do:
       (define u (get 'user id))
       (dict-get u "name")
       (dict-set u "active" "false")
   The primitive returns VStr JSON (or VNil if the input wasn't a valid
   object, or key wasn't found). The underlying parse-and-emit is
   Yojson; the canonical round-trip preserves well-formed objects. *)

(* (dict-get json-str key) -> VStr value | VNil.
   Looks up [key] in the JSON object at [json-str]. Returns the value
   as a VStr (numbers and booleans stringified via Yojson); VNil if
   the key is missing or the input isn't a JSON object. *)
let prim_dict_get args =
  match args with
  | [ VStr json; VStr key ] ->
      (try
        let j = Yojson.Safe.from_string json in
        (match j with
         | `Assoc kvs ->
             (match List.assoc_opt key kvs with
              | Some (`String s) -> VStr s
              | Some (`Int n) -> VStr (string_of_int n)
              | Some (`Intlit s) -> VStr s
              | Some (`Float f) -> VStr (string_of_float f)
              | Some (`Bool b) -> VStr (string_of_bool b)
              | Some (`Null) -> VNil
              | Some (#Yojson.Safe.t as v) ->
                  VStr (Yojson.Safe.to_string v)
              | None -> VNil)
         | _ -> VNil)
      with _ -> VNil)
  | _ -> raise (Interp_error (User_error "dict-get needs 2 strings"))

(* (dict-set json-str key value) -> VStr new-json.
   Builds a JSON object with [key] = [value] (a string; numbers
   detected and emitted as JSON numeric where parseable). Returns the
   updated JSON as a VStr. If [json-str] isn't a JSON object, starts
   a fresh {} and adds the new key. *)
let prim_dict_set args =
  match args with
  | [ VStr json; VStr key; VStr value ] ->
      let base =
        try
          (match Yojson.Safe.from_string json with
           | `Assoc kvs -> kvs
           | _ -> [])
        with _ -> []
      in
      let v_json =
        (* Try parsing the value as JSON (so callers can pass nested
           objects/arrays); if it doesn't parse, emit as a JSON string. *)
        try Yojson.Safe.from_string value
        with _ -> `String value
      in
      let updated =
        (key, v_json) ::
        (List.filter (fun (k, _) -> k <> key) base)
      in
      VStr (Yojson.Safe.to_string (`Assoc updated))
  | _ -> raise (Interp_error (User_error "dict-set needs 3 strings"))

(* (dict-keys json-str) -> VList of VStr keys.
   Returns the keys of the JSON object at [json-str] (insertion order
   preserved by Yojson's `Assoc). Empty list if input isn't an object. *)
let prim_dict_keys args =
  match args with
  | [ VStr json ] ->
      (try
        (match Yojson.Safe.from_string json with
         | `Assoc kvs ->
```

```
            VList (List.map (fun (k, _) -> VStr k) kvs)
        | _ -> VList [])
      with _ -> VList [])
  | _ -> raise (Interp_error (User_error "dict-keys needs 1 string"))

(* (render-html html-str) -> VStr tagged-for-raw-rendering.
   Marks the returned VStr as raw HTML so http.ml renders it
   unescaped (instead of pretty-printing the value inside <pre>).
   The tag is a private prefix sentinel; http.ml checks for it. *)
let html_sentinel = "__html__:"
let redirect_sentinel = "__redirect__:"

let prim_render_html args =
  match args with
  | [ VStr s ] -> VStr (html_sentinel ^ s)
  | _ -> raise (Interp_error (User_error "render-html needs 1 string"))

(* Expose the sentinel so http.ml can import it (single source of
   truth for the marker). *)
let is_raw_html (v : value) : string option =
  match v with
  | VStr s when String.length s >= String.length html_sentinel
             && String.sub s 0 (String.length html_sentinel) =
    html_sentinel ->
      Some (String.sub s (String.length html_sentinel)
             (String.length s - String.length html_sentinel))
  | _ -> None

let redirect_value (path : string) : value =
  VStr (redirect_sentinel ^ path)

let is_redirect (v : value) : string option =
  match v with
  | VStr s when String.length s >= String.length redirect_sentinel
             && String.sub s 0 (String.length redirect_sentinel)
                = redirect_sentinel ->
      Some (String.sub s (String.length redirect_sentinel)
             (String.length s - String.length redirect_sentinel))
  | _ -> None

(* (string-eq a b) -> VBool. String equality for request-method checks etc.
    *)
let prim_string_eq args =
  match args with
  | [ VStr a; VStr b ] -> VBool (String.equal a b)
  | _ -> raise (Interp_error (User_error "string-eq needs 2 strings"))

(* (string-trim s) -> VStr. Trim leading/trailing whitespace. *)
let prim_string_trim args =
  match args with
  | [ VStr s ] -> VStr (String.trim s)
  | _ -> raise (Interp_error (User_error "string-trim needs 1 string"))

(* JSON <-> Lisp values for outline trees and generic data.
   Objects become VStr of JSON object (use dict-get); arrays -> VList. *)
let rec json_to_value (j : Yojson.Safe.t) : value =
  match j with
  | `Null -> VNil
  | `Bool b -> VBool b
  | `Int n -> VInt (Int64.of_int n)
  | `Intlit s -> (try VInt (Int64.of_string s) with _ -> VStr s)
  | `Float f -> VStr (string_of_float f)
  | `String s -> VStr s
  | `List xs -> VList (List.map json_to_value xs)
  | `Assoc _ as o -> VStr (Yojson.Safe.to_string o)

let rec value_to_json (v : value) : Yojson.Safe.t =
  match v with
  | VNil -> `Null
  | VBool b -> `Bool b
  | VInt n -> `Intlit (Int64.to_string n)
  | VStr s ->
      (* Nested JSON object/array strings stay structured when emitted. *)
```

```
      let len = String.length s in
      if len >= 2
         && ((s.[0] = '{' && s.[len - 1] = '}')
             || (s.[0] = '[' && s.[len - 1] = ']'))
      then (try Yojson.Safe.from_string s with _ -> `String s)
      else `String s
  | VList xs -> `List (List.map value_to_json xs)
  | VPair (a, b) ->
      let rec unwind acc = function
        | VPair (x, y) -> unwind (x :: acc) y
        | VNil -> List.rev acc
        | VList ys -> List.rev_append acc ys
        | tail -> List.rev (tail :: acc)
      in
      `List (List.map value_to_json (unwind [] (VPair (a, b))))
  | other -> `String (Pricklypear_lisp.Reader.pp_value other)

let prim_json_parse args =
  match args with
  | [ VStr s ] ->
      (try json_to_value (Yojson.Safe.from_string s)
       with _ -> raise (Interp_error (User_error "json-parse: invalid
    JSON")))
  | _ -> raise (Interp_error (User_error "json-parse needs 1 string"))

let prim_json_emit args =
  match args with
  | [ v ] -> VStr (Yojson.Safe.to_string (value_to_json v))
  | _ -> raise (Interp_error (User_error "json-emit needs 1 value"))

(* (html-escape s) -> VStr. Escape amp lt gt quote apos for HTML
    text/attrs. *)
let rec prim_html_escape args =
  match args with
  | [ VStr s ] ->
      let buf = Buffer.create (String.length s + 8) in
      String.iter
        (function
          | '&' -> Buffer.add_string buf "&amp;"
          | '<' -> Buffer.add_string buf "&lt;"
          | '>' -> Buffer.add_string buf "&gt;"
          | '"' -> Buffer.add_string buf "&quot;"
          | '\'' -> Buffer.add_string buf "&#39;"
          | c -> Buffer.add_char buf c)
        s;
      VStr (Buffer.contents buf)
  | [ v ] ->
      (* coerce then escape *)
      let s =
        match v with
        | VStr s -> s
        | VInt n -> Int64.to_string n
        | VBool true -> "true"
        | VBool false -> "false"
        | VNil -> ""
        | _ -> Pricklypear_lisp.Reader.pp_value v
      in
      prim_html_escape [ VStr s ]
  | _ -> raise (Interp_error (User_error "html-escape needs 1 value"))

(* (hx-get url) -> VStr tagged attribute helper (lightweight; authors may
   also embed Hx attrs inside render-html). *)
let prim_hx_get args =
  match args with
  | [ VStr url ] -> VStr ("hx-get:" ^ url)
  | _ -> raise (Interp_error (User_error "hx-get needs a string url"))

let prim_hx_post args =
  match args with
  | [ VStr url ] -> VStr ("hx-post:" ^ url)
  | _ -> raise (Interp_error (User_error "hx-post needs a string url"))

(* (count list) -> int. Demonstrates a prim that walks the evaluator's
```

```
   own value type. *)
let prim_count args =
  match args with
  | [ VList items ] -> VInt (Int64.of_int (List.length items))
  | [ VNil ] -> VInt 0L
  | _ -> raise (Interp_error (User_error "count needs a list"))

let prim_date_today (_args : value list) : value =
  let tm = Unix.localtime (Unix.time ()) in
  VStr
    (Printf.sprintf "%04d-%02d-%02d"
       (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday)

(* (parse-int s|n) -> VInt. Soft-fail to 0 on bad string. *)
let prim_parse_int (args : value list) : value =
  match args with
  | [ VInt n ] -> VInt n
  | [ VStr s ] ->
      (try VInt (Int64.of_int (int_of_string (String.trim s)))
       with _ -> VInt 0L)
  | _ -> raise (Interp_error (User_error "parse-int needs string or int"))

(* (date-weekday y m d) -> VInt 0=Sunday .. 6=Saturday (local). *)
let prim_date_weekday (args : value list) : value =
  match args with
  | [ VInt y; VInt m; VInt d ] ->
      let y = Int64.to_int y and m = Int64.to_int m and d = Int64.to_int d
      in
      let tm =
        { Unix.tm_sec = 0
        ; tm_min = 0
        ; tm_hour = 12
        ; tm_mday = d
        ; tm_mon = m - 1
        ; tm_year = y - 1900
        ; tm_wday = 0
        ; tm_yday = 0
        ; tm_isdst = false
        }
      in
      let _, tm' = Unix.mktime tm in
      VInt (Int64.of_int tm'.Unix.tm_wday)
  | _ ->
      raise (Interp_error (User_error "date-weekday needs year month day
     ints"))

(* (date-format y m d) -> VStr "YYYY-MM-DD". *)
let prim_date_format (args : value list) : value =
  match args with
  | [ VInt y; VInt m; VInt d ] ->
      VStr
        (Printf.sprintf "%04d-%02d-%02d"
           (Int64.to_int y) (Int64.to_int m) (Int64.to_int d))
  | _ -> raise (Interp_error (User_error "date-format needs year month
     day"))

(** (date-add-days "YYYY-MM-DD" n) | (date-add-days n) with today.
    Local calendar day arithmetic via Unix.mktime (flashcards SRS). *)
let prim_date_add_days (args : value list) : value =
  let add_from_ymd y m d n =
    let tm =
      { Unix.tm_sec = 0
      ; tm_min = 0
      ; tm_hour = 12
      ; tm_mday = d + n
      ; tm_mon = m - 1
      ; tm_year = y - 1900
      ; tm_wday = 0
      ; tm_yday = 0
      ; tm_isdst = false
      }
    in
    let _, tm' = Unix.mktime tm in
```

```
    VStr
      (Printf.sprintf "%04d-%02d-%02d"
         (tm'.Unix.tm_year + 1900)
         (tm'.Unix.tm_mon + 1)
         tm'.Unix.tm_mday)
  in
  let parse_ymd s =
    let s = String.trim s in
    if String.length s < 10 then None
    else
      try
        let y = int_of_string (String.sub s 0 4) in
        let m = int_of_string (String.sub s 5 2) in
        let d = int_of_string (String.sub s 8 2) in
        Some (y, m, d)
      with _ -> None
  in
  match args with
  | [ VStr day; VInt n ] ->
      (match parse_ymd day with
       | Some (y, m, d) -> add_from_ymd y m d (Int64.to_int n)
       | None ->
           raise (Interp_error (User_error "date-add-days: bad date
   string")))
  | [ VInt n ] ->
      let tm = Unix.localtime (Unix.time ()) in
      add_from_ymd
        (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday
        (Int64.to_int n)
  | [ VStr day; VStr n ] ->
      (match parse_ymd day, int_of_string_opt (String.trim n) with
       | Some (y, m, d), Some n -> add_from_ymd y m d n
       | _ ->
           raise (Interp_error (User_error "date-add-days: bad args")))
  | _ ->
      raise
        (Interp_error
           (User_error "date-add-days needs (day n) or (n) for today+n"))

(** (date-diff-days a b) -> VInt days (b - a), local calendar.
    Args are YYYY-MM-DD strings. *)
let prim_date_diff_days (args : value list) : value =
  let parse s =
    let s = String.trim s in
    if String.length s < 10 then None
    else
      try
        Some
          ( int_of_string (String.sub s 0 4)
          , int_of_string (String.sub s 5 2)
          , int_of_string (String.sub s 8 2) )
      with _ -> None
  in
  let epoch_days y m d =
    let tm =
      { Unix.tm_sec = 0; tm_min = 0; tm_hour = 12; tm_mday = d
      ; tm_mon = m - 1; tm_year = y - 1900; tm_wday = 0; tm_yday = 0
      ; tm_isdst = false }
    in
    let t, _ = Unix.mktime tm in
    t /. 86400.
  in
  match args with
  | [ VStr a; VStr b ] ->
      (match parse a, parse b with
       | Some (y1, m1, d1), Some (y2, m2, d2) ->
           let n =
             int_of_float
               (Float.round (epoch_days y2 m2 d2 -. epoch_days y1 m1 d1))
           in
           VInt (Int64.of_int n)
       | _ ->
           raise (Interp_error (User_error "date-diff-days: bad date")))
```

```
  | _ ->
      raise
        (Interp_error (User_error "date-diff-days needs two YYYY-MM-DD
     strings"))

(** FSRS-4.5 schedule step (open-spaced-repetition defaults).

    (fsrs-schedule stability difficulty elapsed_days rating)
      stability/difficulty: float or int; use 0 0 for a new card
      elapsed_days: days since last review (0 for first)
      rating: 1..4 or again|hard|good|easy

    Returns (list S' D' interval_days) where interval_days is int >= 0
    (0 means relearn today). request_retention fixed at 0.9. *)
let fsrs_w : float array =
  [| 0.40255; 1.18385; 3.173; 15.69105; 7.1949; 0.5345; 1.4604; 0.0046;
     1.54575; 0.1192; 1.01925; 1.9395; 0.11; 0.29605; 2.2698; 0.2315;
     2.9898; 0.51655; 0.6621 |]

let fsrs_clamp_d (d : float) : float = max 1. (min 10. d)

let fsrs_init_stability (grade : int) : float =
  (* w[0..3] for Again..Easy *)
  max 0.1 fsrs_w.(grade - 1)

let fsrs_init_difficulty (grade : int) : float =
  fsrs_clamp_d (fsrs_w.(4) -. exp (fsrs_w.(5) *. float_of_int (grade - 1))
     +. 1.)

let fsrs_next_difficulty (d : float) (grade : int) : float =
  let delta = -. fsrs_w.(6) *. float_of_int (grade - 3) in
  let d' = d +. delta *. (10. -. d) /. 9. in
  let d0_good = fsrs_init_difficulty 3 in
  fsrs_clamp_d (fsrs_w.(7) *. d0_good +. (1. -. fsrs_w.(7)) *. d')

let fsrs_retrievability ~(s : float) ~(elapsed : float) : float =
  if s <= 0. then 0.
  else (1. +. elapsed /. (9. *. s)) ** (-1.)

let fsrs_next_stability_success ~(s : float) ~(d : float) ~(r : float)
    ~(grade : int) : float =
  let hard_pen = if grade = 2 then fsrs_w.(15) else 1. in
  let easy_bon = if grade = 4 then fsrs_w.(16) else 1. in
  let s' =
    s
    *. (exp fsrs_w.(8)
        *. (11. -. d)
        *. (s ** (-. fsrs_w.(9)))
        *. (exp (fsrs_w.(10) *. (1. -. r)) -. 1.)
        *. hard_pen
        *. easy_bon
        +. 1.)
  in
  max s' s  (* stability should not decrease on success in v1 *)

let fsrs_next_stability_fail ~(s : float) ~(d : float) ~(r : float) : float
    =
  let s' =
    fsrs_w.(11)
    *. (d ** (-. fsrs_w.(12)))
    *. (((s +. 1.) ** fsrs_w.(13)) -. 1.)
    *. exp (fsrs_w.(14) *. (1. -. r))
  in
  min s (max 0.1 s')

let fsrs_interval_days ~(s : float) : int =
  (* For R=0.9, t ? S (FSRS-4 power formula). *)
  max 0 (int_of_float (Float.round (max 0. s)))

let fsrs_parse_grade = function
  | VInt n ->
      let g = Int64.to_int n in
      if g < 1 || g > 4 then None else Some g
```

```
  | VStr s ->
      (match String.lowercase_ascii (String.trim s) with
       | "again" | "1" -> Some 1
       | "hard" | "2" -> Some 2
       | "good" | "3" -> Some 3
       | "easy" | "4" -> Some 4
       | _ -> None)
  | _ -> None

let float_arg v =
  match num_of_value v with
  | NInt n -> Int64.to_float n
  | NFloat f -> f

let prim_fsrs_schedule (args : value list) : value =
  match args with
  | [ s_v; d_v; elapsed_v; grade_v ] ->
      (match fsrs_parse_grade grade_v with
       | None ->
           raise
             (Interp_error
                (User_error
                   "fsrs-schedule: rating must be 1..4 or
    again|hard|good|easy"))
       | Some grade ->
           let s0 = float_arg s_v in
           let d0 = float_arg d_v in
           let elapsed = max 0. (float_arg elapsed_v) in
           let is_new = s0 <= 0. || d0 <= 0. in
           let s, d =
             if is_new then
               (fsrs_init_stability grade, fsrs_init_difficulty grade)
             else
               let r = fsrs_retrievability ~s:s0 ~elapsed in
               let d' = fsrs_next_difficulty d0 grade in
               let s' =
                 if grade = 1 then
                   fsrs_next_stability_fail ~s:s0 ~d:d0 ~r
                 else
                   fsrs_next_stability_success ~s:s0 ~d:d0 ~r ~grade
               in
               (s', d')
           in
           let interval =
             if grade = 1 then 0
             else if is_new && grade = 2 then 1
             else if is_new && grade = 3 then max 1 (fsrs_interval_days ~s)
             else if is_new && grade = 4 then max 2 (fsrs_interval_days ~s)
             else max 1 (fsrs_interval_days ~s)
           in
           VList [ VFloat s; VFloat d; VInt (Int64.of_int interval) ])
  | _ ->
      raise
        (Interp_error
           (User_error
              "fsrs-schedule needs stability difficulty elapsed rating"))
```

50 image/lib/repl_endpoint.ml

```
(** Pricklypear Image -- Internal REPL endpoint.

    Per spec [image.repl-endpoint]: the image does NOT run a full SSH server.
    Instead it exposes an internal TCP socket on [repl_port] that speaks a
    minimal line protocol. The supervisor connects here and forwards the
    user's PTY stream.

    Protocol (minimal, framed):
      Connect: supervisor opens TCP to image:repl_port
      Auth: none (internal network; supervisor already authed the user)
      Exchange: lines of UTF-8. User input flows in; REPL output flows out.
      The session.ml tracks this connection as a session; on disconnect
      session ends.

    For the minimal image: accept ONE connection, run a basic read-eval-print
    loop on it, drop the connection on EOF. The supervisor will provide
    line buffering and framing later. *)
open Lwt.Syntax
open Lwt.Infix

let default_port = 7878

let pp_error (e : Pricklypear_lisp.Ast.interp_error) : string =
  let open Pricklypear_lisp.Ast in
  match e with
  | Unbound_var s -> Printf.sprintf "unbound variable: %s" s
  | Not_a_function v -> Printf.sprintf "not a function: %s"
                          (Pricklypear_lisp.Reader.pp_value v)
  | Arity_mismatch (n, e_doc, g) ->
      Printf.sprintf "arity mismatch in %s: expected %d, got %d" n e_doc g
  | Wrong_arg_count (n, e_doc, g) ->
      Printf.sprintf "wrong arg count in %s: expected %d, got %d" n e_doc g
  | Type_mismatch (s, _) -> Printf.sprintf "type mismatch: %s" s
  | Budget_exceeded -> "budget exceeded"
  | Prim_error (n, m) -> Printf.sprintf "primitive %s: %s" n m
  | Parse_error (msg, l, c) ->
      Printf.sprintf "parse error: %s (line %d, col %d)" msg l c
  | User_error m -> Printf.sprintf "user error: %s" m

let prompt (s : Session.t) : string =
  Printf.sprintf "%s> " (Session.branch_name s)

(* Run a single REPL turn on the given connected channel pair.
   Returns false if the connection is closed (EOF), true to continue. *)
let repl_turn
    ~(session : Session.t)
    ~(ic : Lwt_io.input_channel)
    ~(oc : Lwt_io.output_channel) : bool Lwt.t =
  let* () = Lwt_io.write oc (prompt session) in
  let* () = Lwt_io.flush oc in
  let* line = Lwt_io.read_line_opt ic in
  match line with
  | None -> Lwt.return_false
  | Some "" -> Lwt.return_true  (* empty line: just re-prompt *)
  | Some input ->
      Lwt.catch
        (fun () ->
          match input with
          | ":quit" | ":q" -> Lwt.return_false
          | _ ->
              let* result = Session.eval_one session input in
              let output = Pricklypear_lisp.Reader.pp_value result in
              let* () = Lwt_io.write oc (output ^ "\n") in
              let* () = Lwt_io.flush oc in
              Lwt.return_true)
        (fun exn ->
          let msg =
            match exn with
            | Pricklypear_lisp.Ast.Interp_error e -> pp_error e
```

```
            | _ -> Printexc.to_string exn
          in
          let* () = Lwt_io.write oc ("error: " ^ msg ^ "\n") in
          let* () = Lwt_io.flush oc in
          Lwt.return_true)

(* Accept connections on [port]. For the minimal image we handle ONE
   at a time (blocking accept). TODO: concurrent sessions.

   HARDENING: when the supervisor closes the proxy fd on `:quit` (or
   on its own exit), Lwt_io.close raises EBADF because (a) the fd
   was already closed by the peer, OR (b) `ic` and `oc` share the
   underlying fd so closing one invalidates the other. We wrap all
   post-disconnect cleanup in Lwt.catch to swallow these expected
   errors so the image survives REPL disconnect -- HTTP server keeps
   serving, and boot-on-connect images can accept a new REPL session. *)
let serve ~(session : Session.t) ?(port = default_port) () : unit Lwt.t =
  let addr = Lwt_unix.ADDR_INET (Unix.inet_addr_loopback, port) in
  let sock = Lwt_unix.socket Lwt_unix.PF_INET Lwt_unix.SOCK_STREAM 0 in
  Lwt_unix.setsockopt sock Lwt_unix.SO_REUSEADDR true;
  let* () = Lwt_unix.bind sock addr in
  Lwt_unix.listen sock 5;
  Printf.eprintf "[repl] listening on 127.0.0.1:%d\n" port;
  flush stderr;
  let rec loop () =
    let* (fd, _peer) = Lwt_unix.accept sock in
    let ic = Lwt_io.of_fd ~mode:Lwt_io.Input fd in
    let oc = Lwt_io.of_fd ~mode:Lwt_io.Output fd in
    let* () = Lwt_io.write oc "Pricklypear REPL. Type :quit to exit.\n" in
    let* () = Lwt_io.flush oc in
    let rec turn_loop () =
      let* cont = repl_turn ~session ~ic ~oc in
      if cont then turn_loop () else Lwt.return_unit
    in
    (* Run the turn loop; on any exception (expected: EBADF on peer-close,
       or actually any user-lob exn) just log and proceed to cleanup. *)
    let* () = Lwt.catch turn_loop
      (fun exn ->
        (* Don't log normal peer-disconnect EBADF as a scary error. *)
        (match exn with
         | Unix.Unix_error (Unix.EBADF, _, _) ->
             Printf.eprintf "[repl] peer disconnected\n"
         | _ ->
             Printf.eprintf "[repl] connection error: %s\n"
               (Printexc.to_string exn));
        flush stderr;
        Lwt.return_unit)
    in
    (* Cleanup: close ic and oc. These share the underlying fd, so the
       second close may raise EBADF. Swallow it. *)
    let* () = Lwt.catch (fun () -> Lwt_io.close ic)
      (fun _ -> Lwt.return_unit) in
    let* () = Lwt.catch (fun () -> Lwt_io.close oc)
      (fun _ -> Lwt.return_unit) in
    let* () = Lwt.catch (fun () -> Lwt_unix.close fd)
      (fun _ -> Lwt.return_unit) in
    loop ()
  in
  loop ()
```

51 image/lib/session.ml

```
(** Pricklypear Image -- REPL session.

    A per-connection session holds the env (function defs loaded for the
    session's branch, plus any (define)'d locals), the manifest (identity
    claim asserted by supervisor), and the store handle.

    Per spec [image.repl-experience]:
      - (define ...) persists to the store: survives image reboot.
      - REPL-local let bindings are session-local: NOT persisted.
      - Switching branch (checkout) keeps REPL locals but changes
     visibility. *)
open Lwt.Syntax
include Session_types

let load_defs (t : t) : unit Lwt.t =
  let* defs = Store.list_function_names_for_load t.store in
  let* _ =
    Lwt_list.iter_s
      (fun (name, body, _author) ->
        (* Always (re)define -- parents first, children last ? override. *)
        Lwt.catch
          (fun () ->
             (* Eval the stored AST directly -- NEVER pp_expr round-trip.
                Complex bodies (ui/home, notes) break under print/parse. *)
             let* _ =
               Pricklypear_lisp.Eval.eval
                 ~budget:(Pricklypear_lisp.Eval.make_budget ())
                 ~prims:t.prims t.env
                 (Pricklypear_lisp.Ast.Define (name, body))
             in
             Lwt.return_unit)
          (fun exn ->
             Printf.eprintf "[session] load def %s failed: %s\n%!"
               name (Printexc.to_string exn);
             Lwt.return_unit))
      defs
  in
  (* Also load view_bindings into the in-memory routes table. The
     recursive-CTE returns ancestors first, so a route defined in both
     the child and parent: child's wins (Hashtbl.replace on the
     first-seen only -- skip further matches). *)
  let* routes = Store.list_routes t.store in
  let seen_r = Hashtbl.create 16 in
  List.iter
    (fun (route, handler, visibility) ->
      if not (Hashtbl.mem seen_r route) then begin
        Hashtbl.replace seen_r route ();
        Hashtbl.replace t.routes route (handler, visibility)
      end)
    routes;
  Lwt.return_unit

let load_defs_async (t : t) : unit =
  Lwt.async (fun () -> load_defs t)

(** Clear env + routes and reload from the store for the current branch.
    Re-binds kernel prims from [t.kernel_catalog] first so checkout does
    not lose +/define/load-library/etc. *)
let rehydrate (t : t) : unit Lwt.t =
  Hashtbl.clear t.env.bindings;
  Hashtbl.clear t.routes;
  List.iter
    (fun (name, _) ->
      Pricklypear_lisp.Env.define t.env name
        (Pricklypear_lisp.Ast.VPrim name))
    t.kernel_catalog;
  Pricklypear_lisp.Env.define t.env "load-string"
    (Pricklypear_lisp.Ast.VPrim "load-string");
  Pricklypear_lisp.Env.define t.env "load-library"
    (Pricklypear_lisp.Ast.VPrim "load-library");
  load_defs t
```

```
71
72  (** Switch store + manifest to [branch_id], then rehydrate env. *)
73  let checkout_to (t : t) ~(branch_id : string) ~(branch_ref : string)
74    : unit Lwt.t =
75    let* () = Store.checkout_branch t.store ~new_branch_id:branch_id in
76    t.manifest <-
77      { t.manifest with
78        Pricklypear_common.Manifest.branch_id = branch_id;
79        branch_name = branch_ref };
80    (* Drop shell caches -- ui/nav and ui/css are branch-visible. Next
81       refresh_ui_shell (request launch / load) repopulates from new env. *)
82    t.ui_nav_html <- None;
83    t.ui_css <- None;
84    rehydrate t
85
86  let split_branch_ref (s : string) : string * string =
87    match String.split_on_char '/' s with
88    | [ o; n ] -> o, n
89    | [ n ] -> "system", n  (* bare name = system/<name> *)
90    | _ -> "system", s
91
92  let register_session_control_prims (t : t) (tbl : Primitives.t) : unit =
93    (* (fork 'parent 'child) -> VStr new-branch-id | VStr "no such parent" *)
94    Primitives.register tbl "fork"
95      (fun args ->
96        if t.manifest.username = "anonymous" then
97          Lwt.return (Pricklypear_lisp.Ast.VStr "fork: denied for anonymous")
98        else if t.agent_branch_lock then
99          Lwt.return
100           (Pricklypear_lisp.Ast.VStr
101              "fork blocked: agent is locked to the current branch")
102       else
103       match args with
104       | [ Pricklypear_lisp.Ast.VStr parent_name;
105           Pricklypear_lisp.Ast.VStr child_name ] ->
106           let parent_owner, parent_short = split_branch_ref parent_name in
107           let child_owner, child_short = split_branch_ref child_name in
108           let* parent_id =
109             Store.lookup_branch_id_by_name
110               t.store ~owner:parent_owner ~name:parent_short
111           in
112           (match parent_id with
113            | None ->
114                Lwt.return (Pricklypear_lisp.Ast.VStr
115                              (Printf.sprintf "no such parent: %s"
    parent_name))
116          | Some pid ->
117              let* new_id =
118                Store.fork_branch t.store ~parent_id:pid
119                  ~owner:child_owner ~child_name:child_short
120              in
121              (match new_id with
122               | Some id ->
123                   Lwt.return (Pricklypear_lisp.Ast.VStr id)
124               | None ->
125                   Lwt.return (Pricklypear_lisp.Ast.VStr "fork failed")))
126       | _ ->
127           Lwt.return (Pricklypear_lisp.Ast.VStr "fork: bad args (need 2
    strings)"));
128
129   (* (merge 'source 'target) -> VStr "merged N rows" | VStr "denied".
130      POLICY (per spec [branching.merge]): merging into `system/*`
131      requires the session username to be "admin"; merges into the
132      caller's own `owner/*` branches are free. *)
133   Primitives.register tbl "merge"
134     (fun args ->
135       if t.manifest.username = "anonymous" then
136         Lwt.return (Pricklypear_lisp.Ast.VStr "merge: denied for anonymous")
137       else if t.agent_branch_lock then
138         Lwt.return
139           (Pricklypear_lisp.Ast.VStr
140              "merge blocked: agent is locked to the current branch")
141       else
```

```
    match args with
    | [ Pricklypear_lisp.Ast.VStr source_ref;
        Pricklypear_lisp.Ast.VStr target_ref ] ->
        let src_owner, src_short = split_branch_ref source_ref in
        let tgt_owner, tgt_short = split_branch_ref target_ref in
        let authorized =
          (tgt_owner <> "system")
          || t.is_admin
          || (t.manifest.username = "admin")
        in
        if not authorized then
          Lwt.return (Pricklypear_lisp.Ast.VStr
                        "denied: admin-only merge to system branch")
        else
          let* src_id =
            Store.lookup_branch_id_by_name
              t.store ~owner:src_owner ~name:src_short
          in
          let* tgt_id =
            Store.lookup_branch_id_by_name
              t.store ~owner:tgt_owner ~name:tgt_short
          in
          (match src_id, tgt_id with
           | Some s, Some tg ->
               let* n = Store.merge_branch t.store
                          ~source_id:s ~target_id:tg in
               (match n with
                | Some count ->
                    Lwt.return (Pricklypear_lisp.Ast.VStr
                                  (Printf.sprintf "merged %d rows" count))
                | None ->
                    Lwt.return (Pricklypear_lisp.Ast.VStr "merge failed"))
           | _ ->
               Lwt.return (Pricklypear_lisp.Ast.VStr
                             "source or target branch not found"))
    | _ ->
        Lwt.return (Pricklypear_lisp.Ast.VStr "merge: bad args (need 2
   strings)"));

(* (route "/path" handler [visibility]) -> VStr "/path".
   Optional third arg 'public | "public" | 'private | "private".
   Handler-only re-register keeps prior visibility (store ON CONFLICT).
   Anonymous sessions may not register routes. *)
let parse_vis = function
  | Pricklypear_lisp.Ast.VStr s ->
      (match String.lowercase_ascii (String.trim s) with
       | "public" -> Some "public"
       | "private" -> Some "private"
       | _ -> None)
  | _ -> None
in
Primitives.register tbl "route"
  (fun args ->
    if t.manifest.username = "anonymous" then
      Lwt.return (Pricklypear_lisp.Ast.VStr "route: denied for anonymous")
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr route_path;
          Pricklypear_lisp.Ast.VStr handler ] ->
          let vis =
            match Hashtbl.find_opt t.routes route_path with
            | Some (_, v) -> v
            | None -> "private"
          in
          (* handler-only: pass None so DB keeps visibility on UPDATE;
             memory keeps prior or private for new. *)
          Hashtbl.replace t.routes route_path (handler, vis);
          let* _ =
            Store.register_route t.store ~route:route_path ~handler ()
          in
          Lwt.return (Pricklypear_lisp.Ast.VStr route_path)
      | [ Pricklypear_lisp.Ast.VStr route_path;
          Pricklypear_lisp.Ast.VStr handler;
```

```
            vis_v ] ->
            (match parse_vis vis_v with
             | None ->
                 Lwt.return
                   (Pricklypear_lisp.Ast.VStr
                      "route: visibility must be public or private")
             | Some vis ->
                 Hashtbl.replace t.routes route_path (handler, vis);
                 let* _ =
                   Store.register_route t.store ~route:route_path ~handler
                     ~visibility:vis ()
                 in
                 Lwt.return (Pricklypear_lisp.Ast.VStr route_path))
        | _ ->
            Lwt.return (Pricklypear_lisp.Ast.VStr "route: bad args"));

  (* (public-route path handler) sugar *)
  Primitives.register tbl "public-route"
    (fun args ->
      if t.manifest.username = "anonymous" then
        Lwt.return (Pricklypear_lisp.Ast.VStr "public-route: denied for
     anonymous")
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr route_path;
            Pricklypear_lisp.Ast.VStr handler ] ->
            Hashtbl.replace t.routes route_path (handler, "public");
            let* _ =
              Store.register_route t.store ~route:route_path ~handler
                ~visibility:"public" ()
            in
            Lwt.return (Pricklypear_lisp.Ast.VStr route_path)
        | _ ->
            Lwt.return (Pricklypear_lisp.Ast.VStr "public-route: bad
     args"));

  (* (route-visibility path) -> "public" | "private" | () *)
  Primitives.register tbl "route-visibility"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr route_path ] ->
          (match Hashtbl.find_opt t.routes route_path with
           | Some (_, vis) ->
               Lwt.return (Pricklypear_lisp.Ast.VStr vis)
           | None -> Lwt.return Pricklypear_lisp.Ast.VNil)
      | _ -> Lwt.return Pricklypear_lisp.Ast.VNil);

  (* (checkout 'branch-ref) -> VStr "checked out" | "no such branch".
     Mutates t.store.branch_id + t.manifest, then re-hydrates the env
     from the new branch's function_defs (sync -- so the next form sees
     the right bindings). Blocked while agent_branch_lock is set. *)
  Primitives.register tbl "checkout"
    (fun args ->
      if t.manifest.username = "anonymous" then
        Lwt.return (Pricklypear_lisp.Ast.VStr "checkout: denied for
     anonymous")
      else if t.agent_branch_lock then
        Lwt.return
          (Pricklypear_lisp.Ast.VStr
             "checkout blocked: agent is locked to the current branch")
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr branch_ref ] ->
            let owner, name = split_branch_ref branch_ref in
            let* found =
              Store.lookup_branch_id_by_name t.store ~owner ~name
            in
            (match found with
             | None -> Lwt.return (Pricklypear_lisp.Ast.VStr "no such
     branch")
             | Some new_id ->
                 let* () = checkout_to t ~branch_id:new_id ~branch_ref in
                 Lwt.return (Pricklypear_lisp.Ast.VStr "checked out"))
```

```
        | _ ->
            Lwt.return (Pricklypear_lisp.Ast.VStr "checkout: bad args"));

  (* Private prim used by the with-transaction special form in eval.ml.
     NOT USER-VISIBLE: it's looked up under name __with_transaction__ by
     the special form handler, never `(with-transaction-call ...)` from
     Lisp.
     Also bound into the env as a VPrim name by the auto-bind loop, but
     a `(with-transaction-call ...)` call would just route through here.
     The prim receives one arg: a VLambda value. It wraps Eval.apply
     inside Store.with_dynamic_transaction, so prims called from the
     body of the lambda see the in-transaction pgx conn via
     Store.current_conn. Lands it29, option A from BORGE-1783438027.

     Returns the lambda body's result. Nested transactions raise a clean
     Interp_error (see with_dynamic_transaction). *)
  Primitives.register tbl "__with_transaction__"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VLambda (params, body, clo_env) as lam ] ->
          (* Enforce zero-arg lambda for clarity: with-transaction bodies
             shouldn't take args. *)
          let _ = params in
          let _ = clo_env in
          Store.with_dynamic_transaction t.store
            (fun () ->
               let budget = Pricklypear_lisp.Eval.make_budget () in
               Pricklypear_lisp.Eval.apply
                 ~budget ~prims:t.prims lam [])
      | _ ->
          Lwt.return (Pricklypear_lisp.Ast.VStr
                        "__with_transaction__: bad args (need a lambda)"))

(** If model returns a bare expression body, wrap as (lambda (x) ...). *)

(* ---- AI / agent tools (Agent_tools) -- re-export for stable Session.*
     API ---- *)
let define_binding = Agent_tools.define_binding
let install_ai_function = Agent_tools.install_ai_function
let function_source = Agent_tools.function_source
let package_binding_names = Agent_tools.package_binding_names
let export_package = Agent_tools.export_package
let propose_function = Agent_tools.propose_function
let default_agent_seed = Agent_tools.default_agent_seed
let agent_session_new = Agent_tools.agent_session_new
let agent_say = Agent_tools.agent_say
let agent_history = Agent_tools.agent_history
let agent_sessions = Agent_tools.agent_sessions
let agent_session_delete = Agent_tools.agent_session_delete
let agent = Agent_tools.agent
let agent_task = Agent_tools.agent_task
let llm_rewrite = Agent_tools.llm_rewrite
let llm_inner = Agent_tools.llm_inner
let llm_outer = Agent_tools.llm_outer
let llm_seed = Agent_tools.llm_seed
let pp_interp_exn = Agent_tools.pp_interp_exn

let register_crud_prims (t : t) (tbl : Primitives.t) : unit =
  (* (deftype 'name 'field1 'field2 ...) -> VStr "defined type <name>".
     All args are quoted symbols (which evaluate to VStr). Persists
     the declaration into type_defs on the session's branch. *)
  let deny_anon name =
    Lwt.return
      (Pricklypear_lisp.Ast.VStr (name ^ ": denied for anonymous"))
  in
  Primitives.register tbl "deftype"
    (fun args ->
      if t.manifest.username = "anonymous" then deny_anon "deftype"
      else Crud.deftype t.store ~args ~author:t.manifest.username);

  (* (get 'type id) -> VStr fields-json | VNil *)
  Primitives.register tbl "get"
    (fun args ->
```

```
      match args with
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VStr id ] ->
          Crud.get t.store ~type_name ~id
      | _ -> Lwt.return (Pricklypear_lisp.Ast.VStr "get: bad args (need
    type, id)"));

  (* (create 'type "{fields-json}") -> VStr new-id | VStr "create failed" *)
  Primitives.register tbl "create"
    (fun args ->
      if t.manifest.username = "anonymous" then deny_anon "create"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr type_name;
            Pricklypear_lisp.Ast.VStr fields_json ] ->
            Crud.create t.store
              ~type_name ~fields_json
              ~author:t.manifest.username
        | _ ->
            Lwt.return
              (Pricklypear_lisp.Ast.VStr "create: bad args (need type,
    fields)"));

  (* (update 'type id "{fields-json}") -> VStr id | VNil *)
  Primitives.register tbl "update"
    (fun args ->
      if t.manifest.username = "anonymous" then deny_anon "update"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr type_name;
            Pricklypear_lisp.Ast.VStr id;
            Pricklypear_lisp.Ast.VStr fields_json ] ->
            Crud.update t.store ~type_name ~id ~fields_json
        | _ ->
            Lwt.return
              (Pricklypear_lisp.Ast.VStr
                 "update: bad args (need type, id, fields)"));

  (* (update-field 'type id "field" "value") -> VStr id | VNil.
      Ergonomic shorthand: composes get + dict-set + update in one prim.
      Lands it18 (2026-07-06); resolves the it14 "verbose round-trip
    syntax" TODO. *)
  Primitives.register tbl "update-field"
    (fun args ->
      if t.manifest.username = "anonymous" then deny_anon "update-field"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr type_name;
            Pricklypear_lisp.Ast.VStr id;
            Pricklypear_lisp.Ast.VStr field;
            Pricklypear_lisp.Ast.VStr value ] ->
            Crud.update_field t.store ~type_name ~id ~field ~value
        | _ ->
            Lwt.return
              (Pricklypear_lisp.Ast.VStr
                 "update-field: bad args (need type, id, field, value)"));

  (* (delete 'type id) -> VStr id | VNil *)
  Primitives.register tbl "delete"
    (fun args ->
      if t.manifest.username = "anonymous" then deny_anon "delete"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr type_name;
            Pricklypear_lisp.Ast.VStr id ] ->
            Crud.delete t.store ~type_name ~id
        | _ ->
            Lwt.return
              (Pricklypear_lisp.Ast.VStr "delete: bad args (need type,
    id)"));

  (* (select-for-update 'type id) -> VStr fields-json | VNil.
     Acquires an explicit SELECT ... FOR UPDATE row lock on the
```

```
423     named row, for the duration of the surrounding transaction.
424     Per spec [storage.transactions] (prim lands it17, 2026-07-06).
425     See Crud.select_for_update doc for transactional-scoping caveat:
426     outside a transaction, this is a hinted-get with no real lock. *)
427   Primitives.register tbl "select-for-update"
428     (fun args ->
429       match args with
430       | [ Pricklypear_lisp.Ast.VStr type_name;
431           Pricklypear_lisp.Ast.VStr id ] ->
432           Crud.select_for_update t.store ~type_name ~id
433       | _ -> Lwt.return (Pricklypear_lisp.Ast.VStr "select-for-update: bad
      args (need type, id)"));
434
435   (* (rows 'type [limit]) -> VList of (id . fields) pairs.
436     With no limit, defaults to 100. NOTE: prim name is `rows`, NOT
437     `list` -- the latter is the pure VList-constructor prim. Earlier
438     tentative naming collided and broke `(list 1 2 3)`; renamed to
439     avoid that. *)
440   Primitives.register tbl "rows"
441     (fun args ->
442       match args with
443       | [ Pricklypear_lisp.Ast.VStr type_name ] ->
444           Crud.list t.store ~type_name ()
445       | [ Pricklypear_lisp.Ast.VStr type_name;
446           Pricklypear_lisp.Ast.VInt n ] ->
447           Crud.list t.store ~type_name ~limit:(Int64.to_int n) ()
448       | _ -> Lwt.return (Pricklypear_lisp.Ast.VStr "rows: bad args (need
      type[, limit])"));
449
450   (* (find-row 'type "field" "value") -> fields JSON | () *)
451   Primitives.register tbl "find-row"
452     (fun args ->
453       match args with
454       | [ Pricklypear_lisp.Ast.VStr type_name;
455           Pricklypear_lisp.Ast.VStr field;
456           Pricklypear_lisp.Ast.VStr value ] ->
457           Crud.find_row t.store ~type_name ~field ~value
458       | _ ->
459           Lwt.return
460             (Pricklypear_lisp.Ast.VStr
461                "find-row: need type, field, value"));
462
463   (* (find-rows 'type "f" "v" ? [limit]) -> list of (id . fields) *)
464   Primitives.register tbl "find-rows"
465     (fun args ->
466       let rec parse_filters acc = function
467         | [] -> Ok (List.rev acc, 500)
468         | [ Pricklypear_lisp.Ast.VInt n ] ->
469             Ok (List.rev acc, Int64.to_int n)
470         | Pricklypear_lisp.Ast.VStr f :: Pricklypear_lisp.Ast.VStr v ::
      rest ->
471             parse_filters ((f, v) :: acc) rest
472         | _ -> Error "find-rows: bad filter args"
473       in
474       match args with
475       | Pricklypear_lisp.Ast.VStr type_name :: rest ->
476           (match parse_filters [] rest with
477            | Error e -> Lwt.return (Pricklypear_lisp.Ast.VStr e)
478            | Ok (filters, limit) ->
479                Crud.find_rows t.store ~type_name ~filters ~limit ())
480       | _ ->
481           Lwt.return
482             (Pricklypear_lisp.Ast.VStr
483                "find-rows: need type and field/value pairs"));
484
485   (* (find-text 'type "field" "needle" [limit]) *)
486   Primitives.register tbl "find-text"
487     (fun args ->
488       match args with
489       | [ Pricklypear_lisp.Ast.VStr type_name;
490           Pricklypear_lisp.Ast.VStr field;
491           Pricklypear_lisp.Ast.VStr needle ] ->
492           Crud.find_text t.store ~type_name ~field ~needle ()
```

```
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VStr field;
          Pricklypear_lisp.Ast.VStr needle;
          Pricklypear_lisp.Ast.VInt n ] ->
          Crud.find_text t.store ~type_name ~field ~needle
            ~limit:(Int64.to_int n) ()
      | _ ->
          Lwt.return
            (Pricklypear_lisp.Ast.VStr
               "find-text: need type, field, needle [, limit]"));

  (* (find-random 'type) -> (id . fields) | () *)
  Primitives.register tbl "find-random"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr type_name ] ->
          Crud.find_random t.store ~type_name
      | _ ->
          Lwt.return
            (Pricklypear_lisp.Ast.VStr "find-random: need type"));

  (* (list-branches) -> VList of "owner/name" strings *)
  Primitives.register tbl "list-branches"
    (fun _args ->
      let* bs = Store.list_branches t.store in
      let names =
        List.map
          (fun (b : Pricklypear_common.Branch.t) ->
            Pricklypear_lisp.Ast.VStr
              (b.Pricklypear_common.Branch.owner ^ "/" ^
               b.Pricklypear_common.Branch.name))
          bs
      in
      Lwt.return (Pricklypear_lisp.Ast.VList names));

  (* (current-branch) -> VStr *)
  Primitives.register_sync tbl "current-branch"
    (fun _ -> Pricklypear_lisp.Ast.VStr t.manifest.branch_name);

  (* Durable agent sessions (ai-authoring.agent-portal). *)
  Primitives.register tbl "agent-session-new"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr seed ] ->
          agent_session_new t ~seed ~tools:true ()
      | [ Pricklypear_lisp.Ast.VStr seed; Pricklypear_lisp.Ast.VBool tools
     ] ->
          agent_session_new t ~seed ~tools ()
      | _ ->
          Lwt.return
            (Pricklypear_lisp.Ast.VStr
               "agent-session-new: need seed-str [, tools-bool]"));

  Primitives.register tbl "agent-say"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr sid; Pricklypear_lisp.Ast.VStr prompt ]
     ->
          agent_say t ~session_id:sid ~prompt ()
      | [ Pricklypear_lisp.Ast.VStr sid; Pricklypear_lisp.Ast.VStr prompt;
          Pricklypear_lisp.Ast.VInt n ] ->
          agent_say t ~session_id:sid ~prompt ~max_steps:(Int64.to_int n) ()
      | _ ->
          Lwt.return
            (Pricklypear_lisp.Ast.VStr
               "agent-say: need session-id, prompt [, max-steps]"));

  Primitives.register tbl "agent-history"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr sid ] -> agent_history t ~session_id:sid
      | _ ->
          Lwt.return
```

```
564             (Pricklypear_lisp.Ast.VStr "agent-history: need session-id"));
565
566   Primitives.register tbl "agent-sessions"
567     (fun args ->
568       match args with
569       | [] -> agent_sessions t ()
570       | [ Pricklypear_lisp.Ast.VInt n ] ->
571           agent_sessions t ~limit:(Int64.to_int n) ()
572       | _ ->
573           Lwt.return
574             (Pricklypear_lisp.Ast.VStr "agent-sessions: optional limit
    int"));
575
576   Primitives.register tbl "agent-session-delete"
577     (fun args ->
578       match args with
579       | [ Pricklypear_lisp.Ast.VStr sid ] ->
580           agent_session_delete t ~session_id:sid
581       | _ ->
582           Lwt.return
583             (Pricklypear_lisp.Ast.VStr "agent-session-delete: need
    session-id"));
584
585   (* Temporary aliases: new session + one say. Prefer
       agent-session-new/say. *)
586   Primitives.register tbl "agent"
587     (fun args ->
588       match args with
589       | [ Pricklypear_lisp.Ast.VStr seed;
590           Pricklypear_lisp.Ast.VStr prompt ] ->
591           agent t ~seed ~prompt ()
592       | _ ->
593           Lwt.return
594             (Pricklypear_lisp.Ast.VStr
595                "agent: need 2 strings (seed, prompt)"));
596
597   Primitives.register tbl "agent-task"
598     (fun args ->
599       match args with
600       | [ Pricklypear_lisp.Ast.VStr prompt ] ->
601           agent_task t ~prompt ()
602       | [ Pricklypear_lisp.Ast.VStr prompt; Pricklypear_lisp.Ast.VInt n ] ->
603           agent_task t ~max_steps:(Int64.to_int n) ~prompt ()
604       | _ ->
605           Lwt.return
606             (Pricklypear_lisp.Ast.VStr
607                "agent-task: need prompt-str [, max-steps int]"));
608
609   (* (propose-function 'name "spec text") -> VStr name | error string.
610      Calls OpenAI-compat LLM, defines the function on the CURRENT branch
611      with author = agent:<model> (AI provenance). Never merges to system. *)
612   Primitives.register tbl "propose-function"
613     (fun args ->
614       match args with
615       | [ Pricklypear_lisp.Ast.VStr name;
616           Pricklypear_lisp.Ast.VStr spec ] ->
617           propose_function t ~name ~spec
618       | _ ->
619           Lwt.return
620             (Pricklypear_lisp.Ast.VStr
621                "propose-function: bad args (need name-str, spec-str)"));
622
623   (* (install-function name body-src) -> install AI-authored body (effect
       gate). *)
624   Primitives.register tbl "install-function"
625     (fun args ->
626       match args with
627       | [ Pricklypear_lisp.Ast.VStr name;
628           Pricklypear_lisp.Ast.VStr body ] ->
629           install_ai_function t ~name ~body_src:body
630       | _ ->
631           Lwt.return
632             (Pricklypear_lisp.Ast.VStr
```

```
633               "install-function: need name-str, body-str"));
634
635   (* (function-source name) -> VStr pretty body | VNil *)
636   Primitives.register tbl "function-source"
637     (fun args ->
638       match args with
639       | [ Pricklypear_lisp.Ast.VStr name ] ->
640           let* src = function_source t name in
641           (match src with
642            | Some s -> Lwt.return (Pricklypear_lisp.Ast.VStr s)
643            | None -> Lwt.return Pricklypear_lisp.Ast.VNil)
644       | _ ->
645           Lwt.return
646             (Pricklypear_lisp.Ast.VStr "function-source: need name-str"));
647
648   (* (package-bindings pkg) -> VList of name strings *)
649   Primitives.register tbl "package-bindings"
650     (fun args ->
651       match args with
652       | [ Pricklypear_lisp.Ast.VStr pkg ] ->
653           let* names = package_binding_names t pkg in
654           Lwt.return
655             (Pricklypear_lisp.Ast.VList
656                (List.map (fun n -> Pricklypear_lisp.Ast.VStr n) names))
657       | _ ->
658           Lwt.return
659             (Pricklypear_lisp.Ast.VStr "package-bindings: need
    package-str"));
660
661   (* (export-package pkg) -> VStr lib.pp text *)
662   Primitives.register tbl "export-package"
663     (fun args ->
664       match args with
665       | [ Pricklypear_lisp.Ast.VStr pkg ] ->
666           let* text = export_package t pkg in
667           Lwt.return (Pricklypear_lisp.Ast.VStr text)
668       | _ ->
669           Lwt.return
670             (Pricklypear_lisp.Ast.VStr "export-package: need package-str"));
671
672   (* Coding tools: freeze AST rewrites into store *)
673   Primitives.register tbl "llm-inner"
674     (fun args ->
675       match args with
676       | [ Pricklypear_lisp.Ast.VStr name;
677           Pricklypear_lisp.Ast.VStr concept ] ->
678           llm_inner t ~name ~concept
679       | _ ->
680           Lwt.return
681             (Pricklypear_lisp.Ast.VStr "llm-inner: need name-str,
    concept-str"));
682
683   Primitives.register tbl "llm-outer"
684     (fun args ->
685       match args with
686       | [ Pricklypear_lisp.Ast.VStr name;
687           Pricklypear_lisp.Ast.VStr concept ] ->
688           llm_outer t ~name ~concept
689       | _ ->
690           Lwt.return
691             (Pricklypear_lisp.Ast.VStr "llm-outer: need name-str,
    concept-str"));
692
693   Primitives.register tbl "llm-rewrite"
694     (fun args ->
695       match args with
696       | [ Pricklypear_lisp.Ast.VStr name;
697           Pricklypear_lisp.Ast.VStr concept ] ->
698           llm_rewrite t ~mode:"replace" ~name ~concept
699       | [ Pricklypear_lisp.Ast.VStr name;
700           Pricklypear_lisp.Ast.VStr concept;
701           Pricklypear_lisp.Ast.VStr mode ] ->
702           llm_rewrite t ~mode ~name ~concept
```

```
      | _ ->
          Lwt.return
            (Pricklypear_lisp.Ast.VStr
               "llm-rewrite: need name, concept [, mode replace|wrap]"));

  Primitives.register tbl "llm-seed"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr requirement ] ->
          llm_seed t ~requirement ()
      | [ Pricklypear_lisp.Ast.VStr requirement;
          Pricklypear_lisp.Ast.VInt n ] ->
          llm_seed t ~requirement ~max_steps:(Int64.to_int n) ()
      | _ ->
          Lwt.return
            (Pricklypear_lisp.Ast.VStr
               "llm-seed: need requirement-str [, max-steps int]"))


let method_to_string (m : http_method) : string =
  match m with `GET -> "GET" | `POST -> "POST"

let require_request (t : t) : request =
  match t.current_request with
  | Some r -> r
  | None ->
      raise
        (Pricklypear_lisp.Ast.Interp_error
           (Pricklypear_lisp.Ast.User_error "no active request"))

(** Register request-* + redirect prims. Closures capture [t] and read
    [current_request] at call time (handler-scoped dynamic binding). *)
let register_request_prims (t : t) (tbl : Primitives.t) : unit =
  Primitives.register_sync tbl "request-method" (fun _args ->
      let r = require_request t in
      Pricklypear_lisp.Ast.VStr (method_to_string r.method_));
  Primitives.register_sync tbl "request-path" (fun _args ->
      let r = require_request t in
      Pricklypear_lisp.Ast.VStr r.path);
  Primitives.register_sync tbl "request-body" (fun _args ->
      let r = require_request t in
      Pricklypear_lisp.Ast.VStr r.body);
  Primitives.register_sync tbl "request-param" (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr key ] ->
          let r = require_request t in
          (match List.assoc_opt key r.params with
           | Some v -> Pricklypear_lisp.Ast.VStr v
           | None -> Pricklypear_lisp.Ast.VNil)
      | _ ->
          raise
            (Pricklypear_lisp.Ast.Interp_error
               (Pricklypear_lisp.Ast.User_error "request-param needs 1
    string")));
  Primitives.register_sync tbl "request-header" (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr key ] ->
          let r = require_request t in
          let k = String.lowercase_ascii key in
          (match List.assoc_opt k r.headers with
           | Some v -> Pricklypear_lisp.Ast.VStr v
           | None -> Pricklypear_lisp.Ast.VNil)
      | _ ->
          raise
            (Pricklypear_lisp.Ast.Interp_error
               (Pricklypear_lisp.Ast.User_error "request-header needs 1
    string")));
  Primitives.register_sync tbl "hx-request?" (fun _args ->
      let r = require_request t in
      match List.assoc_opt "hx-request" r.headers with
      | Some v when v <> "" && v <> "false" -> Pricklypear_lisp.Ast.VBool
    true
      | _ -> Pricklypear_lisp.Ast.VBool false);
```

```
  Primitives.register_sync tbl "request-params" (fun _args ->
      let r = require_request t in
      let j =
        `Assoc
          (List.map (fun (k, v) -> (k, `String v)) r.params)
      in
      Pricklypear_lisp.Ast.VStr (Yojson.Safe.to_string j));
  Primitives.register_sync tbl "redirect" (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr path ] ->
          if String.length path = 0 || path.[0] <> '/' then
            raise
              (Pricklypear_lisp.Ast.Interp_error
                 (Pricklypear_lisp.Ast.User_error
                    "redirect path must be absolute on host (start with /)"))
          else if String.contains path ':' then
            raise
              (Pricklypear_lisp.Ast.Interp_error
                 (Pricklypear_lisp.Ast.User_error
                    "redirect rejects off-site URLs"))
          else Primitives.redirect_value path
      | _ ->
          raise
            (Pricklypear_lisp.Ast.Interp_error
               (Pricklypear_lisp.Ast.User_error "redirect needs 1 string path")))

(* Evaluate a single s-expression string as one REPL turn.
   Returns either the resulting value or an error message string.

   Per spec [image.repl-experience]: (define name val) ALSO persists to
   the store: the env binding is in-memory (fast), the store binding
   survives image reboot. The store call is non-fatal on PG-down (the
   env binding still happened).

   Lwt ABI (post-refactor): fork/merge/route/checkout are NORMAL PRIMS
   registered via [register_session_control_prims] -- they return their
   real DB result wrapped in a promise. Only [define] keeps an AST-level
   intercept: it's read-level special (the name arg is a Var that must
   not be evaluated), so it can't become a regular prim. *)
let rec eval_one (t : t) (input : string) : Pricklypear_lisp.Ast.value Lwt.t =
  try
    let expr = Pricklypear_lisp.Reader.read_one input in
    (* Pre-eval intercept: (define name <body>) is the only AST-level
       special case left, because the name argument is a Var that must
       not be evaluated. Everything else (fork/merge/route/checkout)
       dispatches through Eval.apply via the prim_table. *)
    let* v =
      Pricklypear_lisp.Eval.eval_string ~prims:t.prims t.env input
    in
    (match expr with
     | Pricklypear_lisp.Ast.App
         (Pricklypear_lisp.Ast.Var "define",
          [ Pricklypear_lisp.Ast.Var name; body_expr ]) ->
         (* Persist the body AST to the store for reload-on-boot. The
            env binding already happened via eval's Define case; this
            DB write is non-fatal on PG-down. Anonymous may not persist. *)
         if t.manifest.username <> "anonymous" then
           Lwt.async (fun () ->
             Store.persist_function t.store name body_expr
               t.manifest.username);
         let* () =
           if name = "ui/nav" || name = "ui/home" || name = "ui/css"
              || name = "ui/theme"
              || String.starts_with ~prefix:"theme/" name
           then refresh_ui_shell t
           else Lwt.return_unit
         in
         Lwt.return v
     | _ -> Lwt.return v)
  with
```

```
  | Pricklypear_lisp.Ast.Interp_error e as exn -> Lwt.fail exn
  | Pricklypear_lisp.Reader.Reader_error (msg, line, col) ->
      Lwt.fail (Pricklypear_lisp.Ast.Interp_error
                  (Pricklypear_lisp.Ast.Parse_error (msg, line, col)))
  | exn -> Lwt.fail exn


(* --- Library loader: same .pp text as in git libs/ --- *)

and eval_form (t : t) (expr : Pricklypear_lisp.Ast.expr)
    : Pricklypear_lisp.Ast.value Lwt.t =
  (* Like eval_one but takes AST -- avoids pp_expr string round-trip. *)
  let* v =
    Pricklypear_lisp.Eval.eval ~budget:(Pricklypear_lisp.Eval.make_budget
    ())
      ~prims:t.prims t.env expr
  in
  (match expr with
   | Pricklypear_lisp.Ast.App
       (Pricklypear_lisp.Ast.Var "define",
        [ Pricklypear_lisp.Ast.Var name; body_expr ]) ->
       if t.manifest.username <> "anonymous" then
         Lwt.async (fun () ->
           Store.persist_function t.store name body_expr
    t.manifest.username);
       let* () =
         if name = "ui/nav" || name = "ui/home" || name = "ui/css"
            || name = "ui/theme"
            || String.starts_with ~prefix:"theme/" name
         then refresh_ui_shell t
         else Lwt.return_unit
       in
       Lwt.return v
   | _ -> Lwt.return v)

and load_string (t : t) (src : string) : (string, string) result Lwt.t =
  Loader.load_string ~eval_form:(eval_form t) src

and load_library (t : t) (name : string) : (string, string) result Lwt.t =
  Loader.load_library ~eval_form:(eval_form t) name

and refresh_ui_shell (t : t) : unit Lwt.t =
  (* Product shell: nav HTML + theme CSS (image.product-theme). *)
  let* () =
    match Pricklypear_lisp.Env.lookup t.env "ui/nav" with
    | None ->
        t.ui_nav_html <- None;
        Lwt.return_unit
    | Some _ ->
        Lwt.catch
          (fun () ->
            let* v = eval_one t "(ui/nav)" in
            (match Primitives.is_raw_html v with
             | Some s -> t.ui_nav_html <- Some s
             | None ->
                 (match v with
                  | Pricklypear_lisp.Ast.VStr s -> t.ui_nav_html <- Some s
                  | _ ->
                      t.ui_nav_html <-
                        Some (Pricklypear_lisp.Reader.pp_value v)));
            Lwt.return_unit)
          (fun exn ->
            Printf.eprintf "[session] refresh_ui_shell (ui/nav) failed:
    %s\n%!"
              (Printexc.to_string exn);
            t.ui_nav_html <- None;
            Lwt.return_unit)
  in
  match Pricklypear_lisp.Env.lookup t.env "ui/css" with
  | None ->
      t.ui_css <- None;
      Lwt.return_unit
  | Some _ ->
```

```
      Lwt.catch
        (fun () ->
          let* v = eval_one t "(ui/css)" in
          (match Primitives.is_raw_html v with
           | Some s -> t.ui_css <- Some s
           | None ->
               (match v with
                | Pricklypear_lisp.Ast.VStr s -> t.ui_css <- Some s
                | _ ->
                    t.ui_css <-
                      Some (Pricklypear_lisp.Reader.pp_value v)));
          Lwt.return_unit)
        (fun exn ->
          Printf.eprintf "[session] refresh_ui_shell (ui/css) failed:
    %s\n%!"
            (Printexc.to_string exn);
          t.ui_css <- None;
          Lwt.return_unit)

and seed_one (t : t) (name : string) : unit Lwt.t =
  let* r = load_library t name in
  (match r with
   | Ok msg -> Printf.eprintf "[session] %s: %s\n%!" name msg
   | Error e -> Printf.eprintf "[session] %s load: %s\n%!" name e);
  Lwt.return_unit

and seed_stdlib (t : t) : unit Lwt.t =
  (* Package defaults always land on system/main -- NEVER on the user's
     personal branch. Personal inherits main via the recursive CTE; user
     (define)s write overrides on personal and survive reseed/deploy.

     Canonical source is libs/*.pp in git. Explicit (load-library ...) on
     the current branch still overwrites that branch (intentional). *)
  Store.with_branch t.store ~branch_id:Store.system_main_id (fun () ->
    let* () = seed_one t "std" in
    let* () = seed_one t "weather" in
    let* () = seed_one t "ui" in
    let* () = seed_one t "layout" in
    let* () = seed_one t "welcome" in
    let* () = seed_one t "ai" in
    let* () = seed_one t "outline" in
    let* () = seed_one t "notes" in
    let* () = seed_one t "todo" in
    let* () = seed_one t "contacts" in
    let* () = seed_one t "bookmarks" in
    let* () = seed_one t "calendar" in
    let* () = seed_one t "flashcards" in
    let* () = seed_one t "bible" in
    let* () = seed_one t "dabar" in
    Lwt.return_unit)

(** Ensure username/personal exists as an EMPTY child of system/main.
    Inheritance (not copy) means main defaults stay visible; personal
    only stores the user's overrides. *)
and ensure_personal_branch (t : t) : string option Lwt.t =
  let owner = t.manifest.username in
  let* main_id_opt =
    Store.lookup_branch_id_by_name t.store ~owner:"system" ~name:"main"
  in
  let parent_id =
    match main_id_opt with
    | Some id -> id
    | None -> Store.system_main_id
  in
  Store.ensure_branch t.store ~parent_id ~owner ~name:"personal"

and checkout_personal (t : t) : unit Lwt.t =
  let owner = t.manifest.username in
  let branch_ref = owner ^ "/personal" in
  let* found =
    Store.lookup_branch_id_by_name t.store ~owner ~name:"personal"
  in
  match found with
```

```
  | None ->
      Printf.eprintf
        "[session] personal branch missing; staying on %s\n%!"
        t.manifest.branch_name;
      (* Still rehydrate whatever branch we have. *)
      rehydrate t
  | Some id ->
      Printf.eprintf "[session] checkout %s (%s)\n%!" branch_ref id;
      checkout_to t ~branch_id:id ~branch_ref

and recipe_hello (t : t) : (string, string) result Lwt.t =
  load_library t "hello"

and recipe_todo (t : t) : (string, string) result Lwt.t =
  load_library t "todo"

and recipe_bible (t : t) : (string, string) result Lwt.t =
  (* Types + API + /bible only -- corpus via scripts/seed_bible_bsb.py *)
  load_library t "bible"

and recipe_dabar (t : t) : (string, string) result Lwt.t =
  (* Study layer -- requires bible/*; Strong's/interlinear via seed scripts
     *)
  let* r = load_library t "bible" in
  (match r with
   | Error e -> Lwt.return (Error e)
   | Ok _ -> load_library t "dabar")

and recipe_notes (t : t) : (string, string) result Lwt.t =
  (* Outline-first notes -- outline pure lib then notes package *)
  let* r = load_library t "outline" in
  (match r with
   | Error e -> Lwt.return (Error e)
   | Ok _ -> load_library t "notes")


let make ~(manifest : Pricklypear_common.Manifest.t) ~(store : Store.t) ()
    : t =
  let env = Pricklypear_lisp.Env.create () in
  let prim_table = Primitives.make_with_defaults () in
  (* Pre-allocate the session record so the DB-touching control prims
     (fork/merge/route/checkout) can capture [t] as a closure (they
     need access to t.store + t.manifest) and register themselves on
     the same prim_table. *)
  let routes = Hashtbl.create 16 in
  let t =
    ref {
      manifest;
      store;
      env;
      prims = Primitives.to_eval_table prim_table;  (* temporary; replaced
    below *)
      routes;
      started_at = Unix.gettimeofday ();
      kernel_catalog = [];  (* filled after all prims register *)
      ui_nav_html = None;
      ui_css = None;
      current_request = None;
      agent_branch_lock = false;
      is_admin = (manifest.username = "admin");
      auth_token = None;
    }
  in
  (* Register session-aware primitives (whoami, current-author,
     list-bindings). username_fn re-reads t.manifest so per-request
     identity attach updates (whoami) without re-registering. *)
  Primitives.register_session_prims prim_table
    ~username_fn:(fun () -> (!t).manifest.username) ~env;
  (* Register the DB-touching control primitives on the prim_table,
     then rebind t.prims to the populated table. These prims capture
     (!t), so they read/write store+manifest on each invocation. *)
  register_session_control_prims !t prim_table;
  (* HTTP request-context prims (it33): request-method/path/param(s)/body +
```

```
    redirect. Dynamic scope via current_request while call_handler runs. *)
register_request_prims !t prim_table;
(* Register CRUD primitives (get/create/update/delete/list) over
   the shared data_rows table. Same pattern as control prims:
   they capture (!t) and return their real DB result via Lwt. *)
register_crud_prims !t prim_table;
(* AI + list-branches / current-branch registered inside
   register_crud_prims above (they capture t). *)
(* Make primitives reachable by name in the env, so `(+)` etc.
   resolve. We define each as VPrim <name>; eval.ml's apply already
   dispatches VPrim via lookup_prim. Comes AFTER control prims are
   registered, so fork/merge/route/checkout auto-bind too. *)
Hashtbl.iter
  (fun name _ ->
    Pricklypear_lisp.Env.define env name (Pricklypear_lisp.Ast.VPrim
   name))
  prim_table;
Primitives.register prim_table "load-string"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr src ] ->
        let* r = load_string !t src in
        (match r with
         | Ok msg -> Lwt.return (Pricklypear_lisp.Ast.VStr msg)
         | Error e ->
             Lwt.return (Pricklypear_lisp.Ast.VStr ("load-string: " ^ e)))
    | _ ->
        Lwt.return
          (Pricklypear_lisp.Ast.VStr "load-string: need 1 string"));
Primitives.register prim_table "load-library"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr name ] ->
        let* r = load_library !t name in
        let* () = refresh_ui_shell !t in
        (match r with
         | Ok msg -> Lwt.return (Pricklypear_lisp.Ast.VStr msg)
         | Error e ->
             Lwt.return (Pricklypear_lisp.Ast.VStr ("load-library: " ^
   e)))
    | _ ->
        Lwt.return
          (Pricklypear_lisp.Ast.VStr "load-library: need 1 string"));
let final_prims = Primitives.to_eval_table prim_table in
let kernel_catalog = Primitives.catalog prim_table in
(* Mutate the prims field in place (rather than building a fresh
   record via { (!t) with prims = ... }). Closures captured by
   register_session_control_prims / register_crud_prims read
   t.prims at CALL time (e.g. __with_transaction__'s Eval.apply
   ~prims:t.prims) and need the populated table, not the temporary
   snapshot from step 3. Lands it29: this was latent until
   with-transaction's body needed to call back into the prim
   registry itself. The earlier control prims (fork/merge/route/
   checkout) didn't trip this because they only invoke Store.*
   functions, never Eval.apply against the prims table. *)
(!t).prims <- final_prims;
(!t).kernel_catalog <- kernel_catalog;
(* Bind load prims into env (registered after the bulk VPrim bind). *)
Pricklypear_lisp.Env.define env "load-string" (Pricklypear_lisp.Ast.VPrim
   "load-string");
Pricklypear_lisp.Env.define env "load-library"
   (Pricklypear_lisp.Ast.VPrim "load-library");
let session : t = !t in
(* Boot order (personal-branch safety):
     1. ensure username/personal exists (empty child of system/main)
     2. seed_stdlib writes package defaults ONLY to system/main
     3. checkout personal + rehydrate env (inherits main via CTE)
     4. refresh ui shell from the personal-visible ui/nav
   User (define)s land on personal and are never wiped by reseed. *)
Lwt.async (fun () ->
  let* _ = ensure_personal_branch session in
  let* () = seed_stdlib session in
  let* () = checkout_personal session in
```

```
    let* () = refresh_ui_shell session in
    Lwt.return_unit);
  session


let uptime_seconds (t : t) : int =
  int_of_float (Unix.gettimeofday () -. t.started_at)

let branch_name (t : t) : string = t.manifest.branch_name
let username (t : t) : string = t.manifest.username
let is_admin (t : t) : bool = t.is_admin
let store (t : t) : Store.t = t.store
let branch_id (t : t) : string = t.manifest.branch_id
let auth_token (t : t) : string option = t.auth_token

(** Attach authenticated identity for this request (auth.web-login).
    Updates manifest user/username, is_admin, optional raw token.
    When [checkout_personal] is true, ensures + checks out
    username/personal (login / first attach). *)
let attach_identity
    (t : t)
    ~(user_id : string)
    ~(username : string)
    ~(is_admin : bool)
    ?(auth_token : string option = None)
    ?(checkout_personal_now = false)
    () : unit Lwt.t =
  let prev = t.manifest.username in
  t.manifest <-
    { t.manifest with user_id; username };
  t.is_admin <- is_admin;
  t.auth_token <- auth_token;
  if checkout_personal_now || prev <> username then
    let* _ = ensure_personal_branch t in
    checkout_personal t
  else
    Lwt.return_unit

(** Anonymous request identity (auth.route-visibility).
    system/main only; no personal garden; no admin. *)
let attach_anonymous (t : t) : unit Lwt.t =
  t.manifest <-
    { t.manifest with
      user_id = "anonymous"
    ; username = "anonymous"
    };
  t.is_admin <- false;
  t.auth_token <- None;
  (* Prefer system/main for package defaults. *)
  let* found =
    Store.lookup_branch_id_by_name t.store ~owner:"system" ~name:"main"
  in
  match found with
  | None -> Lwt.return_unit
  | Some id ->
      if t.manifest.branch_id = id then Lwt.return_unit
      else checkout_to t ~branch_id:id ~branch_ref:"system/main"

let clear_auth_token (t : t) : unit =
  t.auth_token <- None

(** Resolve owner/name ref and checkout. *)
let checkout_ref (t : t) (branch_ref : string) : (unit, string) result
    Lwt.t =
  let owner, name = split_branch_ref branch_ref in
  let ref' = owner ^ "/" ^ name in
  let* found = Store.lookup_branch_id_by_name t.store ~owner ~name in
  match found with
  | None -> Lwt.return (Error "no such branch")
  | Some id ->
      let* () = checkout_to t ~branch_id:id ~branch_ref:ref' in
      Lwt.return (Ok ())

```

```
(** Delete a function row on the current branch (unoverride). Rehydrates. *)
let unoverride (t : t) (name : string) : (bool, string) result Lwt.t =
  let bid = Store.current_branch_id t.store in
  let* deleted =
    Store.delete_function_on_branch t.store ~branch_id:bid ~name
  in
  let* () = rehydrate t in
  let* () = refresh_ui_shell t in
  Lwt.return (Ok deleted)

(** Promote selected names from source ref onto system/main (admin policy).
    *)
let promote_to_main (t : t) ~(source_ref : string) ~(names : string list)
  : (int, string) result Lwt.t =
  if names = [] then Lwt.return (Error "no names selected")
  else
    let owner, name = split_branch_ref source_ref in
    let* src =
      Store.lookup_branch_id_by_name t.store ~owner ~name
    in
    match src with
    | None -> Lwt.return (Error "no such source branch")
    | Some source_id ->
        (* Policy: only admin username may write system/main via promote
     UI. *)
        if (not t.is_admin)
           && t.manifest.username <> "admin"
           && t.manifest.username <> "system" then
          Lwt.return (Error "promote to system/main requires admin")
        else
          let* n =
            Store.promote_functions t.store
              ~source_id ~target_id:Store.system_main_id ~names
          in
          (* If we're on main, rehydrate so env sees new bodies. *)
          let* () =
            if Store.current_branch_id t.store = Store.system_main_id then
              rehydrate t
            else Lwt.return_unit
          in
          Lwt.return (Ok n)

(** Full fork: parent_ref -> owner/child_name (uses Store.fork_branch
    copy). *)
let fork_ref (t : t) ~(parent_ref : string) ~(child_ref : string)
  : (string, string) result Lwt.t =
  let p_owner, p_name = split_branch_ref parent_ref in
  let c_owner, c_name = split_branch_ref child_ref in
  let* parent_id =
    Store.lookup_branch_id_by_name t.store ~owner:p_owner ~name:p_name
  in
  match parent_id with
  | None -> Lwt.return (Error "no such parent branch")
  | Some pid ->
      let* created =
        Store.fork_branch t.store ~parent_id:pid ~owner:c_owner
          ~child_name:c_name
      in
      (match created with
       | None -> Lwt.return (Error "fork failed")
       | Some id -> Lwt.return (Ok id))

(* Look up a route's handler name. Returns None if the route isn't
   registered on this branch. Called by http.ml's request_handler. *)
let lookup_route (t : t) (route : string) : string option =
  match Hashtbl.find_opt t.routes route with
  | Some (handler, _) -> Some handler
  | None -> None

let lookup_route_meta (t : t) (route : string)
  : (string * string) option =
  Hashtbl.find_opt t.routes route

```

```
let route_visibility (t : t) (route : string) : string option =
  match Hashtbl.find_opt t.routes route with
  | Some (_, vis) -> Some vis
  | None -> None

let is_anonymous (t : t) : bool = t.manifest.username = "anonymous"

let kernel_catalog (t : t) : (string * string) list = t.kernel_catalog

let ui_nav_html (t : t) : string option = t.ui_nav_html

let set_ui_nav_html (t : t) (s : string option) : unit = t.ui_nav_html <- s

let ui_css (t : t) : string option = t.ui_css

let set_ui_css (t : t) (s : string option) : unit = t.ui_css <- s


let list_pages (t : t) : (string * string) list =
  Hashtbl.fold
    (fun route (handler, _) acc -> (route, handler) :: acc) t.routes []
  |> List.sort (fun (a, _) (b, _) -> String.compare a b)

let list_pages_meta (t : t) : (string * string * string) list =
  Hashtbl.fold
    (fun route (handler, vis) acc -> (route, handler, vis) :: acc) t.routes
     []
  |> List.sort (fun (a, _, _) (b, _, _) -> String.compare a b)

let set_route (t : t) ~(route : string) ~(handler : string)
    ?visibility () : unit Lwt.t =
  let vis =
    match visibility with
    | Some v -> Store.normalize_visibility v
    | None ->
        (match Hashtbl.find_opt t.routes route with
         | Some (_, v) -> v
         | None -> "private")
  in
  Hashtbl.replace t.routes route (handler, vis);
  let* _ =
    match visibility with
    | Some v ->
        Store.register_route t.store ~route ~handler
          ~visibility:(Store.normalize_visibility v) ()
    | None -> Store.register_route t.store ~route ~handler ()
  in
  Lwt.return_unit

(* Invoke a named handler function (looked up via env). The handler
   must be a zero-arg VLambda or a VPrim name. Returns its value, or
   raises Interp_error if the handler doesn't resolve.
   When [~request] is provided, request-* prims see that context for
   the duration of the call (cleared in finalize). *)
let call_handler (t : t) ?request (handler_name : string)
  : Pricklypear_lisp.Ast.value option Lwt.t =
  match Pricklypear_lisp.Env.lookup t.env handler_name with
  | None -> Lwt.return_none
  | Some _fn ->
      (* Evaluate (handler-name) -- zero args. Re-uses eval's apply path
         so VLambda closures AND VPrim dispatch both work. *)
      let run () =
        Lwt.catch
          (fun () ->
             let* v =
               Pricklypear_lisp.Eval.eval_string ~prims:t.prims t.env
                 ("(" ^ handler_name ^ ")")
             in
             Lwt.return (Some v))
          (fun exn ->
             Printf.eprintf "[session] call_handler %s: %s\n%!"
               handler_name (pp_interp_exn exn);
             Lwt.return_none)
```

```
      in
      (match request with
       | None -> run ()
       | Some req ->
           t.current_request <- Some req;
           Lwt.finalize run (fun () ->
               t.current_request <- None;
               Lwt.return_unit))
```

52 image/lib/session_types.ml

```
(** Session record and HTTP request types (shared by Session +
    Agent_tools). *)
type http_method = [ `GET | `POST ]

type request = {
  method_ : http_method;
  path : string;
  params : (string * string) list;  (** form fields win over query *)
  body : string;
  (** Lowercase header names ? values (for HX-Request, etc.). *)
  headers : (string * string) list;
}

type t = {
  mutable manifest : Pricklypear_common.Manifest.t;
  store : Store.t;
  env : Pricklypear_lisp.Ast.env;
  mutable prims : Pricklypear_lisp.Eval.prim_table;
  (* In-memory route table mirroring view_bindings for the branch.
     Populated by load_defs_async at session-make time; (route ...)
     updates also live-persist to view_bindings AND update this table.
     Value is (handler, visibility) where visibility is public|private.
     http.ml reads from here on every request (no DB round-trip). *)
  routes : (string, string * string) Hashtbl.t;
  mutable started_at : float;
  (** Kernel prim catalog for Functions browser: (name, summary) list. *)
  mutable kernel_catalog : (string * string) list;
  (** Cached HTML from ui/nav when defined; None = use OCaml default nav. *)
  mutable ui_nav_html : string option;
  (** Cached CSS from (ui/css) when defined; None = emergency_css in http.
     *)
  mutable ui_css : string option;
  (** Active HTTP request for user-route dispatch; None in REPL. *)
  mutable current_request : request option;
  (** When true, (checkout)/(fork)/(merge) are blocked so the agent
      cannot leave the current branch mid-task. *)
  mutable agent_branch_lock : bool;
  (** Admin flag from users.is_admin (auth identity). *)
  mutable is_admin : bool;
  (** Raw pp_session cookie for the active HTTP request, if any. *)
  mutable auth_token : string option;
}

(* Load persisted function_defs + routes into the env.
   Returns when finished (caller may then seed libs/ which overwrite).

   Load order is critical: many personal pins are *values* evaluated at
   define-time (e.g. ui/menu = (list (menu/link ?))), not lambdas.
   Those bodies need parent helpers (menu/link, menu/group, ?) already
   bound. Store.list_function_names returns effective defs only (closest
   branch wins) but does NOT topological-order dependencies across the
   parent chain. We therefore load the full chain parents-first
   (depth DESC) and redefine so child overrides win last.

   Historical bug: depth ASC + first-wins loaded personal ui/menu BEFORE
   system menu/link ? define failed ? ui/menu unbound ? (ui/nav) fails ?
   silent fallback to OCaml default_nav_nodes ("navbar disappeared"). *)
```

53 image/lib/store.ml

```
(** Pricklypear Image -- Storage layer (branch-aware queries + fork/merge).
    Connection/tx substrate is [Store_db], included below. *)
include Store_db
open Lwt.Syntax

(* Branch-aware lookup. Walks the branch parent chain via a recursive
   CTE so that a definition in [main] is visible from a child branch,
   and a definition in the child OVERRIDES the same name in main
   (per spec storage.no-inheritance: OVERRIDE, not UNION).

   Returns the definition's parsed AST and author. Errors/misses are
   logged and return None so the REPL keeps working with no DB. *)
let lookup_function (t : t) (name : string)
  : (Pricklypear_lisp.Ast.expr * string) option Lwt.t =
  match t.conn with
  | None -> Lwt.return_none
  | Some conn ->
      Lwt.catch
        (fun () ->
          (* Recursive CTE: start at the session's branch and walk parents.
             Return the FIRST row matching [name] when ordered by depth
             ascending (closest ancestor wins). *)
          let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.body, fd.author FROM function_defs fd " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "WHERE fd.name = $2 " ^
            "ORDER BY chain.depth ASC LIMIT 1" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string name ]
              sql in
          (match rows with
           | [] -> Lwt.return_none
           | row :: _ ->
               (* Each row is [cell; cell] where cell = v option;
                  first is jsonb body, second is author text. *)
               (match row with
                | [ body_cell; author_cell ] ->
                    (match Pgx.Value.to_string body_cell,
                             Pgx.Value.to_string author_cell with
                     | Some body_str, Some author ->
                        (match Yojson.Safe.from_string body_str with
                         | j ->
                             (match Pricklypear_lisp.Ast.expr_of_json j with
                              | Ok e -> Lwt.return (Some (e, author))
                              | Error msg ->
                                  Printf.eprintf
                                    "[store] lookup_function: parse %s\n"
    msg;
                                  Lwt.return_none)
                         | exception Yojson.Json_error msg ->
                             Printf.eprintf
                               "[store] lookup_function: json %s\n" msg;
                             Lwt.return_none)
                     | _ -> Lwt.return_none)
                | _ -> Lwt.return_none))
        )
        (fun exn ->
          Printf.eprintf "[store] lookup_function error: %s\n"
            (Printexc.to_string exn);
          Lwt.return_none)

(* List all function names visible to this branch (via the same
   recursive-CTE override walk). Used at session-make time to load
```

```
   persisted definitions into the env. *)
(** Effective function defs for the current branch: parent-chain walk,
    closest definition wins (depth ASC, first occurrence kept).
    [depth] is 0 for rows owned by the current branch, >0 if inherited.
    Callers that only need name/body/author can ignore depth. *)
let list_function_names_meta (t : t)
  : (string * Pricklypear_lisp.Ast.expr * string * int) list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.name, fd.body, fd.author, chain.depth " ^
            "FROM function_defs fd " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC, fd.name ASC" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql in
          let seen = Hashtbl.create 64 in
          let acc =
            List.filter_map
              (fun row ->
                match row with
                | [ name_c; body_cell; author_cell; depth_c ] ->
                    (match Pgx.Value.to_string name_c,
                           Pgx.Value.to_string body_cell,
                           Pgx.Value.to_string author_cell,
                           Pgx.Value.to_string depth_c with
                     | Some name, Some body_str, Some author, Some d_s ->
                         if Hashtbl.mem seen name then None
                         else begin
                           Hashtbl.replace seen name ();
                           let depth =
                             int_of_string_opt d_s |> Option.value
    ~default:0
                           in
                           (try
                              let j = Yojson.Safe.from_string body_str in
                              (match Pricklypear_lisp.Ast.expr_of_json j
    with
                               | Ok e -> Some (name, e, author, depth)
                               | Error _ -> None)
                            with Yojson.Json_error _ -> None)
                         end
                     | _ -> None)
                | _ -> None)
              rows
          in
          Lwt.return acc)
        (fun exn ->
          Printf.eprintf "[store] list_function_names error: %s\n"
            (Printexc.to_string exn);
          Lwt.return [])

(** Effective defs only (deduped). Same as [list_function_names_meta]
    without depth. *)
let list_function_names (t : t)
  : (string * Pricklypear_lisp.Ast.expr * string) list Lwt.t =
  let* rows = list_function_names_meta t in
  Lwt.return (List.map (fun (n, e, a, _) -> (n, e, a)) rows)

(** Full branch-chain defs for env rehydrate: parents first (depth DESC),
    then children. Includes ALL rows (not deduped) so Session.load_defs can
```

```
    redefine -- last write wins (child override). See session.load_defs
     note. *)
let list_function_names_for_load (t : t)
  : (string * Pricklypear_lisp.Ast.expr * string) list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.name, fd.body, fd.author, chain.depth " ^
            "FROM function_defs fd " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "ORDER BY chain.depth DESC, fd.name ASC" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql in
          let acc =
            List.filter_map
              (fun row ->
                match row with
                | [ name_c; body_cell; author_cell; _depth_c ] ->
                    (match Pgx.Value.to_string name_c,
                           Pgx.Value.to_string body_cell,
                           Pgx.Value.to_string author_cell with
                     | Some name, Some body_str, Some author ->
                         (try
                            let j = Yojson.Safe.from_string body_str in
                            (match Pricklypear_lisp.Ast.expr_of_json j with
                             | Ok e -> Some (name, e, author)
                             | Error _ -> None)
                          with Yojson.Json_error _ -> None)
                     | _ -> None)
                | _ -> None)
              rows
          in
          Lwt.return acc)
        (fun exn ->
          Printf.eprintf "[store] list_function_names_for_load error: %s\n"
            (Printexc.to_string exn);
          Lwt.return [])

(* ---- own-branch stats / overrides (no recursive CTE) ---- *)

let count_functions_on_branch (t : t) ~(branch_id : string) : int Lwt.t =
  match t.conn with
  | None -> Lwt.return 0
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT COUNT(*)::text FROM function_defs WHERE branch_id = $1"
    in
          let* rows =
            Pg.execute conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          (match rows with
           | [ [ c ] ] ->
               (match Pgx.Value.to_string c with
                | Some s -> Lwt.return (int_of_string_opt s |> Option.value
    ~default:0)
                | None -> Lwt.return 0)
           | _ -> Lwt.return 0))
        (fun exn ->
          Printf.eprintf "[store] count_functions_on_branch: %s\n"
            (Printexc.to_string exn);
```

```
          Lwt.return 0)

let count_routes_on_branch (t : t) ~(branch_id : string) : int Lwt.t =
  match t.conn with
  | None -> Lwt.return 0
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT COUNT(*)::text FROM view_bindings WHERE branch_id = $1"
      in
          let* rows =
            Pg.execute conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          (match rows with
           | [ [ c ] ] ->
               (match Pgx.Value.to_string c with
                | Some s -> Lwt.return (int_of_string_opt s |> Option.value
      ~default:0)
                | None -> Lwt.return 0)
           | _ -> Lwt.return 0))
        (fun exn ->
          Printf.eprintf "[store] count_routes_on_branch: %s\n"
            (Printexc.to_string exn);
          Lwt.return 0)

(** Own rows only: (name, author, updated_at). *)
let list_own_functions (t : t) ~(branch_id : string)
  : (string * string * string) list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT name, author, updated_at::text FROM function_defs " ^
            "WHERE branch_id = $1 ORDER BY name" in
          let* rows =
            Pg.execute conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          Lwt.return
            (List.filter_map
               (fun row ->
                 match row with
                 | [ n; a; u ] ->
                     (match Pgx.Value.to_string n,
                            Pgx.Value.to_string a,
                            Pgx.Value.to_string u with
                      | Some name, Some author, Some updated ->
                          Some (name, author, updated)
                      | _ -> None)
                 | _ -> None)
               rows))
        (fun exn ->
          Printf.eprintf "[store] list_own_functions: %s\n"
            (Printexc.to_string exn);
          Lwt.return [])

let list_own_routes (t : t) ~(branch_id : string)
  : (string * string) list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT route, handler FROM view_bindings WHERE branch_id = $1
      " ^
            "ORDER BY route" in
          let* rows =
            Pg.execute conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          Lwt.return
```

```
277             (List.filter_map
278                (fun row ->
279                  match row with
280                  | [ r; h ] ->
281                      (match Pgx.Value.to_string r, Pgx.Value.to_string h
    with
282                       | Some route, Some handler -> Some (route, handler)
283                       | _ -> None)
284                  | _ -> None)
285                rows))
286          (fun exn ->
287            Printf.eprintf "[store] list_own_routes: %s\n"
288              (Printexc.to_string exn);
289            Lwt.return [])
290
291 (** Child own names left-joined to parent: name, child_updated,
    parent_updated opt. *)
292 let list_overrides (t : t) ~(branch_id : string) ~(parent_id : string
    option)
293   : (string * string * string option) list Lwt.t =
294   match t.conn, parent_id with
295   | None, _ -> Lwt.return []
296   | Some _, None ->
297       (* Root branch: every own def is "new". *)
298       let* own = list_own_functions t ~branch_id in
299       Lwt.return (List.map (fun (n, _, u) -> (n, u, None)) own)
300   | Some conn, Some pid ->
301       Lwt.catch
302         (fun () ->
303           let sql =
304             "SELECT c.name, c.updated_at::text, p.updated_at::text " ^
305             "FROM function_defs c " ^
306             "LEFT JOIN function_defs p " ^
307             "  ON p.branch_id = $2 AND p.name = c.name " ^
308             "WHERE c.branch_id = $1 ORDER BY c.name" in
309           let* rows =
310             Pg.execute conn
311               ~params:[ Pgx.Value.of_string branch_id;
312                         Pgx.Value.of_string pid ]
313               sql
314           in
315           Lwt.return
316             (List.filter_map
317                (fun row ->
318                  match row with
319                  | [ n; cu; pu ] ->
320                      (match Pgx.Value.to_string n, Pgx.Value.to_string cu
    with
321                       | Some name, Some child_u ->
322                           let parent_u =
323                             match Pgx.Value.to_string pu with
324                             | Some "" | None -> None
325                             | Some s -> Some s
326                           in
327                           Some (name, child_u, parent_u)
328                       | _ -> None)
329                  | _ -> None)
330                rows))
331         (fun exn ->
332           Printf.eprintf "[store] list_overrides: %s\n"
333             (Printexc.to_string exn);
334           Lwt.return [])
335
336 let lookup_function_on_branch (t : t) ~(branch_id : string) ~(name : string)
337   : (Pricklypear_lisp.Ast.expr * string * string) option Lwt.t =
338   match t.conn with
339   | None -> Lwt.return_none
340   | Some conn ->
341       Lwt.catch
342         (fun () ->
343           let sql =
344             "SELECT body::text, author, updated_at::text FROM function_defs
    " ^
```

```
            "WHERE branch_id = $1 AND name = $2" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string branch_id;
                        Pgx.Value.of_string name ]
              sql
          in
          (match rows with
           | [ [ body_c; author_c; upd_c ] ] ->
               (match Pgx.Value.to_string body_c,
                      Pgx.Value.to_string author_c,
                      Pgx.Value.to_string upd_c with
                | Some body_str, Some author, Some updated ->
                    (try
                       match Pricklypear_lisp.Ast.expr_of_json
                               (Yojson.Safe.from_string body_str) with
                       | Ok e -> Lwt.return (Some (e, author, updated))
                       | Error _ -> Lwt.return_none
                     with _ -> Lwt.return_none)
                | _ -> Lwt.return_none)
           | _ -> Lwt.return_none))
        (fun exn ->
          Printf.eprintf "[store] lookup_function_on_branch: %s\n"
            (Printexc.to_string exn);
          Lwt.return_none)

let delete_function_on_branch (t : t) ~(branch_id : string) ~(name : string)
  : bool Lwt.t =
  match t.conn with
  | None -> Lwt.return false
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "DELETE FROM function_defs WHERE branch_id = $1 AND name = $2 "
    ^
            "RETURNING name" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string branch_id;
                        Pgx.Value.of_string name ]
              sql
          in
          Lwt.return (rows <> []))
        (fun exn ->
          Printf.eprintf "[store] delete_function_on_branch: %s\n"
            (Printexc.to_string exn);
          Lwt.return false)

(** Copy selected function names from source branch to target (upsert). *)
let promote_functions (t : t) ~(source_id : string) ~(target_id : string)
    ~(names : string list) : int Lwt.t =
  match t.conn with
  | None -> Lwt.return 0
  | Some _ when names = [] -> Lwt.return 0
  | Some conn ->
      Lwt.catch
        (fun () ->
          Lwt_list.fold_left_s
            (fun acc name ->
              let* src =
                lookup_function_on_branch t ~branch_id:source_id ~name
              in
              match src with
              | None -> Lwt.return acc
              | Some (body, author, _) ->
                  let body_json =
                    Yojson.Safe.to_string
                      (Pricklypear_lisp.Ast.expr_to_json body) in
                  let sql =
                    "INSERT INTO function_defs " ^
                    "  (branch_id, name, body, author) " ^
                    "VALUES ($1, $2, $3::jsonb, $4) " ^
```

```
                    "ON CONFLICT (branch_id, name) DO UPDATE " ^
                    "SET body = EXCLUDED.body, author = EXCLUDED.author, " ^
                    "    updated_at = now() RETURNING name" in
                  let* rows =
                    Pg.execute conn
                      ~params:[ Pgx.Value.of_string target_id;
                                Pgx.Value.of_string name;
                                Pgx.Value.of_string body_json;
                                Pgx.Value.of_string author ]
                      sql
                  in
                  Lwt.return (if rows = [] then acc else acc + 1))
              0 names)
          (fun exn ->
            Printf.eprintf "[store] promote_functions: %s\n"
              (Printexc.to_string exn);
            Lwt.return 0)

(* Persist a function def. INSERT ... ON CONFLICT (branch_id, name)
   DO UPDATE so redefining a name in the same branch upserts. The body
   is serialized to JSONB. *)
let persist_function (t : t) (name : string)
    (body : Pricklypear_lisp.Ast.expr) (author : string)
  : unit Lwt.t =
  match t.conn with
  | None -> Lwt.return_unit  (* no DB; env binding still happened in-memory
     *)
  | Some conn ->
      Lwt.catch
        (fun () ->
          let body_json =
            Yojson.Safe.to_string
              (Pricklypear_lisp.Ast.expr_to_json body) in
          let sql =
            "INSERT INTO function_defs (branch_id, name, body, author) " ^
            "VALUES ($1, $2, $3::jsonb, $4) " ^
            "ON CONFLICT (branch_id, name) DO UPDATE " ^
            "SET body = EXCLUDED.body, author = EXCLUDED.author, " ^
            "    updated_at = now()" in
          let* _ =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string name;
                        Pgx.Value.of_string body_json;
                        Pgx.Value.of_string author ]
              sql in
          Lwt.return_unit)
        (fun exn ->
          Printf.eprintf "[store] persist_function error: %s\n"
            (Printexc.to_string exn);
          Lwt.return_unit)

let list_branches (t : t) : Pricklypear_common.Branch.t list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "SELECT id::text, COALESCE(parent_id::text, ''), " ^
            "       owner, name, created_at::text, " ^
            "       COALESCE(last_synced::text, '') " ^
            "FROM branches ORDER BY owner, name" in
          let* rows = Pg.execute conn sql in
          Lwt.return
            (List.filter_map
               (fun row ->
                 match row with
                 | [ id_c; pid_c; owner_c; name_c; created_c; synced_c ] ->
                     (match Pgx.Value.to_string id_c,
                              Pgx.Value.to_string owner_c,
                              Pgx.Value.to_string name_c,
                              Pgx.Value.to_string created_c with
```

```
                      | Some id, Some owner, Some name, Some created ->
                          let parent_id =
                            match Pgx.Value.to_string pid_c with
                            | Some "" | None -> None
                            | Some s -> Some s
                          in
                          let last_synced =
                            match Pgx.Value.to_string synced_c with
                            | Some "" | None -> None
                            | Some s -> Some s
                          in
                          let base =
                            Pricklypear_common.Branch.make
                              ~id ~parent_id ~owner ~name
                              ~created_at:created ()
                          in
                          Some
                            (match last_synced with
                             | Some s -> { base with last_synced = Some s }
                             | None -> base)
                      | _ -> None)
                 | _ -> None)
               rows))
        (fun exn ->
          Printf.eprintf "[store] list_branches error: %s\n"
            (Printexc.to_string exn);
          Lwt.return [])

(* Look up a branch id by (owner, name). Used by (fork ...) to find
   the parent's UUID from its Lisp-level reference. Returns None if
   the branch isn't in the DB (or no DB). *)
let lookup_branch_id_by_name (t : t) ~(owner : string) ~(name : string)
  : string option Lwt.t =
  match t.conn with
  | None -> Lwt.return_none
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql = "SELECT id::text FROM branches WHERE owner = $1 AND
    name = $2" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string owner;
                        Pgx.Value.of_string name ]
              sql
          in
          (match rows with
           | [ [ id_cell ] ] ->
               Lwt.return (Pgx.Value.to_string id_cell)
           | _ -> Lwt.return_none))
        (fun exn ->
          Printf.eprintf "[store] lookup_branch_id error: %s\n"
            (Printexc.to_string exn);
          Lwt.return_none)

(* Switch the store's branch_id: subsequent lookups/persists/registers
   will target the new branch. Per spec [image.repl-experience]:
   "(checkout branch-id)" keeps REPL locals (env) but changes visibility
   (which (define)d functions are effective). We mutate the field.

   Re-loading the env is the caller's responsibility (session.eval_one
   does it after we return successfully). *)
let checkout_branch (t : t) ~(new_branch_id : string) : unit Lwt.t =
  t.branch_id <- new_branch_id;
  Lwt.return_unit

(* Get the branch_id the store is currently bound to. *)
let current_branch_id (t : t) : string = t.branch_id

(** Canonical system/main UUID -- seeded by migrations/001_branches.sql. *)
let system_main_id = "00000000-0000-0000-0000-000000000001"

(** Temporarily rebind [t.branch_id] for the duration of [f], then restore.
```

```
    Used so seed_stdlib can write package defaults to system/main without
    polluting the user's personal branch. *)
let with_branch (t : t) ~(branch_id : string) (f : unit -> 'a Lwt.t) : 'a
    Lwt.t =
  let old = t.branch_id in
  t.branch_id <- branch_id;
  Lwt.finalize f (fun () -> t.branch_id <- old; Lwt.return_unit)

(** Ensure a branch row exists; create an EMPTY child (parent pointer only)
    if missing. Does NOT copy function_defs/types/routes -- inheritance via
    the recursive CTE makes parent defs visible, and personal overrides are
    just rows on the child. This is the right shape for user/personal. *)
let ensure_branch (t : t) ~(parent_id : string) ~(owner : string)
    ~(name : string) : string option Lwt.t =
  let* existing = lookup_branch_id_by_name t ~owner ~name in
  match existing with
  | Some id -> Lwt.return (Some id)
  | None ->
      (match t.conn with
       | None -> Lwt.return_none
       | Some conn ->
           Lwt.catch
             (fun () ->
               let sql =
                 "INSERT INTO branches (id, parent_id, owner, name) " ^
                 "VALUES (gen_random_uuid(), $1, $2, $3) " ^
                 "ON CONFLICT (owner, name) DO UPDATE SET name =
    EXCLUDED.name " ^
                 "RETURNING id::text" in
               let* rows =
                 Pg.execute conn
                   ~params:[ Pgx.Value.of_string parent_id;
                             Pgx.Value.of_string owner;
                             Pgx.Value.of_string name ]
                   sql
               in
               (match rows with
                | [ [ id_cell ] ] -> Lwt.return (Pgx.Value.to_string
    id_cell)
                | _ ->
                    (* Race: another image created it; re-lookup. *)
                    lookup_branch_id_by_name t ~owner ~name))
             (fun exn ->
               Printf.eprintf "[store] ensure_branch error: %s\n"
                 (Printexc.to_string exn);
               lookup_branch_id_by_name t ~owner ~name))

(* Fork a branch: create a new branch with parent_id as its parent,
   then copy all function_defs, type_defs, view_bindings from the
   parent into the new branch.

   Per spec [branching.fork]:
     (fork 'main 'alice/experiment)
   -> INSERT INTO branches (id, parent_id, owner, name) VALUES ...
   -> INSERT INTO function_defs (branch_id, name, body, ...)
      SELECT new_uuid, name, body, ... FROM function_defs
      WHERE branch_id = parent_id
   -> Same for type_defs, view_bindings

   Cost: O(num code rows in parent). For ~100 fns + ~20 types,
   a few hundred row inserts -- well under a second.

   Forking does NOT copy data table rows (those live in main's
   data tables and are shared across all branches).

   Returns the new branch_id as a string (or raises/fails-closed).
   The fork is one logical operation; the 3 copy INSERTs run in a
   single SERIALIZABLE transaction via [with_transaction] (it16).
   If any statement fails, the entire fork is rolled back. *)
(* Persist a route binding: (route "/path" 'handler-fn-name).
   INSERT ... ON CONFLICT (branch_id, route) DO UPDATE -- so re-defining
   a route overrides the existing binding. *)
let normalize_visibility (v : string) : string =
```

```
  match String.lowercase_ascii (String.trim v) with
  | "public" -> "public"
  | _ -> "private"

(** Persist a route. [visibility_opt]:
    - Some v ? set visibility (public|private)
    - None ? on INSERT default private; on UPDATE keep existing visibility
      (handler-only re-register does not clobber public). *)
let register_route (t : t) ~(route : string) ~(handler : string)
    ?visibility ()
  : unit Lwt.t =
  match t.conn with
  | None -> Lwt.return_unit
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql, params =
            match visibility with
            | Some vis ->
                let vis = normalize_visibility vis in
                ( "INSERT INTO view_bindings " ^
                  "  (branch_id, route, handler, visibility) " ^
                  "VALUES ($1, $2, $3, $4) " ^
                  "ON CONFLICT (branch_id, route) DO UPDATE " ^
                  "SET handler = EXCLUDED.handler, " ^
                  "    visibility = EXCLUDED.visibility"
                , [ Pgx.Value.of_string t.branch_id
                  ; Pgx.Value.of_string route
                  ; Pgx.Value.of_string handler
                  ; Pgx.Value.of_string vis ] )
            | None ->
                ( "INSERT INTO view_bindings " ^
                  "  (branch_id, route, handler, visibility) " ^
                  "VALUES ($1, $2, $3, 'private') " ^
                  "ON CONFLICT (branch_id, route) DO UPDATE " ^
                  "SET handler = EXCLUDED.handler"
                , [ Pgx.Value.of_string t.branch_id
                  ; Pgx.Value.of_string route
                  ; Pgx.Value.of_string handler ] )
          in
          let* _ = Pg.execute conn ~params sql in
          Lwt.return_unit)
        (fun exn ->
          Printf.eprintf "[store] register_route error: %s\n"
            (Printexc.to_string exn);
          Lwt.return_unit)

(* List all route bindings visible to this branch (recursive-CTE walk,
   same as list_function_names -- annotations from ancestors are visible
   via inheritance, child's definition OVERRIDES parent's). Returns
   (route, handler, visibility) ordered with closest ancestor first. *)
let list_routes (t : t) : (string * string * string) list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT vb.route, vb.handler, " ^
            "COALESCE(vb.visibility, 'private') " ^
            "FROM view_bindings vb " ^
            "JOIN chain ON vb.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql
```

```
          in
          Lwt.return
            (List.filter_map
               (fun row ->
                 match row with
                 | [ route_c; handler_c; vis_c ] ->
                     (match Pgx.Value.to_string route_c,
                            Pgx.Value.to_string handler_c with
                      | Some r, Some h ->
                          let vis =
                            match Pgx.Value.to_string vis_c with
                            | Some v -> normalize_visibility v
                            | None -> "private"
                          in
                          Some (r, h, vis)
                      | _ -> None)
                 | [ route_c; handler_c ] ->
                     (* Pre-migration fallback *)
                     (match Pgx.Value.to_string route_c,
                            Pgx.Value.to_string handler_c with
                      | Some r, Some h -> Some (r, h, "private")
                      | _ -> None)
                 | _ -> None)
               rows))
        (fun exn ->
          Printf.eprintf "[store] list_routes error: %s\n"
            (Printexc.to_string exn);
          Lwt.return [])

(* ---- type_defs ----

   Per spec [storage.types]: user declares a type with (deftype 'name
   field1 field2 ...). The declaration is INSERTed into type_defs with
   branch_id = current branch and fields = JSON array of strings (the
   field names, in declared order).

   The data DOESN'T live in a per-type table: packed in migration 003's
   note that generating one PG table per Lisp type would require
   runtime DDL breaking additive-migrations-only. Instead, all data
   rows are stored in the shared data_rows(type_name, fields jsonb)
   table, and type_defs is pure declaration metadata. (Type
   declarations MAY eventually guide field-level validation; for now
   they're documentation.)

   The recursive-CTE override walk applies to type_defs the same as
   function_defs / view_bindings: ancestors' types are visible, child's
   definition wins on name conflict. *)

let persist_type (t : t) ~(name : string) ~(fields : string list)
    ~(author : string) : unit Lwt.t =
  match t.conn with
  | None -> Lwt.return_unit
  | Some conn ->
      Lwt.catch
        (fun () ->
          let fields_json =
            Yojson.Safe.to_string
              (`List (List.map (fun s -> `String s) fields)) in
          let sql =
            "INSERT INTO type_defs (branch_id, name, fields, author) " ^
            "VALUES ($1, $2, $3::jsonb, $4) " ^
            "ON CONFLICT (branch_id, name) DO UPDATE " ^
            "SET fields = EXCLUDED.fields, author = EXCLUDED.author, " ^
            "    updated_at = now()" in
          let* _ =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string name;
                        Pgx.Value.of_string fields_json;
                        Pgx.Value.of_string author ]
              sql
          in
          Lwt.return_unit)
```

```
        (fun exn ->
          Printf.eprintf "[store] persist_type error: %s\n"
            (Printexc.to_string exn);
          Lwt.return_unit)

(* List all type declarations visible to this branch via the recursive
   CTE override walk. Returns (name, fields) pairs with closest
   ancestor first; caller-side Hashtbl first-occurrence dedup wins. *)
let list_types (t : t) : (string * string list) list Lwt.t =
  match t.conn with
  | None -> Lwt.return []
  | Some conn ->
      Lwt.catch
        (fun () ->
          let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT td.name, td.fields::text FROM type_defs td " ^
            "JOIN chain ON td.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC" in
          let* rows =
            Pg.execute conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql
          in
          Lwt.return
            (List.filter_map
               (fun row ->
                 match row with
                 | [ name_cell; fields_cell ] ->
                     (match Pgx.Value.to_string name_cell,
                            Pgx.Value.to_string fields_cell with
                      | Some name, Some fs ->
                          (try
                             let fs' =
                               match Yojson.Safe.from_string fs with
                               | `List lst ->
                                   List.filter_map
                                     (fun j ->
                                       match j with
                                       | `String s -> Some s
                                       | _ -> None)
                                     lst
                               | _ -> []
                             in Some (name, fs')
                           with _ -> None)
                      | _ -> None)
                 | _ -> None)
               rows))
        (fun exn ->
          Printf.eprintf "[store] list_types error: %s\n"
            (Printexc.to_string exn);
          Lwt.return [])

let fork_branch (t : t) ~(parent_id : string) ~(owner : string)
    ~(child_name : string) : string option Lwt.t =
  (* All four SQL statements (branch row + 3 copies) run inside a
     single SERIALIZABLE transaction via [with_transaction]. If any
     statement raises, pgx AUTOMATICALLY ROLLS BACK and propagates the
     exception to [with_transaction]'s caller. We catch the exception
     OUTSIDE [with_transaction] (after pgx has rolled back) so a partial
     fork doesn't leave a half-forked branch row in PG; the caller sees
     [None] with an [e]printf diagnostic, REPL stays alive.*)
  Lwt.catch
    (fun () ->
      with_transaction t (fun conn ->
        match conn with
        | None -> Lwt.return_none  (* no DB; can't fork *)
        | Some conn ->
```

```
            (* 1. Create the new branch row, returning its id. *)
            let insert_branch_sql =
              "INSERT INTO branches (id, parent_id, owner, name) " ^
              "VALUES (gen_random_uuid(), $1, $2, $3) " ^
              "RETURNING id::text" in
            let* rows =
              Pg.execute conn
                ~params:[ Pgx.Value.of_string parent_id;
                          Pgx.Value.of_string owner;
                          Pgx.Value.of_string child_name ]
                insert_branch_sql
            in
            (match rows with
             | [ [ id_cell ] ] ->
                 (match Pgx.Value.to_string id_cell with
                  | Some new_branch_id ->
                      (* 2. Copy function_defs from parent. *)
                      let copy_fns_sql =
                        "INSERT INTO function_defs " ^
                        "  (branch_id, name, body, author, updated_at) " ^
                        "SELECT $1, name, body, author, updated_at " ^
                        "FROM function_defs WHERE branch_id = $2" in
                      let* _ =
                        Pg.execute conn
                          ~params:[ Pgx.Value.of_string new_branch_id;
                                    Pgx.Value.of_string parent_id ]
                          copy_fns_sql
                      in
                      (* 3. Copy type_defs from parent. *)
                      let copy_types_sql =
                        "INSERT INTO type_defs " ^
                        "  (branch_id, name, fields, migrated, schema_vers,
    " ^
                        "   author, updated_at) " ^
                        "SELECT $1, name, fields, migrated, schema_vers, " ^
                        "       author, updated_at " ^
                        "FROM type_defs WHERE branch_id = $2" in
                      let* _ =
                        Pg.execute conn
                          ~params:[ Pgx.Value.of_string new_branch_id;
                                    Pgx.Value.of_string parent_id ]
                          copy_types_sql
                      in
                      (* 4. Copy view_bindings from parent (incl.
    visibility). *)
                      let copy_views_sql =
                        "INSERT INTO view_bindings " ^
                        "  (branch_id, route, handler, visibility) " ^
                        "SELECT $1, route, handler, " ^
                        "  COALESCE(visibility, 'private') " ^
                        "FROM view_bindings WHERE branch_id = $2" in
                      let* _ =
                        Pg.execute conn
                          ~params:[ Pgx.Value.of_string new_branch_id;
                                    Pgx.Value.of_string parent_id ]
                          copy_views_sql
                      in
                      Lwt.return (Some new_branch_id)
                  | None -> Lwt.return_none)
             | _ -> Lwt.return_none)))
      (fun exn ->
        Printf.eprintf "[store] fork_branch error: %s\n"
          (Printexc.to_string exn);
        Lwt.return_none)

(* Merge a source branch's function_defs into a target branch.

   Per spec [branching.merge]:
     (merge 'alice/experiment 'alice/main)
   -> copy all function_defs from source to target (upsert).
   Policy: admin-only merges to main; per-user can merge into their
   own branches. Policy is enforced at the Lisp intercept layer
   (session.ml); Store.merge_branch just does the SQL.
```

```
919
920     Initial implementation: copies ALL function_defs from source to
921     target via INSERT ... ON CONFLICT (branch_id, name) DO UPDATE.
922     TODO: diff-based merge (only fns whose updated_at > parent's
923     last_synced_with_child, OR whose name was added in child).
924     TODO: detect conflicts (parent ALSO modified same name since fork)
925     and per-function conflict resolution.
926     TODO: schema-merge for type_defs -- only main accepts schema changes
927     via special migration-governed merge.
928
929     Wrapped in a SERIALIZABLE transaction via [with_transaction] since
930     it16 (previously best-effort atomic -- carried-forward since it9). *)
931 let merge_branch (t : t) ~(source_id : string) ~(target_id : string)
932   : int option Lwt.t =
933   Lwt.catch
934     (fun () ->
935       with_transaction t (fun conn ->
936         match conn with
937         | None -> Lwt.return_none  (* no DB; can't merge *)
938         | Some conn ->
939             (* Upsert all function_defs from source into target. Returns
940                the count of merged rows via Pg's INSERT RETURNING. *)
941             let sql =
942               "INSERT INTO function_defs (branch_id, name, body, author,
    updated_at) " ^
943               "SELECT $1, name, body, author, updated_at " ^
944               "FROM function_defs WHERE branch_id = $2 " ^
945               "ON CONFLICT (branch_id, name) DO UPDATE " ^
946               "  SET body = EXCLUDED.body, author = EXCLUDED.author, " ^
947               "      updated_at = EXCLUDED.updated_at " ^
948               "RETURNING name" in
949             let* rows =
950               Pg.execute conn
951                 ~params:[ Pgx.Value.of_string target_id;
952                           Pgx.Value.of_string source_id ]
953                 sql
954             in
955             Lwt.return (Some (List.length rows))))
956     (fun exn ->
957       Printf.eprintf "[store] merge_branch error: %s\n"
958         (Printexc.to_string exn);
959       Lwt.return_none)
```

54 image/lib/store_db.ml

```
(** Pricklypear Image -- Store connection, config, transactions.
    Query/branch APIs live in [Store] (includes this module). *)
open Lwt.Syntax

(* A [Pgx_lwt.Io_intf.S] implementation backed by [Lwt_unix]. Pgx_lwt's
   Make functor requires this; it provides the low-level read/write/flush
   primitives over a TCP or Unix-domain socket. *)
module Io_unix = struct
  type in_channel = Lwt_io.input_channel
  type out_channel = Lwt_io.output_channel
  type sockaddr =
    | Unix of string
    | Inet of string * int

  let output_char = Lwt_io.write_char
  let output_string oc s = Lwt_io.write oc s
  let flush = Lwt_io.flush
  let input_char = Lwt_io.read_char
  let really_input ic buf pos len = Lwt_io.read_into_exactly ic buf pos len
  let close_in = Lwt_io.close
  let getlogin () = Lwt.return "pricklypear"
  let open_connection = function
    | Unix path ->
        let sock =
          Lwt_unix.socket Lwt_unix.PF_UNIX Lwt_unix.SOCK_STREAM 0 in
        let* () = Lwt_unix.connect sock (Lwt_unix.ADDR_UNIX path) in
        Lwt.return (Lwt_io.of_fd ~mode:Lwt_io.Input sock,
                    Lwt_io.of_fd ~mode:Lwt_io.Output sock)
    | Inet (host, port) ->
        let sock =
          Lwt_unix.socket Lwt_unix.PF_INET Lwt_unix.SOCK_STREAM 0 in
        let addr = Lwt_unix.ADDR_INET (Unix.inet_addr_of_string host, port)
     in
        let* () = Lwt_unix.connect sock addr in
        Lwt.return (Lwt_io.of_fd ~mode:Lwt_io.Input sock,
                    Lwt_io.of_fd ~mode:Lwt_io.Output sock)
end

(* Concrete pgx connection module, built with our Lwt_unix Io. *)
module Pg = Pgx_lwt.Make (Io_unix)

type config = {
  database : string;
  host : string;
  port : int;
  user : string;
  password : string;
}

let env_or name dft = match Sys.getenv_opt name with Some s -> s | None ->
     dft

let config_from_env () : config =
  {
    database = env_or "PP_DB_NAME" (env_or "PGDATABASE" "pricklypear");
    host = env_or "PP_DB_HOST" (env_or "PGHOST" "127.0.0.1");
    port = int_of_string (env_or "PP_DB_PORT" (env_or "PGPORT" "5433"));
    user = env_or "PP_DB_USER" (env_or "PGUSER" "pricklypear_image");
    password = env_or "PP_DB_PASS" "";
  }

type t = {
  cfg : config;
  mutable conn : Pg.t option;
  mutable branch_id : string;  (* (checkout 'branch) reassigns it *)
  (* Session-level dynamic transaction binding (option A from
     BORGE-1783438027, lands it29). Points to the transaction-scoped
     pgx connection while inside (with-transaction ...), so all DB
     prims called from within the body see the SAME connection (and
     thus the same transaction -- including row locks acquired by
     select-for-update). None when not inside a transaction, in
```

```
70      which case prims fall back to [conn_of] (the default pool conn). *)
71    mutable current_tx : Pg.t option;
72  }
73
74  let make ~cfg ~branch_id () : t = { cfg; conn = None; branch_id; current_tx
        = None }
75
76  (* Expose the optional pool connection to sibling modules. [conn_of]
77     always returns the long-lived pool connection; prims should prefer
78       [current_conn t]
79     which returns the in-transaction connection (if set) or falls
80     back to the pool conn. *)
81  let conn_of (t : t) : Pg.t option = t.conn
82
83  (* Active connection for DB prims to use: in-transaction connection if
84     we're inside a (with-transaction ...) body, else the default pool
85     connection. Returns None when there's no DB at all. *)
86  let current_conn (t : t) : Pg.t option =
87    match t.current_tx with
88    | Some tx -> Some tx
89    | None -> t.conn
90
91  (* Run [f] inside a Postgres transaction with SERIALIZABLE isolation
92     (per AGENTS.md / spec [storage.transactions]: SERIALIZABLE by
93     default for mutations). [f] receives the upgraded transactional
94     connection (pgx returns a new `t` representing the in-transaction
95     state); all execute calls against that connection participate in the
96     transaction.
97
98     If [f] raises, pgx automatically ROLLS BACK and propagates the
99     exception to with_transaction's caller. Per pgx's mli it's an error
100    to call commit/rollback manually inside [f].
101
102    On no-DB (t.conn = None), still calls [f] with None so the caller
103    can implement their no-DB fallback (rare case; stores without DB
104    aren't writable).
105
106    Resolves the carried-forward TODO on fork_branch and merge_branch.
107    Both now run all their SQL statements as a SINGLE atomic
108    transaction. *)
109 let with_transaction ?(isolation = Pgx.Isolation.Serializable)
110     (t : t) (f : Pg.t option -> 'a Lwt.t) : 'a Lwt.t =
111   match t.conn with
112   | None -> f None
113   | Some conn ->
114       Pg.with_transaction ~isolation conn (fun tx_conn -> f (Some tx_conn))
115
116 (** Session-scoped dynamic transaction binding (option A from
117     BORGE-1783438027, lands it29). Used by the (with-transaction ...)
118     Lisp prim: sets [t.current_tx] to the transactional pgx connection
119     BEFORE [f] runs, so DB prims called from within the body consult
120     [current_conn] (which they already use) and see the SAME
121     transaction -- including row locks acquired via
122     (select-for-update 'type id).
123
124     The cleanup at the end is a [finally]-shaped try-with that clears
125     [t.current_tx] whether [f] succeeds or raises. CRITICAL: the
126     error handler retries the body's value-type raise AFTER restoring
127     the [current_tx = None] invariant; pgx still sees the propagated
128     exception via its own finally and ROLLS BACK the transaction.
129
130     Nested transactions (a (with-transaction ...) inside another) are
131     NOT supported -- at the pgx level [Pg.with_transaction] on the
132     in-transaction conn would error. We raise [Interp_error] if
133     someone tries it. *)
134 let with_dynamic_transaction
135     ?(isolation = Pgx.Isolation.Serializable) (t : t)
136     (f : unit -> 'a Lwt.t) : 'a Lwt.t =
137   match t.conn with
138   | None -> f ()
139   | Some conn ->
140       if t.current_tx <> None then
141         Lwt.fail (Pricklypear_lisp.Ast.Interp_error
```

```
                    (Pricklypear_lisp.Ast.User_error
                       "with-transaction: nested transactions not
    supported"))
      else
        Pg.with_transaction ~isolation conn
          (fun tx_conn ->
             t.current_tx <- Some tx_conn;
             Lwt.finalize f
               (fun () -> t.current_tx <- None; Lwt.return_unit))


let connect (t : t) : unit Lwt.t =
  match t.conn with
  | Some _ -> Lwt.return_unit
  | None ->
      Lwt.catch
        (fun () ->
          let* conn =
            Pg.connect
              ~host:t.cfg.host ~port:t.cfg.port
              ~user:t.cfg.user ~password:t.cfg.password
              ~database:t.cfg.database ()
          in
          t.conn <- Some conn;
          Printf.eprintf "[store] connected to PG %s:%d/%s as %s\n"
            t.cfg.host t.cfg.port t.cfg.database t.cfg.user;
          Lwt.return_unit)
        (fun exn ->
          Printf.eprintf "[store] PG connect failed: %s\n"
            (Printexc.to_string exn);
          Lwt.return_unit)
```

55 image/bin/main.ml

```
(** Pricklypear Image -- entry point.

    A single plain OCaml / Lwt_unix binary (not a Mirage unikernel).
    All configuration is via env vars; see [Boot.manifest_from_env]
    and [Store.config_from_env]. *)
let () =
  Lwt_main.run (Pricklypear_image_lib.Boot.start ())
```

56 lisp/lib/ast.ml

```
(** Pricklypear Lisp -- AST.

    The AST IS the source (homoiconicity). Function bodies are stored as
    JSONB rows in [function_defs.body], parsed back into [expr] values by
    the reader and evaluated by [Eval].

    Per spec [lisp-interpreter.ast]: this is a Scheme-flavored Lisp-1
    (lexically scoped, first-class closures, primitives dispatched by
    name through a table the IMAGE layer registers at boot).

    The interpreter proper is PURE: it takes an env and an expr and
    returns a value or raises an [Interp_error]. No I/O here. Side
    effects go through [Prim] calls, wired to real Postgres/HTTP by the
    image layer. *)

(** A reference into the persistent store. Carries its target type so
    the interpreter can verify derefs at call time before querying. *)
type type_name = string
type id = string  (* uuid-ish; opaque to the interpreter *)

type expr =
  | Var of string
  | Lit of value
  | App of expr * expr list
  | Let of string * expr * expr
  | Letrec of (string * expr) list * expr
  | If of expr * expr * expr
  | Lambda of string list * expr
  | Prim of string * expr list
  | Seq of expr * expr
  | Quote of expr
  | Define of string * expr   (* top-level binding *)

(** Values are what evaluation produces. The interesting cases are
    [VRef] (a tagged reference into the store, so values can point into
    the persistent object graph without copying it) and [VLambda]
    (closures capture their defining env). *)
and value =
  | VInt of int64
  | VFloat of float                (* IEEE float; arithmetic promotes
     int+float *)
  | VStr of string
  | VBool of bool
  | VNil
  | VPair of value * value         (* cons cells *)
  | VLambda of string list * expr * env
  | VPrim of string                 (* primitive name; resolved at apply *)
  | VRef of type_name * id
  | VList of value list             (* sugar for nested pairs; reader
     interns *)

and env = {
  bindings : (string, value) Hashtbl.t;
  parent : env option;
}

(** Raised on interpreter errors. The REPL catches these, prints them
    with location if available, and continues the session. *)
type interp_error =
  | Unbound_var of string
  | Not_a_function of value
  | Arity_mismatch of string * int * int  (* (name, expected, got) *)
  | Wrong_arg_count of string * int * int
  | Type_mismatch of string * value  (* (expected, got) *)
  | Budget_exceeded
  | Prim_error of string * string    (* (prim_name, message) *)
  | Parse_error of string * int * int (* (message, line, col) *)
  | User_error of string

exception Interp_error of interp_error

```

```
let _ = Obj.repr (fun (_ : interp_error) -> ())  (* keep type referenced *)

(** Helper constructors for building AST in tests *)
let var s = Var s
let int n = Lit (VInt (Int64.of_int n))
let float f = Lit (VFloat f)
let str s = Lit (VStr s)
let bool b = Lit (VBool b)
let nil = Lit VNil

(** JSON serialization of [expr].

    Function bodies are stored as JSONB. Round-trips via [of_json]. *)
let rec expr_to_json (e : expr) : Yojson.Safe.t =
  match e with
  | Var s -> `Assoc [ ("var", `String s) ]
  | Lit v -> `Assoc [ ("lit", value_to_json v) ]
  | App (f, args) ->
      `Assoc [ ("app", `List (expr_to_json f :: List.map expr_to_json
     args)) ]
  | Let (name, val_expr, body) ->
      `Assoc [
        ("let", `List [ `String name; expr_to_json val_expr; expr_to_json
    body ]);
      ]
  | Letrec (bindings, body) ->
      `Assoc [
        ( "letrec",
          `List [
            `List (List.map (fun (n, e) -> `List [ `String n; expr_to_json
    e ]) bindings);
            expr_to_json body;
          ] );
      ]
  | If (test, then_e, else_e) ->
      `Assoc [ ("if", `List [ expr_to_json test; expr_to_json then_e;
     expr_to_json else_e ]) ]
  | Lambda (params, body) ->
      `Assoc [ ("lambda", `List [ `List (List.map (fun s -> `String s)
     params); expr_to_json body ]) ]
  | Prim (name, args) ->
      `Assoc [ ("prim", `List [ `String name; `List (List.map expr_to_json
     args) ]) ]
  | Seq (a, b) -> `Assoc [ ("seq", `List [ expr_to_json a; expr_to_json b
    ]) ]
  | Quote e -> `Assoc [ ("quote", expr_to_json e) ]
  | Define (name, val_expr) ->
      `Assoc [ ("define", `List [ `String name; expr_to_json val_expr ]) ]

and value_to_json (v : value) : Yojson.Safe.t =
  match v with
  | VInt n -> `Assoc [ ("int", `String (Int64.to_string n)) ]
  | VFloat f -> `Assoc [ ("float", `Float f) ]
  | VStr s -> `Assoc [ ("str", `String s) ]
  | VBool b -> `Assoc [ ("bool", `Bool b) ]
  | VNil -> `Assoc [ ("nil", `Null) ]
  | VPair (a, b) -> `Assoc [ ("pair", `List [ value_to_json a;
     value_to_json b ]) ]
  | VLambda _ -> `Assoc [ ("lambda", `String "#<lambda>") ]  (* closures
     aren't serializable *)
  | VPrim name -> `Assoc [ ("prim", `String name) ]
  | VRef (t, i) -> `Assoc [ ("ref", `List [ `String t; `String i ]) ]
  | VList vs -> `Assoc [ ("list", `List (List.map value_to_json vs)) ]

(** JSON deserialization of [expr]. Inverse of [expr_to_json]. *)
let rec expr_of_json (j : Yojson.Safe.t) : (expr, string) result =
  match j with
  | `Assoc [ ("var", `String s) ] -> Ok (Var s)
  | `Assoc [ ("lit", v) ] ->
      (match value_of_json v with
       | Ok v -> Ok (Lit v)
       | Error e -> Error e)
  | `Assoc [ ("app", `List (f :: args)) ] ->
```

```
      (match expr_of_json f with
       | Ok f ->
           let args' =
             List.fold_left
               (fun acc a ->
                 match acc, expr_of_json a with
                 | Ok lst, Ok e -> Ok (lst @ [ e ])
                 | Error e, _ -> Error e
                 | _, Error e -> Error e)
               (Ok []) args
           in
           (match args' with Ok lst -> Ok (App (f, lst)) | Error e -> Error e)
       | Error e -> Error e)
  | `Assoc [ ("let", `List [ `String name; v; b ]) ] ->
      (match expr_of_json v, expr_of_json b with
       | Ok v, Ok b -> Ok (Let (name, v, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("letrec", `List [ `List bindings; b ]) ] ->
      let bindings' =
        List.fold_left
          (fun acc bj ->
            match acc, bj with
            | Ok lst, `List [ `String n; e ] ->
                (match expr_of_json e with
                 | Ok e -> Ok (lst @ [ (n, e) ])
                 | Error e -> Error e)
            | Ok _, _ -> Error "letrec binding not a [name, expr] pair"
            | Error e, _ -> Error e)
          (Ok []) bindings
      in
      (match bindings', expr_of_json b with
       | Ok bs, Ok b -> Ok (Letrec (bs, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("if", `List [ t; th; el ]) ] ->
      (match expr_of_json t, expr_of_json th, expr_of_json el with
       | Ok t, Ok th, Ok el -> Ok (If (t, th, el))
       | Error e, _, _
       | _, Error e, _
       | _, _, Error e -> Error e)
  | `Assoc [ ("lambda", `List [ `List params; b ]) ] ->
      let params' =
        List.fold_left
          (fun acc p ->
            match acc, p with
            | Ok lst, `String s -> Ok (lst @ [ s ])
            | Ok _, _ -> Error "lambda param not a string"
            | Error e, _ -> Error e)
          (Ok []) params
      in
      (match params', expr_of_json b with
       | Ok ps, Ok b -> Ok (Lambda (ps, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("prim", `List [ `String name; `List args ]) ] ->
      let args' =
        List.fold_left
          (fun acc a ->
            match acc, expr_of_json a with
            | Ok lst, Ok e -> Ok (lst @ [ e ])
            | Error e, _ -> Error e
            | _, Error e -> Error e)
          (Ok []) args
      in
      (match args' with Ok lst -> Ok (Prim (name, lst)) | Error e -> Error e)
  | `Assoc [ ("seq", `List [ a; b ]) ] ->
      (match expr_of_json a, expr_of_json b with
       | Ok a, Ok b -> Ok (Seq (a, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("quote", e) ] ->
      (match expr_of_json e with Ok e -> Ok (Quote e) | Error e -> Error e)
  | `Assoc [ ("define", `List [ `String name; v ]) ] ->
      (match expr_of_json v with
```

```
      | Ok v -> Ok (Define (name, v))
      | Error e -> Error e)
  | _ -> Error ("unrecognized expr JSON: " ^ Yojson.Safe.to_string j)

and value_of_json (j : Yojson.Safe.t) : (value, string) result =
  match j with
  | `Assoc [ ("int", `String s) ] ->
      (try Ok (VInt (Int64.of_string s))
       with _ -> Error ("bad int: " ^ s))
  | `Assoc [ ("float", `Float f) ] -> Ok (VFloat f)
  | `Assoc [ ("float", `Int n) ] -> Ok (VFloat (float_of_int n))
  | `Assoc [ ("float", `String s) ] ->
      (try Ok (VFloat (float_of_string s))
       with _ -> Error ("bad float: " ^ s))
  | `Assoc [ ("str", `String s) ] -> Ok (VStr s)
  | `Assoc [ ("bool", `Bool b) ] -> Ok (VBool b)
  | `Assoc [ ("nil", `Null) ] -> Ok VNil
  | `Assoc [ ("pair", `List [ a; b ]) ] ->
      (match value_of_json a, value_of_json b with
       | Ok a, Ok b -> Ok (VPair (a, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("prim", `String name) ] -> Ok (VPrim name)
  | `Assoc [ ("ref", `List [ `String t; `String i ]) ] -> Ok (VRef (t, i))
  | `Assoc [ ("list", `List lst) ] ->
      let lst' =
        List.fold_left
          (fun acc v ->
            match acc, value_of_json v with
            | Ok l, Ok v -> Ok (l @ [ v ])
            | Error e, _ -> Error e
            | _, Error e -> Error e)
          (Ok []) lst
      in
      (match lst' with Ok lst -> Ok (VList lst) | Error e -> Error e)
  | _ -> Error ("unrecognized value JSON: " ^ Yojson.Safe.to_string j)
```

57 lisp/lib/effect.ml

```
(** Capability effects for Pricklypear Lisp programs.

    Separate from Lwt async plumbing and from OCaml 5 algebraic effects.
    See spec [lisp-interpreter.effects].

    Effects label what a term *needs* (capabilities), not when it runs. *)

open Ast

(** [effect] is an OCaml 5 keyword; use [eff]. *)
type eff =
  | Pure
  | ReadDb
  | WriteDb
  | Branch
  | Publish
  | Ai
  | Html
  | Txn

let eff_to_string = function
  | Pure -> "pure"
  | ReadDb -> "read-db"
  | WriteDb -> "write-db"
  | Branch -> "branch"
  | Publish -> "publish"
  | Ai -> "ai"
  | Html -> "html"
  | Txn -> "txn"

let eff_of_string = function
  | "pure" -> Some Pure
  | "read-db" -> Some ReadDb
  | "write-db" -> Some WriteDb
  | "branch" -> Some Branch
  | "publish" -> Some Publish
  | "ai" -> Some Ai
  | "html" -> Some Html
  | "txn" -> Some Txn
  | _ -> None

let eff_rank = function
  | Pure -> 0
  | Html -> 1
  | ReadDb -> 2
  | WriteDb -> 3
  | Txn -> 4
  | Publish -> 5
  | Branch -> 6
  | Ai -> 7

module EffSet = struct
  type t = eff list  (* unique, sorted by rank then name *)

  let empty : t = []

  let normalize (xs : eff list) : t =
    let xs = List.filter (function Pure -> false | _ -> true) xs in
    let xs =
      List.sort_uniq
        (fun a b ->
          let c = compare (eff_rank a) (eff_rank b) in
          if c <> 0 then c else String.compare (eff_to_string a)
     (eff_to_string b))
        xs
    in
    xs

  let singleton e = normalize [ e ]

  let union a b = normalize (a @ b)
```

```
71
72    let unions sets = List.fold_left union empty sets
73
74    let mem e set = List.mem e set
75
76    let is_empty = function [] -> true | _ -> false
77
78    let is_pure = is_empty
79
80    let subset (a : t) (b : t) : bool =
81      List.for_all (fun e -> mem e b) a
82
83    let to_list s = s
84
85    let pp (s : t) : string =
86      if is_pure s then "pure"
87      else String.concat "+" (List.map eff_to_string s)
88
89    let badge (s : t) : string =
90      (* short UI badge *)
91      if is_pure s then "pure"
92      else if mem Ai s then "ai"
93      else if mem Publish s then "page"
94      else if mem Branch s then "branch"
95      else if mem WriteDb s then "db-write"
96      else if mem ReadDb s then "db-read"
97      else if mem Html s then "html"
98      else pp s
99  end
100
101 type effect_set = EffSet.t
102
103 (** Default pure prim names (image may add more). *)
104 let pure_prims = [
105   "+"; "-"; "*"; "="; "<"; ">"; "<="; ">="; "not";
106   "list"; "car"; "cdr"; "cons"; "null?"; "pair?"; "count";
107   "string-append"; "str"; "type-of"; "as-str";
108   "string-eq"; "string-trim"; "string-length"; "substring";
        "string-contains?";
109   "html-escape"; "json-parse"; "json-emit";
110   "hx-request?"; "request-header";
111   "dict-get"; "dict-set"; "dict-keys";
112   "hx-get"; "hx-post";
113 ]
114
115 let read_db_prims = [
116   "get"; "rows"; "select-for-update";
117   "find-row"; "find-rows"; "find-text"; "find-random";
118   "list-branches"; "current-branch"; "list-bindings";
119   "whoami"; "current-author";
120   "function-source"; "package-bindings"; "export-package";
121 ]
122
123 let write_db_prims = [
124   "create"; "update"; "update-field"; "delete"; "deftype";
125 ]
126
127 let branch_prims = [ "fork"; "merge"; "checkout" ]
128 let publish_prims = [ "route" ]
129 let html_prims = [ "render-html" ]
130 (* Author tools that freeze into function_defs (Ai + WriteDb). *)
131 let ai_write_prims = [
132   "propose-function"; "install-function"; "agent-task";
133   "llm-inner"; "llm-outer"; "llm-rewrite"; "llm-seed";
134 ]
135 (* Portal that only talks to the model (Ai only). *)
136 let ai_prims = "agent" :: ai_write_prims
137
138 let effects_for_prim (name : string) : effect_set =
139   if List.mem name pure_prims then EffSet.empty
140   else if List.mem name read_db_prims then EffSet.singleton ReadDb
141   else if List.mem name write_db_prims then EffSet.singleton WriteDb
142   else if List.mem name branch_prims then EffSet.singleton Branch
```

```
  else if List.mem name publish_prims then EffSet.singleton Publish
  else if List.mem name html_prims then EffSet.singleton Html
  else if name = "agent" then EffSet.singleton Ai
  else if List.mem name ai_write_prims then
    EffSet.union (EffSet.singleton Ai) (EffSet.singleton WriteDb)
  else if name = "with-transaction" || name = "__with_transaction__" then
     EffSet.singleton Txn
  else
    (* Unknown name: not a registered capability prim. User fns resolved
       via known_fn; bare unknowns contribute nothing (avoid painting every
       call as db-write). *)
    EffSet.empty

type prim_effect_lookup = string -> effect_set
type known_fn_lookup = string -> effect_set option

let default_prim_lookup = effects_for_prim

(** Conservative AST effect inference.
    [known_fn] resolves user function names when known; unknown globals
    contribute no extra effects (overapprox only on prims). Higher-order
    unknown application does not explode to top -- callers may wrap. *)
let rec infer
    ?(prims = default_prim_lookup)
    ?(known_fn : known_fn_lookup = fun _ -> None)
    (e : expr) : effect_set =
  let go = infer ~prims ~known_fn in
  match e with
  | Var name ->
      (match known_fn name with
       | Some s -> s
       | None ->
           (* bare var might be a prim bound in env *)
           if List.mem name pure_prims
              || List.mem name read_db_prims
              || List.mem name write_db_prims
              || List.mem name branch_prims
              || List.mem name publish_prims
              || List.mem name html_prims
              || List.mem name ai_prims
           then prims name
           else EffSet.empty)
  | Lit _ -> EffSet.empty
  | Quote _ -> EffSet.empty
  | Lambda (_params, body) -> go body
  | If (t, a, b) -> EffSet.unions [ go t; go a; go b ]
  | Let (_n, v, body) -> EffSet.union (go v) (go body)
  | Letrec (bindings, body) ->
      let bs = List.map (fun (_, e') -> go e') bindings in
      EffSet.unions (go body :: bs)
  | Seq (a, b) -> EffSet.union (go a) (go b)
  | Define (_n, v) -> go v
  | Prim (name, args) ->
      EffSet.union (prims name) (EffSet.unions (List.map go args))
  | App (head, args) ->
      let head_fx =
        match head with
        | Var name ->
            (match known_fn name with
             | Some s -> s
             | None -> prims name)
        | _ -> go head
      in
      let special =
        match head with
        | Var "with-transaction" -> EffSet.singleton Txn
        | _ -> EffSet.empty
      in
      EffSet.unions (head_fx :: special :: List.map go args)

(** Default AI allowlist for generated code (install_function bodies).

    WriteDb is allowed so agents can build real apps (create/update/delete/
```

```
    deftype on data_rows). Publish (route) stays denied here -- use the
    dedicated register_route tool so route registration is explicit and
    not smuggled inside a lambda body. Elevate with PP_AI_ALLOW_WRITE for
    Branch/Publish/Ai extras. *)
let ai_default_allow : effect_set =
  EffSet.normalize [ ReadDb; WriteDb; Html ]

let policy_allows ~(allow : effect_set) (got : effect_set) : bool =
  (* Pure always allowed; every effect in got must be in allow,
     except Pure which is absent from sets. WriteDb not allowed unless in
     allow. *)
  EffSet.subset got allow
  || (EffSet.is_pure got)
```

58 lisp/lib/env.ml

```
(** Pricklypear Lisp -- Lexical environment.

    Lexically scoped bindings. Each env has a parent (set at closure
    capture time) and a hashtable of name -> value. Lookup walks up the
    parent chain.

    Per spec [lisp-interpreter.eval]: the evaluator is pure w.r.t. I/O;
    the env is mutable (binding updates), which is fine -- it's local to
    a session, not persisted. Persistence happens via [(define ...)]
    which, in the IMAGE layer, ALSO writes to the store. *)
open Ast

type env = Ast.env

let create ?(parent = None) () : env =
  { bindings = Hashtbl.create 16; parent }

let child (e : env) : env = create ~parent:(Some e) ()

let rec lookup (e : env) (name : string) : value option =
  match Hashtbl.find_opt e.bindings name with
  | Some v -> Some v
  | None ->
      (match e.parent with
       | Some p -> lookup p name
       | None -> None)

let define (e : env) (name : string) (v : value) : unit =
  Hashtbl.replace e.bindings name v

let set_existing (e : env) (name : string) (v : value) : bool =
  (* Like [define] but only succeeds if the name is already bound
     somewhere in the parent chain. Returns false if not bound. *)
  let rec find_env e =
    match Hashtbl.find_opt e.bindings name with
    | Some _ -> Some e
    | None -> Option.bind e.parent find_env
  in
  match find_env e with
  | Some e' -> Hashtbl.replace e'.bindings name v; true
  | None -> false

(* Walk all bindings in [e] and its ancestor chain, applying [f].
   Used by reflection primitives like (list-bindings). *)
let rec fold (e : env) (f : string -> value -> 'a -> 'a) (acc : 'a) : 'a =
  let local = Hashtbl.fold f e.bindings acc in
  match e.parent with
  | Some p -> fold p f local
  | None -> local
```

59 lisp/lib/eval.ml

```
(** Pricklypear Lisp -- Evaluator.

    Lwt-typed. Takes an env and an expr, returns a value (or raises
    [Interp_error]) wrapped in a promise `value Lwt.t`. All side effects
    go through [Prim] calls, which the IMAGE layer wires to real
    Postgres/HTTP. The interpreter itself does no I/O; pure primitives
    just wrap their results in [Lwt.return].

    Per spec [lisp-interpreter.eval]:

      - Cooperative evaluation: the eval loop is stepped with a
        [Budget.t] counter checked at every Prim invocation and every
        recursive eval depth. When exceeded, that ONE call aborts; the
        session continues. Prevents "user's bad function wedges the image".

      - Special forms [lambda/let/letrec/if/define/quote] handled in
        eval, not the reader.

      - [define] at top-level adds binding to the session env AND
        (in the IMAGE layer) persists to the store. Here it just binds;
        the image layer's primitive wrapper handles persistence.

      - Primitives are dispatched by name through a [prim_table] the
        IMAGE layer registers at boot. The interpreter is Lwt-typed so
        that DB-touching primitives (fork, merge, get, create, etc.)
        can return promises directly, with no special-case intercept
        machinery at the REPL. Pure primitives wrap their results in
        [Lwt.return].

    Per directive: threading Lwt through eval is the unblock for the
    data half of the OS. Reads (`(get 'type id)`) need the DB result as
    the return value; the sync ABI `value list -> value` could not
    provide that. *)
open Lwt
open Lwt.Syntax
open Ast

(** A budget tracks a per-call step limit. Counted down at each eval
    step. When zero, raises [Interp_error Budget_exceeded]. *)
type budget = {
  mutable steps : int;
  limit : int;
}

let default_limit = 100_000

let make_budget ?(limit = default_limit) () : budget = { steps = 0; limit }

let tick (b : budget) : unit =
  b.steps <- b.steps + 1;
  if b.steps > b.limit then raise (Interp_error Budget_exceeded)

(** A primitive: takes evaluated args, returns a value promise.
    Pure primitives wrap results in [Lwt.return]; DB-touching
    primitives return the DB promise directly. *)
type prim_fn = value list -> value Lwt.t

type prim_table = (string, prim_fn) Hashtbl.t

let make_prim_table () : prim_table = Hashtbl.create 32
let register_prim (tbl : prim_table) (name : string) (fn : prim_fn) : unit =
  Hashtbl.replace tbl name fn
let lookup_prim (tbl : prim_table) (name : string) : prim_fn option =
  Hashtbl.find_opt tbl name

(** Special-form recognition: an [App] whose head is [Var name] for one of
    these names is dispatched as the special form, not as a function call.
     *)
let is_special_form (name : string) : bool =
  match name with
  | "lambda" | "let" | "letrec" | "if" | "define" | "quote" | "seq"
```

```
  | "prim" | "with-transaction" -> true
  | _ -> false

(* Quote any expr into a first-class value (full s-expr quote).
   Compounds become nested VLists so quoted data is usable in Lisp. *)
let rec quote_to_value (e : expr) : value =
  match e with
  | Lit v -> v
  | Var s -> VStr s
  | App (h, args) ->
      VList (quote_to_value h :: List.map quote_to_value args)
  | Quote e' -> VList [ VStr "quote"; quote_to_value e' ]
  | If (t, th, el) ->
      VList [ VStr "if"; quote_to_value t; quote_to_value th;
    quote_to_value el ]
  | Lambda (params, body) ->
      VList [ VStr "lambda"
            ; VList (List.map (fun p -> VStr p) params)
            ; quote_to_value body ]
  | Let (name, v, body) ->
      VList [ VStr "let"
            ; VList [ VList [ VStr name; quote_to_value v ] ]
            ; quote_to_value body ]
  | Letrec (bindings, body) ->
      VList [ VStr "letrec"
            ; VList (List.map (fun (n, e') ->
                VList [ VStr n; quote_to_value e' ]) bindings)
            ; quote_to_value body ]
  | Define (name, v) ->
      VList [ VStr "define"; VStr name; quote_to_value v ]
  | Seq (a, b) ->
      VList [ VStr "seq"; quote_to_value a; quote_to_value b ]
  | Prim (name, args) ->
      VList (VStr name :: List.map quote_to_value args)

let rec eval
  ~(budget : budget)
  ~(prims : prim_table)
  (env : env)
  (e : expr)
  : value Lwt.t =
  tick budget;
  match e with
  | Var name ->
      (match Env.lookup env name with
       | Some v -> Lwt.return v
       | None -> raise (Interp_error (Unbound_var name)))
  | Lit v -> Lwt.return v
  | Quote e' ->
      (* Quote any s-expr into a value form (BORGE-1783383905):
         Lit -> value, Var -> VStr name, App -> VList of quoted parts,
         other expr constructors -> nested VList encoding. *)
      Lwt.return (quote_to_value e')
  | If (test, then_e, else_e) ->
      let* v = eval ~budget ~prims env test in
      (match v with
       | VBool b -> if b then eval ~budget ~prims env then_e
                     else eval ~budget ~prims env else_e
       | VNil -> eval ~budget ~prims env else_e
       | _ -> eval ~budget ~prims env then_e)  (* non-nil, non-false is
    truthy *)
  | Seq (a, b) ->
      let* _ = eval ~budget ~prims env a in
      eval ~budget ~prims env b
  | Let (name, val_expr, body) ->
      let* v = eval ~budget ~prims env val_expr in
      let env' = Env.child env in
      Env.define env' name v;
      eval ~budget ~prims env' body
  | Letrec (bindings, body) ->
      let env' = Env.child env in
      (* pre-bind all names to VNil, then evaluate each binding *)
      List.iter (fun (n, _) -> Env.define env' n VNil) bindings;
```

```
      let* _ =
        Lwt_list.iter_s
          (fun (n, e) -> let* v = eval ~budget ~prims env' e in
                         Env.define env' n v; Lwt.return_unit)
          bindings
      in
      eval ~budget ~prims env' body
  | Lambda (params, body) -> Lwt.return (VLambda (params, body, env))
  | Define (name, val_expr) ->
      let* v = eval ~budget ~prims env val_expr in
      Env.define env name v;
      Lwt.return v  (* define returns the bound value *)
  | Prim (name, args) ->
      (* Evaluate args left-to-right in sequence (let* over list). *)
      let* arg_vals =
        Lwt_list.map_s (eval ~budget ~prims env) args
      in
      (match lookup_prim prims name with
       | Some fn -> fn arg_vals
       | None -> raise (Interp_error (Prim_error (name, "unregistered
    primitive"))))
  | App (head, args) ->
      (* First: is this a special form? Special forms are recognized
         when the head is a [Var] naming one. Note: [args] here is the
         list AFTER the head; we prepend head when dispatching so the
         special-form handler sees the full argument list. *)
      (match head with
       | Var name when is_special_form name ->
           eval_special_form ~budget ~prims env name (head :: args)
       | _ ->
           (* ordinary application *)
           let* f = eval ~budget ~prims env head in
           let* arg_vals = Lwt_list.map_s (eval ~budget ~prims env) args in
           apply ~budget ~prims f arg_vals)

and eval_special_form
  ~(budget : budget)
  ~(prims : prim_table)
  (env : env)
  (name : string)
  (full_args : expr list)
  : value Lwt.t =
  (* these mirror the [Let/Letrec/If/Seq/Lambda/Define/Quote] expr cases,
     but for the surface-syntax App form (when read from text). The
     reader produces App(Var "if", [t; th; el]) rather than If(...). *)
  match name, full_args with
  | "lambda", [ _head; params_expr; body ] ->
      (* full_args = [Var "lambda"; params_expr; body]. Params can be:
         - ()  -> Lit VNil (zero params)
         - (p1 p2 ...) -> App(Var p1, [Var p2; ...]) -- collect head + args
    *)
      let params =
        match params_expr with
        | Lit VNil -> []
        | App (Var p, rest) ->
            (* head is the first param, rest are the rest *)
            p :: List.map (fun e ->
              match e with
              | Var s -> s
              | _ -> raise (Interp_error (User_error "lambda param not a
    symbol")))
              rest
        | _ -> raise (Interp_error (User_error "malformed lambda params"))
      in
      Lwt.return (VLambda (params, body, env))
  | "let", _ ->
      (* (let ((name val) ...) body) -- full_args = [Var "let"; bindings;
    body] *)
      (match full_args with
       | [ _; bindings_expr; body ] ->
           (match bindings_expr with
            | Lit VNil ->
                (* (let () body) *)
```

```
              eval ~budget ~prims env body
          | App (App (Var _, _) as first_binding, rest_bindings) ->
              (* bindings list: head is the first binding, rest are more
   *)
              let all_bindings = first_binding :: rest_bindings in
              let env' = Env.child env in
              let* _ =
                Lwt_list.iter_s
                  (fun bp ->
                    match bp with
                    | App (Var name, [ val_expr ]) ->
                        let* v = eval ~budget ~prims env val_expr in
                        Env.define env' name v;
                        Lwt.return_unit
                    | _ -> raise (Interp_error (User_error "bad let
   binding")))
                  all_bindings
              in
              eval ~budget ~prims env' body
          | App (Var name, [ val_expr ]) when full_args = [] ->
              (* single binding without outer parens -- not standard;
   skip *)
              raise (Interp_error (User_error "let bindings missing"))
          | _ -> raise (Interp_error (User_error "bad let bindings")))
      | _ -> raise (Interp_error (User_error "let needs bindings + body")))
  | "if", [ _; test; then_e; else_e ] ->
      let* v = eval ~budget ~prims env test in
      (match v with
       | VBool b -> eval ~budget ~prims env (if b then then_e else else_e)
       | VNil -> eval ~budget ~prims env else_e
       | _ -> eval ~budget ~prims env then_e)
  | "define", [ _; Var name; val_expr ] ->
      let* v = eval ~budget ~prims env val_expr in
      Env.define env name v;
      Lwt.return v
  | "quote", [ _; e' ] ->
      Lwt.return (quote_to_value e')
  | "seq", [ _; a; b ] ->
      let* _ = eval ~budget ~prims env a in
      eval ~budget ~prims env b
  | "prim", _ ->
      (* (prim name args...) -- full_args = [Var "prim"; Var name; args...]
   *)
      (match full_args with
       | [ _; Var prim_name; body ] ->
           let* arg_vals =
             Lwt_list.map_s (eval ~budget ~prims env) [body]
           in
           (match lookup_prim prims prim_name with
            | Some fn -> fn arg_vals
            | None -> raise (Interp_error (Prim_error (prim_name,
   "unregistered primitive"))))
       | _ -> raise (Interp_error (User_error "prim needs name + arg")))
  | "letrec", _ ->
      raise (Interp_error (User_error "letrec via App form not yet
   supported"))
  | "with-transaction", _ ->
      (* (with-transaction (lambda () body...)) -> runs the lambda body
         inside the dynamic transaction binding established by the
         image layer's private __with_transaction__ prim. The prim
         receives the VLambda value, calls it via Eval.apply inside
         Store.with_dynamic_transaction (so prims called from the body
         consult the in-transaction pgx conn via current_conn).

         Optional isolation arg (v1.5): (with-transaction 'read-committed
         ...) -- currently ignored, defaults to SERIALIZABLE.

         We DON'T handle the body directly in lisp/ (the transactional
         binding is I/O -- pgx -- lives in the image layer); we delegate
         to the registered __with_transaction__ prim. If no image-layer
         prim is registered (e.g. pure lisp tests), we raise a clean
         error so the absence is observable. *)
      (match full_args with
```

```
      | [ _; body_expr ] ->
          let* lambda_v = eval ~budget ~prims env body_expr in
          (match lambda_v with
           | VLambda _ ->
               (match lookup_prim prims "__with_transaction__" with
                | Some fn -> fn [lambda_v]
                | None ->
                    raise (Interp_error
                             (Prim_error ("with-transaction",
                                          "no __with_transaction__ prim registered"))))
           | _ ->
               raise (Interp_error
                        (User_error "with-transaction body must be a lambda")))
      | [ _; _isolation_arg; body_expr ] ->
          (* (with-transaction 'isolation body) -- isolation honored in v1.5 *)
          let* lambda_v = eval ~budget ~prims env body_expr in
          (match lambda_v with
           | VLambda _ ->
               (match lookup_prim prims "__with_transaction__" with
                | Some fn -> fn [lambda_v]
                | None ->
                    raise (Interp_error
                             (Prim_error ("with-transaction",
                                          "no __with_transaction__ prim registered"))))
           | _ ->
               raise (Interp_error
                        (User_error "with-transaction body must be a lambda")))
      | _ ->
          raise (Interp_error
                   (User_error "with-transaction needs a lambda body")))
  | _ ->
      raise (Interp_error (User_error ("malformed special form: " ^ name)))

and apply
  ~(budget : budget)
  ~(prims : prim_table)
  (f : value)
  (args : value list)
  : value Lwt.t =
  match f with
  | VLambda (params, body, clo_env) ->
      let expected = List.length params
      and got = List.length args in
      if expected <> got then
        raise (Interp_error (Arity_mismatch ("lambda", expected, got)));
      let call_env = Env.child clo_env in
      List.iter2 (fun p a -> Env.define call_env p a) params args;
      eval ~budget ~prims call_env body
  | VPrim name ->
      (match lookup_prim prims name with
       | Some fn -> fn args
       | None -> raise (Interp_error (Prim_error (name, "unregistered primitive"))))
  | other -> raise (Interp_error (Not_a_function other))

(** Convenience: evaluate an s-expression string with a fresh budget. *)
let eval_string
  ~(prims : prim_table)
  (env : env)
  (s : string)
  : value Lwt.t =
  let budget = make_budget () in
  let e = Reader.read_one s in
  eval ~budget ~prims env e
```

60 lisp/lib/reader.ml

```
(** Pricklypear Lisp -- S-expression reader.

    Parses surface s-expression syntax -> Ast.expr. The AST IS the
    source (homoiconicity): a function stored as JSONB is its AST, and
    the s-expr syntax is one of several possible editors (along with
    the htmx tree-editor that edits the AST directly).

    Per spec [lisp-interpreter.reader]:
      - Standard s-expressions: (foo bar baz), 'symbol, "string", 123,
        #t, #f, ()
      - Comments: ; to end of line
      - Docstrings: a leading string in a body position (deferred; reader
        just produces the AST, docstring extraction is eval-time)
      - Special forms [lambda/let/letrec/if/define/quote] are recognized
        at EVAL time, not read time -- the reader produces plain App
        nodes; eval dispatches them.

    The reader tracks line/col for error messages. *)
open Ast

type reader_error = string * int * int  (* (message, line, col) *)
exception Reader_error of reader_error

let is_space c = c = ' ' || c = '\t' || c = '\n' || c = '\r'
let is_digit c = c >= '0' && c <= '9'
let is_symbol_start c =
  (c <> '(' && c <> ')' && c <> '\'' && c <> '"'
  && c <> ';' && not (is_space c))
let is_symbol_cont c = is_symbol_start c || is_digit c || c = '#'

(* # is allowed as a token start for #t/#f (and later #\char etc). *)
let is_token_start c = c = '#' || is_symbol_start c

let is_token_cont c = is_symbol_cont c

type state = {
  mutable input : string;
  mutable pos : int;
  mutable line : int;
  mutable col : int;
}

let make_state (s : string) : state =
  { input = s; pos = 0; line = 1; col = 1 }

let peek (st : state) : char option =
  if st.pos >= String.length st.input then None
  else Some st.input.[st.pos]

let advance (st : state) : char option =
  match peek st with
  | None -> None
  | Some c ->
      st.pos <- st.pos + 1;
      if c = '\n' then (st.line <- st.line + 1; st.col <- 1)
      else st.col <- st.col + 1;
      Some c

let skip_ws_and_comments (st : state) : unit =
  let rec loop () =
    match peek st with
    | None -> ()
    | Some ' ' | Some '\t' | Some '\n' | Some '\r' ->
        ignore (advance st); loop ()
    | Some ';' ->
        (* line comment *)
        (match peek st with
         | Some '\n' | None -> ()
         | _ -> ignore (advance st); loop ())
    | _ -> ()
  in
```

```
  loop ()

let read_string (st : state) : string =
  (* assumes current char is the opening quote *)
  ignore (advance st);  (* consume the opening quote *)
  let buf = Buffer.create 16 in
  let rec loop () =
    match peek st with
    | None ->
        raise (Reader_error ("unterminated string", st.line, st.col))
    | Some '"' ->
        ignore (advance st);
        Buffer.contents buf
    | Some '\\' ->
        (* Escape: consume the backslash, then the next char is the
           escape letter. Two-step: advance past '\\', then peek+advance
           the escape char (NOT (match advance st with ...) -- that
           would match the '\\' we just peeked, not the next char). *)
        ignore (advance st);  (* consume '\\' (we peeked it) *)
        (match peek st with
         | None ->
             raise (Reader_error ("unterminated escape", st.line, st.col))
         | Some 'n' -> ignore (advance st); Buffer.add_char buf '\n'
         | Some 't' -> ignore (advance st); Buffer.add_char buf '\t'
         | Some '\\' -> ignore (advance st); Buffer.add_char buf '\\'
         | Some '"' -> ignore (advance st); Buffer.add_char buf '"'
         | Some c ->
             raise (Reader_error ("bad escape \\" ^ String.make 1 c,
    st.line, st.col)));
        loop ()
    | Some c ->
        Buffer.add_char buf c;
        ignore (advance st);
        loop ()
  in
  loop ()

let read_atom (st : state) : expr =
  (* reads a non-list, non-quote atom. Returns an [expr]: [Var] for bare
     symbols (which eval will look up), [Lit] for numbers, booleans,
     and (already-read) strings. *)
  let buf = Buffer.create 8 in
  let rec loop () =
    match peek st with
    | Some c when is_symbol_cont c ->
        Buffer.add_char buf c;
        ignore (advance st);
        loop ()
    | _ -> ()
  in
  loop ();
  let s = Buffer.contents buf in
  if s = "" then
    raise (Reader_error ("empty token", st.line, st.col))
  else if s = "#t" then Lit (VBool true)
  else if s = "#f" then Lit (VBool false)
  else
    (* Numbers: int first; if token has '.' or exponent, try float.
       Bare symbols otherwise (Var). *)
    let has_dot_or_exp =
      String.contains s '.'
      || String.contains s 'e'
      || String.contains s 'E'
    in
    if has_dot_or_exp then
      (try Lit (VFloat (float_of_string s))
       with _ -> Var s)
    else
      (try Lit (VInt (Int64.of_string s))
       with _ -> Var s)

let rec read_list (st : state) : expr list =
  (* current char is '('; consume it *)
```

```
  ignore (advance st);
  let rec loop acc =
    skip_ws_and_comments st;
    match peek st with
    | None ->
        raise (Reader_error ("unterminated list", st.line, st.col))
    | Some ')' ->
        ignore (advance st);
        List.rev acc
    | Some _ ->
        let e = read_expr st in
        loop (e :: acc)
  in
  loop []

and read_expr (st : state) : expr =
  skip_ws_and_comments st;
  match peek st with
  | None -> raise (Reader_error ("unexpected end of input", st.line,
     st.col))
  | Some '(' ->
      (* list -> App or special form (handled at eval; reader produces App
         with the first element as operator, or Lit for empty list) *)
      let items = read_list st in
      (match items with
       | [] -> Lit VNil  (* () is nil *)
       | op :: args -> App (op, args))
  | Some '\'' ->
      (* quote sugar: 'x -> (quote x) *)
      ignore (advance st);
      let e = read_expr st in
      App (Var "quote", [ e ])
  | Some '"' -> Lit (VStr (read_string st))
  | Some c when is_token_start c ->
      (* atom: symbol (-> Var), number, or #t/#f (-> Lit) *)
      read_atom st
  | Some c ->
      raise (Reader_error
               ("unexpected character " ^ String.make 1 c, st.line, st.col))

(** [read_one s] reads exactly one s-expression from [s]. Raises
    [Reader_error] on parse error or if there's trailing junk after the
    expression. *)
let read_one (s : string) : expr =
  let st = make_state s in
  let e = read_expr st in
  skip_ws_and_comments st;
  (match peek st with
   | None -> ()
   | Some _ ->
       raise (Reader_error ("trailing input after expression", st.line,
     st.col)));
  e

(** [read_all s] reads as many top-level expressions as are in [s]. *)
let read_all (s : string) : expr list =
  let st = make_state s in
  let rec loop acc =
    skip_ws_and_comments st;
    match peek st with
    | None -> List.rev acc
    | Some _ ->
        let e = read_expr st in
        loop (e :: acc)
  in
  loop []

(** [is_complete s] returns true if [s] has balanced parens and closed
    strings, i.e. the REPL can evaluate it. Used by the REPL to know
    whether to prompt for more input vs. evaluate now. *)
let is_complete (s : string) : bool =
  let st = make_state s in
  let depth = ref 0 in
```

```
  let in_string = ref false in
  let in_comment = ref false in
  try
    while peek st |> Option.is_some do
      let c = Option.get (peek st) in
      if !in_comment then begin
        if c = '\n' then in_comment := false;
        ignore (advance st)
      end else if !in_string then begin
        (match c with
         | '"' -> in_string := false
         | _ -> ());
        ignore (advance st)
      end else begin
        (match c with
         | ';' -> in_comment := true
         | '"' -> in_string := true
         | '(' -> incr depth
         | ')' -> decr depth
         | _ -> ());
        ignore (advance st)
      end
    done;
    not !in_string && !depth = 0
    (* in_comment at EOF is fine: a line comment is implicitly
       closed by end-of-input. Only an unclosed string or
       unbalanced parens means "need more input". *)
  with _ -> false

(** Pretty-print a value for the REPL. *)
let rec pp_value (v : value) : string =
  match v with
  | VInt n -> Int64.to_string n
  | VFloat f ->
      (* Prefer a readable form; ensure a decimal so re-read is float. *)
      let s = Printf.sprintf "%.12g" f in
      if String.contains s '.' || String.contains s 'e' || String.contains
     s 'E'
      then s
      else s ^ ".0"
  | VStr s -> "\"" ^ String.escaped s ^ "\""
  | VBool b -> if b then "#t" else "#f"
  | VNil -> "()"
  | VPair (a, b) ->
      (* Conventional list printer: a chain of VPairs ending in VNil
         prints as (a b c ...); an improper tail prints as
         (a b . tail). A VList tail is treated as the proper
         continuation (VList is just sugar for nested pairs). *)
      let rec collect acc = function
        | VPair (x, rest) -> collect (x :: acc) rest
        | VNil -> Some (List.rev acc, None)
        | VList items -> Some (List.rev acc @ items, None)
        | tail -> Some (List.rev acc, Some tail)
      in
      (match collect [] (VPair (a, b)) with
       | Some (items, None) ->
           "(" ^ String.concat " " (List.map pp_value items) ^ ")"
       | Some (items, Some tail) ->
           "(" ^ String.concat " " (List.map pp_value items)
           ^ " . " ^ pp_value tail ^ ")"
       | None -> assert false)
  | VLambda _ -> "#<lambda>"
  | VPrim name -> "#<prim:" ^ name ^ ">"
  | VRef (t, i) -> "#<ref:" ^ t ^ ":" ^ String.sub i 0 (min 8
     (String.length i)) ^ ">"
  | VList vs -> "(" ^ String.concat " " (List.map pp_value vs) ^ ")"

(** Flat one-line print (for length checks / compact leaves). *)
let rec pp_expr_flat (e : expr) : string =
  match e with
  | Var s -> s
  | Lit v -> pp_value v
  | App (f, args) ->
```

```
      "(" ^ pp_expr_flat f
      ^ (if args = [] then ""
         else " " ^ String.concat " " (List.map pp_expr_flat args))
      ^ ")"
  | Let (n, v, b) ->
      "(let ((" ^ n ^ " " ^ pp_expr_flat v ^ ")) " ^ pp_expr_flat b ^ ")"
  | Letrec (bs, b) ->
      "(letrec ("
      ^ String.concat " "
          (List.map (fun (n, e) -> "(" ^ n ^ " " ^ pp_expr_flat e ^ ")") bs)
      ^ ") " ^ pp_expr_flat b ^ ")"
  | If (t, th, el) ->
      "(if " ^ pp_expr_flat t ^ " " ^ pp_expr_flat th ^ " " ^ pp_expr_flat
    el
      ^ ")"
  | Lambda (ps, b) ->
      "(lambda (" ^ String.concat " " ps ^ ") " ^ pp_expr_flat b ^ ")"
  | Prim (n, args) ->
      "(prim " ^ n
      ^ (if args = [] then ""
         else " " ^ String.concat " " (List.map pp_expr_flat args))
      ^ ")"
  | Seq (a, b) -> "(seq " ^ pp_expr_flat a ^ " " ^ pp_expr_flat b ^ ")"
  | Quote e -> "'" ^ pp_expr_flat e
  | Define (n, e) -> "(define " ^ n ^ " " ^ pp_expr_flat e ^ ")"

let pp_indent (depth : int) : string = String.make (depth * 2) ' '

(** Prefer one line when short and shallow; otherwise break. *)
let rec expr_is_compact (e : expr) : bool =
  let flat = pp_expr_flat e in
  if String.length flat <= 56 then
    match e with
    | Var _ | Lit _ -> true
    | Quote e' -> expr_is_compact e'
    | App (f, args) ->
        List.length args <= 5
        && expr_is_compact f
        && List.for_all expr_is_compact args
    | Lambda (ps, b) -> List.length ps <= 4 && expr_is_compact b
    | Prim (_, args) -> List.for_all expr_is_compact args
    | If (t, th, el) ->
        expr_is_compact t && expr_is_compact th && expr_is_compact el
    | Seq (a, b) -> expr_is_compact a && expr_is_compact b
    | Let (_, v, b) -> expr_is_compact v && expr_is_compact b
    | Letrec (bs, b) ->
        List.length bs <= 2
        && List.for_all (fun (_, e) -> expr_is_compact e) bs
        && expr_is_compact b
    | Define (_, e) -> expr_is_compact e
  else false

(** Pretty-print an expr with indentation for the Functions browser /
    editor. *)
let rec pp_expr_at (depth : int) (e : expr) : string =
  if depth > 0 && expr_is_compact e then pp_expr_flat e
  else
    let ind1 = pp_indent (depth + 1) in
    match e with
    | Var s -> s
    | Lit v -> pp_value v
    | Quote e' ->
        if expr_is_compact e' then "'" ^ pp_expr_flat e'
        else "(quote\n" ^ ind1 ^ pp_expr_at (depth + 1) e' ^ ")"
    | App (f, []) -> "(" ^ pp_expr_at depth f ^ ")"
    | App (f, args) ->
        if expr_is_compact e then pp_expr_flat e
        else
          let head = pp_expr_at (depth + 1) f in
          let body =
            List.map
              (fun a -> ind1 ^ pp_expr_at (depth + 1) a)
              args
```

```
          in
          "(" ^ head ^ "\n" ^ String.concat "\n" body ^ ")"
    | Lambda (ps, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(lambda (" ^ String.concat " " ps ^ ")\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | If (t, th, el) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(if\n"
          ^ ind1 ^ pp_expr_at (depth + 1) t ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) th ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) el ^ ")"
    | Let (n, v, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(let ((" ^ n ^ "\n"
          ^ ind1 ^ "  " ^ pp_expr_at (depth + 2) v ^ "))\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | Letrec (bs, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          let binds =
            List.map
              (fun (n, e') ->
                ind1 ^ "(" ^ n ^ "\n"
                ^ pp_indent (depth + 2) ^ pp_expr_at (depth + 2) e' ^ ")")
              bs
          in
          "(letrec (\n"
          ^ String.concat "\n" binds ^ ")\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | Seq (a, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(seq\n"
          ^ ind1 ^ pp_expr_at (depth + 1) a ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | Prim (n, args) ->
        if expr_is_compact e then pp_expr_flat e
        else if args = [] then "(prim " ^ n ^ ")"
        else
          "(prim " ^ n ^ "\n"
          ^ String.concat "\n"
              (List.map
                 (fun a -> ind1 ^ pp_expr_at (depth + 1) a)
                 args)
          ^ ")"
    | Define (n, body) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(define " ^ n ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) body ^ ")"

let pp_expr (e : expr) : string = pp_expr_at 0 e
```

61 image/test/ai_store_test.ml

```
(** Round-trip tests for ai_store (needs Postgres from scripts/dev.sh).

    Skips cleanly when PP_DB is unreachable so unit CI without PG still
    builds. Live smoke is the primary verification. *)
open Lwt.Syntax

let env_or name dft =
  match Sys.getenv_opt name with Some s when s <> "" -> s | _ -> dft

let with_store f =
  let cfg = Pricklypear_image_lib.Store.config_from_env () in
  let store =
    Pricklypear_image_lib.Store.make
      ~cfg
      ~branch_id:"00000000-0000-0000-0000-000000000001"
      ()
  in
  Lwt_main.run
    (let* () = Pricklypear_image_lib.Store.connect store in
     match Pricklypear_image_lib.Store.conn_of store with
     | None ->
         Printf.printf "SKIP: no postgres (set PP_DB_* / run
    scripts/dev.sh)\n%!";
         Lwt.return_unit
     | Some _ -> f store)

let test_round_trip () =
  with_store (fun store ->
    let open Pricklypear_image_lib in
    let owner = "ai_store_test_" ^ string_of_int (Random.int 1_000_000) in
    let* created =
      Ai_store.create_session store ~owner ~seed:"system seed" ~tools:true
        ~title:"t1" ~branch_ref:"system/main" ~model:"test-model" ()
    in
    match created with
    | Error e -> Alcotest.fail e
    | Ok sess ->
        let* r1 =
          Ai_store.append_messages store ~session_id:sess.id
            [ Llm_client.Msg { role = Llm_client.User; content = "hello" }
            ; Llm_client.Msg { role = Llm_client.Assistant; content = "hi" }
            ]
        in
        (match r1 with Error e -> Alcotest.fail e | Ok () -> ());
        let tc : Llm_client.tool_call =
          { id = "call_1"; name = "list_bindings"; arguments = "{}" }
        in
        let* r2 =
          Ai_store.append_messages store ~session_id:sess.id
            [ Llm_client.Assistant_tools
                { content = Some "checking"; tool_calls = [ tc ] }
            ; Llm_client.Tool_result
                { tool_call_id = "call_1"; content = "(no bindings)" }
            ]
        in
        (match r2 with Error e -> Alcotest.fail e | Ok () -> ());
        let* hist = Ai_store.list_messages store ~session_id:sess.id in
        Alcotest.(check int) "4 messages" 4 (List.length hist);
        let api = Ai_store.messages_for_api sess hist in
        (match api with
         | Llm_client.Msg { role = Llm_client.System; content } :: _ ->
             Alcotest.(check string) "seed injected" "system seed" content
         | _ -> Alcotest.fail "expected system seed first");
        Alcotest.(check int) "api msgs = 1+history" 5 (List.length api);
        let* got = Ai_store.get_session store ~owner ~id:sess.id in
        (match got with
         | None -> Alcotest.fail "get_session missing"
         | Some s ->
             Alcotest.(check string) "owner" owner s.owner;
             Alcotest.(check bool) "tools" true s.tools);
        let* listed = Ai_store.list_sessions store ~owner () in
```

```
        Alcotest.(check bool) "listed" true (List.length listed >= 1);
        let* del = Ai_store.delete_session store ~owner ~id:sess.id in
        Alcotest.(check bool) "deleted" true del;
        let* gone = Ai_store.get_session store ~owner ~id:sess.id in
        Alcotest.(check bool) "gone" true (gone = None);
        Lwt.return_unit)

let () =
  Random.self_init ();
  Alcotest.run "ai_store"
    [ ( "round-trip"
      , [ Alcotest.test_case "create/append/list/delete" `Quick
    test_round_trip
        ] )
    ]
```

62 image/test/auth_password_test.ml

```
(** Pure unit tests for interim password hashing + token hash. *)

let () = Mirage_crypto_rng_unix.use_default ()

let hash_roundtrip () =
  let h = Pricklypear_image_lib.Auth_password.hash_password "s3cret" in
  Alcotest.(check bool) "prefix" true
    (String.length h > 10
     && String.sub h 0 7 = "sha256$");
  Alcotest.(check bool) "verify ok" true
    (Pricklypear_image_lib.Auth_password.verify_password ~stored:h
     "s3cret");
  Alcotest.(check bool) "verify bad" false
    (Pricklypear_image_lib.Auth_password.verify_password ~stored:h "wrong")

let distinct_salts () =
  let a = Pricklypear_image_lib.Auth_password.hash_password "same" in
  let b = Pricklypear_image_lib.Auth_password.hash_password "same" in
  Alcotest.(check bool) "different hashes" true (a <> b);
  Alcotest.(check bool) "both verify" true
    (Pricklypear_image_lib.Auth_password.verify_password ~stored:a "same"
     && Pricklypear_image_lib.Auth_password.verify_password ~stored:b
     "same")

let malformed_reject () =
  Alcotest.(check bool) "empty" false
    (Pricklypear_image_lib.Auth_password.verify_password ~stored:"" "x");
  Alcotest.(check bool) "garbage" false
    (Pricklypear_image_lib.Auth_password.verify_password
     ~stored:"not-a-hash" "x");
  Alcotest.(check bool) "partial" false
    (Pricklypear_image_lib.Auth_password.verify_password
       ~stored:"sha256$ab" "x")

let secure_equal_tests () =
  Alcotest.(check bool) "eq" true
    (Pricklypear_image_lib.Auth_password.secure_equal "abc" "abc");
  Alcotest.(check bool) "neq" false
    (Pricklypear_image_lib.Auth_password.secure_equal "abc" "abd");
  Alcotest.(check bool) "len" false
    (Pricklypear_image_lib.Auth_password.secure_equal "ab" "abc")

let token_hash_deterministic () =
  let h1 = Pricklypear_image_lib.Auth_password.sha256_hex "tok" in
  let h2 = Pricklypear_image_lib.Auth_password.sha256_hex "tok" in
  Alcotest.(check string) "det" h1 h2;
  Alcotest.(check bool) "diff" true
    (h1 <> Pricklypear_image_lib.Auth_password.sha256_hex "tok2")

let () =
  Alcotest.run "auth_password"
    [ ("hash",
       [ ("roundtrip", `Quick, hash_roundtrip);
         ("distinct salts", `Quick, distinct_salts);
         ("malformed reject", `Quick, malformed_reject);
         ("secure_equal", `Quick, secure_equal_tests);
         ("token hash det", `Quick, token_hash_deterministic) ]) ]
```

63 image/test/packages_test.ml

```
(** Packages prefix helpers *)
module P = Pricklypear_image_lib.Packages

let test_package_of_name () =
  Alcotest.(check string) "todo/list" "todo" (P.package_of_name
     "todo/list");
  Alcotest.(check string) "nested" "todo/ui" (P.package_of_name
     "todo/ui/row");
  Alcotest.(check string) "root" "" (P.package_of_name "hi-page")

let test_local_name () =
  Alcotest.(check string) "list" "list" (P.local_name "todo/list");
  Alcotest.(check string) "root local" "hi-page" (P.local_name "hi-page")

let test_join () =
  Alcotest.(check string) "join" "todo/list"
    (P.join_name ~package:"todo" ~local:"list");
  Alcotest.(check string) "root join" "greet"
    (P.join_name ~package:"" ~local:"greet");
  Alcotest.(check string) "strip slash" "todo/list"
    (P.join_name ~package:"todo/" ~local:"list")

let test_encode () =
  Alcotest.(check string) "encode" "todo%2Flist" (P.name_encode
     "todo/list");
  Alcotest.(check string) "decode" "todo/list" (P.name_decode "todo%2Flist")

let test_group () =
  let defs =
    [ ("hi-page", (), "a")
    ; ("todo/list", (), "a")
    ; ("todo/add", (), "a")
    ; ("calendar/today", (), "b")
    ]
  in
  let g = P.group_by_package defs in
  let pkgs = List.map fst g in
  Alcotest.(check (list string)) "pkg order" [""; "calendar"; "todo"] pkgs;
  let todo = List.assoc "todo" g in
  Alcotest.(check int) "todo count" 2 (List.length todo)

let () =
  Alcotest.run "pricklypear-packages"
    [ ( "packages"
      , [ ("package_of_name", `Quick, test_package_of_name)
        ; ("local_name", `Quick, test_local_name)
        ; ("join", `Quick, test_join)
        ; ("encode", `Quick, test_encode)
        ; ("group", `Quick, test_group)
        ] )
    ]
```

64 image/test/primitives_test.ml

```
(** Pricklypear Image -- primitive tests (no DB).

    Tests the PURE primitives in image/lib/primitives.ml by evaluating
    Lisp forms against a session prim_table + env built the same way
    Session.make builds them. We don't actually call Session (which
    needs manifest + store); instead we build the table directly.

    The interpreter is Lwt-typed (post-refactor), so each test drive
    uses [Lwt_main.run] to obtain the value from the promise. Pure
    prims wrap their results in [Lwt.return] via [register_sync]. *)
open Pricklypear_lisp.Ast

(* Build a prim_table + an env with all prims auto-defined as VPrim,
   so `(+)` etc. resolve. Mirrors image/lib/session.ml's setup. *)
let make_test_env () =
  let prim_table = Pricklypear_image_lib.Primitives.make_with_defaults () in
  let env = Pricklypear_lisp.Env.create () in
  Hashtbl.iter
    (fun name _ ->
      Pricklypear_lisp.Env.define env name (VPrim name))
    prim_table;
  (env, Pricklypear_image_lib.Primitives.to_eval_table prim_table)

(* Helper: eval a Lisp form string against the test env, return the
   resulting value (driving the Lwt promise via [Lwt_main.run]).
   Rethrows Interp_error so the test message shows it. *)
let eval s =
  let (env, prims) = make_test_env () in
  Lwt_main.run (Pricklypear_lisp.Eval.eval_string ~prims env s)

(* Helper: eval to a string-pretty form. *)
let eval_str s =
  Pricklypear_lisp.Reader.pp_value (eval s)

(* --- arithmetic --- *)
let arithmetic_tests = [
  ("+ variadic sums", `Quick, fun () ->
    Alcotest.(check string) "sum" "42" (eval_str "(+ 1 2 39)"));
  ("+ identity", `Quick, fun () ->
    Alcotest.(check string) "0-arg add" "0" (eval_str "(+)"));
  ("- binary", `Quick, fun () ->
    Alcotest.(check string) "sub" "5" (eval_str "(- 10 5)"));
  ("- unary negates", `Quick, fun () ->
    Alcotest.(check string) "neg" "-7" (eval_str "(- 7)"));
  ("* variadic products", `Quick, fun () ->
    Alcotest.(check string) "prod" "24" (eval_str "(* 2 3 4)"));
  ("* identity", `Quick, fun () ->
    Alcotest.(check string) "1 mul" "1" (eval_str "(*)"));
  ("= numeric equality true", `Quick, fun () ->
    Alcotest.(check string) "eq true" "#t" (eval_str "(= 5 5)"));
  ("= numeric equality false", `Quick, fun () ->
    Alcotest.(check string) "eq false" "#f" (eval_str "(= 5 6)"));
  ("<  less-than true", `Quick, fun () ->
    Alcotest.(check string) "lt true" "#t" (eval_str "(< 3 4)"));
  (">  greater-than false", `Quick, fun () ->
    Alcotest.(check string) "gt" "#f" (eval_str "(> 3 4)"));
  ("<= true on equal", `Quick, fun () ->
    Alcotest.(check string) "le eq" "#t" (eval_str "(<= 4 4)"));
  (">= false on less", `Quick, fun () ->
    Alcotest.(check string) "ge" "#f" (eval_str "(>= 3 5)"));
  ("not flips bool", `Quick, fun () ->
    Alcotest.(check string) "not true" "#f" (eval_str "(not #t)"));
  ("not nil -> true", `Quick, fun () ->
    Alcotest.(check string) "not nil" "#t" (eval_str "(not ())"));
]

(* --- list/pair operations --- *)
let list_tests = [
  ("list variadic", `Quick, fun () ->
    Alcotest.(check string) "list" "(1 2 3)" (eval_str "(list 1 2 3)"));
  ("list zero-arg is nil", `Quick, fun () ->
```

```
    Alcotest.(check string) "empty list" "()" (eval_str "(list)"));
  ("car of list", `Quick, fun () ->
    Alcotest.(check string) "car head" "1" (eval_str "(car (list 1 2 3))"));
  ("cdr of list", `Quick, fun () ->
    Alcotest.(check string) "cdr tail" "(2 3)" (eval_str "(cdr (list 1 2
     3))"));
  ("car of pair", `Quick, fun () ->
    Alcotest.(check string) "car pair" "1" (eval_str "(car (cons 1 2))"));
  ("cdr of pair", `Quick, fun () ->
    Alcotest.(check string) "cdr pair" "2" (eval_str "(cdr (cons 1 2))"));
  ("cons onto list", `Quick, fun () ->
    Alcotest.(check string) "cons" "(0 1 2)" (eval_str "(cons 0 (list 1
     2))"));
  ("null? empty", `Quick, fun () ->
    Alcotest.(check string) "null?()" "#t" (eval_str "(null? ())"));
  ("null? on non-empty", `Quick, fun () ->
    Alcotest.(check string) "null? non-empty" "#f"
      (eval_str "(null? (list 1 2))"));
  ("pair? on pair", `Quick, fun () ->
    Alcotest.(check string) "pair? pair" "#t"
      (eval_str "(pair? (cons 1 2))"));
  ("pair? on empty list", `Quick, fun () ->
    Alcotest.(check string) "pair? empty" "#f"
      (eval_str "(pair? (list))"));
  ("count on list", `Quick, fun () ->
    Alcotest.(check string) "count" "3" (eval_str "(count (list 1 2 3))"));
  ("count on nil", `Quick, fun () ->
    Alcotest.(check string) "count nil" "0" (eval_str "(count ())"));
]

(* --- string / reflection --- *)
let string_tests = [
  ("string-append", `Quick, fun () ->
    Alcotest.(check string) "concat" "\"foobarbaz\""
      (eval_str "(string-append \"foo\" \"bar\" \"baz\")"));
  ("str of int", `Quick, fun () ->
    Alcotest.(check string) "str int" "\"42\""
      (eval_str "(str 42)"));
  ("str of bool", `Quick, fun () ->
    Alcotest.(check string) "str bool" "\"#t\""
      (eval_str "(str #t)"));
  ("type-of int", `Quick, fun () ->
    Alcotest.(check string) "type int" "\"int\""
      (eval_str "(type-of 5)"));
  ("type-of str", `Quick, fun () ->
    Alcotest.(check string) "type str" "\"str\""
      (eval_str "(type-of \"hello\")"));
  ("type-of list", `Quick, fun () ->
    Alcotest.(check string) "type list" "\"list\""
      (eval_str "(type-of (list 1 2))"));
  ("type-of bool", `Quick, fun () ->
    Alcotest.(check string) "type bool" "\"bool\""
      (eval_str "(type-of #f)"));
]

(* --- composition: define + call + nested arithmetic --- *)
let composition_tests = [
  ("let-bind persists through lambda", `Quick, fun () ->
    Alcotest.(check string) "let persists"
      "25"
      (eval_str "(let ((square (lambda (x) (* x x)))) (square 5))"));
  ("let + arithmetic", `Quick, fun () ->
    Alcotest.(check string) "let nested" "30"
      (eval_str "(let ((x 5) (y 6)) (* x y))"));
  ("if true branch", `Quick, fun () ->
    Alcotest.(check string) "if true" "\"yes\""
      (eval_str "(if #t \"yes\" \"no\")"));
  ("if false via nil", `Quick, fun () ->
    Alcotest.(check string) "if nil" "\"no\""
      (eval_str "(if () \"yes\" \"no\")"));
  ("nested arithmetic inside lambda body", `Quick, fun () ->
    Alcotest.(check string) "nested"
      "12"
```

```
      (eval_str "((lambda (a b) (+ (* a 2) (* b 3))) 3 2)"));
]

(* === IT13 PROBE: reader parses JSON-form strings with escaped quotes ===
   This is a real test for a real bug found while building CRUD primitives:
   the reader's escape handler was reading the '\\' twice (peeks '\\',
   then advances which returns '\\', then matches '\\' as the escape
   letter -- so `\"` added '\\' to buf and the following '"' closed the
   string). After the fix, `"\"name\":\"alice\""` should read back as
   the string `{"name":"alice"}`. *)
let json_string_probe () =
  let json_str = "{\"name\":\"alice\",\"age\":30}" in   (* the JSON data *)
  let lisp_src = "\"" ^ String.escaped json_str ^ "\"" in  (* lisp string
    literal *)
  let e = Pricklypear_lisp.Reader.read_one lisp_src in
  (match e with
   | Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VStr s) ->
       Alcotest.(check string)
         "reader parses JSON-form string (escapes work)"
         json_str s
   | _ ->
       Alcotest.failf "expected Lit VStr, got %s"
         (Pricklypear_lisp.Reader.pp_expr e))

let probe_tests = [
  ("reader parses JSON string with \\\" escapes", `Quick,
    json_string_probe);
]

(* Helper: read key back from JSON-string via Yojson. Used by the
   dict-set test to verify the new key round-trips through a parse. *)
let dict_get_back (json : string) (key : string) : string option =
  try
    (match Yojson.Safe.from_string json with
     | `Assoc kvs ->
         (match List.assoc_opt key kvs with
          | Some (`String s) -> Some s
          | Some (`Int n) -> Some (string_of_int n)
          | _ -> None)
     | _ -> None)
  with _ -> None

(* --- dict primitives (operate on VStr JSON; pure, no DB) --- *)
let dict_tests = [
  ("dict-get string value", `Quick, fun () ->
    Alcotest.(check string) "get name"
      "\"alice\"" (* pp_value of VStr adds outer quotes *)
      (eval_str "(dict-get \"{\\\"name\\\":\\\"alice\\\",\\\"age\\\":30}\"
    \"name\")"));
  ("dict-get int value stringified", `Quick, fun () ->
    Alcotest.(check string) "get age"
      "\"30\"" (* pp_value of VStr adds outer quotes *)
      (eval_str "(dict-get \"{\\\"name\\\":\\\"alice\\\",\\\"age\\\":30}\"
    \"age\")"));
  ("dict-get missing key is nil", `Quick, fun () ->
    Alcotest.(check string) "missing key"
      "()"
      (eval_str "(dict-get \"{\\\"a\\\":1}\" \"nope\")"));
  ("dict-set adds new key", `Quick, fun () ->
    (* The exact key order may vary; just check the chosen key is present
    and decodes back. *)
    let v = eval "(dict-set \"{\\\"a\\\":1}\" \"b\" \"2\")" in
    (match v with
     | VStr s ->
         let r = dict_get_back s "b" in
         Alcotest.(check (option string)) "b present" (Some "2") r
     | _ ->
         Alcotest.failf "expected VStr, got %s"
    (Pricklypear_lisp.Reader.pp_value v)));
  ("dict-keys lists 2 keys", `Quick, fun () ->
    let v = eval "(dict-keys
    \"{\\\"name\\\":\\\"alice\\\",\\\"age\\\":30}\")" in
    (match v with
```

```
    | VList [ VStr _; VStr _ ] -> ()
    | _ ->
        Alcotest.failf "expected list of 2 keys, got %s"
          (Pricklypear_lisp.Reader.pp_value v)));
]

(* --- render-html primitive (tagged VStr) --- *)
let render_html_tests = [
  ("render-html tags the string", `Quick, fun () ->
    let v = eval "(render-html \"<b>hi</b>\")" in
    Alcotest.(check bool) "is raw html struck"
      true
      (Pricklypear_image_lib.Primitives.is_raw_html v <> None));
  ("render-html raw content extractable", `Quick, fun () ->
    let v = eval "(render-html \"<h2>title</h2>\")" in
    Alcotest.(check (option string)) "content"
      (Some "<h2>title</h2>")
      (Pricklypear_image_lib.Primitives.is_raw_html v));
  ("plain VStr not raw html", `Quick, fun () ->
    Alcotest.(check (option string)) "not raw"
      None
      (Pricklypear_image_lib.Primitives.is_raw_html (VStr "plain")));
]

let () =
  Alcotest.run "pricklypear-image-primitives" [
    "arithmetic", arithmetic_tests;
    "list", list_tests;
    "string", string_tests;
    "probe", probe_tests;
    "dict", dict_tests;
    "render-html", render_html_tests;
    "composition", composition_tests;
  ]
```

65 lisp/bin/read_test.ml

```
let () =
  let path = Sys.argv.(1) in
  let ic = open_in path in
  let s = really_input_string ic (in_channel_length ic) in
  close_in ic;
  let forms = Pricklypear_lisp.Reader.read_all s in
  List.iteri (fun i e ->
    Printf.printf "%d: %s\n" i (Pricklypear_lisp.Reader.pp_expr e)) forms
```

66 lisp/bin/repl.ml

```
(** Pricklypear Lisp -- standalone dev REPL.

   No image, no Postgres, no SSH. Reads s-expressions from stdin,
   evaluates them, prints results. Useful for testing the interpreter
   without booting the full image.

   Per spec [architecture.package-layout]: lisp/bin/repl.ml is a dev
   convenience, not the production REPL (which lives in the image and
   is reached over the supervisor-fronted SSH -> image REPL endpoint).

   The interpreter is Lwt-typed (post-refactor), so this dev REPL drives
   each turn through [Lwt_main.run (Eval.eval_string ...)]. *)
open Lwt
open Lwt.Syntax
open Pricklypear_lisp

let prompt = "lisp> "

(* Run one evaluation, print the result or error, return either cleanly.
   [loop] loops on the result. *)
let run_one env prims input : unit =
  (try
     let v = Lwt_main.run (Eval.eval_string ~prims env input) in
     print_endline (Reader.pp_value v)
   with
   | Reader.Reader_error (msg, ln, col) ->
     Printf.printf "Read error (line %d, col %d): %s\n" ln col msg
   | (Ast.Interp_error _ as exn) ->
     Printf.printf "Error: %s\n" (Printexc.to_string exn))

let rec loop env prims =
  print_string prompt;
  flush stdout;
  match input_line stdin with
  | exception End_of_file -> print_endline "bye"
  | line ->
    if String.trim line = "" then loop env prims
    else
      let () =
        if not (Reader.is_complete line) then
          (* keep reading until parens balance *)
          let buf = Buffer.create 64 in
          Buffer.add_string buf line;
          let rec more () =
            print_string "  ... ";
            flush stdout;
            (match input_line stdin with
             | exception End_of_file -> print_endline "(aborted)"
             | l ->
                Buffer.add_char buf '\n';
                Buffer.add_string buf l;
                if Reader.is_complete (Buffer.contents buf) then ()
                else more ())
          in
          more ();
          run_one env prims (Buffer.contents buf)
        else
          run_one env prims line
      in
      loop env prims

let () =
  let env = Env.create () in
  let prims = Eval.make_prim_table () in
  (* Register a couple of demo primitives for the dev REPL. Post-refactor,
     prim_fn = `value list -> value Lwt.t`, so pure prims wrap their
     results in `Lwt.return`. *)
  Eval.register_prim prims "+"
    (fun args ->
      (* variadic + over ints *)
      let n =
```

```
      List.fold_left (fun acc v ->
        match v with Ast.VInt i -> Int64.add acc i | _ -> acc) 0L args
    in
    Lwt.return (Ast.VInt n));
Eval.register_prim prims "println"
  (fun args ->
    List.iter (fun v -> print_endline (Reader.pp_value v)) args;
    Lwt.return Ast.VNil);
print_endline "Pricklypear Lisp dev REPL. Ctrl-D to quit.";
loop env prims
```

67 lisp/test/effect_test.ml

```
(** Capability effect inference tests -- pure, no services. *)
open Pricklypear_lisp

let infer s =
  let e = Reader.read_one s in
  Effect.infer e

let pp_fx e = Effect.EffSet.pp e

let checks = [
  ("pure arithmetic", `Quick, fun () ->
    let fx = infer "(* x x)" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("lambda pure", `Quick, fun () ->
    let fx = infer "(lambda (x) (* x x))" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("get is read-db", `Quick, fun () ->
    let fx = infer "(get 'note id)" in
    Alcotest.(check bool) "read" true (Effect.EffSet.mem Effect.ReadDb fx);
    Alcotest.(check bool) "not write" false (Effect.EffSet.mem
     Effect.WriteDb fx));
  ("create is write-db", `Quick, fun () ->
    let fx = infer "(create 'note \"{}\")" in
    Alcotest.(check bool) "write" true (Effect.EffSet.mem Effect.WriteDb
     fx));
  ("route is publish", `Quick, fun () ->
    let fx = infer "(route \"/x\" \"f\")" in
    Alcotest.(check bool) "pub" true (Effect.EffSet.mem Effect.Publish fx));
  ("render-html is html", `Quick, fun () ->
    let fx = infer "(render-html \"<b>x</b>\")" in
    Alcotest.(check bool) "html" true (Effect.EffSet.mem Effect.Html fx));
  ("propose-function is ai", `Quick, fun () ->
    let fx = infer "(propose-function \"f\" \"spec\")" in
    Alcotest.(check bool) "ai" true (Effect.EffSet.mem Effect.Ai fx));
  ("compose read+html", `Quick, fun () ->
    let fx = infer "(lambda () (render-html (dict-get (get 'n id) \"t\")))"
     in
    Alcotest.(check bool) "read" true (Effect.EffSet.mem Effect.ReadDb fx);
    Alcotest.(check bool) "html" true (Effect.EffSet.mem Effect.Html fx));
  ("badge pure", `Quick, fun () ->
    Alcotest.(check string) "b" "pure" (Effect.EffSet.badge
     Effect.EffSet.empty));
  ("policy allows pure", `Quick, fun () ->
    Alcotest.(check bool) "ok" true
      (Effect.policy_allows ~allow:Effect.ai_default_allow
     Effect.EffSet.empty));
  ("policy allows write-db for app scaffolding", `Quick, fun () ->
    let got = Effect.EffSet.singleton Effect.WriteDb in
    Alcotest.(check bool) "allow write" true
      (Effect.policy_allows ~allow:Effect.ai_default_allow got));
  ("policy denies publish without elevate", `Quick, fun () ->
    let got = Effect.EffSet.singleton Effect.Publish in
    Alcotest.(check bool) "deny publish" false
      (Effect.policy_allows ~allow:Effect.ai_default_allow got));
  ("policy allows read+write+html", `Quick, fun () ->
    let got =
      Effect.EffSet.normalize
        [ Effect.ReadDb; Effect.WriteDb; Effect.Html ]
    in
    Alcotest.(check bool) "ok" true
      (Effect.policy_allows ~allow:Effect.ai_default_allow got));
]

let () =
  Alcotest.run "pricklypear-lisp-effects" [ "effects", checks ]
```

68 lisp/test/eval_test.ml

```
(** Tests for the Pricklypear Lisp interpreter.

   These are PURE tests -- no Postgres, no HTTP, no SSH. They verify the
   reader, AST round-trip, and evaluator. Per spec
   [lisp-interpreter.eval] / [workflow.build]:
     cd lisp/ && dune build && dune runtest
   needs no services. *)
open Pricklypear_lisp

(* --- Reader tests --- *)

let reads_as name input expected =
  ("reader: " ^ name, `Quick, fun () ->
    match Reader.read_one input with
    | Lit v -> Alcotest.(check string)
                 (Reader.pp_value expected) (Reader.pp_value expected)
                 (Reader.pp_value v)
    | other -> Alcotest.failf "expected Lit, got %s" (Reader.pp_expr other))

let reader_tests = [
  reads_as "int" "42" (Ast.VInt 42L);
  reads_as "negative int" "-7" (Ast.VInt (-7L));
  ("reader: float literal", `Quick, fun () ->
    match Reader.read_one "3.14" with
    | Lit (Ast.VFloat f) ->
        if abs_float (f -. 3.14) > 1e-9 then
          Alcotest.failf "float value %g" f
    | other -> Alcotest.failf "expected VFloat lit, got %s" (Reader.pp_expr
     other));
  reads_as "true" "#t" (Ast.VBool true);
  reads_as "false" "#f" (Ast.VBool false);
  reads_as "string" "\"hello\"" (Ast.VStr "hello");
  reads_as "nil" "()" Ast.VNil;
]

let reader_list_tests = [
  ("reader: list of ints parses", `Quick, fun () ->
    (* (1 2 3) reads as App(Lit 1, [Lit 2; Lit 3]); head is separate *)
    match Reader.read_one "(1 2 3)" with
    | App (_, [ Lit (Ast.VInt _); Lit (Ast.VInt _) ]) -> ()
    | other -> Alcotest.failf "expected App with 2 li int args, got %s"
                                 (Reader.pp_expr other));
  ("reader: nested list parses", `Quick, fun () ->
    match Reader.read_one "(foo (bar) baz)" with
    | Ast.App _ -> ()
    | other -> Alcotest.failf "expected App, got %s" (Reader.pp_expr
     other));
]

let reader_completeness_tests = [
  ("is_complete: balanced parens", `Quick, fun () ->
    Alcotest.(check bool) "balanced" true (Reader.is_complete "(foo bar)"));
  ("is_complete: unbalanced parens", `Quick, fun () ->
    Alcotest.(check bool) "unbalanced" false (Reader.is_complete "(foo
     bar"));
  ("is_complete: unclosed string", `Quick, fun () ->
    Alcotest.(check bool) "unclosed" false (Reader.is_complete "\"hello"));
  ("is_complete: comment only", `Quick, fun () ->
    Alcotest.(check bool) "comment" true (Reader.is_complete "; comment
     only"));
]

(* --- AST JSON round-trip tests --- *)

let roundtrip name e =
  ("ast roundtrip: " ^ name, `Quick, fun () ->
    let j = Ast.expr_to_json e in
    (match Ast.expr_of_json j with
     | Ok e' ->
        Alcotest.(check string)
          (Reader.pp_expr e) (Reader.pp_expr e) (Reader.pp_expr e')
```

```
    | Error err -> Alcotest.failf "roundtrip failed: %s" err))

let ast_roundtrip_tests = [
  roundtrip "var" (Ast.Var "x");
  roundtrip "int" (Ast.int 5);
  roundtrip "string" (Ast.str "hello");
  roundtrip "if" (Ast.If (Ast.var "x", Ast.int 1, Ast.int 2));
  roundtrip "lambda" (Ast.Lambda (["x"], Ast.var "x"));
  roundtrip "let" (Ast.Let ("y", Ast.int 5, Ast.var "y"));
  roundtrip "prim" (Ast.Prim ("query", [Ast.str "event"]));
  roundtrip "define" (Ast.Define ("greet", Ast.Lambda (["name"], Ast.Var
     "name")));
  roundtrip "app" (Ast.App (Ast.Var "f", [Ast.int 1; Ast.int 2]));
]

(* --- Evaluator tests --- *)

(* The interpreter is Lwt-typed: Eval.eval returns `value Lwt.t`. Each test
   body that calls the evaluator uses [Lwt_main.run] to drive the promise;
   reader/AST-roundtrip tests are pure and don't need Lwt wrapping. *)

let evals_to name expr expected =
  ("eval: " ^ name, `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let budget = Eval.make_budget () in
    let got = Lwt_main.run (Eval.eval ~budget ~prims env expr) in
    Alcotest.(check string)
      (Reader.pp_value expected) (Reader.pp_value expected)
      (Reader.pp_value got))

let eval_tests = [
  evals_to "int literal" (Ast.int 42) (Ast.VInt 42L);
  ("eval: define binds var", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let _ = Lwt_main.run (Eval.eval_string ~prims env "(define x 99)") in
    (match Env.lookup env "x" with
     | Some (Ast.VInt n) ->
        Alcotest.(check int64) "x's value" 99L n
     | other -> Alcotest.failf "expected Some VInt, got %s"
                                (match other with None -> "None"
                                 | Some v -> Reader.pp_value v)));
  ("eval: if-true branch", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "(if #t 1 2)") in
    Alcotest.(check string) "1" "1" (Reader.pp_value got));
  ("eval: if-false branch", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "(if #f 1 2)") in
    Alcotest.(check string) "2" "2" (Reader.pp_value got));
  ("eval: lambda application", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "((lambda (x) x)
     42)") in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval: quote compound list", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "'(a b c)") in
    (match got with
     | Ast.VList [Ast.VStr "a"; Ast.VStr "b"; Ast.VStr "c"] -> ()
     | other -> Alcotest.failf "expected (a b c) list, got %s"
                  (Reader.pp_value other)));
  ("eval: quote nested", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "'(1 (2 3))") in
    (match got with
     | Ast.VList [Ast.VInt 1L; Ast.VList [Ast.VInt 2L; Ast.VInt 3L]] -> ()
```

```
    | other -> Alcotest.failf "expected nested quote, got %s"
                (Reader.pp_value other)));
  ("eval: primitive dispatch", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "add-one"
      (fun args -> match args with
        | [Ast.VInt n] -> Lwt.return (Ast.VInt (Int64.add n 1L))
        | _ -> Lwt.return Ast.VNil);
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "(prim add-one
     41)") in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval: let binds locally", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Lwt_main.run (Eval.eval_string ~prims env "(let ((y 5)) y)")
     in
    Alcotest.(check string) "5" "5" (Reader.pp_value got));
  ("eval: budget exceeded", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let budget = Eval.make_budget ~limit:10 () in
    (* build a deep nest to burn through 10 steps *)
    let rec nest n acc =
      if n = 0 then acc else nest (n-1) (Ast.Seq (Ast.int 1, acc))
    in
    let e = nest 20 (Ast.int 0) in
    try
      let _ = Lwt_main.run (Eval.eval ~budget ~prims env e) in
      Alcotest.fail "expected budget exceeded"
    with Ast.Interp_error Ast.Budget_exceeded -> ());
  ("eval: unbound var raises", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    try
      let _ = Lwt_main.run (Eval.eval_string ~prims env "nope") in
      Alcotest.fail "expected Unbound_var"
    with Ast.Interp_error (Ast.Unbound_var name) ->
      Alcotest.(check string) "nope" "nope" name);
]

let pretty_print_tests = [
  ("pp_expr: nested list breaks lines", `Quick, fun () ->
    let src =
      "(list (menu/link \"home\" \"/\") (menu/group \"apps\" (list
     (menu/link \"todos\" \"/todos\"))))"
    in
    let e = Reader.read_one src in
    let pp = Reader.pp_expr e in
    if not (String.contains pp '\n') then
      Alcotest.failf "expected newlines in:\n%s" pp;
    if not (String.contains pp 'a') then
      Alcotest.fail "missing content";
    (* Round-trip: pretty output still parses to same flat shape *)
    let e' = Reader.read_one pp in
    Alcotest.(check string) "flat equal"
      (Reader.pp_expr_flat e) (Reader.pp_expr_flat e'));
  ("pp_expr: short form stays one line", `Quick, fun () ->
    let e = Reader.read_one "(+ 1 2)" in
    let pp = Reader.pp_expr e in
    Alcotest.(check bool) "no newline" false (String.contains pp '\n');
    Alcotest.(check string) "compact" "(+ 1 2)" pp);
]

let () =
  Alcotest.run "pricklypear-lisp" [
    "reader", reader_tests @ reader_list_tests @ reader_completeness_tests;
    "ast-roundtrip", ast_roundtrip_tests;
    "pretty-print", pretty_print_tests;
    "eval", eval_tests;
  ]
```